Build a Production SaaS Platform From Scratch

A rebuild-from-zero course in architecture, engineering practice, and the decisions behind them

Perezosoft Platform · generated 2026-07-08

Contents

Preface

What this book is

This is a course disguised as a book. Over ten parts and fifty lessons, you will build — by typing every hand-written line yourself — a production-grade, multi-tenant SaaS platform: custom JWT authentication with passwordless sign-in, OAuth, and MFA; tenant isolation enforced by the type system *and* by the database; billing with Stripe; reliable asynchronous work via a transactional outbox; role-based access control; file storage; GDPR export and erasure; a public API; outbound webhooks; an admin back-office; full observability — and at the end you will deploy it to a real host behind a real CI/CD pipeline that you also built.

The code is real. Every excerpt in this book is drawn from a working reference platform (Perezosoft Platform), not invented for the page. A coverage gate — a script, in the same spirit as the tests you'll write — maps every file of that platform to the lesson that builds it, so nothing is skipped and nothing appears out of thin air.

But the code is not the point. The point is the two habits that separate an architect from a very fast typist:

1. **Making invariants structural.** Not "remember to filter by tenant" but "a query *cannot* forget to filter by tenant." Not "please don't call this API in feature code" but "the build fails if you do." You will meet this move dozens of times — in query filters, interceptors, marker interfaces, architecture tests, CI gates, database policies — until reaching for it is a reflex.
1. **Recording decisions.** Every lesson ends with an *Architecture Decision* box: the fork that was faced, the option chosen, the options rejected, and the price paid. You will write your own decision log from lesson 0.1 onward, before any code exists. Six months from now, "why is it like this?" will have an answer.

Who this is for

You are comfortable in C# — classes, interfaces, async/await, LINQ hold no fear — and you can find your way around a terminal and git. You have probably built applications, perhaps professionally, but you want the *architecture* to stop feeling like folklore: why layers, why seams, why tests first, why all these patterns with strange names, and — above all — when *not* to use them.

You do not need prior experience with multi-tenancy, EF Core internals, Stripe, Docker, OAuth, or GitHub Actions. Each is taught when the platform needs it, motivated by a concrete problem you will feel before you solve it.

What this book is not

Honesty up front: this is a course in **systems design and engineering practice**, not computational problem-solving. You will find no algorithm puzzles and almost no performance engineering here — no profilers, and caching is deliberately deferred (a recorded decision you'll read about). If you want to get better at data structures, this is the

wrong book; if you want to get better at *building software that survives contact with production and with other people*, it is the right one.

The rhythm

Every lesson follows the same beat, and the beat is the curriculum:

1. **Motivate** — a concrete problem, ideally demonstrated by a failing or *leaking* test.
2. **Red** — write the test that pins the behavior you want. It fails.
3. **Green** — write the minimum code to pass it.
4. **Refactor & harden** — extract the seam; add the guardrail that makes the mistake impossible to repeat.

1. **Run it** — commands and expected output; every lesson ends with the app working.
2. **Architecture Decision** — the fork, the choice, the road not taken.
3. **Checkpoint** — a git tag; what you should now be able to do.

Test-driven development is not a chapter in this book; it is the loop you will live roughly fifty times, exactly as the reference platform was actually built.

How to use this book

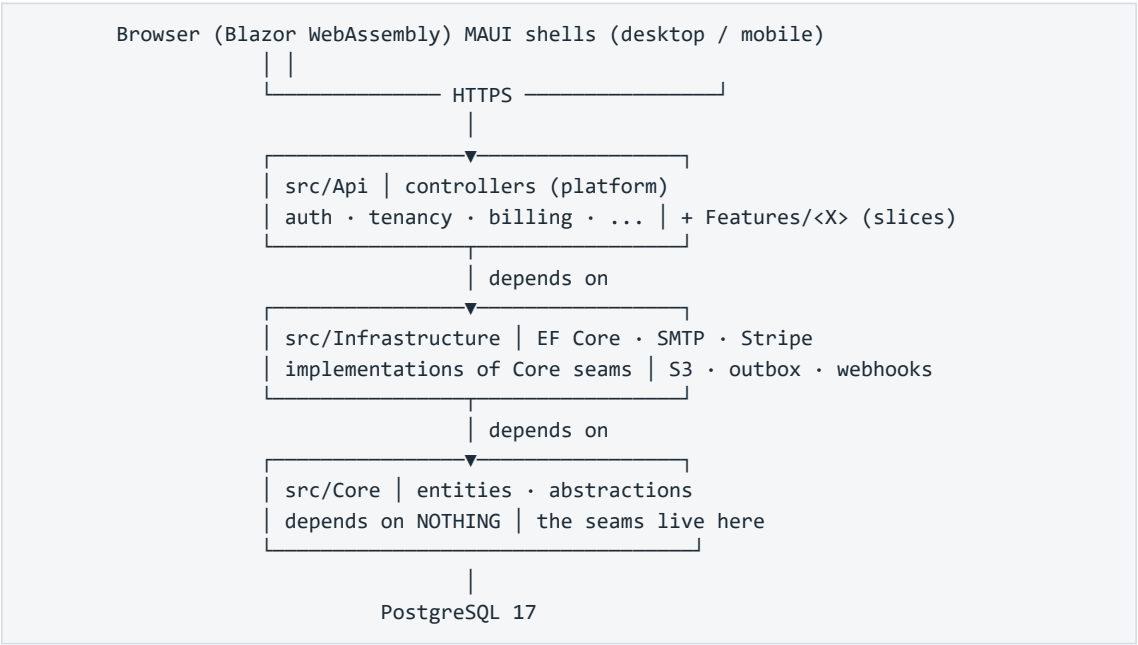
The companion repository. Alongside the book there is a companion repo built in teaching order, with one tagged checkpoint per lesson (`lesson-1.3`, `lesson-2.6`, …). Type everything yourself — that is the course — but when you are stuck, `git diff` your work against the checkpoint, or check it out and keep moving. Falling behind the typing is recoverable; skipping the understanding is not.

Don't skip Part 0. Lesson 0.2 pins your toolchain and infrastructure so that the other forty-nine lessons behave identically on your machine. Most abandoned courses die at "works on my machine"; we kill that failure mode first.

When a lesson cites a rule (R1–R35) or an ADR, that is a pointer into the reference platform's own governance documents — the machine-enforced invariants and the decision log. You are not just building an app; you are building the *system that keeps the app honest*, and those citations show you where each piece of it came from.

Reading order is dependency order. Later lessons lean on earlier seams without re-explaining them. If Part 5 feels steep, the fix is usually two parts back, not a re-read of Part 5.

The platform at a glance



One sentence to keep for the whole book: **app data belongs to the tenant, not the user** — and by the end of Part 2 you will have made it impossible, at three separate layers, for any line of feature code to violate that sentence.

Part 0 — Orientation

Lesson 0.1 — The mental model & the decision record

> **Part 0 · Orientation.** No code in this lesson — deliberately. You're about to type several hundred files over ~48 lessons. The two things that determine whether that makes you an architect or a very tired typist are (1) the mental model you hang each file on, and (2) the habit of recording *why* before *what*. Both are installed here.

Goal: you can draw the system from memory — its layers, its one non-negotiable invariant, and its unit of work — and you've written your project's first ADR.

Concepts & principles: multi-tenancy (tenant ≠ user); clean architecture's dependency rule; vertical slices vs. horizontal layers; Architecture Decision Records.

Maps to: ADR-C1/C2/C3, ADR-004 · `docs/DECISIONS.md`, `docs/WAYS_OF_WORKING.md` in the reference repo.

---

1. What you are going to build

A production-grade, multi-tenant SaaS **platform**: custom JWT auth (passwordless + OAuth + MFA), tenant isolation enforced by the type system, billing with Stripe, reliable async via a transactional outbox, RBAC, file storage, GDPR export/erasure, a public API, outbound webhooks, an admin back-office, observability — deployed to a real host at the end. Not a to-do app. The kind of chassis a real product bolts features onto.

You will build it **test-first, in vertical slices**, and every architectural rule will be enforced by a test you wrote — not by a comment hoping to be read.

2. The four ideas everything else hangs on

Idea 1 — Tenant ≠ user

The single most important sentence in this course: **app data belongs to the tenant, not the user.** A *tenant* is the paying unit — a household, a team, a company. Users are members of a tenant; they come and go, and the tenant's data stays. Get this backwards and you'll design tables, queries, and permissions around the wrong axis — and unwinding that later is a rewrite, not a refactor.

One consequence you'll meet in Lesson 2.6 and never stop meeting: every read and write of app data must be scoped to the current tenant, and that scoping must be **impossible to forget**, not merely encouraged.

Idea 2 — The dependency rule (clean architecture, minus the ceremony)

Three production layers, dependencies pointing strictly inward:

```
Api ──► Infrastructure ──► Core
(HTTP, auth, (EF Core, SMTP, (entities, abstractions —
 controllers, Stripe, S3, the depends on NOTHING)
 features) implementations)
```

`Core` holds the entities and the *seams* — small interfaces like `IEmailSender`, `IFileStorage`, `IBillingProvider`. `Infrastructure` implements them with real technology (MailKit, S3, Stripe). `Api` orchestrates. The payoff is swappability where it matters (Mailpit in dev, Brevo in prod — same seam) and testability everywhere. The UI (Blazor WebAssembly) is *just another API client* — it never touches the database.

Idea 3 — Clean platform + vertical slices (the hybrid)

Pure layered architecture scatters one feature across four folders. Pure feature-folders duplicate the platform in every slice. This codebase does both, deliberately, and the split is the point:

- The **platform** — auth, tenancy, billing, email, persistence — is the durable chassis,

organized in clean layers. Built once, reused by everything.

- Each **app feature** lives in ONE folder (`src/Api/Features/<Feature>/`) containing its

endpoints, handler, DTOs, and data contributor. Features *reuse* the platform; they never reach into each other.

When you wonder "where does this file go?", the question to ask is: *chassis or feature?*

Idea 4 — Vertical slices as the unit of work

Every increment of work cuts through all layers — DB to UI — and leaves the app working. You'll never build "all the tables" then "all the endpoints." Each lesson in this course is itself a slice: it ends with a green build, passing tests, and a tagged checkpoint.

3. The decision record — the habit that makes it stick

Here's what separates a codebase you can *maintain* from one you can only *fear*: six months from now, the question is never "what does this code do?" (you can read it), it's **"why is it like this — and is it safe to change?"**

An **Architecture Decision Record** answers that in ~10 lines, written at the moment of decision:

```
## ADR-002 — Custom JWT + rotating refresh tokens (not ASP.NET Core Identity)

**Date:** 2026-06-XX · **Status:** Accepted

**Context:** We need auth for web + native clients with passwordless and OAuth.
Identity brings cookie-centric defaults, schema we don't control, and hides the
token lifecycle we need to reason about (MFA step-up, rotation, native flows).

**Decision:** Issue our own short-lived JWT access tokens + rotating refresh
tokens; store only token hashes.

**Consequences:** We own token security (rotation, reuse detection — must test
it ourselves). In exchange: full control of claims (tenant_id drives data
isolation), one auth model across web/native, no framework fight when MFA lands.
```

Note what an ADR is *not*: not documentation of how the code works (the code shows that), and never deleted when wrong — a reversed decision gets a **new dated ADR** superseding the old one. The history of your reasoning is itself an asset.

The reference repo runs on this: 20 app ADRs + amendments in `docs/DECISIONS.md`. Every lesson ahead ends with an **Architecture Decision box** — the fork faced, the option chosen, the roads not taken. By Part 4 you should be predicting them before reading them; that reflex *is* the architectural skill this course exists to build.

4. Exercise — write your ADR-000 and ADR-001

Create your project directory (empty — the solution comes in Lesson 1.1) and start the decision log:

```
mkdir my-saas && cd my-saas && git init
mkdir docs && $EDITOR docs/DECISIONS.md
```

Write two ADRs yourself — don't copy:

1 **ADR-000 — Why I'm building this.** Context (what you want to learn), decision (build the platform from scratch, test-first), consequences (weeks of effort; deep ownership of every line).

1 **ADR-001 — Local secrets live in `.env`, never in the repo.** You haven't built the loading mechanism yet (that's 0.2 and 1.4) — which is exactly the point: **the decision precedes the code.** Context: secrets in committed files leak (git history is forever). Decision: a gitignored `.env` is the single local source of truth; a committed `.env.example` documents every key; production uses real environment variables. Consequences: one more setup step per machine; zero secrets in history — enforced by a scanner you'll add in the next lesson.

5. Checkpoint

```
git add docs/DECISIONS.md && git commit -m "docs: ADR-000 and ADR-001 — the decision log starts before the code"
git tag lesson-0.1
```

You should now be able to: state the tenant≠user invariant and its consequence for every future query; draw the three layers and their dependency direction; say which of the two organizational modes (chassis vs. feature folder) a given file belongs to; and write an ADR that captures context → decision → consequences in under a page.

Next: *Lesson 0.2 — A reproducible machine*, where your ADR-001 gets its mechanical teeth: `.env.example`, docker-compose, and a secret scanner.

Lesson 0.2 — A reproducible machine

> **Part 0 · Orientation.** Most from-scratch courses die at "install PostgreSQL." This lesson exists so yours can't. At the end, your machine runs the exact infrastructure the app will use for the next 47 lessons — pinned, containerized, verifiable with three commands — and your ADR-001 (secrets never in the repo) is enforced by machinery, not memory.

Goal: `docker compose up -d` gives you a healthy Postgres 17 and a mail trap; `dotnet --version` prints the pinned SDK; a secret can't reach a commit.

Concepts & principles: pinned toolchains ("latest stable, never previews" — and never "whatever was on the machine"); infrastructure-as-code for dev; the `.env` contract (ADR-001); fail-loud health checks; defense-in-depth for secrets.

Maps to: ADR-001, ADR-C13 · Foundation theme R20–R22 (the config contract this enables) · repo: `docker-compose.yml`, `.env.example`, `.gitignore`, `.gitleaks.toml`.

Prerequisites: 0.1 (your repo exists and ADR-001 is written).

---

1. Motivate — "works on my machine" is an architecture failure

Two developers clone the same repo. One has Postgres 15 installed as a Windows service from a project two jobs ago; the other has nothing. One gets a subtle JSON-function bug that doesn't reproduce; the other gets a connection refused. Neither failure is *in the code* — and that's the point: **your dev environment is part of your architecture**, and undeclared dependencies are its silent killers.

The fix is the same one you'll apply to code all course long: make the implicit explicit, and make violations loud.

2. Install the pinned toolchain

Three tools go on the machine itself; everything else runs in containers:

Tool	Version	Check
.NET SDK	**10.0.3xx** (the line this course pins)	`dotnet --version`
Docker Desktop / Engine	current stable	`docker version`
EF Core tooling	current stable	`dotnet tool install --global dotnet-ef` (first used in 1.3)

Pin the SDK **in the repo**, not in a README nobody reads — create `global.json`:

```
{
  "sdk": { "version": "10.0.301", "rollForward": "latestFeature" }
}
```

Now a machine with the wrong SDK fails at `dotnet build` with an explicit message instead of failing three lessons later with an inscrutable one. This is the course's recurring move — *fail*

loud and early — applied for the first time, to the toolchain itself.

3. Infrastructure as one file — docker-compose

The app needs PostgreSQL and — less obviously — an **SMTP trap**. From Lesson 2.3 onward the platform sends real email (magic links, OTPs, invitations), and you need somewhere for it to land in dev that isn't a real inbox. Mailpit accepts SMTP on one port and shows every message in a web UI on another — and, critically for Lesson 3.6, exposes an **HTTP API your tests will query** to fish out OTP codes. Two infrastructure decisions, one committed file — create `docker-compose.yml`:

```
services:
  db:
    image: postgres:17
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${DB_NAME:-dev_db}
      POSTGRES_USER: ${DB_USER:-dev}
      POSTGRES_PASSWORD: ${DB_PASSWORD:-devpassword}
    ports:
      - "${DB_PORT:-5432}:5432"
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-dev} -d ${DB_NAME:-dev_db}"]
      interval: 5s
      timeout: 5s
      retries: 5
      start_period: 10s

  mail:
    image: axllent/mailpit:latest
    restart: unless-stopped
    ports:
      - "${MAIL_SMTP_PORT:-1025}:1025" # SMTP — the app sends here
      - "${MAIL_UI_PORT:-8025}:8025" # Web UI + API — you and your tests read here
    healthcheck:
      test: ["CMD-SHELL", "wget -q --spider http://localhost:8025 || exit 1"]
      interval: 5s
      timeout: 5s
      retries: 5
      start_period: 5s

volumes:
  db_data:
```

Read it as a set of decisions, because each line is one:

- **`postgres:17`** — a pinned major version, mirroring `global.json`'s job for the SDK.

"Latest stable, never previews" also means *never unpinned*.

- **`${DB_PORT:-5432}`** — every value is overridable via environment, with a working

default. A developer whose port 5432 is squatted by that old Windows service sets `DB_PORT=5433` in `.env` and moves on. (The reference repo defaults to 5433 for exactly that reason — port collisions are the #1 setup killer.)

- **`healthcheck`** — "container started" ≠ "Postgres accepting connections." Health

checks make readiness explicit, and anything that later `depends_on` these services waits for *healthy*, not merely *running*. You'll re-meet this exact idea as the API's own `/health` endpoint in Lesson 4.5.

- **`volumes: db_data`** — your data survives `docker compose down`. (Destroying it becomes a deliberate act: `docker compose down -v`.)

4. The `.env` contract — ADR-001 gets teeth

Your ADR-001 said: secrets live in a gitignored `.env`; a committed `.env.example` documents every key. Build both halves now.

`.gitignore` (start; it grows with the solution in 1.1):

```
.env
bin/
obj/
storage/
```

`.env.example` — **committed**, and it is *documentation with a job*:

```
# Copy this file to .env and fill in your values. .env is gitignored — never commit it.
# Single source of local-dev config + secrets: read by docker-compose (containers) AND
# by the API via DotNetEnv (from Lesson 1.4). See docs/DECISIONS.md ADR-001.

# ── Containers (docker-compose) ──────────────────────────
DB_NAME=dev_db
DB_USER=dev
DB_PASSWORD=devpassword
DB_PORT=5433

# Mailpit (SMTP trap)
MAIL_SMTP_PORT=1025
MAIL_UI_PORT=8025
```

Then `cp .env.example .env` and confirm `git status` does **not** list `.env`.

Notice the shape of the contract: *every* key the system reads appears here with an inline comment saying what it does, whether it's required, and what happens when unset. In Lesson 1.4 you'll write a test (`DocAndConfigSyncTests`) that **fails the build when a config key exists in code but not in this file** — turning "please document your config" from a review nag into a compile-adjacent guarantee (R20). The habit starts now.

Docker-compose reads `.env` automatically. So one file drives both the containers and — once DotNetEnv arrives in 1.4 — the app. Two consumers, one source of truth, zero drift.

The second layer: a secret scanner

`.gitignore` is advisory — it does nothing if someone hardcodes a connection string in a `.cs` file. Add gitleaks config, `.gitleaks.toml`:

```
# Secret scanning — the backstop behind ADR-001. Default rules + our allowlist.
[extend]
useDefault = true

[allowlist]
description = "Dev-only placeholder values that look like secrets but aren't"
paths = ['''\.env\.example''']
```

Run it locally (`gitleaks detect`) whenever you like; in Lesson 8.3 it becomes a CI gate. Layers, again: `.gitignore` prevents the accident, gitleaks catches what slips past, CI makes it policy. No single layer is trusted alone — the same defense-in-depth shape you'll apply to tenancy (filter + interceptor + arch test) from Part 2 on.

5. Run it — verify everything

```
docker compose up -d
docker compose ps # both services: status "healthy" (wait a few seconds)
dotnet --version # 10.0.3xx — matches global.json
docker compose exec db psql -U dev -d dev_db -c "select version();" # PostgreSQL 17.x
```

Open http://localhost:8025 — Mailpit's empty inbox. In Lesson 2.4, your app's first magic-link email lands here; in 3.6, your Playwright tests read OTP codes out of it through its API.

6. Architecture Decision

ARCHITECTURE DECISION

The fork: where does dev infrastructure live? (a) Installed natively per machine, (b) a shared cloud dev database, (c) containers defined in the repo.

Chosen: containers (ADR-C13). The infrastructure's *versions and topology* are code, reviewed like code; a new machine is productive in minutes; two projects with different Postgres majors coexist.

Rejected: native installs — unpinned, colliding, undocumented by nature. Shared cloud dev DB — couples every developer's iteration loop to a network and to each other's data (and costs money before the first feature exists).

The trade: Docker becomes a hard dev dependency. Accepted knowingly — it's also the test-isolation strategy (Testcontainers, Lesson 1.3) and the deployment unit (8.2), so the dependency pays three times.

7. Checkpoint

```
git add global.json docker-compose.yml .gitignore .gitleaks.toml .env.example
git commit -m "chore: reproducible dev environment — pinned SDK, compose infra, env
contract"
git tag lesson-0.2
```

You should now be able to: bring the full dev infrastructure up and down with one command; explain why the SDK and Postgres versions are pinned *in the repo* rather than in a README; state the `.env` / `.env.example` contract and the two layers that enforce it; and articulate why health checks matter before anything depends on these services.

Next: *Lesson 1.1 — Solution, projects & supply chain* — the .NET solution appears, with its dependencies pointing the right way and its packages locked.

Part 1 — The walking skeleton

Lesson 1.1 — Solution, projects & supply chain

> **Part 1 · The walking skeleton.** Nothing runs yet — that's next lesson. Here we lay out the solution so that the architecture from Lesson 0.1 isn't a diagram you promised to respect but a set of project references the compiler enforces. We also make a decision most projects defer until it hurts: exactly which versions of everything we depend on, pinned so a build today and a build in six months produce the same bytes.

Goal: a solution with every project in place, dependencies physically pointing inward, one central version table, and a locked, reproducible restore.

Concepts & principles: the dependency rule made mechanical; Central Package Management; lockfiles and supply-chain hygiene; warnings-as-errors from day one.

Maps to: ADR-C3/C4/C5 · R25–R27 · repo: `Perezosoft.slnx`, `Directory.Build.props`, `Directory.Packages.props`, `*/*.csproj`.

Prerequisites: 0.2 (pinned SDK; `dotnet --version` prints 10.0.3xx).

---

1. Motivate — the layout is the first enforcement

Every codebase claims to have an architecture. The honest question is: *what happens if someone violates it?* If the answer is "a reviewer might notice," you have a convention. If the answer is "it does not compile," you have a structure.

Project references are the cheapest structural enforcement .NET offers. When `Core` has no reference to `Infrastructure`, a domain entity *cannot* reach out to EF Core or SMTP — not "should not," cannot. The dependency rule from Lesson 0.1 stops being a poster on the wall the moment the reference graph encodes it. That's what we build in this lesson.

The second problem we solve is quieter and nastier. `dotnet add package` with no version pinned means your build depends on *when* you ran it. Two developers restore on different days and get different transitive graphs; CI builds an artifact subtly unlike the one you tested; and when a compromised package version hits NuGet — it has happened — nothing in your build even notices. Reproducibility isn't a nicety; it's the precondition for being able to say "this is what we shipped."

2. Create the skeleton

The solution uses the modern XML solution format (`.slnx` — human-readable, merge-friendly, supported by .NET 10 tooling):

```
dotnet new sln --format slnx -n Perezosoft
dotnet new classlib -o src/Core -n Perezosoft.Core
dotnet new classlib -o src/Infrastructure -n Perezosoft.Infrastructure
dotnet new web -o src/Api -n Perezosoft.Api
dotnet new blazorwasm -o src/Web -n Perezosoft.Web
dotnet new razorclasslib -o src/Shared.Ui -n Perezosoft.Shared.Ui
dotnet new xunit -o tests/Core.Tests -n Perezosoft.Core.Tests
dotnet new xunit -o tests/Api.Tests -n Perezosoft.Api.Tests
dotnet new nunit -o tests/E2E.Tests -n Perezosoft.E2E.Tests
dotnet sln add src/Core src/Infrastructure src/Api src/Web src/Shared.Ui \
              tests/Core.Tests tests/Api.Tests tests/E2E.Tests
```

Delete the template's sample classes (`Class1.cs`, the Blazor counter page can stay for now — it goes when the real UI arrives in 3.4). Then wire the references — this is the architecture, so read it slowly:

```
dotnet add src/Infrastructure reference src/Core # Infra implements Core seams
dotnet add src/Api reference src/Core src/Infrastructure # Api composes both
dotnet add src/Web reference src/Shared.Ui # Web hosts the shared UI
dotnet add tests/Core.Tests reference src/Core
dotnet add tests/Api.Tests reference src/Api src/Infrastructure src/Core
```

What's deliberately **absent** is the point: `Core` references *nothing* of ours. `Infrastructure` cannot see `Api`. `Web` cannot see `Core`, `Infrastructure`, or `Api` at all — the UI will talk to the API over HTTP like any other client (Golden Rule: the clean API boundary), and the compiler now holds it to that. Try it: add `using Perezosoft.Infrastructure;` to a file in `Core` — it does not build. That error message is the architecture working.

Each project file stays nearly empty. Here is the real `Perezosoft.Core.csproj` in its entirety:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

Nine lines. Everything else lives at the solution root, where it applies to *all* projects at once — which is our next move.

3. One build policy for the whole solution

MSBuild reads a file named `Directory.Build.props` from every ancestor directory of a project. Put one at the repo root and every project inherits it — a single place for build policy, impossible for a new project to forget. This is the reference platform's actual file:

```
<Project>

  <!--
    Solution-wide build hygiene. Nullable reference types on, and warnings are
    errors so the invariants the analyzers and the compiler check can't rot into
    ignored noise. This is half of "solid by construction" — the architecture
    tests are the other half, and CI runs both.
  -->
  <PropertyGroup>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <ImplicitUsings>enable</ImplicitUsings>
    <!--
      Supply-chain hardening. Restore writes a packages.lock.json next to each
      project pinning the full transitive graph; those files are committed and CI
      restores in locked mode so any drift fails the build. Paired with Central
      Package Management (Directory.Packages.props).
    -->
    <RestorePackagesWithLockFile>true</RestorePackagesWithLockFile>
  </PropertyGroup>

</Project>
```

Two of these settings deserve a hard look, because both are *decisions about failure*:

`TreatWarningsAsErrors` — a warning is the compiler telling you something is probably wrong. On a codebase that tolerates warnings, the count creeps from 3 to 300, and the one that mattered drowns. Turning warnings into errors on day one costs nothing (there is no code yet!) and means the count stays at zero forever. Adopting this on an *existing* codebase is a month of cleanup — which is exactly why it must happen now, in the empty solution. Several rules you'll meet later (like nullability soundness) are only real because a violation refuses to compile.

`Nullable` — nullable reference types turn "I forgot this could be null" from a production `NullReferenceException` into a compile-time squiggle — and with the previous setting, into a build failure. The domain modeling you'll do from Part 2 onward leans on this constantly: `string?` in an entity is a *documented decision* that the value is optional, checked by the compiler, rather than tribal knowledge.

4. Central Package Management — one version table

Without CPM, every `.csproj` carries its own `<PackageReference Include="X" Version="Y">` and the same package drifts across projects (the reference repo actually hit this: two test projects referencing different versions of the test SDK). With CPM, versions live in **one** committed table, `Directory.Packages.props`, and project files reference packages *without* versions. An excerpt of the real table:

```
<Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
  </PropertyGroup>

  <ItemGroup>
    <!-- App / API -->
    <PackageVersion Include="DotNetEnv" Version="3.2.0" />
    <PackageVersion Include="Microsoft.AspNetCore.Authentication.JwtBearer"
Version="10.0.9" />

    <!-- Infrastructure -->
    <PackageVersion Include="MailKit" Version="4.17.0" />
    <PackageVersion Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.2" />

    <!-- Tests -->
    <PackageVersion Include="xunit" Version="2.9.3" />
    <PackageVersion Include="Testcontainers.PostgreSql" Version="4.12.0" />
    <!-- Consolidated: E2E referenced 17.14.0, the xUnit projects 17.14.1 → highest wins.
-->
    <PackageVersion Include="Microsoft.NET.Test.Sdk" Version="17.14.1" />
  </ItemGroup>
</Project>
```

Create yours with just the packages the walking skeleton needs (the test SDK, xunit); every later lesson that says "add package X" means: add the `<PackageVersion>` row here, add a version-less `<PackageReference>` in the project that uses it. One table to review when you upgrade; one diff line when a version changes; zero possibility of the same package at two versions (that's rule R27, and it's enforced by construction).

Notice that comment about `17.14.0` vs `17.14.1` — it's a fossil of the exact drift CPM exists to prevent, found *during* the consolidation and preserved in the reference repo as a comment. Real codebases carry these scars; good ones label them.

5. Lock it — restore becomes reproducible

`RestorePackagesWithLockFile` (already in our props file) makes restore write a `packages.lock.json` beside each project: the **full transitive closure** — every package your packages depend on, recursively, with exact versions and content hashes. Commit these files. Then the magic flag:

```
dotnet restore --use-lock-file # first restore: writes the lockfiles
git add **/packages.lock.json
dotnet restore --locked-mode # from now on (and in CI): verify, don't resolve
```

In locked mode, restore doesn't *resolve* versions — it *verifies* them. If anything in the graph differs from the lockfile (a floating version moved, a transitive dependency was swapped, a hash doesn't match), restore **fails loudly** instead of silently building something new. When lesson 1.6 stands up CI, `--locked-mode` is one of its first gates: a supply-chain change becomes a red build and a reviewable diff, not an invisible drift.

The mental model: your dependency graph is now *code*. It changes by commit, with review, or not at all.

6. Run it

```
dotnet build
# 0 Warning(s), 0 Error(s) — and it must stay that way: warnings ARE errors now
dotnet restore --locked-mode
# succeeds — graph matches the committed lockfiles
```

And prove a gate actually gates (pain now, so you trust it later): edit a `packages.lock.json` by hand — change any version digit — and run `dotnet restore --locked-mode` again. It fails with `NU1004`. Revert. That failure is what a supply-chain surprise will look like: loud, early, and in your terminal instead of in production.

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how do we manage dependency versions? (a) Per-project references with whatever versions were current at add-time; (b) a shared props file with version *variables*; (c) Central Package Management + committed lockfiles + locked-mode CI.

Chosen: (c) — CPM for one reviewable version table (R25), lockfiles for the transitive graph, locked restore so drift fails the build (R27). Also decided here: warnings-as-errors and nullable from the empty solution, because both are nearly free now and prohibitively expensive to retrofit.

Rejected: (a) is how version drift and silent supply-chain changes happen — it's the default, which is the problem. (b) centralizes the *numbers* but not the *graph*: transitive dependencies still float.

The trade: every dependency bump is now an explicit two-file diff (table + lock), and a new package is a two-step add. Accepted: that friction *is* the review point.

8. Checkpoint

```
git add -A && git commit -m "feat: solution skeleton — projects, inward refs, CPM, locked restore"
git tag lesson-1.1
```

You should now be able to: explain why `Core` referencing nothing is the architecture's first enforcement; add a package the CPM way; say what a lockfile pins that a version number doesn't; and articulate why warnings-as-errors is a day-one decision, not a someday cleanup.

Next: *Lesson 1.2 — First endpoint, first test* — the API says hello, and the Red→Green loop you'll live fifty times gets its first turn.

Lesson 1.2 — First endpoint, first test

> **Part 1 · The walking skeleton.** The solution compiles but does nothing. By the end of this lesson it answers HTTP — and, more importantly, a test *proves* it answers HTTP by booting the real application. This is the lesson where you live the Red→Green→Refactor loop for the first time, on the smallest possible stakes, so that when the stakes are tenant isolation or billing you already trust the rhythm.

Goal: `GET /health` returns 200, and a test in `Api.Tests` boots the actual app in-process and asserts it.

Concepts & principles: the walking skeleton; Red→Green→Refactor; in-process integration testing with `WebApplicationFactory`; testing the real composition, not a hand-assembled copy.

Maps to: ADR-C14 · repo: `src/Api/Program.cs`, `tests/Api.Tests/Infrastructure/IntegrationTestFactory.cs`, `tests/Api.Tests/Integration/HarnessSmokeTests.cs`, `src/Api/Perezosoft.Api.http`.

Prerequisites: 1.1 (solution builds, CPM in place).

---

1. Motivate — why a "hello world" deserves a whole lesson

A *walking skeleton* is the thinnest possible slice that exercises the whole path: request in, response out, test proving it. It looks trivial. It isn't, because it forces every piece of plumbing to exist and cooperate **before** any feature depends on that plumbing: the web host boots, routing routes, the test project can reach the app, CI (next lessons) can run it all. Every integration problem you flush out now is one that won't ambush you inside a complicated lesson later.

There's also a decision hiding in "a test proving it" — *what kind* of test? We could unit-test a handler function and call it a day. But the failures that hurt in web apps are rarely inside one function; they're in the *composition* — DI registrations missing, middleware ordered wrong, routes not mapped, config not read. So our default harness boots the **real** `Program` — the actual composition root, not a hand-assembled lookalike — in-process, and talks to it over real HTTP semantics. The reference platform's harness makes the reasoning explicit; from its actual doc comment:

> Boots the REAL app (`Program`) via `WebApplicationFactory<T>` … so tests exercise the full pipeline …, not hand-assembled services. … [this] de-risks routing/DI refactors by asserting routes stay stable end-to-end.

A test against a hand-wired `ServiceCollection` keeps passing while the real app is broken. A test against `Program` cannot.

2. Red — the failing test comes first

In `tests/Api.Tests`, add the package references (CPM style — versions come from the table you made in 1.1): `Microsoft.AspNetCore.Mvc.Testing`, and reference the Api project. Then write the test **before** the endpoint exists:

```
// tests/Api.Tests/HealthEndpointTests.cs
using System.Net;
using Microsoft.AspNetCore.Mvc.Testing;

namespace Perezosoft.Api.Tests;

public class HealthEndpointTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly WebApplicationFactory<Program> _factory;

    public HealthEndpointTests(WebApplicationFactory<Program> factory) => _factory =
factory;

    [Fact]
    public async Task Health_endpoint_responds_200()
    {
        var client = _factory.CreateClient(); // in-process; no port, no network

        var response = await client.GetAsync("/health");

        Assert.Equal(HttpStatusCode.OK, response.StatusCode);
    }
}
```

Run it:

```
dotnet test tests/Api.Tests
# Failed! Health_endpoint_responds_200
# Assert.Equal() Failure: Expected OK, Actual NotFound
```

Red — the app boots (WebApplicationFactory found `Program` and started the real host) but there is no `/health` endpoint, so the request falls through to a 404. That's a *properly* failing test: it fails for exactly the reason it should, and the failure message describes the missing behavior.

One wrinkle to know about before it bites: our `Program.cs` is top-level statements (no explicit class), and the class the compiler generates for it is **internal**. On current SDKs the web template arranges for the test host to reach it anyway, so the test above compiles and runs — but if you ever hit `error CS0246: The type or namespace name 'Program' could not be found`, the fix is one line in `Perezosoft.Api.csproj`, and the reference platform carries it as standing hygiene:

```
<ItemGroup>
  <InternalsVisibleTo Include="Perezosoft.Api.Tests" />
</ItemGroup>
```

`InternalsVisibleTo` is a scalpel: it grants *one named assembly* access to internals, instead of making `Program` public for the whole world. You'll see this same "smallest-possible opening" instinct again and again — it's the security posture of the entire platform in miniature. Add it now even though the build didn't force you to; the next person's SDK might.

3. Green — the minimum that passes

`src/Api/Program.cs`, in full at this stage of its life:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddHealthChecks();

var app = builder.Build();

app.MapHealthChecks("/health");

app.Run();
```

```
dotnet test tests/Api.Tests
# Passed! - 1 test
```

That's the loop: a failing test that *specifies*, then the minimum code that *satisfies*. Resist the urge to add more — no controllers, no Swagger, no logging config. Every one of those will arrive in the lesson where a test demands it. (For the curious: this file grows to ~300 lines by Part 8 in the reference repo, and every block of it traces to a lesson.)

Why `AddHealthChecks()`/`MapHealthChecks` rather than a hand-rolled `app.MapGet("/health", …)`? Because this endpoint has a *career* ahead of it: in Lesson 4.5 it splits into liveness (`/health`) and readiness (`/health/ready`) with a real database probe behind a tag filter, and in Lesson 8.3 the deploy pipeline's smoke gate calls it to decide whether a release goes out. Using the framework's health-check pipeline from day one means that growth is additive. The reference repo's final form, for a preview of where this line of code is headed:

```
app.MapHealthChecks("/health", new HealthCheckOptions { Predicate = _ => false });
app.MapHealthChecks("/health/ready", new HealthCheckOptions { Predicate = check =>
check.Tags.Contains("ready") });
```

4. Refactor & harden — a scratchpad and a habit

No production refactor needed yet (there's barely any production). But two small work-habits start here.

The `.http` scratchpad. Create `src/Api/Perezosoft.Api.http` — using the https port from *your* `src/Api/Properties/launchSettings.json` (`dotnet new web` assigns a random one; the reference repo happens to sit on 7160):

```
@host = https://localhost:7160

GET {{host}}/health
```

Visual Studio and VS Code (REST Client) execute these files in-editor. Every lesson that adds an endpoint adds a request here — by Part 7 it's a living, executable catalog of the whole API surface, versioned with the code. It's the cheapest API documentation that can't go stale, because you *use* it to poke the app.

Run the real thing. Tests passing in-process is necessary, not sufficient — actually boot it once:

```
dotnet run --project src/Api
# listening on https://localhost:xxxx
curl -k https://localhost:xxxx/health
# Healthy
```

The response body `Healthy` comes from the health-check middleware's default writer. In-process tests + a real boot = the two views you'll keep cross-checking all course.

5. Run it

```
dotnet build # 0 warnings, 0 errors — still, and always
dotnet test tests/Api.Tests
# 1 passed
```

6. Architecture Decision

ARCHITECTURE DECISION

The fork: what does the default test harness boot? (a) Nothing — unit-test handlers as plain functions; (b) a hand-assembled `ServiceCollection` with the pieces each test needs; (c) the real `Program` via `WebApplicationFactory`, in-process.

Chosen: (c) as the default for anything HTTP-shaped (ADR-C14's integration layer). The composition root is where web apps actually break — missing registrations, middleware order, unmapped routes — and only booting the real one tests it. In-process means no ports, no network flakes, parallel-safe.

Rejected: (a) alone leaves the wiring untested (we still unit-test pure logic — Core.Tests exists for exactly that). (b) is the seductive trap: it *looks* like an integration test but silently diverges from the real app — a test passing against a copy proves nothing about the original.

The trade: booting the real app makes some things awkward on purpose — config the app reads at startup must genuinely exist (you'll feel this in 1.4, and the reference harness has a documented seam for it: the JWT secret must be an *environment variable* because `Program` reads it before the factory's config hooks run). That awkwardness is information: it's your startup contract made visible.

7. Checkpoint

```
git add -A && git commit -m "feat: walking skeleton — health endpoint + real-app test
harness"
git tag lesson-1.2
```

You should now be able to: explain what a walking skeleton de-risks; write the failing test first and feel why the order matters; say why `WebApplicationFactory<Program>` beats a hand-assembled service collection; and explain what `InternalsVisibleTo` grants and why it's preferable to making things public.

Next: *Lesson 1.3 — Database & the test container* — Postgres joins the skeleton, and the tests get a real database that appears and disappears on demand.

Lesson 1.3 — Database & the test container

> **Part 1 · The walking skeleton.** The skeleton gets a spine: PostgreSQL via EF Core, a first migration, and — the real prize — a test fixture that gives every test run a genuine Postgres that appears in seconds and vanishes without trace. The single most consequential testing decision of the course happens here: **we do not use the EF in-memory provider. Ever.** By the end you'll know exactly why.

Goal: `AppDbContext` exists with one throwaway entity, a migration creates its schema, and a test writes to and reads from a *real* Postgres spun up by the test run itself.

Concepts & principles: EF Core migrations discipline; Testcontainers; why in-memory database fakes lie; model-derived test resets (isolation that can't be forgotten).

Maps to: ADR-C6/C7 · R11 · repo: `src/Infrastructure/Persistence/AppDbContext.cs`, `src/Api/AppDbContextFactory.cs`, `tests/Api.Tests/Infrastructure/PostgresFixture.cs`, `tests/Api.Tests/Infrastructure/PostgresTestBase.cs`, `tests/Api.Tests/MigrationsTests.cs`.

Prerequisites: 1.2 (harness runs); 0.2 (Docker running — Testcontainers needs the daemon, and you already have it for compose).

---

1. Motivate — your tests are only as honest as their database

Here's the trap nearly every .NET team falls into once. EF Core ships an in-memory provider; it's right there; tests using it need no Docker and run fast. So the test suite grows on it. Then one day a query that passed a thousand test runs fails in production, because the in-memory provider isn't a database — it's a `List<T>` wearing a trench coat. The reference platform's fixture states the consequence as a matter of record, in its actual doc comment:

> Real Postgres (not the EF in-memory provider) is required because several services branch on `Database.IsRelational()` and use relational-only constructs — `ExecuteUpdateAsync`, savepoints — that the in-memory provider silently skips.

"Silently skips" is the killer phrase. And the list is longer than that comment: the in-memory provider has no transactions worth the name, no unique constraints, no `SKIP LOCKED` (Lesson 4.1 depends on it), case-insensitive string comparisons where Postgres is case-sensitive — and, fatally for this course, its handling of **global query filters** (Lesson 2.6, the keystone of tenant isolation) diverges from a real relational provider. A tenant-isolation test that passes in-memory proves *nothing*.

The classical escape was a shared "test database" on somebody's server — slow to reset, polluted by yesterday's runs, fought over by CI agents. Testcontainers dissolves the dilemma: the test run itself starts a pristine `postgres:17` in a Docker container, uses it, and destroys it. Real engine, zero residue.

2. The context and its first entity

In `Infrastructure`, add the EF packages (CPM rows + version-less references — `Npgsql.EntityFrameworkCore.PostgreSQL`; `Microsoft.EntityFrameworkCore.Design` goes in the Api project for tooling). Then the context — humble at this stage:

```
// src/Infrastructure/Persistence/AppDbContext.cs
using Microsoft.EntityFrameworkCore;

namespace Perezosoft.Infrastructure.Persistence;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    protected override void OnModelCreating(ModelBuilder builder)
    {
        base.OnModelCreating(builder);
        // Per-entity mapping lives in one IEntityTypeConfiguration<TEntity> class each,
        // discovered from this assembly — no central registry to forget.
        builder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
    }
}
```

That `ApplyConfigurationsFromAssembly` line is a pattern you met in 0.2 (health checks) and will meet a dozen more times: **discovery over registration**. Entity configurations will be *found* because they exist, not because someone remembered to list them. (In 2.6, tenant filters attach the same way — by reflection over the model — and in this lesson's fixture, the reset list will too. One idea, three appearances already.)

For a first entity we need something disposable — the real platform's first entities (User, Tenant) arrive with Part 2 and deserve their own lessons. Use a placeholder `Probe` entity with an `Id` and a `Note` — its only job is to prove the pipeline, and you'll delete it in 2.1 without ceremony. Give it an `IEntityTypeConfiguration<Probe>` so the discovery pattern gets exercised.

Now scroll your build output before moving on — there's a landmine here worth defusing in daylight. Depending on package versions, adding the EF stack can produce:

```
warning MSB3277: Found conflicts between different versions of
"Microsoft.EntityFrameworkCore" that could not be resolved.
```

…and the build still says **succeeded**. Didn't we make warnings errors in 1.1? We did — for *compiler* warnings. `MSB3277` is an **MSBuild** warning, a different pipeline that `TreatWarningsAsErrors` does not cover. Your zero-warnings bar has a blind spot, and this is the classic thing that hides in it: the Npgsql provider pulling one EF Core version while the Design/test packages pull another. The fix is to pin the shared assembly explicitly — add `Microsoft.EntityFrameworkCore.Relational` at the same version as your other EF packages to the CPM table and reference it from `Api.Tests` (the reference repo carries exactly this pin, with a comment saying why). The transferable lesson: **"the build is green" and "the build is clean" are different claims** — read the output when you add a dependency, because version-unification warnings are how runtime `MethodNotFound` surprises introduce themselves politely first.

3. Migrations — schema changes as reviewable code

The EF tooling is a separate global tool, not part of the SDK — install it once (0.2's toolchain table now has a third row):

```
dotnet tool install --global dotnet-ef
dotnet ef migrations add InitialCreate --project src/Infrastructure --startup-project
src/Api
```

...and it fails, or worse, half-works, because EF's design-time tooling wants to *boot your app* to obtain a configured `DbContext` — and your app (from 1.4 onward) will refuse to boot without validated config. The reference platform closes this with a **design-time factory**, and its doc comment explains the whole story:

> Design-time factory so EF tooling ... can build the context WITHOUT booting the API host — routing through the host runs startup config validation (e.g. `Jwt:Secret`) that isn't present at design time. The connection string comes from `ConnectionStrings__DefaultConnection`, the SAME single source the running app uses ... There is no hardcoded fallback.

```
// src/Api/AppDbContextFactory.cs (abridged from the real file)
public class AppDbContextFactory : IDesignTimeDbContextFactory<AppDbContext>
{
    public AppDbContext CreateDbContext(string[] args)
    {
        try { DotNetEnv.Env.TraversePath().Load(); } catch { /* no .env present */ }

        var connectionString =
            Environment.GetEnvironmentVariable("ConnectionStrings__DefaultConnection")
            ?? throw new InvalidOperationException(
                "ConnectionStrings__DefaultConnection is not set. Add it to .env ...");

        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseNpgsql(connectionString).Options;
        return new AppDbContext(options);
    }
}
```

Note what it *refuses* to do: no hardcoded `Host=localhost;...` fallback. The connection string lives in exactly one place — your `.env` (ADR-001, given teeth in 0.2) — and if it's missing, the error says precisely what to fix. A fallback would "work" and thereby hide misconfiguration until it mattered.

Which means two files gain a line right now, honoring 0.2's contract that every key the system reads is documented. In `.env` (so the tooling works on your machine) and in `.env.example` (so it works on the next machine):

```
# REQUIRED — Postgres connection for the API + EF tooling. Mirror the DB_* values above
# (note the port: 5433 if you kept 0.2's collision-avoiding default).
ConnectionStrings__DefaultConnection="Host=localhost;Port=5433;Database=dev_db;Username=dev
;Password=devpassword"
```

(When 1.4's doc-sync gate arrives, an undocumented key like this would fail the build — we're just early.)

Run the migration command again; a `Migrations/` folder appears in Infrastructure. Read the generated file once, top to bottom — you'll rarely need to again, but you should know what the tool writes on your behalf: an `Up()` building your tables, a `Down()` reversing it, and a model snapshot the *next* migration diffs against. Migrations are why "the schema" is never a rumor: it's the sum of committed, reviewed, replayable scripts.

4. The fixture — a real database that can't leak between tests

Now the centerpiece. Add `Testcontainers.PostgreSql` to `Api.Tests` and build the fixture (this is the reference platform's real code, trimmed of the tenancy parts that arrive in Part 2):

```
// tests/Api.Tests/Infrastructure/PostgresFixture.cs
public sealed class PostgresFixture : IAsyncLifetime
{
    private readonly PostgreSqlContainer _container = new
PostgreSqlBuilder("postgres:17").Build();

    public string ConnectionString => _container.GetConnectionString();

    public async Task InitializeAsync()
    {
        await _container.StartAsync();
        await using var db = CreateContext();
        // Build the schema from the EF model (fast; no migration history needed for
tests).
        await db.Database.EnsureCreatedAsync();
    }

    public Task DisposeAsync() => _container.DisposeAsync().AsTask();

    public AppDbContext CreateContext()
    {
        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseNpgsql(ConnectionString).Options;
        return new AppDbContext(options);
    }

    /// <summary>Truncates every table so each test starts from a clean slate. The table
    /// list is DERIVED from the EF model, so a new entity is reset automatically — no
    /// hand-maintained list to forget.</summary>
    public async Task ResetAsync()
    {
        await using var db = CreateContext();
        var tables = db.Model.GetEntityTypes()
            .Select(e => e.GetTableName())
            .Where(name => name is not null)
            .Distinct()
            .Select(name => $"\"{name}\"");
        var sql = "TRUNCATE TABLE " + string.Join(", ", tables) + " RESTART IDENTITY
CASCADE;";
        await db.Database.ExecuteSqlRawAsync(sql);
    }
}

[CollectionDefinition(PostgresCollection.Name)]
public sealed class PostgresCollection : ICollectionFixture<PostgresFixture>
{
    public const string Name = "postgres";
}
```

Three design decisions in forty lines, each load-bearing:

One container per run, not per test. Container startup costs seconds; a suite of hundreds of tests can't pay it each time. The xUnit *collection fixture* shares one container across every test class marked `[Collection(PostgresCollection.Name)]`, and xUnit runs a collection's members serially — so sharing is safe by construction.

Reset between tests, and make the reset un-forgettable. Shared container means shared state, so every test must start clean. The rookie version is a hand-maintained list of tables to truncate — which goes stale the day someone adds an entity and forgets the list (this exact bug is why rule R11 exists; the reference repo found it in an audit). The fix is the discovery pattern again: **derive the table list from `db.Model.GetEntityTypes()`**. New entity → automatically in the reset. And rather than trusting each test to call `ResetAsync()`, a tiny base class makes isolation structural:

```
// tests/Api.Tests/Infrastructure/PostgresTestBase.cs — real file, in full
public abstract class PostgresTestBase(PostgresFixture fixture) : IAsyncLifetime
{
    protected PostgresFixture Fixture { get; } = fixture;

    public Task InitializeAsync() => Fixture.ResetAsync(); // before EVERY test
    public Task DisposeAsync() => Task.CompletedTask;
}
```

Inherit it and there is no reset call to forget. Same move as the tenant filter to come: correct by default, not correct by discipline.

`EnsureCreatedAsync` here, migrations elsewhere. The fixture builds schema straight from the model — fast, no history table. But that means the *migrations themselves* could rot untested. So one more test exists solely to apply every migration, in order, against a scratch database (`MigrationsTests` in the reference repo — and the integration harness from 1.2 will boot the app with its real startup-migration path in Part 2). Schema-from-model for speed; migrations proven separately; nothing unverified.

5. Red → Green — prove the loop against real Postgres

```
// tests/Api.Tests/ProbePersistenceTests.cs
[Collection(PostgresCollection.Name)]
public class ProbePersistenceTests(PostgresFixture fixture) : PostgresTestBase(fixture)
{
    [Fact]
    public async Task Probe_roundtrips_through_real_postgres()
    {
        await using (var db = Fixture.CreateContext())
        {
            db.Add(new Probe { Note = "hello from a container" });
            await db.SaveChangesAsync();
        }
        await using (var db2 = Fixture.CreateContext()) // fresh context: no cache tricks
        {
            var probe = await db2.Set<Probe>().SingleAsync();
            Assert.Equal("hello from a container", probe.Note);
        }
    }
}
```

```
dotnet test tests/Api.Tests
# Passed! - 2 tests (health + probe) ... first run pulls postgres:17, be patient
```

Watch `docker ps` during the run: a postgres container blinks into existence, then is gone. No cleanup script, nothing to remember, nothing left behind.

6. Architecture Decision

ARCHITECTURE DECISION

The fork: what database do tests run against? (a) EF in-memory provider; (b) SQLite in-memory ("at least it's relational"); (c) a shared long-lived test server; (d) Testcontainers — throwaway real Postgres per run.

Chosen: (d), with schema-from-model for speed and a separate migrations-apply test. Tests must exercise the engine production runs, because this platform *leans* on Postgres-specific behavior: global query filters as the tenancy wall, `SKIP LOCKED` job claiming, savepoints, and eventually row-level security. (ADR-C7/C13; R11 for the derived reset.)

Rejected: (a) silently skips the exact constructs we depend on — a green suite proving nothing. (b) is a different engine with different SQL, different concurrency, no RLS: it converts "provider lies" into "dialect lies". (c) reintroduces shared mutable state across runs and machines — the thing test isolation exists to kill.

The trade: Docker becomes a hard requirement for running the suite, and the first test run pays an image pull. Accepted consciously in 0.2 — the same daemon already runs your dev infrastructure, and CI (1.6) has it natively.

7. Checkpoint

```
git add -A && git commit -m "feat: EF Core + Postgres — first migration, Testcontainers fixture, model-derived reset"
git tag lesson-1.3
```

You should now be able to: list three concrete ways the in-memory provider lies; explain the container-per-run / reset-per-test economy and why the reset list is derived, not written; say what a design-time factory is for and why it must not have a connection fallback; and describe how migrations stay verified even though the fixture doesn't use them.

Next: *Lesson 1.4 — Configuration & the options pattern* — the app learns to read config, validate it at startup, and fail loudly at boot rather than mysteriously at 3 a.m.

Lesson 1.4 — Configuration & the options pattern

> **Part 1 · The walking skeleton.** The app can answer HTTP and talk to Postgres; now it learns to be *configured* — and, more to the point, to refuse to start when it isn't. Configuration is where "works on my machine" goes to hide: a missing key that defaults to something plausible, a secret pasted into a JSON file, a setting the code reads that nobody documented. This lesson closes all three holes, and closes them with machinery, not vigilance.

Goal: the API loads `.env` in dev, exposes typed, validated settings objects, fails at *startup* (not first-use) on missing/invalid config, and a test fails the build when code reads a config key that isn't documented.

Concepts & principles: the config hierarchy; secrets never in the repo (ADR-001 mechanized); typed settings with construction-time validation; fail-fast startup; the doc-sync gate.

Maps to: ADR-001 · R20–R22 · repo: `src/Api/Configuration/SettingsProvider.cs`, `SettingsRegistration.cs`, `src/Api/Program.cs` (the DotNetEnv line), `tests/Api.Tests/DocAndConfigSyncTests.cs`, `.env.example`, `appsettings.json`.

Prerequisites: 1.3; the `.env` contract from 0.2.

---

1. Motivate — the 3 a.m. config bug

Picture the failure mode we're designing against. A service reads `config["Jwt:Secret"]` deep inside a request handler. On the machine where it's missing, nothing happens at startup — the app boots, health checks pass, the deploy is declared good. Hours later the first user tries to sign in and gets a 500, and the stack trace points at token-signing code, three layers away from the actual problem: a key nobody set. Now multiply by every config value in the system.

The root defect isn't the missing key — machines will always occasionally be misconfigured. The defect is **when** and **how** the system tells you. Config errors should surface at *boot*, in a message that names the key and says where to set it. That transformation — from runtime mystery to startup message — is what this lesson builds.

And beneath it, a second rule from Lesson 0.1's ADR-001: secrets never live in committed files. Which raises the practical question: where *does* the app get them, and how do committed files and secret files cooperate?

2. The config hierarchy — one mental model

ASP.NET Core builds `IConfiguration` from layered sources, later layers overriding earlier ones. Our arrangement (dev):

```
appsettings.json committed — non-secret defaults, shape documentation
appsettings.Development.json committed — dev-only tweaks (verbose logging, etc.)
environment variables NOT committed — secrets & machine-specifics …
        ▲
        └── in dev, populated from .env by DotNetEnv at startup
```

The elegant part is the last line. Production sets real environment variables; dev keeps them in the gitignored `.env` from 0.2, and a single line at the very top of `Program.cs` — before anything reads config — loads them. From the real file:

```
// Local dev: load secrets/config from the repo-root .env (the single local source of
// truth — see docs/DECISIONS.md). TraversePath walks up to find it regardless of the
// working dir; the try/catch makes it a no-op when there's no .env (e.g. production,
// which uses real environment variables).
try { DotNetEnv.Env.TraversePath().Load(); } catch { /* no .env present */ }

var builder = WebApplication.CreateBuilder(args);
```

One mechanism, two environments: to the app, dev and prod look identical — env vars either way. Nested keys use the `Section__Sub` double-underscore convention (`Jwt__Secret` in `.env` becomes `Jwt:Secret` to `IConfiguration`), which is the standard env-var transport for hierarchical config.

`appsettings.json` keeps the **non-secret** shape: section names, defaults worth committing, values a reviewer should see. It also serves as *documentation of what exists* — which the gate in §5 exploits.

3. Typed settings — validation at construction

Reading `config["Jwt:Secret"]` at call sites scatters string literals through the code, defers errors to first use, and re-parses on every read. The platform's pattern: one small class per config section, reading and validating **in its constructor**, registered as a singleton. The real `JwtSettings`:

```
// src/Api/Configuration/SettingsProvider.cs
public class JwtSettings : IJwtSettings
{
    public string SecretKey { get; }
    public string Issuer { get; }
    public int ExpiryMinutes { get; }

    public JwtSettings(IConfiguration config)
    {
        SecretKey = config["Jwt:Secret"]
            ?? throw new InvalidOperationException(
                "Jwt:Secret not configured (set Jwt__Secret in .env for dev)");
        Issuer = config["Jwt:Issuer"] ?? "Perezosoft";
        ExpiryMinutes = config.GetValue("Jwt:ExpiryMinutes", 60);

        const int MinSecretKeyLength = 32;
        if (SecretKey.Length < MinSecretKeyLength)
            throw new InvalidOperationException(
                $"Jwt:Secret must be at least {MinSecretKeyLength} characters");
    }
}
```

Read the error messages as UX, because they are: each names the key **and the fix** ("set `Jwt__Secret` in `.env` for dev"). Whoever hits this — possibly you, in six months, on a new laptop — gets handed the solution, not a scavenger hunt. And notice the *semantic* validation: not just "present" but "at least 32 characters," because an HMAC key shorter than that is a security bug that would otherwise hide until token validation mysteriously failed.

Registration happens once, in one place, and the singletons are constructed **eagerly** so a bad value kills the boot:

```
// src/Api/Configuration/SettingsRegistration.cs (abridged — real file)
public static IJwtSettings AddAppSettings(this IServiceCollection services, IConfiguration
config)
{
    // Build JwtSettings once; the same instance backs both the DI singleton and the
    // JWT-bearer TokenValidationParameters — a single source of truth.
    var jwtSettings = new JwtSettings(config);

    services.AddSingleton<IJwtSettings>(jwtSettings);
    services.AddSingleton<IRefreshTokenSettings>(new RefreshTokenSettings(config));
    services.AddSingleton<IApplicationSettings>(new ApplicationSettings(config));
    // ...
    return jwtSettings;
}
```

Two subtleties worth pausing on. The settings are constructed with `new`, *right here*, not lazily by the container — so startup **is** the validation pass; a missing key can't lurk until first resolve. And `JwtSettings` is built into a local and *returned*, because `Program.cs` needs the same values to configure JWT-bearer validation — one instance, two consumers, zero chance of drift between "the settings DI hands out" and "the settings auth actually validates with." (This is the platform's blessed shape per rule R22; .NET's `IOptions` + `ValidateOnStart` is a fine alternative — the *decision* is eager validation and one pattern everywhere, not which library performs it.)

Consumers then depend on `IJwtSettings` — an interface, so tests hand in a stub without touching `IConfiguration` at all. Config-reading is quarantined to these constructors the way MailKit is quarantined to `Infrastructure/Email`: one place, one pattern.

And your 1.2 test just broke — good. `WebApplicationFactory` boots the *real* `Program`, which now demands `Jwt:Secret` at startup; the health test dies with the very `InvalidOperationException` you wrote. This is fail-fast config working exactly as designed — the test boots the app, the app validates its config, no exceptions for test mode. The seam is the one the reference harness documents: `Program` reads config at `CreateBuilder` time, *before* the factory's config hooks run, so the value must exist as an **environment variable** that early. A static constructor on the test class is the cleanest place:

```
public class HealthEndpointTests : IClassFixture<WebApplicationFactory<Program>>
{
    static HealthEndpointTests() =>
        Environment.SetEnvironmentVariable("Jwt__Secret",
            "integration-test-jwt-secret-key-0123456789"); // ≥32 chars; value irrelevant
    // ...
}
```

The dummy only needs to *satisfy the validation* (length), because nothing in this test signs or verifies a token. When the integration harness grows up in Part 2, this seam moves into its

constructor — same idea, one shared home.

4. Defaults are decisions

Look again: `Issuer` defaults to `"Perezosoft"`, `ExpiryMinutes` to `60` — but `SecretKey` throws. Why the asymmetry? Because **a default is a decision that absence is safe**. A missing issuer name has a sensible universal answer. A missing *secret* does not — any default would be catastrophic, so absence must be fatal. When you write a settings class, sort every value into one of these bins deliberately: safe default, or refuse to boot. (Part 7 adds a third bin for optional *features*: absent config means the feature is **off** — R21's config-gated default-off. Same principle: absence maps to the safe state, never the surprising one.)

5. The gate — config that can't go undocumented

Now the structural guarantee. Nothing so far stops a developer from reading `config["Fancy:NewKnob"]` in code and telling no one — until someone deploys without it. The reference platform closes this with a test that cross-references **code against documentation**, from the real `DocAndConfigSyncTests`:

```
[Fact]
public void ConfigKeys_ReadInCode_AreDocumented()
{
    // R20: every Section:Key literal read via IConfiguration is documented — either in
    // an appsettings*.json (as a nested path) or in .env.example (as Section__Key).
    // Catches config drift where code reads a key nobody declared.
    var documented = DocumentedConfigPaths(); // parsed from appsettings*.json +
.env.example

    var readKeys = new HashSet<string>(StringComparer.Ordinal);
    var access = new Regex(
        @"(?:GetSection|GetValue<[^>]*>|GetValue|configuration|config|Configuration)\s*[\(\
[]\s*""([A-Za-z][A-Za-z0-9]*(?::[A-Za-z0-9]+)+)""");
    foreach (var f in SourceFiles(Path.Combine(RepoRoot(), "src")))
        foreach (Match m in access.Matches(File.ReadAllText(f)))
            readKeys.Add(m.Groups[1].Value);

    var undocumented = readKeys.Where(k => !documented.Any(d =>
            d.Equals(k) || d.StartsWith(k + ":") || k.StartsWith(d + ":")))
        .OrderBy(k => k).ToList();

    Assert.True(undocumented.Count == 0,
        $"Config keys read in code but not in appsettings*.json or .env.example:
{string.Join(", ", undocumented)}");
}
```

It's a source scan — grep with a build verdict. Read a key in code without adding it to `.env.example` or an `appsettings*.json`, and the suite goes red with the key's name in the failure message. (One adaptation when you write `RepoRoot()`: anchor the upward directory walk on a file your repo actually has at its root — `global.json` works perfectly and 0.2 guaranteed it exists.) The documentation *cannot* drift from the code, because drift fails CI. This is the third face of one idea you now hold: discovery over registration (1.3), isolation by base class (1.3), documentation by gate (here). None of them ask a human to remember anything.

Add the corresponding entries to `.env.example` as you go — from 0.2 it's the contract: every key, a comment saying what it does, required-or-default status. `Jwt__Secret=` goes in now

(empty value, comment "REQUIRED — ≥32 chars").

6. Run it — watch it fail correctly

The best test of fail-fast config is failing it on purpose:

```
# comment out Jwt__Secret in your .env, then:
dotnet run --project src/Api
# Unhandled exception: System.InvalidOperationException:
# Jwt:Secret not configured (set Jwt__Secret in .env for dev)
```

Boot refused, key named, fix stated. Restore the key; boots clean. Then run the suite — including your new doc-sync test — and prove the gate: read a fake key (`config["Nope:Missing"]`) anywhere in `src/`, watch the test name it, delete the line.

```
dotnet test tests/Api.Tests # all green, including ConfigKeys_ReadInCode_AreDocumented
```

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how does configuration reach the code? (a) `config["key"]` at call sites; (b) .NET's options pattern (`IOptions<T>` + `BindConfiguration` + `ValidateOnStart`); (c) hand-rolled typed settings classes validating in their constructors, registered eagerly as singletons.

Chosen: (c) as this platform's single blessed pattern (R22), with ADR-001's `.env` as the dev secret source and a doc-sync test (R20) making documentation mandatory. Constructor validation gives fail-at-boot with zero framework ceremony, interfaces make consumers trivially testable, and *one pattern everywhere* means a new reader learns it once.

Rejected: (a) scatters literals, defers failure to first use, and is exactly what the R20 gate exists to catch. (b) is genuinely fine — `ValidateOnStart` achieves the same fail-fast — but brings `IOptions<T>` indirection at every consumer and *two* ways to do one thing once any hand-rolled class exists. Uniformity won. The pattern choice matters less than the properties: eager, validated, typed, documented.

The trade: hand-rolled means we own the validation code, and each new section is a small class rather than one `Bind` call. Accepted for a platform this size; revisit if settings classes multiply into the dozens.

8. Checkpoint

```
git add -A && git commit -m "feat: typed validated config — .env loading, fail-fast
settings, doc-sync gate"
git tag lesson-1.4
```

You should now be able to: draw the config layering and say where secrets live in dev vs prod; explain why validation belongs in the constructor and registration must be eager; sort a config value into "safe default" vs "refuse to boot" and defend the sort; and describe how the doc-sync gate turns documentation from a request into an invariant.

Next: *Lesson 1.5 — The error envelope* — deciding, once, what every error in the system looks like from the outside.

Lesson 1.5 — The error envelope

> **Part 1 · The walking skeleton.** The shortest lesson in the course, on purpose — the artifact is five lines. But it's a five-line *treaty*: a decision about what every error in the system looks like from the outside, made once, before there are any errors to regret. Small decisions made early are cheap; the same decisions retrofitted across forty endpoints are a migration project.

Goal: one shared error shape for the whole API; a written rule that exception internals never cross the boundary; and clarity on what an error *code* is for versus an error *message*.

Concepts & principles: the error envelope; machine-readable vs human-readable error channels; information leakage through error text; deciding API contracts before the API grows.

Maps to: R16, R18 · repo: `src/Api/Services/ErrorResponse.cs`, its uses across every controller.

Prerequisites: 1.2 (an API exists to have errors).

---

1. Motivate — the anatomy of a leaked stack trace

Two failure modes, one root cause.

Failure one: the inconsistent zoo. Without a decision, each endpoint invents its own error shape: `{ "error": "..." }` here, `{ "message": "..." }` there, `{ "errors": [ ... ] }` from whoever wrote validation, a bare string from the intern's endpoint, HTML from the framework when nobody handled it at all. Every client — your own Blazor app first of all — grows a thicket of "well, if the body has *this* shape…" parsing. The API's error surface becomes something you *reverse-engineer* rather than read.

Failure two: the confession. Somewhere, a handler does the natural thing:

```
catch (Exception ex)
{
    return StatusCode(500, new { error = ex.Message }); // ← the natural thing. Don't.
}
```

Now consider what `ex.Message` can contain, on a bad day: `"28P01: password authentication failed for user \"app_rt\""` — a Postgres error naming an internal role. `"Connection refused (10.0.3.17:5432)"` — internal topology. `"The token 'eyJhbGciOi…' is malformed"` — a credential fragment. An attacker probing your API doesn't need a vulnerability report; your exceptions *are* one, streamed on demand. This is why rule R16 exists and is checked in review: **client-facing error bodies never contain raw exception text; detail stays in server logs**, where you have access control and the attacker doesn't.

Both failures have the same fix: decide the shape once, make the safe thing the easy thing.

2. Green — the envelope

The reference platform's entire `ErrorResponse.cs`:

```
namespace Perezosoft.Api.Services;

/// <summary>The standard API error envelope. Serializes to
/// <c>{ "error": ..., "message": ... }</c> (camelCase) — the shape every error
/// response uses.</summary>
public record ErrorResponse(string Error, string Message);
```

That's the artifact. Its power is entirely in the *convention it anchors*. Usage, from the real `AuthController`:

```
return Unauthorized(new ErrorResponse("no_refresh_token", "Refresh token not found"));
return Unauthorized(new ErrorResponse("invalid_refresh_token", "Refresh token is invalid or
expired"));
return StatusCode(500, new ErrorResponse("refresh_failed", "Failed to refresh token"));
```

Two fields, two audiences, and the discipline is in keeping them apart:

`error` is for machines. A stable, snake_case code — `invalid_refresh_token`, `seat_limit_reached` — that clients branch on. It's *API contract*: renaming one is a breaking change, exactly like renaming a JSON property. When the Blazor client (3.4) decides whether to bounce a user to login or show "try again," it switches on this field, never on prose.

`message` is for humans. A safe, self-contained sentence for logs and debugging. It describes the *category* of failure in words you chose — never words an exception chose. Note what `"Failed to refresh token"` doesn't say: no exception type, no stack frame, no connection string. The interesting detail went to the server log at the `catch` site, correlated by the request's trace ID (which Lesson 4.5 makes automatic).

Why not ProblemDetails (RFC 7807), the standards-blessed answer? It's a reasonable fork — see the decision box. The short version: the envelope predates it here, is smaller, and the *rule* (one shape, no leaks) matters far more than which shape.

3. Harden — a shape is only a shape if it's the only one

A convention with exceptions is a suggestion. Two enforcement layers make this one hold:

Status codes carry the first bit of meaning. The envelope rides on top of correct HTTP semantics, and this platform is unusually disciplined about three of them — you'll build each: **401** "we don't know who you are" (2.1), **403** "we know you, your *role* may not do this" (6.1, `RequirePermission`), **402** "we know you, your *plan* doesn't include this" (5.1, `RequireEntitlement`). Three different problems, three different client reactions (sign in / hide the button / show the upgrade page), distinguishable before reading a byte of body. The envelope then says *which* permission, *which* limit.

A scan for defectors. Rule R18: all error responses use the shared helper — no anonymous `{ error = ... }` objects. It's enforced the same way as 1.4's config gate: a source scan that fails when an anonymous error shape appears in a controller. You now have three of these grep-with-a-verdict tests (MailKit containment, config sync, error shape) — cheap, blunt, effective. When you write your own (3.3 formalizes the habit into the architecture-test project), keep the pattern: scan for the *forbidden* form, fail with the file name and the fix.

One honest caveat: model-binding failures (malformed JSON, missing required fields) produce ASP.NET's built-in `ValidationProblemDetails` before your code runs. The platform

leaves that be — framework-generated 400s are consistent *with each other*, and bending the framework's validation pipeline into the envelope isn't worth the ceremony. The envelope rule governs *your* error paths. Record the exception to the rule as part of the rule; an undocumented exception is how conventions rot.

4. Run it

```
curl -k https://localhost:7160/api/nope
# 404 — and once real endpoints exist, error bodies all read:
# { "error": "not_found", "message": "..." }
```

The real payoff is invisible today. It arrives every time this course adds an endpoint and there is exactly one way an error can look — and it arrives hardest in 3.4, when the client's error handling is a single small function instead of a parser museum.

5. Architecture Decision

ARCHITECTURE DECISION

The fork: what do API errors look like? (a) Whatever each endpoint returns; (b) RFC 7807 ProblemDetails everywhere; (c) a minimal shared envelope (`{ error, message }`) + correct status codes, with framework validation left native.

Chosen: (c). The binding decisions are *one shape* and *no exception text* (R16, R18); a two-field record delivers both with zero ceremony, and the machine-readable `error` code gives clients a stable contract the status code alone can't.

Rejected: (a) isn't a decision, it's an absence of one — and it's what naturally happens if this lesson doesn't exist. (b) is genuinely respectable: standard, extensible, tooling-friendly. It lost on weight — `type` URIs and extension members nobody would populate — and on the platform's preference for the smallest thing that holds the rule. Migrating later is mechanical precisely because every error already flows through one type.

The trade: a custom shape means API consumers read *your* docs, not an RFC. Accepted for a platform whose first-party clients dominate; the public API surface (7.3) documents the envelope in its OpenAPI schema.

6. Checkpoint

```
git add -A && git commit -m "feat: shared error envelope — one shape, no exception leakage"
git tag lesson-1.5
```

You should now be able to: name the two audiences of an error and which field serves each; explain why `ex.Message` in a response body is a security defect, not a style issue; distinguish 401/402/403 and say who reacts to each; and defend deciding contract shapes *before* the surface grows.

Next: *Lesson 1.6 — CI from commit one* — everything this part built (locked restore, warnings-as-errors, the gates) starts running on every push, forever.

Lesson 1.6 — CI from commit one

> **Part 1 · The walking skeleton.** The final lesson of the skeleton, and the one that makes everything before it *permanent*. So far, the gates you've built — locked restore, warnings-as-errors, the doc-sync scan — run when you remember to run them. This lesson puts them on a machine that never forgets: a pipeline that executes on every push, on a clean runner, with no local state to flatter you. The pipeline is born small and — like the architecture-test suite it partners with — **grows a job per part** for the rest of the course.

Goal: a GitHub Actions workflow that restores in locked mode, builds with warnings-as-errors, runs the tests (Testcontainers included), scans for secrets and copyleft licenses — red on any violation, on every push and pull request.

Concepts & principles: CI as a growing gate; clean-room verification ("if it only works on your machine, it doesn't work"); supply-chain gates in the pipeline; the PR checklist as process.

Maps to: R26, R27 (+ every earlier rule, now enforced remotely) · repo: `.github/workflows/ci.yml` (jobs `build-test`, `secret-scan`, `license-scan`), `.github/forbidden-licenses.json`, `.github/pull_request_template.md`.

Prerequisites: 1.1–1.5 (there must be something worth gating); a GitHub repo to push to.

---

1. Motivate — a gate you add at the end is a gate you never designed for

Here's what happens to teams that "add CI later." Later arrives; someone writes the workflow; it fails — of course it fails, nothing was ever built on a clean machine. The tests assume a database that isn't there, the build has 87 warnings that were always going to be errors *someday*, a connection string is hardcoded somewhere, and one package resolves differently than it did in March. Fixing all of it is now a project, so the workflow gets watered down — warnings allowed, the flaky tests skipped — and the team learns that CI is a formality. The gate exists, but the gate gates nothing.

Now run the tape the other way. The pipeline exists from commit one, when the entire system is a health endpoint and a probe entity. It's green in five minutes of work, because there's nothing to be red *about*. From then on, every lesson that adds a capability adds its check while the diff is small and the author is present. The pipeline never has a catching-up crisis, because it never fell behind. That's the whole argument, and it's the same argument as warnings-as-errors in 1.1: **adopt the standard when meeting it is free.**

There's a second, subtler payoff. Your machine has your `.env`, your Docker images, your half-remembered global tools. A green build there is a claim contaminated by local state. CI is a *clean room*: fresh runner, no `.env`, nothing but the repo and the workflow. When it's green, the claim "anyone can build this from a checkout" has actually been tested — which is, notice, the same claim Lesson 0.2 made about dev machines. CI is 0.2's reproducibility promise, verified continuously.

2. The workflow — build-test, from the real file

Create `.github/workflows/ci.yml`. This is the reference platform's actual first job, and every step is a decision you already made — now enforced remotely:

```
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - uses: actions/setup-dotnet@v5
        with:
          dotnet-version: "10.0.x"

      # Supply-chain: restore in LOCKED mode against the committed packages.lock.json
      # files. --locked-mode fails if the resolved graph differs from the lockfile, so a
      # dependency change can't slip in without updating the (reviewed) lockfile.
      - name: Restore (locked mode)
        run: |
          dotnet restore src/Api/Perezosoft.Api.csproj --locked-mode
          dotnet restore tests/Core.Tests/Perezosoft.Core.Tests.csproj --locked-mode
          dotnet restore tests/Api.Tests/Perezosoft.Api.Tests.csproj --locked-mode

      # Warnings-as-errors comes from Directory.Build.props — the same build policy
      # locally and here, one definition. --no-restore: reuse the locked restore above
      # (don't let the build re-restore loosely).
      - name: Build (warnings-as-errors)
        run: dotnet build src/Api/Perezosoft.Api.csproj -c Release --no-restore

      # Api.Tests spins up Postgres via Testcontainers — Docker is available on
      # ubuntu-latest, so the same tests run here as locally, unchanged.
      - name: Test (Core + Api)
        run: |
          dotnet test tests/Core.Tests/Perezosoft.Core.Tests.csproj -c Release --no-restore
          dotnet test tests/Api.Tests/Perezosoft.Api.Tests.csproj -c Release --no-restore

      # Fail the build if the EF model has drifted from the latest migration/snapshot.
      - name: No pending EF model changes
        env:
          ConnectionStrings__DefaultConnection:
"Host=localhost;Port=5432;Database=ef_designtime;Username=ci;Password=ci"
        run: |
          dotnet tool install --global dotnet-ef
          dotnet ef migrations has-pending-model-changes \
            --project src/Infrastructure/Perezosoft.Infrastructure.csproj \
            --startup-project src/Api/Perezosoft.Api.csproj
```

Walk the details, because each is a small lesson:

- **`--locked-mode` then `--no-restore`.** The restore step verifies the graph against

the lockfiles (1.1's gate, now unskippable); the build and test steps then *reuse* that restore. Without `--no-restore`, `dotnet build` would quietly re-restore in normal mode — and your

carefully locked graph would be decoration.

- **Testcontainers just works.** The GitHub runner has Docker, so the suite you built in

1.3 — real Postgres and all — runs identically here. This is the moment the Testcontainers decision pays its second dividend: no "CI database" to provision, no service-container YAML to keep in sync with the fixture.

- **The EF drift check** closes a gap you couldn't feel locally: change an entity,

forget `dotnet ef migrations add`, and everything *builds* — the model and the migrations have simply parted ways, to be discovered at the next deploy. `has-pending-model-changes` compares model to snapshot and fails loudly. Note the placeholder connection string: the check opens no DB connection, but the design-time factory (1.3) *requires the key to exist* — no fallback, remember. CI documents that contract by satisfying it explicitly.

3. Two more jobs — the supply-chain gates go remote

Secret scan. 0.2 installed gitleaks as a local backstop; policy is when it runs whether you remember or not. From the real workflow:

```
  secret-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
        with:
          fetch-depth: 0 # full history so the scan covers every commit, not just the tip

      - name: gitleaks
        env:
          GITLEAKS_VERSION: "8.21.2"
        run: |
          curl -sSfL "https://github.com/gitleaks/gitleaks/releases/download/v${GITLEAKS_VE
RSION}/gitleaks_${GITLEAKS_VERSION}_linux_x64.tar.gz" \
            | tar -xz gitleaks
          ./gitleaks detect --no-banner --redact --config .gitleaks.toml
```

Two deliberate choices worth stealing: `fetch-depth: 0` scans **all history**, because a secret committed and then "deleted" is still a leaked secret — git remembers; and `--redact`, because a scanner that prints the secrets it finds into a CI log has just created a second leak. (It runs the pinned OSS binary directly rather than a marketplace action — one less third party in the trust chain of the job whose *purpose* is trust.)

License scan. Rule R26: no copyleft in the dependency graph. Not because copyleft is bad — because for *this* platform (a commercial SaaS template) a GPL dependency imposes obligations the product can't meet, and a transitive one arrives silently with some innocent-looking package. The job inventories every license, direct **and transitive**, and fails on a prohibited list (`.github/forbidden-licenses.json`: GPL/AGPL/LGPL/SSPL):

```
  license-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-dotnet@v5
        with:
          dotnet-version: "10.0.x"
      - name: dotnet-project-licenses (fail on copyleft)
        run: |
          dotnet restore src/Api/Perezosoft.Api.csproj
          dotnet tool install --global dotnet-project-licenses
          export PATH="$PATH:$HOME/.dotnet/tools"
          dotnet-project-licenses --input src/Api/Perezosoft.Api.csproj \
            --include-transitive --use-project-assets-json \
            --forbidden-license-types .github/forbidden-licenses.json
```

Note the shape: a **prohibited-list, not an allow-list** — MIT/Apache/BSD and friends never trip a false positive, so the gate stays quiet until it has something real to say. A gate that cries wolf gets disabled; this one is designed to be believed. It's also a *separate job*, deliberately: a license failure and a build failure are different conversations, and the red X should say which one you're having.

4. The human gate — the PR template

One more file, `.github/pull_request_template.md`, auto-loaded by GitHub into every pull request: the platform's checklist — tests written first, tenant-scoping verified, docs updated, no direct UI→DB access. It gates nothing mechanically; it's the *review script* — the conversation every change must survive, written down. The division of labor is worth stating: **machines check what machines can check** (this lesson, the arch tests), **the checklist scripts what humans must check** (rules like R16's "no exception text" — review-enforced until a scan exists). Process is a gate too; it just runs on people.

5. Run it — push and watch

```
git add .github && git commit -m "ci: build-test + secret-scan + license-scan on every
push"
git push -u origin main
# → GitHub → Actions: three jobs, all green
```

Then prove the gates gate (one minute, permanent trust): bump any package version in `Directory.Packages.props` *without* updating the lockfile, push a branch, and watch `Restore (locked mode)` fail with `NU1004`. Revert. That red X is 1.1's promise, kept by a machine that doesn't know you and doesn't take your word for anything.

Where this file goes from here: e2e journeys join in 3.6; the Docker image build in 8.2; deploy-to-staging with a post-deploy smoke, then gated production, in 8.3; native platform legs in the appendix. In the reference repo this file ends at ~800 lines and fourteen jobs — every one of which you'll have written in the lesson that gave it a reason to exist.

6. Architecture Decision

ARCHITECTURE DECISION

The fork: when does CI arrive, and how much does it check? (a) Later, once there's "something real to test"; (b) now, minimal — build only, expand someday; (c) now, with every gate the codebase already claims to honor: locked restore, warnings-as-errors, tests, EF drift, secrets, licenses.

Chosen: (c). A claimed-but-unchecked standard decays silently; every gate in this job list is one the previous five lessons *already committed to*, so enforcing it remotely costs minutes now and prevents the catch-up crisis later. Growth by accretion — a job per capability, added with the capability — keeps every addition small and owned.

Rejected: (a) is how teams end up with the watered-down formality pipeline — descoped at exactly the moment it's needed most. (b) defers the cheap-now checks to when they're expensive, which is (a) with better intentions.

The trade: every push now takes a few minutes to go green, and a red X blocks merging even when "it's fine, it's just the license tool." That friction is the product. Accepted — and never bypassed with a skip flag; a gate you can wave through is a gate in name only.

7. Checkpoint

```
git tag lesson-1.6
```

Part 1 complete. You have a walking skeleton with a spine, a nervous system, and an immune system: an API answering HTTP, real-database tests, validated config, one error shape — and a clean-room pipeline that re-verifies all of it on every push, forever.

You should now be able to: argue for CI-from-day-one against "we'll add it when there's something to test"; explain `--locked-mode` + `--no-restore` as a pair; say why the secret scan reads full history and redacts; and describe the prohibited-list design and why quiet gates are trustworthy gates.

Next: *Part 2 — Identity & tenancy.* The skeleton gets people: users, tokens, and the tenant model everything else stands on.

Part 2 — Identity & tenancy

Lesson 2.1 — Users & JWT access tokens

> **Part 2 · Identity & tenancy.** The platform meets its first real domain concept — a user — and its most consequential early decision: **we build authentication ourselves** instead of adopting ASP.NET Core Identity. That choice ripples through the next twenty lessons, so this lesson earns it properly: what a JWT actually is, what belongs in one, and why "store hashes, never tokens" is the reflex to install before any token exists.

Goal: a `User` entity persists; the API can issue a signed JWT access token carrying identity claims; a protected endpoint returns 401 without one and 200 with one.

Concepts & principles: JWT anatomy (header/payload/signature); claims as the identity contract; custom auth vs framework Identity (ADR-002); hash-only credential storage; the token generator/hasher seams.

Maps to: ADR-002 · repo: `src/Core/Entities/User.cs`, `src/Api/Services/JwtTokenService.cs`, `JwtClaims.cs`, `TokenGenerator.cs`, `TokenHasher.cs`, `src/Api/Configuration/JwtValidation.cs`, `src/Api/Controllers/AuthControllerBase.cs`, `AuthController.cs` (begins), `tests/Api.Tests/TokenServiceTests.cs`, `UserServiceTests.cs`.

Prerequisites: Part 1 complete (config validation especially — `Jwt:Secret` from 1.4 is about to become load-bearing). Delete the `Probe` entity from 1.3 now; its job is done.

---

1. Motivate — the fork in the road

Type `dotnet new webapp --auth Individual` and you get ASP.NET Core Identity: user tables, password hashing, cookie login, two-factor scaffolding — batteries included. So the burden of proof is on *not* using it. Here's the case the reference platform recorded in ADR-002, and it's worth internalizing because it's really a case about **owning your core domain**:

This platform's authentication is not a checkbox; it's a *product surface*. Passwordless magic links (2.4), OAuth with account-linking rules (2.5), a `tenant_id` claim that drives all data isolation (2.6), MFA step-up on multiple sign-in paths (6.5), native clients with a different token transport (appendix) — every one of these needs precise control of *when tokens are issued, what they carry, and how sessions extend*. Identity can be bent to each of these, but you end up fighting a framework whose defaults are cookie-centric and whose schema you don't own — and the fights compound. Meanwhile the part of Identity that's genuinely hard to hand-roll — password hashing — **we don't need**, because this platform never stores passwords at all: sign-in is passwordless or via OAuth, full stop.

Strip out passwords and "authentication" reduces to two well-understood mechanics: **mint a signed claim set** (this lesson) and **manage its renewal** (2.2). Both sit on primitives the BCL provides. That's a domain worth owning.

The honest counter-case: owning auth means owning its mistakes — rotation, reuse detection, replay windows are now *your* tests (and this course writes them). If your product's auth is a checkbox — internal tool, password login is fine, nothing custom — take Identity and

spend your innovation budget elsewhere. Architecture is choosing what to own; this platform chooses auth, deliberately.

2. The `User` — smaller than you expect

```
// src/Core/Entities/User.cs — the real entity
public class User
{
    public Guid Id { get; set; } = Guid.CreateVersion7();
    public required string Email { get; set; }

    /// <summary>Display name from the OAuth provider, refreshed each sign-in.</summary>
    public string? DisplayName { get; set; }

    /// <summary>True only when the provider asserts a verified email claim.</summary>
    public bool EmailVerified { get; set; }

    /// <summary>Preferred UI language (e.g. "en", "es"), or null to fall back to the
    /// browser/OS culture. A per-user preference that follows the user across
devices.</summary>
    public string? Locale { get; set; }

    // Tenant membership is the source of truth for which tenant a user belongs to;
    // resolve it via TenantMembership (one tenant per user). See ITenantRepository.

    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset UpdatedAt { get; set; }

    /// <summary>OAuth identities linked to this account (one per provider).</summary>
    public ICollection<UserLogin> Logins { get; set; } = new List<UserLogin>();
}
```

Read it for what's **missing**. No password hash — decided above. No `TenantId` — the comment points at the deliberate indirection: membership is a separate entity (2.8), so "which tenant" has one source of truth and a user's account survives leaving a household. No roles — membership owns those too. Every nullable is an ADR in miniature: `DisplayName?` because providers may not send one; `Locale?` because null means "follow the device." And `Guid.CreateVersion7()` — UUIDv7 ids are time-ordered, so Postgres b-tree inserts append instead of splattering randomly (a real index-locality win over v4), while staying globally unique and unguessable enough for URLs. Add the `IEntityTypeConfiguration<User>` (unique index on `Email` — one account per address is a *rule*, so it's a *constraint*), and a migration.

3. What a JWT actually is — sixty seconds of anatomy

A JWT is three base64url segments: `header.payload.signature`. The payload is just JSON claims — **readable by anyone who holds the token** (paste one into any decoder). The signature is an HMAC-SHA256 over the first two segments with your `Jwt:Secret`: it makes the token *tamper-evident*, not *secret*. Two consequences drive everything else:

1. **Never put secrets in claims.** Claims are identity statements, not storage.
2. **Validation is pure computation.** Any API instance can verify a token with just the signing key — no session table, no auth-server round-trip. That's why JWTs scale horizontally for free, and also their weakness: *you can't revoke pure math*. A stolen access token is valid until it expires. The mitigation is the two-token design: access tokens are **short-lived** (60

minutes here) and renewal runs through a *revocable* refresh token — which is exactly Lesson 2.2.

4. Green — the token service

The real issuer, trimmed to its spine:

```
// src/Api/Services/JwtTokenService.cs
public class JwtTokenService(IJwtSettings settings, TimeProvider clock,
                             ILogger<JwtTokenService> logger) : IJwtTokenService
{
    public string IssueAccessToken(Guid userId, string email, string provider,
        string? displayName = null, string? tenantName = null,
        string? locale = null, Guid? tenantId = null)
    {
        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(settings.SecretKey));
        var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var claims = new List<Claim>
        {
            new(ClaimTypes.NameIdentifier, userId.ToString()),
            new(ClaimTypes.Email, email),
            new(JwtClaims.Provider, provider),
            // jti is an opaque uniqueness token, not a DB key — random Guid, not UUIDv7.
            new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
        };
        claims.Add(new Claim(ClaimTypes.Name,
            string.IsNullOrWhiteSpace(displayName) ? email : displayName));
        if (tenantId is { } tid)
            claims.Add(new Claim(JwtClaims.TenantId, tid.ToString()));
        // ... tenant_name, locale: same optional pattern

        var token = new JwtSecurityToken(
            issuer: settings.Issuer,
            audience: settings.Issuer,
            claims: claims,
            expires: clock.GetUtcNow().UtcDateTime.AddMinutes(settings.ExpiryMinutes),
            signingCredentials: credentials);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

Details that are decisions:

- **`IJwtSettings` and `TimeProvider` injected** — 1.4's validated config and the

injectable clock (formally taught in 3.3; note it's *already* here — expiry math must be testable, so ambient `DateTime.UtcNow` never appears).

- **The claim set is the contract.** `JwtClaims` centralizes the custom names as

constants — and its doc comment notes they *mirror the client's* `AppClaims`: these strings cross the wire, so they're API surface, named once. The `tenant_id` claim is optional here and earns its keep in 2.6, where it becomes the input to all data isolation. The `jti` comment repays reading: a *uniqueness* nonce wants randomness (v4), a *database key* wants ordering (v7) — the platform uses each where it belongs.

- **Validation parameters come from one place** (`JwtValidation/settings.CreateParameters()`):

the JWT-bearer middleware and the service's own `ValidateToken` share them, so "what makes a token valid" can't fork into two subtly different answers.

Wire up JWT-bearer in `Program.cs` (`AddAuthentication().AddJwtBearer(...)` using the same `IJwtSettings` instance `AddAppSettings` returned — the single-source trick from 1.4), add `app.UseAuthentication()/UseAuthorization()`, and a first protected endpoint: `GET /api/auth/me` on the nascent `AuthController` returning the caller's claims.

5. Red first, though — the tests that drove it

TDD order restored for the record — these exist before the service does (`TokenServiceTests` in the reference repo):

```
[Fact]
public void Issued_token_roundtrips_and_carries_identity()
{
    var svc = CreateService(); // stub IJwtSettings + FakeTimeProvider
    var token = svc.IssueAccessToken(userId, "a@b.test", provider: "test");

    var principal = svc.ValidateToken(token);

    Assert.NotNull(principal);
    Assert.Equal("a@b.test", principal!.FindFirstValue(ClaimTypes.Email));
}

[Fact]
public void Expired_token_is_rejected()
{
    var clock = new FakeTimeProvider(start);
    var svc = CreateService(clock);
    var token = svc.IssueAccessToken(userId, "a@b.test", "test");

    clock.Advance(TimeSpan.FromMinutes(61)); // past ExpiryMinutes=60

    Assert.Null(svc.ValidateToken(token));
}

[Fact]
public void Tampered_token_is_rejected()
{
    var svc = CreateService();
    var token = svc.IssueAccessToken(userId, "a@b.test", "test");
    var tampered = token[..^4] + "AAAA"; // corrupt the signature tail

    Assert.Null(svc.ValidateToken(tampered));
}
```

Expiry, tamper, roundtrip — the three properties that *are* a token service. The `FakeTimeProvider` makes the expiry test deterministic and instant; that's the injected clock paying rent already. At the HTTP level, the 1.2 harness gets its first auth-shaped assertions: `/api/auth/me` → 401 bare, 200 with a minted token. (In the reference repo those live in the integration harness you'll grow in 2.6's wire tests.)

6. Harden — the generator and hasher seams

Beyond JWTs, the platform is about to need many *opaque* tokens — refresh tokens (2.2), magic-link tokens (2.4), invitations (2.9). Two five-line services, built now, set the pattern for all of them:

```
public class TokenGenerator : ITokenGenerator
{
    public string GenerateToken()
        => Convert.ToBase64String(RandomNumberGenerator.GetBytes(64)); // CSPRNG, 512 bits
}

public class TokenHasher : ITokenHasher
{
    public string HashToken(string token)
        => Convert.ToBase64String(SHA256.HashData(Encoding.UTF8.GetBytes(token)));

    public bool Verify(string rawToken, string storedHash)
    {
        var computed = SHA256.HashData(Encoding.UTF8.GetBytes(rawToken ?? string.Empty));
        byte[] stored;
        try { stored = Convert.FromBase64String(storedHash ?? string.Empty); }
        catch (FormatException) { return false; }
        // FixedTimeEquals is constant-time and returns false (not throws) on a length
        // mismatch, so a malformed stored hash is simply rejected.
        return CryptographicOperations.FixedTimeEquals(computed, stored);
    }
}
```

Three security reflexes, in fifteen lines:

- **`RandomNumberGenerator`, never `Random`.** `Random` is predictable by design; a guessable token is no token. CSPRNG or nothing, for anything security-bearing.
- **The database stores hashes, never tokens.** The rule that names the lesson: when a refresh token or magic link lands in a table, it lands as SHA-256. A leaked database then yields nothing replayable — the attacker holds hashes of credentials, not credentials. (Why plain SHA-256, when passwords demand bcrypt/argon2? Because these are 512-bit *random* values — brute-forcing the preimage is hopeless. Slow hashes exist to protect *low-entropy* secrets. Using the right hash for the entropy at hand is itself the lesson.)
- **`FixedTimeEquals`.** Naïve `==` returns early on the first differing byte; timing the difference leaks how much of a guess was right. Constant-time comparison closes the side channel — and note the *fail-closed* posture of `Verify`: malformed stored hash → `false`, not an exception.

7. Run it

```
dotnet test tests/Api.Tests # token roundtrip/expiry/tamper + 401/200 wire tests
dotnet run --project src/Api
# then, with a token minted via a test helper or the temporary dev endpoint:
curl -k https://localhost:7160/api/auth/me -H "Authorization: Bearer eyJ..."
```

Paste the token into a JWT decoder and *look* at it: your claims, in plain sight. Internalize that — everything in a JWT is public; only its integrity is protected.

8. Architecture Decision

ARCHITECTURE DECISION

The fork: adopt ASP.NET Core Identity, or issue our own JWTs + refresh tokens?

Chosen: custom (ADR-002). The platform is passwordless-or-OAuth — Identity's hardest-won feature (password storage) is dead weight, while the things this product needs precise control over (claim contents driving tenancy, multiple sign-in paths converging on one issue-point, MFA step-up, native token transport) are exactly where a framework's opinions cost most. Token mechanics without passwords are small, well-understood, and — decisive point — *fully testable by us* (expiry, tamper, rotation, reuse all have first-class tests).

Rejected: Identity — for *this* product. The trade is real: we own every auth bug, and several later lessons exist to pay that debt honestly. Also rejected: an external IdP (Auth0/Entra) — a fine choice in many orgs, but it outsources the exact seams this course exists to teach, and its per-user pricing fights a template meant to run free.

The trade: ~1,500 lines of auth code across Part 2 that Identity would have given us "free," in exchange for auth that bends to the product instead of the reverse — and zero framework fights in lessons 2.4, 2.5, 6.5, and the native appendix.

9. Checkpoint

```
git add -A && git commit -m "feat: users + JWT access tokens — custom auth foundation
(ADR-002)"
git tag lesson-2.1
```

You should now be able to: decode a JWT by eye and say which parts are secret (none) and which are protected (integrity); justify custom-auth-here without defending custom-auth-everywhere; explain hash-only storage and why *fast* SHA-256 is correct for *high-entropy* tokens; and name the three properties every token service must prove by test.

Next: *Lesson 2.2 — Rotating refresh tokens & sessions* — how a 60-minute token becomes a 30-day session without becoming a 30-day theft window.

Lesson 2.2 — Rotating refresh tokens & sessions

> **Part 2 · Identity & tenancy.** Lesson 2.1 left a deliberate tension: access tokens expire in 60 minutes (because they're unrevocable), but nobody will use a product that logs them out hourly. The classical resolution is the **refresh token** — and the interesting engineering isn't the happy path, it's the two attacker-facing properties: rotation (a refresh token works *once*) and **reuse detection** (a replayed old token doesn't just fail — it burns the thief's entire access).

Goal: `POST /api/auth/refresh` exchanges a valid refresh token for a fresh access token + a *new* refresh token; the used token is dead; replaying a dead token revokes every session the user has.

Concepts & principles: the two-token session model; rotation; reuse-as-theft-signal; the HttpOnly cookie as token transport (and each cookie attribute as a decision); server-side revocability.

Maps to: ADR-002 · repo: `src/Core/Entities/RefreshToken.cs`, `src/Api/Services/RefreshTokenService.cs`, `SessionService.cs`, `CookieService.cs`, `src/Api/Controllers/AuthController.cs` (`/refresh`, `/logout`), `tests/Api.Tests/SessionServiceTests.cs`, `CookieServiceTests.cs`.

Prerequisites: 2.1 (JWTs, `TokenGenerator`/`TokenHasher`).

---

1. Motivate — the session-length trilemma

Pick two: long sessions, revocability, stateless validation. A pure-JWT design that stretches expiry to 30 days gets long sessions and statelessness — and a stolen token that works for a month with no kill switch. Short-lived JWTs alone get revocability by expiry — and hourly logouts. The two-token model takes all three by *splitting the job*:

- The **access token** (2.1) stays short-lived and stateless — validated by signature on

every request, no database touch, horizontally free.

- The **refresh token** is long-lived (30 days) but *stateful*: an opaque random value

whose hash lives in a table, checked against that table exactly once per hour-ish, when the client exchanges it. Server-side state means server-side control: revoke the row, kill the session.

The pattern's cost is that the refresh token is now the crown jewel — steal one and you own the session for a month. So the whole design bends toward protecting *it*: it rides in an `HttpOnly` cookie JavaScript can't read, it's stored only as a hash, and — the crucial move — it's **single-use**.

2. The entity — a session record wearing a token costume

```
// src/Core/Entities/RefreshToken.cs — the real entity
public class RefreshToken
{
    public Guid Id { get; set; } = Guid.CreateVersion7();
    public Guid UserId { get; set; }
    public required string TokenHash { get; set; } // never the token itself
    public DateTimeOffset IssuedAt { get; set; }
    public DateTimeOffset ExpiresAt { get; set; }
    public bool IsRevoked { get; set; }
    public required string IssuedFromIp { get; set; }

    /// <summary>OAuth provider used at sign-in for this session; carried through
    /// rotation so refreshed JWTs keep an accurate provider claim.</summary>
    public required string Provider { get; set; }
}
```

Notice what this table *is*: the platform's session store. Each row is one device's session; `RevokeAllForUser` is "sign me out everywhere"; `IssuedFromIp` is forensics. And notice `IsRevoked` — rotated tokens are **flagged, not deleted**. That's not housekeeping laziness; it's the entire reuse-detection mechanism, as you're about to see. (A scheduled cleanup job eventually prunes long-dead rows — Lesson 4.4, and now you know why that job exists.)

3. Issuing — the seams compose

```
// src/Api/Services/RefreshTokenService.cs (core of the real file)
public async Task<IssuedRefreshToken> IssueRefreshTokenAsync(
    Guid userId, string ipAddress, string provider, CancellationToken ct = default)
{
    var rawToken = tokenGenerator.GenerateToken(); // 512-bit CSPRNG (2.1)
    var tokenHash = tokenHasher.HashToken(rawToken); // SHA-256 (2.1)
    var now = clock.GetUtcNow();

    var refreshToken = new RefreshToken
    {
        Id = Guid.CreateVersion7(),
        UserId = userId,
        TokenHash = tokenHash,
        IssuedAt = now,
        ExpiresAt = now.AddDays(settings.ExpiryDays),
        IsRevoked = false,
        IssuedFromIp = ipAddress,
        Provider = provider,
    };

    var created = await repository.CreateAsync(refreshToken, ct);
    return new IssuedRefreshToken(rawToken, created); // raw → client; hash → DB
}
```

The return type says the security model in one line: `IssuedRefreshToken(RawToken, Token)` — the raw value goes to the client and is *never seen again by the server as plaintext*; the entity holds only the hash. Its doc comment states the payoff: "a database leak cannot be used to forge cookies." The generator and hasher you built in 2.1 as ten throwaway lines are now load-bearing — that's what a seam buys.

4. The refresh endpoint — where rotation and detection live

The service classifies; the controller responds. First the classifier — a presented token is exactly one of four things:

```
public enum RefreshTokenStatus
{
    Valid, // active, unexpired — safe to rotate
    Expired, // known token past its expiry — reject (no theft signal)
    Unknown, // no such hash — reject as a plain bad token
    Reuse, // a previously-rotated token is being REPLAYED — theft signal
}

public async Task<RefreshTokenInspection> InspectRefreshTokenAsync(string rawToken, ...)
{
    var token = await repository.GetByHashAsync(tokenHasher.HashToken(rawToken), ct);
    if (token is null) return new(RefreshTokenStatus.Unknown, null);
    // A revoked row whose hash is being presented = a rotated-out token replayed → theft.
    if (token.IsRevoked) return new(RefreshTokenStatus.Reuse, token);
    if (token.ExpiresAt <= clock.GetUtcNow()) return new(RefreshTokenStatus.Expired,
token);
    return new(RefreshTokenStatus.Valid, token);
}
```

And the endpoint (abridged from the real `AuthController.Refresh`):

```
var inspection = await refreshTokenService.InspectRefreshTokenAsync(rawToken, ct);
if (inspection.Status == RefreshTokenStatus.Reuse)
{
    // Replay of a rotated-out token   assume theft: revoke every session for the user.
    // Client still gets the generic error below, so the reuse signal isn't leaked.
    await refreshTokenService.RevokeAllUserTokensAsync(inspection.Token!.UserId, ct);
    logger.LogWarning("Refresh-token reuse detected for user {UserId}; revoked all
sessions", ...);
}
if (inspection.Status != RefreshTokenStatus.Valid)
    return Unauthorized(new ErrorResponse("invalid_refresh_token", "Refresh token is
invalid or expired"));

// Rotate: revoke the used token, then issue a fresh session.
await refreshTokenService.RevokeRefreshTokenAsync(validToken.Id, ct);
var session = await sessionService.IssueAsync(user, validToken.Provider, ClientIp, native,
ct);
cookieService.SetRefreshTokenCookie(Response, session.RefreshToken, Request);
return Ok(session.Response);
```

Walk the threat model, because every line answers an attack:

Why rotate at all? If refresh tokens were reusable, a stolen one is a 30-day skeleton key and you'd never know. Rotation makes every token single-use — so after a theft, *two parties* hold single-use tokens: the attacker and the legitimate user. Whichever redeems first wins… and the *second* redemption trips `Reuse`.

Why is `Reuse` a five-alarm signal? Because in a correct client, a rotated-out token is *gone* — replaced the moment it was used. The only party that presents a dead token's hash is someone who copied it earlier: a thief (or, rarely, a client bug — which also deserves investigation). The response is deliberately maximal: revoke **all** the user's sessions. Attacker *and* user get logged out everywhere; the user re-authenticates and is safe; the attacker holds nothing. One stolen token cost the attacker their entire foothold.

Why the same generic 401 for `Reuse` and `Unknown`? Look at the comment: "the reuse signal isn't leaked." If reuse returned a distinguishable response, an attacker could *probe*: present a captured token and learn from the error whether it's been used yet — i.e., whether it's still worth redeeming, or whether their theft has been noticed. Same principle as 1.5's error envelope: what an error *doesn't say* is part of its design. The signal goes where it belongs — the audit log and a `LogWarning` — not to the caller.

Why `Expired` ≠ `Reuse`? An expired-but-never-rotated token is a user who left a laptop closed for a month — normal life, not theft. Treating it as an attack would mass- revoke sessions on every stale device. Classifying failure modes *separately, then responding differently* is the deep skill this endpoint teaches.

5. The cookie — every attribute is a decision

The refresh token travels in a cookie, and the real `CookieService` sets each attribute for an articulable reason:

```
response.Cookies.Append(RefreshTokenCookieName, token, new CookieOptions
{
    HttpOnly = true, // JS can never read it → XSS can't exfiltrate it
    Secure = request.IsHttps, // HTTPS-only transport (see the dev nuance below)
    SameSite = SameSiteMode.Lax, // sent on same-site + top-level nav; CSRF-resistant
    Path = "/api/auth", // sent ONLY to auth endpoints — nothing else needs it
    // MaxAge/Expires: matches the token's ExpiresAt
});
```

- **`HttpOnly`** is the reason the refresh token lives in a cookie at all rather than

localStorage: a script-injection (XSS) can call your API as the user *while the page is open*, but it cannot *steal the durable credential* — the browser withholds it from JavaScript entirely.

- **`Path=/api/auth`** is least-privilege for cookies: the token accompanies exactly the

three endpoints that need it (`refresh`, `logout`, sign-in) and never rides along on every API call, shrinking both attack surface and bytes-per-request. (The real file carries battle scars here — a long comment about expiring a legacy `Path=/` cookie that would otherwise *shadow* the scoped one. Cookies have sharp edges; the comments are the platform remembering a real 3-hour debug.)

- **`SameSite=Lax` + `Secure = request.IsHttps`** encode a dev/prod reality: in dev the

client (`:7008`) and API (`:7160`) are different *ports* — same-site, so `Lax` works — and `Secure` follows the actual scheme because a hardcoded `Secure=true` silently drops the cookie on any non-HTTPS leg. The comment in the real file names the failure: "silently breaks refresh." In production this whole class of problem is *deleted* rather than configured — single-origin hosting (8.1) puts client and API on one origin. That decision is being planted here, two parts early, and this cookie is why.

Notice also what the endpoint does *not* require: `[Authorize]`. Refresh runs on the cookie alone — by design, because its whole purpose is to work *after* the access token expired. (Native clients, which have no cookie jar, send the token in the body under an `X-Native-Client` header — same endpoint, different transport; the appendix picks this up.)

6. Red — the tests that pin the properties

From `SessionServiceTests`/the integration suite — the three that matter, in Gherkin first (the story format from `WAYS_OF_WORKING.md`):

```
Scenario: refresh rotates the token
  Given a signed-in user with refresh token R1
  When the client exchanges R1
  Then it receives a new access token and refresh token R2
  And exchanging R1 again fails

Scenario: replaying a rotated token burns all sessions
  Given tokens R1 (rotated out) and R2 (current) for the same user
  When an attacker presents R1
  Then the response is the same generic 401 as any bad token
  And R2 no longer works either

Scenario: expired token is rejected without the theft response
  Given a token past ExpiresAt that was never rotated
  When the client presents it
  Then the response is 401
  And the user's other sessions remain valid
```

The second scenario is the one teams skip — and it's the *entire point* of rotation. Write it with two clients: rotate via client A, replay the old token via client B, then assert client A's *new* token is also dead. `FakeTimeProvider` drives the third.

7. Run it

```
dotnet test tests/Api.Tests
# then live, via the .http scratchpad: sign in (temporary dev seed), then
# POST {{host}}/api/auth/refresh → 200, Set-Cookie with a NEW value
# POST again with the OLD cookie → 401 invalid_refresh_token (and a warning in the API log)
```

Watching the reuse warning appear in the log while the client sees only a generic 401 — that's the design, observable.

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how do sessions extend? (a) Long-lived JWTs (30-day expiry, no server state); (b) server sessions for everything (session id cookie, every request hits the store); (c) short JWT + rotating single-use refresh token with reuse detection.

Chosen: (c) — the standard modern trade (ADR-002). Per-request validation stays stateless (JWT signature); the stateful check runs once per access-token lifetime, at the refresh exchange, which is also the *choke point* where revocation, rotation, and theft detection all naturally live. Reuse-response = revoke-all: maximal, cheap, and user-recoverable (they just sign in again).

Rejected: (a) — unrevocable month-long credentials; a breach response of "rotate the signing key and log out *every user on the platform*" is not a plan. (b) — a DB hit on every API call buys revocation-latency we don't need at the cost of the JWT's free horizontal scaling; reasonable elsewhere, wrong here.

The trade: more moving parts than either pure option — four token states, cookie path subtleties, a cleanup job (4.4), and the client must handle mid-flight 401s by refreshing and retrying (3.4 builds exactly that). Each part is testable, and each got its test before its code.

9. Checkpoint

```
git add -A && git commit -m "feat: rotating refresh tokens — sessions, reuse detection,
HttpOnly cookie transport"
git tag lesson-2.2
```

You should now be able to: draw the two-token model and say which token is stateless and why; explain rotation as the mechanism that *converts theft into a detectable event*; justify revoke-all-on-reuse and the deliberately indistinguishable 401; and defend every attribute on the refresh cookie, including the path scoping.

Next: *Lesson 2.3 — Email I: the `IEmailSender` seam* — before magic links can be mailed, the platform needs exactly one way to send email.

Lesson 2.3 — Email I: the `IEmailSender` seam

> **Part 2 · Identity & tenancy.** Before the next lesson can mail a magic link, the platform needs to send email — and *how* you introduce that capability is a masterclass in the seam pattern. One interface in Core; MailKit quarantined behind it so hard that a test fails if it leaks; Mailpit as the dev transport so your inbox stays a test fixture; and branded, localized templates with the one logo-embedding technique real mail clients actually render. Email is also this book's best *long-game* example: the five-line interface you write today gets decorated into crash-safety in Lesson 4.2 without touching a single call site.

Goal: `IEmailSender` exists in Core; an SMTP implementation (MailKit) lives in `Infrastructure/Email/` and nowhere else — enforced by a test; a branded test email lands in Mailpit's inbox.

Concepts & principles: the seam (swap-point interface); dependency quarantine with a containment test; SMTP dev/prod duality (Mailpit ↔ Brevo, same code); email HTML as a hostile rendering target; CID inline images; server-side localization by explicit culture.

Maps to: auth rules ("`IEmailSender` is the only way to send email") · repo: `src/Core/Abstractions/IEmailSender.cs`, `src/Infrastructure/Email/SmtpEmailSender.cs`, `SmtpSettings.cs`, `BrandedEmail.cs`, `EmailStrings.resx`, `Assets/logo.png`, `tests/Api.Tests/DocAndConfigSyncTests.cs` (`MailKit_StaysBehindInfrastructureEmail`), `tests/Api.Tests/SmtpSettingsTests.cs`.

Prerequisites: 0.2 (Mailpit is already running in your compose stack); 1.4 (options pattern).

---

1. Motivate — email is a vendor decision wearing a feature costume

"Send an email" sounds like a feature. It's actually a *vendor relationship*: dev wants a fake inbox, staging wants a free SMTP relay (Brevo), production might want SES or Postmark next year, and tests want to assert "an email was sent" without any network at all. If `new SmtpClient()` calls are scattered through your services, every one of those is a migration. If instead there is exactly **one** interface and callers know nothing else, every one of those is a config change or one new class.

That's the seam pattern, and email is its cleanest first outing because the swap pressure is real and immediate — you'll swap transports *today* (Mailpit locally, Brevo when staging arrives in 8.2) without either "side" of the seam noticing.

2. The interface — small, but every word chosen

```
// src/Core/Abstractions/IEmailSender.cs — the real file
public interface IEmailSender
{
    /// <summary>
    /// Sends an HTML email. <paramref name="inlineImages"/> are embedded in the message
    /// (multipart/related) and referenced from the HTML via <c>cid:{ContentId}</c> — the
    /// only logo-embedding approach mainstream clients (Gmail/Outlook) render reliably.
    /// <para>Throws <see cref="EmailSendException"/> if delivery to the SMTP server fails;
    /// a hung server is bounded by a per-send timeout rather than stalling the request
    /// indefinitely.</para>
    /// </summary>
    Task SendAsync(string to, string subject, string htmlBody,
        IReadOnlyList<EmailInlineImage>? inlineImages = null, CancellationToken
cancellationToken = default);
}

public sealed record EmailInlineImage(string ContentId, string FileName, byte[] Content,
string MediaType);

public sealed class EmailSendException(string message, Exception innerException)
    : Exception(message, innerException);
```

Study the *placement* first: this lives in **Core**, which references nothing — so the interface mentions no MailKit type, no `MimeMessage`, nothing SMTP-flavored. It speaks the domain's language ("send this HTML to this address, these images inline") and lets Infrastructure translate. Note also the custom `EmailSendException`: callers catch a *domain* exception, not `MailKit.Net.Smtp.SmtpCommandException` — because catching a vendor's exception type is a vendor reference in disguise, and it would quietly weld call sites to the implementation the interface exists to hide.

3. The implementation — and the sharp edges it rounds off

```
// src/Infrastructure/Email/SmtpEmailSender.cs (the real file, lightly trimmed)
public class SmtpEmailSender(IOptions<SmtpSettings> options, ILogger<SmtpEmailSender>
logger) : IEmailSender
{
    private readonly SmtpSettings _settings = options.Value;

    public async Task SendAsync(string to, string subject, string htmlBody,
        IReadOnlyList<EmailInlineImage>? inlineImages = null, CancellationToken ct =
default)
    {
        var message = new MimeMessage();
        message.From.Add(new MailboxAddress(_settings.FromName, _settings.FromAddress));
        message.To.Add(MailboxAddress.Parse(to));
        message.Subject = subject;

        var builder = new BodyBuilder { HtmlBody = htmlBody };
        if (inlineImages is not null)
            foreach (var image in inlineImages)
            {
                var resource = builder.LinkedResources.Add(
                    image.FileName, image.Content, ContentType.Parse(image.MediaType));
                resource.ContentId = image.ContentId; // referenced from HTML as
cid:{ContentId}
            }
        message.Body = builder.ToMessageBody();

        using var client = new SmtpClient
        {
            // Bound a hung server: without this, a stuck Connect/Send stalls the request
forever.
            Timeout = _settings.TimeoutSeconds * 1000,
            CheckCertificateRevocation = _settings.CheckCertificateRevocation,
        };
        try
        {
            await client.ConnectAsync(_settings.Host, _settings.Port,
SecureSocketOptions.Auto, ct);
            if (!string.IsNullOrEmpty(_settings.Username))
                await client.AuthenticateAsync(_settings.Username, _settings.Password ??
string.Empty, ct);
            await client.SendAsync(message, ct);
            await client.DisconnectAsync(true, ct);
        }
        catch (OperationCanceledException)
        {
            throw; // genuine cancellation — let it propagate, don't mask as a send failure
        }
        catch (Exception ex)
        {
            logger.LogError(ex, "SMTP send to {Host}:{Port} failed", _settings.Host,
_settings.Port);
            throw new EmailSendException($"Failed to send email via
{_settings.Host}:{_settings.Port}.", ex);
        }
    }
}
```

Three details that separate production code from sample code:

- **The timeout.** SMTP servers hang; without a bound, the *user's sign-in request*

hangs with them. The `Timeout` line converts "infinite stall" into "bounded failure" — and its value is config (`SmtpSettings.TimeoutSeconds`), because 1.4 taught us where numbers like this live. (`CheckCertificateRevocation` is likewise a knob — added when a corporate network's blocked CRL endpoint made every send fail; a real scar with a real test, `SmtpSettingsTests`.)

- **The `OperationCanceledException` pass-through.** A cancelled request isn't a failed

email; catching it into `EmailSendException` would lie to callers and pollute logs with phantom SMTP errors. Distinguishing *cancellation* from *failure* is a habit worth stealing for every I/O wrapper you ever write.

- **Auth only when configured.** Mailpit takes no credentials; Brevo requires them. One

`if` makes the same code serve both — the dev/prod duality lives in config, not code.

Add `SmtpSettings` (host/port default to Mailpit — `localhost:1025` — so a fresh checkout sends to the trap with *zero* config), the `.env.example` block for a real provider (commented out, per the contract), and register in DI: `services.AddScoped<IEmailSender, SmtpEmailSender>()`.

4. The containment test — a seam with a fence

An abstraction "everyone should use" lasts until the first hurried Tuesday. The platform makes the quarantine mechanical (from the real `DocAndConfigSyncTests`):

```
[Fact]
public void MailKit_StaysBehindInfrastructureEmail()
{
    // The IEmailSender abstraction is the only way to send email; the concrete
    // MailKit/MimeKit dependency must never leak outside src/Infrastructure/Email/.
    var offenders = SourceFiles(Path.Combine(RepoRoot(), "src"))
        .Where(f => !f.Contains(Path.Combine("Infrastructure", "Email")))
        .Where(f => File.ReadAllText(f) is var t && (t.Contains("using MailKit")
                    || t.Contains("using MimeKit") || t.Contains("MailKit.") ||
t.Contains("MimeKit.")))
        .Select(Path.GetFileName).ToList();

    Assert.True(offenders.Count == 0,
        $"MailKit/MimeKit must stay behind IEmailSender: {string.Join(", ", offenders)}");
}
```

You've now built this move three times (config sync, error shapes, and this) — but note what's different here: this is the first time a *dependency* is fenced rather than a practice. The rule "only Infrastructure/Email knows MailKit exists" stops being a convention the moment this test exists; it's a property of the codebase. When 4.2 wraps this sender in an outbox decorator, and when a future maintainer swaps MailKit for something else entirely, the blast radius is one folder — *provably*.

5. Branded, localized — email HTML is a hostile country

A magic-link email is often the *first* thing your product ever shows a user — it should look like the product. `BrandedEmail` builds the HTML, and its doc comment is a compact field guide to why email templates can't just reuse your web CSS:

> Email HTML is its own world — table-based layout, inline styles, web-safe fonts only — so this does NOT reuse the app's CSS. The logo is embedded via CID (multipart/related), the one approach Gmail and Outlook render reliably (they block data-URI images).

Unpack the two traps it names. **Layout:** mail clients render HTML with engines that are years behind browsers and aggressively sanitized — no external stylesheets, patchy flexbox, `<style>` blocks stripped by some clients. Table layout + inline styles isn't retro taste; it's the *only* dialect with universal support. **The logo:** you can't hotlink it (blocked-by-default remote images) and you can't data-URI it (Gmail/Outlook block those too). The working answer is **CID embedding**: the PNG travels *inside* the message as a `multipart/related` part with a `Content-ID`, and the HTML references `cid:perezosoft-logo`. That's why `IEmailSender` has the `inlineImages` parameter — the seam's shape was dictated by a Gmail rendering rule, which is exactly the kind of knowledge that belongs *behind* a seam.

And localization — note the design, because it differs from UI localization in one important way:

```
private static readonly ResourceManager Rm =
    new("Perezosoft.Infrastructure.Email.EmailStrings", typeof(BrandedEmail).Assembly);

public static CultureInfo ResolveCulture(string? locale)
{
    if (string.IsNullOrWhiteSpace(locale)) return DefaultCulture; // "en"
    try { return CultureInfo.GetCultureInfo(locale.Trim()); }
    catch (CultureNotFoundException) { return DefaultCulture; } // fail soft, not 500
}
```

Emails are rendered **server-side, in an explicit culture** — the requester's UI language for an OTP, the *inviter's* saved locale for an invitation — so the code keys `ResourceManager` by a passed-in culture and never trusts the ambient thread culture (which on a server belongs to nobody in particular). `EmailStrings.resx` + `EmailStrings.es.resx` hold the strings; Lesson 3.5 adds the UI's resx pipeline and they'll rhyme. Also file away the *rebrand trap* the platform's docs call out: this inline logo and palette are brand touchpoints that live nowhere near the UI folder — lesson 9.1's checklist exists because everyone forgets them.

6. Red → Green — Mailpit is your assertable inbox

The test story for email is two-layered. Unit level: a `FakeEmailSender` recording `(to, subject, body)` — services in later lessons assert against it. Integration level: actually send, and *read the result back out of Mailpit* — it exposes a REST API over its inbox (`GET /api/v1/messages`), which is precisely why 0.2 chose it:

```
[Fact]
public async Task Branded_email_arrives_with_inline_logo()
{
    var sender = CreateRealSender(); // pointed at localhost:1025
    var body = BrandedEmail.MagicLink("https://example.test/x",
BrandedEmail.ResolveCulture("es"));

    await sender.SendAsync("probe@test.local", body.Subject, body.Html, body.InlineImages);

    var msg = await Mailpit.LatestMessageTo("probe@test.local"); // Mailpit's REST API
    Assert.Contains("cid:", body.Html);
    Assert.Equal(body.Subject, msg.Subject); // localized subject
}
```

In 3.6 this same trick graduates to E2E: Playwright journeys will fish OTP codes out of Mailpit to complete real sign-ins, browser-to-inbox-to-browser.

7. Run it

```
docker compose up -d # Mailpit already in your stack from 0.2
dotnet test tests/Api.Tests # containment + settings + send tests
# then send one for real (temporary dev endpoint or test), and open:
# http://localhost:8025 → your branded email, logo rendered, in Spanish if you asked
```

Seeing the actual rendered email in Mailpit's UI is worth thirty assertions: fonts, layout, logo — this is what your user's first impression looks like.

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how does the platform send email? (a) SMTP client calls inline where needed; (b) a vendor SDK (SendGrid/SES) used directly; (c) one Core seam (`IEmailSender`), an SMTP implementation behind it, containment enforced by test.

Chosen: (c), with **SMTP as the protocol** rather than any vendor's API — SMTP is the one interface every provider (Mailpit, Brevo, SES, Postmark) speaks, so even the *implementation* is vendor-portable; switching providers is a config change (8.2 does exactly this for staging). The seam keeps callers domain-pure and gives 4.2 its decoration point.

Rejected: (a) welds a vendor and its failure modes into every call site — the anti-seam. (b) buys nicer delivery analytics at the price of a per-vendor rewrite and mocks that mimic a vendor SDK; if those analytics are ever needed, they arrive as *another implementation of the same seam*.

The trade: SMTP is lowest-common-denominator — no template hosting, no delivery webhooks (bounces land in a provider dashboard, not your code). Accepted; a platform can graduate individual needs later without touching a single caller.

9. Checkpoint

```
git add -A && git commit -m "feat: email seam — IEmailSender, MailKit quarantined, branded
localized templates"
git tag lesson-2.3
```

You should now be able to: place an interface on the correct side of the dependency rule and say why the exception type matters as much as the method signature; explain CID embedding and why email HTML can't share the app's CSS; state why emails localize by explicit culture; and write a containment test for any dependency you ever decide to quarantine.

Next: *Lesson 2.4 — Passwordless: magic link + OTP* — the seam gets its first real customer, and the platform gets its first sign-in.

Lesson 2.4 — Passwordless: magic link + OTP

> **Part 2 · Identity & tenancy.** The platform gets its first sign-in — and it's not a password form. Two passwordless credentials, one entity: the **magic link** (a long random token in an emailed URL) and the **OTP** (a short numeric code you type). The magic link is easy. The OTP is where this lesson earns its place, because a 6-digit code has a million possibilities and an attacker has a keyboard — the interesting engineering is entirely in the *brute-force arithmetic* and the lockout design that answers it, including one genuinely subtle bug ("resend-proof lockout") the platform's audit caught.

Goal: a user with nothing but an email address can sign in twice over — click a mailed link, or type a mailed code — and both credentials are single-use, time-limited, stored only as hashes, and brute-force-proof.

Concepts & principles: possession-based auth (email as the factor); high- vs low-entropy credentials and their different defenses; cumulative lockout windows; enumeration-resistant issuance *and* error mapping; account creation at redemption.

Maps to: ADR-C15, auth rules (`LoginToken` + `PasswordlessService`, lifetimes in config) · repo: `src/Core/Entities/LoginToken.cs`, `src/Api/Services/PasswordlessService.cs`, `AuthController` issue/redeem endpoints, `tests/Api.Tests/PasswordlessServiceTests.cs`, `OtpErrorMappingTests.cs`, `tests/Core.Tests/LoginTokenTests.cs`.

Prerequisites: 2.1 (generator/hasher, JWT issue), 2.2 (sessions to mint on success), 2.3 (email to deliver the credentials).

---

1. Motivate — passwords are a liability you can decline

Every password your system stores is a promise: to hash it with an expensive KDF, to survive credential-stuffing from every other site's breach, to run "forgot password" flows that are themselves attack surface, to nag about rotation. Decline the deal and the liability evaporates: **prove control of the inbox instead**. Anyone who can read `ana@example.test`'s email *is* — for this product's threat model — Ana; email is already the recovery root for password systems anyway, so passwordless just removes the middleman.

Two ergonomic forms of the same proof:

	Magic link	OTP
Credential	512-bit random token in a URL	6 random digits
UX	one click, same device	type a code — survives cross-device (read on phone, type on desktop)
Entropy	~10^{77} possibilities	**10^6 possibilities**
Defense	its own entropy	*everything else in this lesson*

That entropy row is the lesson's spine. Nobody brute-forces the link. The code, though — a million possibilities, five tries a code, and unlimited codes on request? Do the math before writing the code; we'll do it in §4.

2. One entity for both — `LoginToken`

```
// src/Core/Entities/LoginToken.cs — the real entity
public class LoginToken
{
    public Guid Id { get; set; } = Guid.CreateVersion7();

    /// <summary>Normalized (lower-cased) email the credential was issued to.</summary>
    public required string Email { get; set; }

    /// <summary>SHA-256 hash of the raw token/code.</summary>
    public required string CodeHash { get; set; }

    /// <summary><see cref="LoginTokenPurpose"/> — magic link or OTP.</summary>
    public required string Purpose { get; set; }

    public DateTimeOffset ExpiresAt { get; set; }

    /// <summary>Set when the credential is redeemed (or locked out); null while
usable.</summary>
    public DateTimeOffset? ConsumedAt { get; set; }

    /// <summary>Failed verification attempts — used to lock out OTP
brute-forcing.</summary>
    public int AttemptCount { get; set; }

    public DateTimeOffset CreatedAt { get; set; }

    // Derived, never stored. The *At(now) overloads are the deterministic core (testable
    // with an explicit clock); the parameterless properties are ambient-time conveniences.
    public bool IsConsumed => ConsumedAt.HasValue;
    public bool IsExpiredAt(DateTimeOffset now) => now >= ExpiresAt;
    public bool IsValidAt(DateTimeOffset now) => !IsConsumed && !IsExpiredAt(now);
}
```

Familiar decisions compounding: hash-only storage (2.1), UUIDv7 keys, lifetimes from config (1.4 — `Auth:MagicLink:TokenLifespanMinutes`, `Auth:Otp:*`). Two new ones:

- **`IsValid` is computed, never stored.** There is no `IsValid` column to forget to update — validity *derives* from `ConsumedAt` + `ExpiresAt` at the moment of asking (Golden Rule: derived values are computed, not stored as stale flags). And note the shape: `IsValidAt(now)` is the deterministic core the unit tests drive with an explicit clock; the ambient-time property is a convenience that delegates. Logic first, sugar second.
- **`ConsumedAt` is a timestamp, not a boolean.** Same storage cost, strictly more information — *when* it was redeemed is forensics the platform gets for free.

3. Issue and redeem — the magic-link half

```
// src/Api/Services/PasswordlessService.cs (real code)
public async Task<string> IssueMagicLinkTokenAsync(string email, CancellationToken ct =
default)
{
    email = Normalize(email); // trim + lower — one canonical form
    await repository.InvalidateActiveAsync(email, LoginTokenPurpose.MagicLink, ct);

    var raw = tokenGenerator.GenerateToken(); // 512-bit CSPRNG (2.1's seam)
    await repository.AddAsync(NewToken(email, LoginTokenPurpose.MagicLink, raw,
                                       settings.MagicLinkLifespanMinutes), ct);
    return raw; // → BrandedEmail → IEmailSender (2.3)
}

public async Task<User?> RedeemMagicLinkAsync(string email, string token, CancellationToken
ct = default)
{
    email = Normalize(email);
    var hash = tokenHasher.HashToken(token);
    var record = await repository.GetActiveByHashAsync(email, LoginTokenPurpose.MagicLink,
hash, ct);
    if (record is null) return null;

    record.ConsumedAt = clock.GetUtcNow(); // single-use: consumed atomically with success
    await repository.UpdateAsync(record, ct);

    return await userService.GetOrCreateByEmailAsync(email, ct); // ← account born HERE
}
```

Three quiet decisions:

- **Issue invalidates predecessors.** One active credential per email per purpose —

"request, request, request" leaves one live token, not a growing pile of valid ones.

- **The account is created at *redemption*, never issuance.** The service's doc comment

states why: "a typo'd or probed email leaves no trace." Anyone can type any address into a login box; if issuance created accounts, your Users table would fill with strangers' addresses and typos — and an attacker could *enumerate* which emails are registered by watching for "account already exists" behavior. Sign-in and sign-up collapse into one flow, and the collapse is a security feature.

- **Redemption looks up by hash** — the raw token exists only in the email and the

redeeming request, never in a query log or a table.

The controller endpoints glue this to 2.3's email and, on success, to 2.2's session issuance: redeem → `SessionService.IssueAsync` → access token + refresh cookie. The routes are worth a glance for a subtlety: `POST /api/auth/magic-link/send` issues, but verify is `GET /api/auth/magic-link/verify` — a **GET**, because it's reached by *clicking a link in an email*, not by a scripted request. (OTP, typed by hand, is symmetric POSTs: `otp/send` and `otp/verify`.) All sign-in paths in this platform converge on that same session-mint point — remember that convergence; MFA (6.5) will exploit it.

4. The OTP half — do the arithmetic first

A 6-digit code: 10^6 possibilities. Guessing has a 1-in-a-million shot per try. Sounds fine — until you multiply. Five attempts per code and free code-reissue gives an attacker who

requests a fresh code every lockout: 5 tries, resend, 5 more tries against a *new* code with a *reset* counter… The per-code counter never trips, and over hours a patient script accumulates thousands of attempts. Worse, each resend gives *this* code its own 1-in-200,000 ($5/10^6$) chance. The naive design is soft.

The platform's answer, from the real `RedeemOtpAsync` — read the comment first, it's the audit finding that shaped the code:

```
// Cumulative, resend-proof lockout: count failed attempts across EVERY code issued to
// this email in the window, not just the current one. Issuing a fresh code
// (AttemptCount=0) can't hand the attacker another budget, and the lock holds even
// against the correct code until the window elapses (CONF-5).
var windowStart = clock.GetUtcNow().AddMinutes(-settings.OtpLockoutWindowMinutes);
var failuresInWindow = await repository.CountFailedAttemptsSinceAsync(
    email, LoginTokenPurpose.Otp, windowStart, ct);
if (failuresInWindow >= settings.OtpMaxAttempts)
    return new OtpResult(OtpStatus.TooManyAttempts, null);

var record = await repository.GetLatestActiveAsync(email, LoginTokenPurpose.Otp, ct);
if (record is null)
    return new OtpResult(OtpStatus.Expired, null); // none active → expired or never issued

// Timing-safe comparison — the OTP code is low-entropy, so don't leak it via
// early-exit string equality.
if (tokenHasher.Verify(code, record.CodeHash)) { /* consume + get-or-create user */ }

record.AttemptCount++; // wrong code — count it
```

The design in three moves:

- **The budget belongs to the *email*, not the code.** Failures are counted across every

code in the window (`CountFailedAttemptsSinceAsync`), so "resend" no longer resets anything. Five failures in fifteen minutes = locked, *even for the correct code*, until the window drains. Redo the math: 5 attempts per 15 minutes = 480/day against a code that *itself expires in 10 minutes*. The attack is dead.

- **The lockout is state the attacker can't launder.** Note what the audit comment

implies about the bug it fixed: any per-code counter is attacker-resettable *by design* (they can always request a fresh code). When you design a lockout, ask: *what action resets my counter, and can the attacker perform it?*

- **Timing-safe verify, even for six digits.** Low-entropy secrets are exactly where a

timing oracle helps most — leaking "first digit right" collapses 10^6 to 10^5 fast. 2.1's `FixedTimeEquals` hasher was built for this moment.

And the numeric code generator — one line worth pausing on:

```
digits[i] = (char)('0' + RandomNumberGenerator.GetInt32(0, 10));
```

`GetInt32(0, 10)` per digit — CSPRNG *and* modulo-bias-free, versus the classic `randomBytes % 10` bug that skews digit distribution. Small crypto details are still crypto details.

5. What the caller learns — collapse the statuses

Internally there are four OTP outcomes. Externally, there are fewer — deliberately. From the real `OtpErrors`:

```
/// "No active code" (Expired) and "wrong code" (Invalid) collapse to the SAME
/// invalid_code so a caller can't probe whether an address has an outstanding OTP;
/// the full status stays server-side.
public static string ClientCode(OtpStatus status) => status switch
{
    OtpStatus.Invalid or OtpStatus.Expired => "invalid_code",
    OtpStatus.TooManyAttempts => "too_many_attempts",
    ...
};
```

Same principle as 2.2's indistinguishable 401: the error surface is *part of the attack surface*. "Does this address have a code outstanding?" is information (it reveals someone is mid-sign-in). Model statuses precisely inside; collapse them at the boundary (1.5's `error` codes); keep the distinction in logs. There's a dedicated test — `OtpErrorMappingTests` — because a *mapping* is exactly the kind of code a refactor breaks silently.

One more boundary defense wired in the controller: issuance is **rate-limited per IP** (`Auth:RateLimit:PasswordlessPermitLimit`, default 5/min) — because "email anyone a code, free, forever" is otherwise a spam cannon pointed at your SMTP reputation.

6. Red — the tests that drove it

The Gherkin scenarios (per `WAYS_OF_WORKING.md`, written before the service):

```
Scenario: magic link signs in and is single-use
  Given "ana@example.test" requested a magic link
  When she redeems it
  Then she is signed in and an account exists for her email
  And redeeming the same link again fails

Scenario: OTP lockout survives a resend
  Given an attacker has failed 5 OTP attempts for "ana@example.test"
  When a fresh code is issued and the attacker tries again — even the CORRECT code
  Then verification returns too_many_attempts until the window elapses

Scenario: expiry is enforced
  Given a magic link issued 16 minutes ago (lifespan 15)
  When it is redeemed
  Then sign-in fails
```

`PasswordlessServiceTests` drives all of these with `FakeTimeProvider` (advance past expiry; drain the lockout window) and the fake email sender from 2.3 capturing raw codes. The resend-proof scenario is the one that *distinguishes this design* — if you write one test today, write that one. In 3.6, the E2E suite re-proves the happy path end-to-end by fishing the real OTP out of Mailpit with a browser driving.

7. Run it

```
dotnet test tests/Api.Tests --filter Passwordless
dotnet run --project src/Api
# .http scratchpad: POST /api/auth/otp/send { "email": "you@test.local" }
# → open http://localhost:8025, read the branded code from Mailpit
# → POST /api/auth/otp/verify { "email": "...", "code": "123456" }
# → 200: access token + refresh cookie. You just signed in with no password in the system.
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: primary sign-in mechanism? (a) Passwords (+email reset); (b) OAuth only ("Sign in with Google, period"); (c) passwordless email credentials + OAuth as an accelerator (next lesson).

Chosen: (c) (ADR-C15). Email possession is the root of trust either way — password reset already reduces to it — so passwordless removes an entire liability class (storage, stuffing, reset flows) rather than defending it. Magic link and OTP share one entity and one service because they are one concept at two entropy levels; the low-entropy one is wrapped in cumulative lockout, rate limiting, timing-safe compare, and enumeration-resistant errors.

Rejected: (a) — maximum liability for zero differentiation; this platform would spend its auth budget defending a mechanism users increasingly don't want. (b) — excludes anyone unwilling to link a big-tech identity and makes *your* account system hostage to *their* account system; fine for internal tools, wrong default for a SaaS template.

The trade: sign-in now depends on email deliverability (SMTP down = login down — which is why 2.3 bounded that timeout, and why 4.2 will make sends crash-safe), and inbox compromise = account compromise. That last is true of password systems too — passwordless is just honest about it. MFA (6.5) exists for the users who need more.

9. Checkpoint

```
git add -A && git commit -m "feat: passwordless sign-in — magic link + OTP, cumulative
resend-proof lockout"
git tag lesson-2.4
```

You should now be able to: run the brute-force arithmetic for any short credential and size a lockout to beat it; explain why the lockout budget must attach to the email, not the code; justify account-creation-at-redemption as enumeration defense; and articulate which OTP statuses collapse at the boundary and why the *mapping itself* deserves a test.

Next: *Lesson 2.5 — OAuth & account linking* — "sign in with Google" in one line per provider, and the takeover-shaped edge case hiding inside account linking.

Lesson 2.5 — OAuth & account linking

> **Part 2 · Identity & tenancy.** "Sign in with Google" looks like a convenience feature. It's actually a *federation decision* — you're accepting identity assertions from a third party — and the dangerous part isn't the OAuth dance (the framework does that); it's the moment two identities claim the same email address. This lesson builds the one-line-per-provider registration the platform advertises, and then spends its real energy where the real risk is: the **account-linking takeover guard** and the fail-closed claim normalization behind it.

Goal: Google and Microsoft sign-in work end-to-end; adding a future provider is one constant + one `.AddXxx()` block; a same-email sign-in from a *new* provider links to the existing account — but **only** when the email is provably verified.

Concepts & principles: OAuth as identity federation (what you delegate, what you keep); provider identity vs email identity; the unverified-email takeover; fail-closed normalization (R17); trust decisions as explicit, testable objects.

Maps to: ADR-002 ("new provider = one line", takeover guard) · R17 · repo: `src/Api/Services/AuthProviders.cs`, `ClaimsExtractor.cs`, `ProviderEmailTrust.cs`, `UserService.cs` (`GetOrCreateUserAsync`), `src/Core/Entities/UserLogin.cs`, `LinkTokenService.cs`, `src/Infrastructure/ServiceCollectionExtensions.cs` (provider registration), tests: `ClaimsExtractorTests`, `ProviderEmailTrustTests`, `AuthProvidersTests`.

Prerequisites: 2.1–2.4 (all sign-in paths converge on the same session mint).

---

1. Motivate — what OAuth actually outsources

When a user clicks "Sign in with Google," you delegate *authentication* — Google proves who they are, with all its infrastructure (password vaults, device signals, its own MFA). What you do **not** outsource is *identity*: the decision "which account in *my* system is this person" remains entirely yours. That's the part frameworks can't do for you, and it's the part with the security cliff.

Here's the cliff, concretely. Ana signed up months ago via magic link as `ana@example.test`. Today someone signs in via OAuth-Provider-X, and X asserts their email is `ana@example.test`. If you match by email and link this new credential to Ana's account — congratulations, you may have just handed Ana's account to an attacker, because **some providers let users type any email into their profile, unverified**. The attacker didn't hack Ana; they hacked *your merge rule*.

Everything in this lesson exists to let the convenient thing (same email = same account, no duplicate accounts) happen *only* when it's safe.

2. The dance itself — buy, don't build

The OAuth redirect flow (authorize → callback → code exchange → claims) is standardized and hostile to hand-rolling; ASP.NET Core's authentication handlers do it properly. The platform's entire per-provider cost, from the real `ServiceCollectionExtensions`:

```
if (!string.IsNullOrEmpty(configuration["Authentication:Google:ClientId"]))
    auth.AddGoogle(google =>
    {
        google.ClientId = configuration["Authentication:Google:ClientId"]!;
        google.ClientSecret = configuration["Authentication:Google:ClientSecret"]!;
        // callback path, scopes...
    });
```

Note the guard clause: **a provider registers only when its ClientId is configured.** That's R21's config-gating instinct applied to auth — an undeployed provider isn't a broken button, it's an absent scheme. `AuthProviders.EnabledAsync` closes the loop by asking the runtime which schemes actually registered, so the login UI renders buttons for exactly the providers this deployment configured — no dead, 500-ing "Sign in with Microsoft" on an instance that never set it up. And `AuthProviders` itself:

```
public static class AuthProviders
{
    public const string Google = "google";
    public const string Microsoft = "microsoft";
    public static readonly IReadOnlyList<string> Supported = [Google, Microsoft];

    /// <summary>The authentication scheme to challenge for a provider, or null if
unsupported.</summary>
    public static string? SchemeFor(string provider) => provider.ToLowerInvariant() switch
    {
        Google => GoogleDefaults.AuthenticationScheme,
        Microsoft => MicrosoftAccountDefaults.AuthenticationScheme,
        _ => null,
    };
}
```

One place for provider names instead of string literals scattered through controllers — "new OAuth provider = one line" is really "one constant, one switch arm, one `.AddXxx()`," and nothing else in the platform changes. That's the promise; keeping it depends on the next two sections *not* being provider-specific.

The callback flow: the provider's handler materializes the external identity as a short-lived **cookie principal on a dedicated "External" scheme** — a carrier, not a session. The callback endpoint reads it, runs the identity resolution below, mints the platform's own session (2.2), and discards the carrier. External assertion in; *our* tokens out; the two never blur.

3. Normalize the claims — fail closed (R17)

Providers emit claims differently: `email_verified` as `"true"`, as `true`, or — the dangerous case — not at all. Normalization is one tiny class, and its posture is the whole point. From the real `ClaimsExtractor`:

```
public bool IsEmailVerified(ClaimsPrincipal principal)
{
    // Fail closed: verified ONLY when the provider explicitly asserts
    // email_verified="true". Absent / empty / any other value   not verified. An
    // absent claim must not be trusted — not every provider asserts it, and the
    // platform advertises "new OAuth provider = one line", so a silent
    // absent- -verified default would hand any such provider the account-takeover
    // merge path.
    var claim = principal.FindFirst("email_verified")?.Value;
    return string.Equals(claim, "true", StringComparison.OrdinalIgnoreCase);
}
```

Read the comment's chain of reasoning — it connects a *marketing promise* to a *security posture*. Because adding a provider is deliberately trivial, the system must assume some future provider will be added carelessly. If absence-of-claim defaulted to "verified," that careless one-liner would silently open the takeover path. Fail-closed normalization means the careless default is the *safe* default. This is rule R17, and it has its own unit tests (`ClaimsExtractorTests`): absent claim → false, `"True"` → true, `"1"` → false, garbage → false. Boring tests; load-bearing boundary.

4. The identity resolution — three cases, one guard

The heart of the lesson, from the real `UserService.GetOrCreateUserAsync`. A provider identity arrives as `(provider, providerUserId, email, emailVerified)` — note that **`providerUserId` is the real identity**; email is metadata that can change or lie:

```
// 1. Known provider identity → existing account
var existingUser = await repository.GetByLoginAsync(provider, providerUserId, ct);
if (existingUser != null)
    return existingUser; // the (provider, id) pair is ground truth

// 2. Same email from a new provider → same account; link the identity.
// Guard: an UNVERIFIED email claim must never attach a new credential to an
// existing account (takeover vector) — refuse outright.
var userByEmail = await repository.GetByEmailAsync(email, ct);
if (userByEmail != null && !emailVerified)
{
    logger.LogWarning("Refused unverified-email merge for {Email} via {Provider}", email,
provider);
    throw new UnverifiedEmailConflictException(email);
}
if (userByEmail != null)
{
    await repository.AddLoginAsync(new UserLogin { UserId = userByEmail.Id,
        Provider = provider, ProviderUserId = providerUserId }, ct);
    return userByEmail; // safe merge: verified email, linked
}

// 3. Brand new user — create the tenant ("household of one"), the user, and an
// owner membership linking them, atomically.
```

The three cases in threat-model terms:

- **Case 1 — the returning user.** `(provider, providerUserId)` matched a `UserLogin`

row: this exact Google account signed in before. No email logic runs at all — even if the user changed their email at Google, they get *their* account. The `UserLogin` entity (one account,

many provider identities) is what makes "Ana signs in with Google on Monday and Microsoft on Tuesday" one person instead of two accounts.

- **Case 2 — the merge, guarded.** New provider identity, known email. Convenience says

link; the guard says *only if the provider vouches for the email*. Unverified → refuse loudly (a typed exception the callback turns into a safe error page — the legitimate owner's account is untouched, and the refusal is logged as the security-relevant event it is).

- **Case 3 — the newcomer.** No matches: create user + their tenant + owner membership

in one transaction. (The "household of one" is a Part 2.8 concept arriving early — every user has a tenant from birth, which is what lets the `tenant_id` claim of 2.6 be non-optional in practice.)

One nuance sits on top: `ProviderEmailTrust`. Microsoft's v2 endpoint *omits* `email_verified` even for accounts whose email Microsoft itself verified (personal accounts). Strictly fail-closed would force those users through case-2 refusal forever. The platform's answer is a tiny, explicit trust object:

```
public bool TrustsEmailWithoutClaim(string provider) => provider.ToLowerInvariant() switch
{
    AuthProviders.Google => true, // Google always asserts the claim anyway
    AuthProviders.Microsoft => IsMicrosoftConsumersTenant(), // personal accounts only
    _ => false, // unknown/future providers stay fail-closed
};
```

Study the *shape* more than the rules: the exception to fail-closed is not an `if` buried in the merge — it's a named, injectable, unit-tested policy object (`ProviderEmailTrustTests`) whose default arm is still `false`, and whose Microsoft arm narrows to exactly the sub-case with a documented justification (consumers-tenant emails are Microsoft-verified) that *reverts automatically* if the deployment flips the configured tenant to work/school accounts. When you must carve an exception into a security rule, this is what the carving should look like: explicit, narrow, self-reverting, tested.

5. Explicit linking — the same guard from the other door

Settings will eventually offer "Connect your Google account" for an already-signed-in user (`LinkTokenService` carries the intent through the OAuth round-trip as — of course — a hashed, single-use, time-limited token; the 2.4 pattern reused). Linking *while authenticated* proves control of the platform account, so the email-match question doesn't arise — but the *other* direction still bites: the OAuth identity being attached must not already belong to another user. Same lesson, different door: every path that can attach a credential to an account needs the takeover question asked. The reference repo keeps both paths' checks in `UserService` so there's one answer.

6. Red — the tests

OAuth's redirect dance is miserable to test through a real provider — so the platform tests the *decisions* as units and the *flow* with a stand-in. The integration harness (1.2/2.6) swaps the External scheme's handler for one that authenticates from test headers — the callback then runs *its real code* against controlled identities:

```
Scenario: same verified email links, doesn't duplicate
  Given an account exists for "ana@example.test" (created via magic link)
  When a Google identity asserting email_verified=true for that email signs in
  Then no new account is created
  And the Google identity is linked to Ana's account

Scenario: unverified email must not merge (the takeover test)
  Given an account exists for "ana@example.test"
  When a provider identity with that email but NO verified claim signs in
  Then sign-in is refused with a safe error
  And Ana's account has no new login attached

Scenario: returning provider identity ignores email drift
  Given a user who previously signed in with Google id "g-123"
  When "g-123" signs in again asserting a different email
  Then they get their original account
```

The middle one is this lesson's reason to exist. If your suite has one OAuth test, make it that one.

7. Run it

Real-provider setup is config, not code: create OAuth apps in Google/Microsoft consoles, put ClientId/Secret in `.env` (the `.env.example` block from 1.4 documents the keys), run, click the button, land signed in. Then verify the *linking* behavior: sign in via OTP as `you@...`, sign out, sign in via Google with the same address — one account, two rows in `UserLogins`.

```
dotnet test tests/Api.Tests --filter "ClaimsExtractor|ProviderEmailTrust|AuthProviders"
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how do provider identities map to accounts? (a) Separate account per provider ("you signed up with Google" forever); (b) auto-merge on email match, always; (c) merge on email match **only when verified** (explicit claim or a narrow, tested trust policy), with `(provider, providerUserId)` as the primary identity.

Chosen: (c) (ADR-002 + R17). Duplicate accounts are a genuine product failure (split data, "where are my things"), so merging matters; the guard makes it safe, and fail-closed normalization keeps it safe against the *next* provider someone adds in one careless line. `ProviderEmailTrust` handles the real-world wrinkle (Microsoft consumers) as an explicit policy object rather than a buried special case.

Rejected: (a) — punts the hard problem onto users, who end up with two half accounts and a support ticket. (b) — is the account-takeover vector, full stop; it's what several real-world breaches were made of.

The trade: legitimate users on providers that don't assert verification (outside the trust list) can't auto-merge — they land in a refusal instead of a silent link. Correct trade: the failure mode is a support case, not a takeover. The error page tells them to sign in with their original method and link explicitly from Settings.

9. Checkpoint

```
git add -A && git commit -m "feat: OAuth sign-in — one-line providers, fail-closed claims, takeover-guarded linking"
git tag lesson-2.5
```

You should now be able to: state precisely what OAuth delegates and what it can never delegate; explain why `(provider, providerUserId)` is identity and email is testimony; reproduce the takeover scenario from memory and name the two lines that prevent it; and justify when a security exception deserves to be a policy object instead of an `if`.

Next: *Lesson 2.6 — Tenancy I: the global query filter.* The `tenant_id` claim has been riding along in every token since 2.1. Time for it to earn its keep — the keystone lesson of the entire course.

Lesson 2.6 — Tenancy I: the global query filter

> **Part 2 · Identity & tenancy.** The keystone of the whole platform. Everything you build after this — billing, files, webhooks, every feature slice — leans on the guarantee installed here: *a query cannot accidentally read another tenant's data.* We earn that guarantee the hard way: build the leak first, prove it with a test, then close it structurally. The `tenant_id` claim that has ridden every JWT since 2.1 finally does its job today.

Goal: every entity marked `ITenantScoped` is filtered to the current tenant automatically, so a handler that "forgets" to scope its query still cannot read another tenant's rows — and an unauthenticated caller sees *nothing*, not *everything*.

Concepts & principles: structural vs conventional enforcement; EF Core global query filters; marker interfaces driving platform behavior; fail-closed defaults; invariant tests (pinning a platform guarantee, not a method's output).

Maps to: ADR-003 · Golden Rule 1 · R2 · repo: `src/Core/Entities/ITenantScoped.cs`, `src/Core/Abstractions/ICurrentTenant.cs`, `src/Api/Services/HttpCurrentTenant.cs`, `src/Infrastructure/Persistence/AppDbContext.cs`, tests: `TenantScopeFilterTests.cs`, `TenantInvariantTests.cs`, `HttpCurrentTenantTests.cs`, `HarnessSmokeTests.cs` (the wire-level proof).

Prerequisites: 2.1 (the `tenant_id` claim), 1.3 (the Postgres fixture — this lesson is *why* tests run against a real relational engine).

---

1. Motivate — build the leak, then look at it

Every SaaS bug that makes the news is some version of "*customer A saw customer B's data.*" The naive defense is discipline: "always add `.Where(x => x.TenantId == me)` to every query." That fails the first time a tired developer forgets — and you cannot grep your way back to confidence across a growing codebase, because absence of a `.Where` is invisible in review.

Let's feel it. Suppose a `Widget` entity and the obvious handler:

```
// first attempt — looks multi-tenant, is not
public class Widget
{
    public Guid Id { get; set; }
    public Guid TenantId { get; set; } // we store it…
    public string Name { get; set; } = "";
}

public Task<List<Widget>> ListAsync() =>
    _db.Set<Widget>().ToListAsync(); // …but nothing filters on it
```

The `TenantId` column exists, so the schema *looks* multi-tenant. Nothing enforces it. A stored column is data; isolation is a *behavior*, and no behavior has been built.

2. Red — a test that proves the leak

Seed two tenants, a widget in each, act as tenant A, and assert you see only A's rows. Against the naive handler this fails — and that failure is the whole point:

```
// tests/Api.Tests/TenantScopeFilterTests.cs (shape of the real test)
[Collection(PostgresCollection.Name)]
public class TenantScopeFilterTests(PostgresFixture fixture) : PostgresTestBase(fixture)
{
    [Fact]
    public async Task Listing_never_returns_another_tenants_rows()
    {
        var (tenantA, tenantB) = (Guid.CreateVersion7(), Guid.CreateVersion7());
        await SeedWidgetAsync(tenantA, "A's widget");
        await SeedWidgetAsync(tenantB, "B's widget");

        await using var db = Fixture.CreateContext(tenantId: tenantA); // act as A
        var visible = await db.Set<Widget>().ToListAsync(); // "forgetful" query

        Assert.Single(visible);
        Assert.Equal("A's widget", visible[0].Name); //   naive: returns BOTH
    }
}
```

Note what this test is *not*: it isn't testing a handler's logic. It's pinning an **invariant of the platform** — "a tenant-scoped read is isolated *no matter how carelessly it's written*." That's why the fix must live in the platform, not in the handler. (And note it runs on the real Postgres fixture: the EF in-memory provider's handling of global query filters diverges — a green isolation test on a fake database proves nothing. Lesson 1.3's decision was made *for this moment*.)

3. Green — make isolation structural

Three pieces. First, the marker — and the real file's doc comment deserves reading in full, because it teaches the *boundary* of the rule, not just the rule:

```
// src/Core/Entities/ITenantScoped.cs — the real file
/// <summary>
/// Marks an entity as belonging to a single tenant. Every entity implementing this is
/// automatically filtered to the current tenant by a global EF query filter (see
/// AppDbContext.OnModelCreating), so feature/domain queries are tenant-scoped by
/// default — you cannot forget to scope a read.
/// <para>
/// Platform entities that are intentionally cross-tenant or used before a tenant is
/// resolved (User, TenantMembership, auth tokens) deliberately do NOT implement this.
/// The rare cross-tenant/pre-auth lookup opts out with IgnoreQueryFilters().
/// </para>
/// </summary>
public interface ITenantScoped
{
    Guid TenantId { get; }
}
```

Second, *who is the current tenant?* A request-scoped abstraction the DbContext can depend on — and read its null contract carefully, it's the fail-closed hinge:

```
// src/Core/Abstractions/ICurrentTenant.cs — the real file
/// <summary>
/// The tenant the current request acts within, resolved from the authenticated
/// principal (the JWT tenant_id claim). This is the single, request-scoped tenancy
/// entry point: it drives the global tenant query filter in the DbContext, and
/// feature slices inject it to read their tenant id.
/// <para>TenantId is null when the caller is unauthenticated or has no tenant; the
/// global filter then matches no tenant-scoped rows (fail closed).</para>
/// </summary>
public interface ICurrentTenant
{
    Guid? TenantId { get; }
}
```

The HTTP implementation (`HttpCurrentTenant`) reads the `tenant_id` claim off the validated JWT — the claim you've been issuing since 2.1. Tests swap in the trivial settable fake from 1.3's fixture.

Third — the move that closes the leak for every entity at once, from the real `AppDbContext`:

```
// src/Infrastructure/Persistence/AppDbContext.cs (the real code)
/// <summary>Tenant the global query filter scopes to. Guid.Empty when there is no
/// current tenant — it matches no real (UUIDv7) row, so unauthenticated/tenant-less
/// callers see no tenant-scoped data (fail closed).</summary>
public Guid CurrentTenantId => _currentTenant.TenantId ?? Guid.Empty;

protected override void OnModelCreating(ModelBuilder builder)
{
    base.OnModelCreating(builder);
    builder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);

    // Tenant isolation as a structural guarantee: every ITenantScoped entity is
    // filtered to CurrentTenantId by default, so feature/domain queries can't forget
    // to scope. Discovered from the model — not hand-listed.
    foreach (var entityType in builder.Model.GetEntityTypes())
        if (typeof(ITenantScoped).IsAssignableFrom(entityType.ClrType))
            ApplyTenantFilterMethod.MakeGenericMethod(entityType.ClrType).Invoke(this,
[builder]);
}

private void ApplyTenantFilter<TEntity>(ModelBuilder builder) where TEntity : class,
ITenantScoped
    => builder.Entity<TEntity>().HasQueryFilter(e => e.TenantId == CurrentTenantId);
```

Mark `Widget : ITenantScoped`, re-run the leak test: **green**. The unchanged, still-careless `ToListAsync()` now emits `WHERE tenant_id = @currentTenant` — the handler never mentioned the tenant; the platform did. One subtlety in that lambda repays study: the filter references `CurrentTenantId` — a *property of the context* — so EF captures it as a parameter evaluated **per query**, not a constant baked at startup. One model, every request correctly scoped.

This is the discovery pattern's fourth appearance (configurations 1.3, fixture reset 1.3, config gate 1.4, now this) and its highest-stakes one: a new entity is protected the moment it implements the marker. No registration step exists to forget.

4. Harden — fail closed, then pin both properties

Look hard at one line: `_currentTenant.TenantId ?? Guid.Empty`. Why not throw? Why not skip filtering? Because tenant ids are UUIDv7 — **`Guid.Empty` matches no real row**. The failure mode of "no tenant" is therefore *see nothing*, never *see everything*. A misconfigured caller gets an empty screen, not a data breach. Pin it:

```
[Fact]
public async Task With_no_current_tenant_scoped_reads_return_nothing()
{
    await SeedWidgetAsync(Guid.CreateVersion7(), "someone's widget");
    await using var db = Fixture.CreateContext(tenantId: null); // nobody
    Assert.Empty(await db.Set<Widget>().ToListAsync()); // fail closed
}
```

Then pin the *coverage* of the rule itself. The filter protects `ITenantScoped` entities — but who checks that entities which *should* be marked, are? Rule R2's invariant test (`TenantInvariantTests` in the reference repo) walks the EF model and asserts every entity is either global-filtered or on an explicit, commented allowlist of deliberately cross-tenant types (`User`, `TenantMembership`, auth tokens, outbox plumbing). Forgetting the marker on a new entity fails the build with the entity's name — the allowlist makes the *exceptions* as reviewable as the rule.

Finally, the wire-level proof. Unit-level isolation could still be undone by a wrong registration or a middleware gap, so the integration harness (1.2) asserts the whole chain — from the real `HarnessSmokeTests`:

```
// Two owners, two tenants. Each calls the identical route with their own token; the
// JWT tenant_id claim drives the global filter, so neither can see the other's data.
var bodyA = await
_factory.CreateClientFor(a).GetFromJsonAsync<HouseholdDto>("/api/household");
var bodyB = await
_factory.CreateClientFor(b).GetFromJsonAsync<HouseholdDto>("/api/household");
Assert.DoesNotContain(bodyA.Members, m => m.UserId == b.UserId);
Assert.DoesNotContain(bodyB.Members, m => m.UserId == a.UserId);
```

Token → claim → `HttpCurrentTenant` → filter → SQL: proven at the wire, per commit.

5. Run it

```
dotnet test tests/Api.Tests --filter "TenantScope|TenantInvariant|HttpCurrentTenant"
# isolation, fail-closed, coverage invariant — the three-legged stool
```

Optional but worth it once: log the SQL (`LogTo(Console.WriteLine)`) and *see* the injected `WHERE` clause on a query whose C# has no `Where` at all.

6. Architecture Decision

ARCHITECTURE DECISION

The fork: how is tenant isolation enforced on reads? (a) Discipline — every query adds `.Where(TenantId == …)`, reviewers catch misses; (b) repository-layer scoping — data access goes through repositories that add the predicate; (c) a global EF query filter keyed on a marker interface, discovered from the model, fail-closed on no tenant.

Chosen: (c) (ADR-003). Isolation enforced by the type system + one central filter can't be forgotten per-query; discovery means new entities are protected by construction; `?? Guid.Empty` turns misconfiguration into an empty result instead of a breach. (b) arrives *as well* in 3.1 — the repository seam adds ergonomics and a sanctioned escape hatch — but the guarantee must not *depend* on everyone using the repository; defense in depth starts by not trusting your own conventions.

Rejected: (a) — it's not a mechanism, it's a hope; the news is full of its failures. Also considered: database-enforced isolation (row-level security) — *not* rejected, deferred: it arrives in 8.4 as the second wall beneath this one, once the platform is worth two walls. And schema-per-tenant / database-per-tenant: real isolation, but operationally heavy (migrations × tenants) and wrong for this platform's "thousands of small tenants" shape.

The trade: query filters are an EF feature — raw SQL escapes them (a rule for later: raw SQL is platform code, never slice code), and the filter must be *provably attached* to every entity that needs it, which is exactly what the R2 invariant test exists to hold. Write-side isolation is a separate problem — deliberately: it's the next lesson.

7. Checkpoint

```
git add -A && git commit -m "feat: tenancy reads — ITenantScoped + global query filter, fail closed (ADR-003)"
git tag lesson-2.6
```

You should now be able to: explain why a stored `TenantId` column is not multi-tenancy; add a global query filter keyed on a marker interface and say why it's discovered, not listed; defend `?? Guid.Empty` as a security decision; distinguish an invariant test from a behavior test; and name which entities deliberately live *outside* the filter and why.

Next: *Lesson 2.7 — Tenancy II: write-side.* The filter scopes what you can *read*. Nothing yet stops a buggy handler from *writing* a row stamped with someone else's tenant — or from updating cross-tenant with `ExecuteUpdate`. Isolation needs both doors.

Lesson 2.7 — Tenancy II: write-side stamping & `EnterTenant`

> **Part 2 · Identity & tenancy.** Lesson 2.6 locked the read door; this lesson locks the write door — because a filter on *reads* does nothing to stop a buggy handler from *inserting* a row stamped with someone else's tenant, or updating a row it loaded through the escape hatch. Isolation is only structural when **both directions** are. And once writes are guarded, a new question appears: how do legitimate tenant-less callers — a Stripe webhook, a future admin console — write into a tenant *without* bypassing all the guarantees? The answer is the platform's most elegant small primitive: `EnterTenant`.

Goal: a tenant-scoped entity written under the wrong tenant never reaches the database; new rows are stamped automatically; and system-authenticated operations enter a tenant scope that gives them the *same* isolation an authenticated request gets.

Concepts & principles: write-side stamping; EF `SaveChangesInterceptor`; the system-context trust level; scoping vs authorizing; the sanctioned escape hatch and its CI ban (R5); ambient context with disposable restore.

Maps to: ADR-003 (amendment) · R5, R28 · repo: `src/Infrastructure/Persistence/TenantStampingInterceptor.cs`, `src/Core/Abstractions/ITenantContext.cs`, `HttpCurrentTenant` (EnterTenant impl), tests: `TenantStampingInterceptorTests.cs`, `EnterTenantScopingTests.cs`, `ArchitectureTests.cs` (the `IgnoreQueryFilters` ban).

Prerequisites: 2.6 (the filter, `ICurrentTenant`, fail-closed `Guid.Empty`).

---

1. Motivate — the door you forgot you left open

The 2.6 test suite proves tenant A cannot *read* tenant B's rows. Write this handler, though, and watch the guarantee evaporate:

```
public async Task TransferAsync(Guid widgetId, Guid toTenantId) // hypothetical bug
{
    var widget = await _db.Set<Widget>().SingleAsync(w => w.Id == widgetId);
    widget.TenantId = toTenantId; // ← quietly re-homes A's data into B
    await _db.SaveChangesAsync(); // ← nothing objects
}
```

Or subtler: a handler builds a `new Widget { TenantId = someIdFromTheRequest }` — trusting client input for the one field that must never come from the client. Or subtler still (this one is the v2 audit's ADV-1 finding): platform code legitimately loads a row *cross-tenant* through the escape hatch, then a later code path innocently `SaveChanges`es it — an update landing under the wrong tenant with no one ever having "written" a tenant id at all.

Reads have one shape (a query) so one filter covers them. Writes have three shapes (insert, update, delete) and arrive through the change tracker — so the write-side guard lives where the change tracker converges: **`SaveChanges`**.

2. Green — the interceptor

EF Core's `SaveChangesInterceptor` sees every pending change just before it becomes SQL. The real `TenantStampingInterceptor`, core logic in full:

```
// src/Infrastructure/Persistence/TenantStampingInterceptor.cs
private static void Stamp(AppDbContext? db)
{
    if (db is null) return;

    var currentTenantId = db.CurrentTenantId;
    if (currentTenantId == Guid.Empty) return; // system/seed/cross-tenant context — not
enforced

    foreach (var entry in db.ChangeTracker.Entries<ITenantScoped>())
    {
        if (entry.State is not (EntityState.Added or EntityState.Modified or
EntityState.Deleted))
            continue;

        var tenantProperty = entry.Property(nameof(ITenantScoped.TenantId));
        var tenantId = (Guid)tenantProperty.CurrentValue!;

        // A new row may leave TenantId unset → stamp the current tenant.
        if (entry.State == EntityState.Added && tenantId == Guid.Empty)
        {
            tenantProperty.CurrentValue = currentTenantId;
            continue;
        }

        // An explicitly-tenanted insert, or any update/delete, must belong to the current
tenant.
        if (tenantId != currentTenantId)
            throw new InvalidOperationException(
                $"Refusing to {entry.State.ToString().ToLowerInvariant()} a
{entry.Entity.GetType().Name} " +
                $"for tenant {tenantId} while the current tenant is {currentTenantId}.
...");
    }
}
```

Two behaviors, both worth naming:

- **Stamping is a gift.** A handler creating an entity simply doesn't set `TenantId` —

the platform stamps it. The *correct* code is the code with *less* in it: feature authors literally cannot get this field wrong because they never touch it. (Notice the interplay with 2.6's fail-closed choice: `Guid.Empty` means "unset," which is exactly why real tenant ids being UUIDv7 matters twice.)

- **Validation is a wall.** Any insert/update/delete whose `TenantId` differs from the

current tenant throws — *before* SQL is generated. The exception message is written for the developer who hits it: what was refused, both tenant ids, and what to do instead. Wrong-tenant writes are now a *test failure during development*, not a data corruption in production.

And one registration subtlety from the real `AppDbContext` — the interceptor is attached in `OnConfiguring`, *inside the context*, not (only) in DI:

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    base.OnConfiguring(optionsBuilder);
    optionsBuilder.AddInterceptors(TenantStampingInterceptor.Instance, ...);
}
```

Why there? Because tests (and tools) construct contexts directly, bypassing DI — and a guard that only exists on the DI path is a guard your test double forgot. Attaching in `OnConfiguring` means **every** `AppDbContext`, however constructed, enforces the rule. The interceptor is deliberately stateless (it reads the tenant from the context being saved), so one shared `Instance` serves them all.

3. The trust boundary — who is `Guid.Empty`, really?

Look again at the early return: *no current tenant → no enforcement*. Isn't that a hole? It's a **trust level**, and drawing it consciously is this lesson's second idea.

A context with no current tenant is the *system* context: migrations, seeders, the outbox dispatcher (Part 4), tenant-dissolution teardown (2.9). System code is exactly the code that legitimately touches many tenants — and it's held to a different standard: it must be *platform* code, reviewed as such, and the read-side twin of this trust level (the `IgnoreQueryFilters()` / `QueryAllTenants()` escape hatch) is **banned from feature slices by an architecture test** (R5 — the scan you'll formalize in 3.3 fails the build if the string appears under `src/Api/Features/`). Feature code gets the guarded world with no exit; platform code gets the trusted world with review. Two tiers, machine-separated.

But there's a third kind of caller, and it's the interesting one: code that has **no JWT but a trusted tenant id**. A Stripe webhook arrives bearing a cryptographic signature (5.2) and *names* a tenant; the future admin console (7.5) impersonates into one. The lazy answer is "run them as system and be careful." The platform's answer is better — from the real `ITenantContext`:

```
// src/Core/Abstractions/ITenantContext.cs
/// Enters a tenant context for system/integration operations that have no JWT but a
/// TRUSTED tenant id — e.g. a Stripe-signature-verified webhook. While the returned
/// scope is active, ICurrentTenant.TenantId resolves to the entered tenant, so the
/// write-stamping interceptor and the global read filter scope to it. Such operations
/// therefore get the SAME structural tenant isolation an authenticated request gets,
/// instead of bypassing it with the cross-tenant escape hatch.
///
/// This primitive only SCOPES; it does not AUTHORIZE. The caller MUST have already
/// authenticated the tenant id by other means (a verified webhook signature, an admin
/// authz check).
public interface ITenantContext
{
    IDisposable EnterTenant(Guid tenantId);
}
```

Usage reads like what it is — a scoped assumption:

```
// inside the (signature-verified) billing webhook handler, Part 5:
using (tenantContext.EnterTenant(tenantIdFromVerifiedEvent))
{
    // in here, reads are filtered to this tenant and writes are stamped/validated
    // exactly as if a user of this tenant had made an authenticated request
    subscription.Status = newStatus;
    await db.SaveChangesAsync();
} // dispose restores the previous tenant (nesting supported)
```

The doc comment's sharpest sentence deserves a highlight: **"only scopes; does not authorize."** `EnterTenant` never decides *whether* you may act as tenant X — the webhook's signature check or the admin's authz check did that. It decides that, having been authorized, you operate under the *full* isolation regime rather than the trusted bypass. The alternative — webhook code running system-level with hand-rolled `WHERE` clauses — is precisely the conventional-discipline world 2.6 was built to abolish. This primitive exists so the platform never has to choose between "no JWT" and "no guardrails."

Implementation-wise, `HttpCurrentTenant` doubles as the `ITenantContext`: `EnterTenant` swaps the ambient tenant and returns an `IDisposable` that restores the previous value (nesting supported) — the same enter/restore shape you saw in the test double back in 1.3, which is no accident: the fixture's `TestCurrentTenant` implements both interfaces so services under test see one coherent ambient tenant.

4. Red — the tests that pin all three behaviors

From the real `TenantStampingInterceptorTests` / `EnterTenantScopingTests`, as Gherkin:

```
Scenario: new rows are stamped
  Given the current tenant is A
  When a Widget with unset TenantId is added and saved
  Then the persisted row's TenantId is A

Scenario: wrong-tenant writes are refused — all three states
  Given the current tenant is A and a row belonging to B (loaded via the escape hatch)
  When the row is modified / deleted / a new row is added with TenantId = B
  Then SaveChanges throws before any SQL runs
  And the message names both tenants

Scenario: EnterTenant gives a webhook the same isolation as a request
  Given no current tenant (system context)
  When the handler enters tenant A, writes, and disposes the scope
  Then the write is stamped/validated as A
  And after dispose the context is system again (and nested scopes restore correctly)
```

The middle scenario is ADV-1's regression test — the audit finding that widened the interceptor from "validate inserts" to "validate *every* state." Audits feed tests; tests hold the line.

One honest boundary the interceptor's own comment draws: **bulk operations (`ExecuteUpdate`/`ExecuteDelete`) bypass the change tracker** — no `SaveChanges`, no interceptor. The global *read* filter still constrains what they touch (they compose onto the filtered query), but the write-side wall has this known gap at the edges, held by review and by 8.4's database-level backstop. Structural enforcement doesn't mean pretending the structure covers everything; it means *knowing exactly where it ends*.

5. Run it

```
dotnet test tests/Api.Tests --filter "TenantStamping|EnterTenant"
```

Then feel the wall once: in any handler, set a `TenantId` to a random Guid and `SaveChanges`. The exception you get — naming the entity, both tenants, and the rule — is the platform teaching your future teammates in the only classroom that scales.

6. Architecture Decision

ARCHITECTURE DECISION

The fork: how are *writes* kept tenant-correct? (a) Trust handlers to set `TenantId` (review catches mistakes); (b) require every write to pass through a repository method that takes the tenant explicitly; (c) a SaveChanges interceptor that stamps unset inserts and refuses cross-tenant writes, plus `EnterTenant` for trusted tenant-less callers.

Chosen: (c) (ADR-003 amendment). The change tracker is the one choke point every tracked write crosses, so enforcement there is complete-by-construction for tracked writes; stamping makes correct code *smaller* than incorrect code; and `EnterTenant` extends the same regime to signature-authenticated work instead of granting it the bypass. Registered in `OnConfiguring` so no construction path escapes.

Rejected: (a) — the same hope 2.6 rejected for reads, with worse blast radius. (b) — helps ergonomics (and 3.1 builds it!) but can't be the *guarantee*: any code holding the DbContext can sidestep a repository, and the platform refuses to found isolation on "everyone used the blessed API."

The trade: an ambient-context primitive (`EnterTenant`) is ambient state — the classic critique applies (invisible in signatures). Contained by its shape: scoped, disposable, nest-restoring, and used only at authenticated entry points, never as a general parameter-passing dodge. And the tracked-write wall knowingly excludes bulk ops — a gap documented in code, patrolled by review, and closed for good by the RLS backstop in 8.4.

7. Checkpoint

```
git add -A && git commit -m "feat: tenancy writes — stamping interceptor + EnterTenant
primitive (ADR-003)"
git tag lesson-2.7
```

You should now be able to: name the three write shapes and where they converge; explain why stamping-plus-refusal beats "set it yourself"; articulate the system trust level and the difference between *scoping* and *authorizing*; and state precisely where the tracked-write guarantee ends and what covers the remainder.

Next: *Lesson 2.8 — Tenants & membership* — the entities behind the claim: what a tenant actually *is*, and the membership model that decides everything from "who's in my household" to "what happens when the owner leaves."

Lesson 2.8 — Tenants & membership

> **Part 2 · Identity & tenancy.** Two lessons of machinery have enforced "the current tenant" without ever defining what a tenant *is*. Time to model it — and the modeling lesson here is about restraint and invariants: the `Tenant` is four properties; all the interesting decisions live in **membership**, the link entity that answers who belongs where, in what role, and — the questions that actually bite — what happens on remove, transfer, and leave. Lifecycle transitions are where domain models earn or lose their integrity.

Goal: `Tenant` and `TenantMembership` exist with their invariants (one tenant per user, exactly one owner per tenant) enforced; the app-facing `/api/household` surface supports get/rename/remove-member/transfer/leave, each transition preserving every invariant.

Concepts & principles: link entities as decision records; invariants as constraints + result enums; the "membership, not `User.TenantId`" indirection; result types over exceptions for domain outcomes; lifecycle design ("nobody is ever tenant-less").

Maps to: ADR-003, ADR-C1/C2 · repo: `src/Core/Entities/Tenant.cs`, `TenantMembership.cs`, `src/Api/Services/TenantService.cs`, `TenantModels.cs`, `src/Api/Controllers/HouseholdController.cs`, `src/Core/Repositories/ITenantRepository.cs`, tests: `UserServiceTests` (tenant-of-one), integration `HarnessSmokeTests` (`/api/household`), later `MemberRoleManagementTests`.

Prerequisites: 2.6/2.7 (the machinery this model feeds), 2.1 (users).

---

1. Motivate — the model is the answer sheet

Product questions this lesson must answer, each of which has burned a real SaaS:

- Can a user belong to two tenants? (Your data model answers even if you never decided.)
- Ana leaves the household — does the shopping list she created leave with her?
- The owner deletes their account — is the household now headless? Orphaned? Gone?
- Two admins demote each other simultaneously — who wins?

Answer these *in entities and constraints* and the platform behaves consistently forever; answer them ad hoc per endpoint and every feature reinvents the rules until two of them disagree. So: model first, endpoints second.

2. The entities — one boring, one load-bearing

```
// src/Core/Entities/Tenant.cs — the real file, in full
public class Tenant
{
    public Guid Id { get; set; } = Guid.CreateVersion7();
    public required string Name { get; set; }
    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset UpdatedAt { get; set; }
}
```

Four properties. That's not underdesign — it's the discipline of putting state where it belongs: billing state will live in a `Subscription` projection (Part 5), member count is *derived* (never a stored counter that drifts — Golden Rule 4), settings arrive when a feature needs them. A tenant is an identity to hang ownership off; resist making it a junk drawer.

```
// src/Core/Entities/TenantMembership.cs — the real file
/// <summary>
/// Links a user to their tenant with a role. A user belongs to exactly one tenant at a
/// time (unique on UserId); a tenant has exactly one owner. Identity (logins, inbox)
/// stays user-scoped; app data is scoped to the tenant. Membership — not User.TenantId —
/// is the source of truth for tenant resolution.
/// </summary>
public class TenantMembership
{
    public Guid Id { get; set; } = Guid.CreateVersion7();
    public Guid TenantId { get; set; }
    public Guid UserId { get; set; }
    public string Role { get; set; } = TenantRoles.Member; // owner | admin | member
    public DateTimeOffset JoinedAt { get; set; } = DateTimeOffset.UtcNow;
}
```

The doc comment is a compressed ADR — unpack its three commitments:

"Exactly one tenant at a time (unique on UserId)." The biggest product decision in Part 2, and it's enforced as a **unique index**, not a code path. Multi-tenant-per-user (Slack-style workspace switching) is a legitimate other product — with tenant pickers, per-tenant tokens, "which tenant is this request for" on every call. This platform's domain (a household product) wants one home per user, and buys enormous simplification with it: the JWT carries *the* `tenant_id`, not a selected one. Know which product you're building; put the answer in the schema.

"Membership — not `User.TenantId` — is the source of truth." A `TenantId` column on `User` would be simpler — and would weld identity to belonging. As a separate entity, membership can carry *belonging-specific* state (`Role`, `JoinedAt`), be deleted without touching the account, and later support history. The user's account — logins, locale, notifications — survives every household transition. Identity is user-scoped; app data is tenant-scoped; membership is the hinge (ADR-C1/C2, in code).

"Exactly one owner." Not a preference — an invariant every transition below must conserve. `TenantRoles` pins the vocabulary (`owner > admin > member`, ordered — the matrix that gives the ordering teeth is RBAC's, Lesson 6.1; today only *owner* gates anything).

Neither entity implements `ITenantScoped` — they're on 2.6's documented allowlist. Membership *is* the thing tenant resolution derives from; filtering it by the thing it defines would be circular. First-hand proof the allowlist isn't a formality.

3. Where tenants come from — the household of one

No "create tenant" endpoint exists. Look back at 2.5's case 3 and 2.4's redemption: both funnel into one private `CreateUserWithTenantAsync`, which writes **user + tenant + owner membership atomically** — three rows, one `IUnitOfWork` transaction, all-or-nothing (a crash mid-way must not birth a user with no household). Every user is thus born an owner of their own tenant-of-one; there is no moment — not one request — in which an authenticated user

has no tenant. That non-existence of a "tenant-less user" state kills an entire class of null-handling and onboarding UX, and it's why 2.6's claim `tenant_id` can be relied on rather than defended against.

This "nobody is ever tenant-less" invariant now becomes the design key for every transition below.

4. Transitions — where models go to die (or not)

The service surface, from the real `TenantService` — notice it speaks in **result enums**, not exceptions:

```
public enum RemoveMemberResult { Removed, NotAMember, CannotRemoveOwner }
public enum TransferResult { Transferred, TargetNotMember, ConcurrentModification }
public enum LeaveOutcome { Left, MustTransferFirst, ConfirmationRequired, Dissolved }
```

A member being non-removable-because-owner isn't *exceptional* — it's a legal outcome the UI must explain. Exceptions are for broken invariants; enums are for domain answers. (The controller maps each to 1.5's envelope: `CannotRemoveOwner` → `{ "error": "cannot_remove_owner", ... }` — machine-readable all the way out.) Now the transitions, each read against the invariants:

Remove a member. From the real doc comment: "*lands them in a fresh tenant-of-one (owner) so they are never tenant-less. Their contributed data stays with the tenant.*" Two decisions in one sentence. Re-homing conserves the no-tenant-less invariant even under an *involuntary* transition. And the data question — the shopping list Ana created — is answered by tenancy itself: app data belongs to the *tenant* (it's `ITenantScoped`, stamped with the tenant's id, 2.7). Ana's authorship was participation, not ownership; she leaves, the list stays. The model made this decision back in 2.6 — this lesson just makes it visible. And `CannotRemoveOwner`: removal would leave the tenant headless; ownership *moves*, never vanishes.

Transfer ownership. Demote-owner + promote-target — two rows that must flip *atomically* (a crash between them = zero or two owners; a transaction makes it all-or-nothing). And note `ConcurrentModification`: transfer is guarded against the racing-admins problem with optimistic concurrency — if the membership rows changed under you, the transfer reports it instead of silently double-applying. Invariants under *concurrency*, not just under happy sequences.

Leave. The richest one — walk `LeaveOutcome`'s four arms as a decision tree: a non-owner leaves (re-homed, data stays); an owner with members can't leave until they transfer (`MustTransferFirst` — the invariant, stated as a 409); a *sole* owner leaving means destroying the tenant, which is irreversible and therefore demands an explicit confirmation round-trip (`ConfirmationRequired` → client re-sends with `confirm_dissolve: true` → `Dissolved`). That confirm-then-wipe dance previews 2.9, where "wipe" gets its real machinery.

5. The app-facing surface — and the rename seam

The controller binds it all to HTTP — `GET /api/household`, `PUT` rename, `DELETE members/{id}`, `POST transfer-ownership`, `POST leave` — with one naming oddity worth explaining rather than hiding: the route says **household**, the code says **tenant**. Deliberate bilingualism. *Tenant* is platform vocabulary — every seam, filter, and rule from 2.6/2.7 speaks it, and it stays stable

across products built from this template. *Household* is this app's word for it — user-facing, changeable per product (a team app renames it "workspace" by touching the controller layer and UI strings only; the rebrand checklist in 9.1 includes exactly this). Keep the dictionary small and one-directional: domain term inside, product term at the boundary.

Owner-only actions (rename, remove, transfer) are enforced at the controller via the membership role — full RBAC arrives in 6.1; today the check is "is caller the owner," which is the only distinction the model needs yet. The wire tests you met in 2.6's harness (`/api/household` returning *your* household, cross-tenant invisibility) now read as what they were all along: this lesson's integration suite.

6. Red — the tests

```
Scenario: removing a member re-homes them
  Given a tenant with owner Ana and member Ben, where Ben created a Widget
  When Ana removes Ben
  Then Ben is the owner of a fresh tenant-of-one
  And the Widget still belongs to Ana's tenant

Scenario: ownership transfer is atomic and conserves single-owner
  Given owner Ana and member Ben
  When Ana transfers ownership to Ben
  Then Ben is owner and Ana is admin — and at no observable point were there zero or two
owners

Scenario: sole owner leaving requires explicit confirmation
  Given Ana alone in her tenant
  When she leaves without confirmation
  Then the outcome is ConfirmationRequired and nothing changed
  When she confirms
  Then the tenant and its data are gone and Ana is re-homed
```

Each scenario is an invariant wearing a story. The membership-lifecycle E2E journey (built with the UI in Part 3/6) replays them through a real browser.

7. Run it

```
dotnet test tests/Api.Tests --filter "Tenant|Household"
# .http: GET /api/household → your tenant-of-one, role "owner", member list of you
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how do users relate to tenants? (a) `User.TenantId` — a column; (b) many-to-many memberships (multi-workspace); (c) a membership link entity with a **unique-per-user** constraint — one tenant at a time, roles on the link.

Chosen: (c) (ADR-003). The link entity keeps identity and belonging separable (accounts survive transitions; roles/joined-at live where they belong) while the unique index keeps the *product's* answer — one home — enforced by the schema. Transitions are modeled as result enums with re-homing so no path strands a user tenant-less; ownership is conserved by construction (transfer-only, atomic).

Rejected: (a) — welds identity to belonging; every lifecycle transition turns into account surgery, and role-on-membership has nowhere to live. (b) — a different product; adopting its complexity (tenant pickers, per-request tenant selection, N×M permission surfaces) without its requirement is pure cost. The link-entity shape deliberately keeps (b) *reachable* — drop the unique index and add a picker — which is the correct way to keep an option open: pay for it only if exercised.

The trade: "one tenant per user" is a real product constraint (no guest memberships across households), and re-homing means removals *create* tenants — tenant count ≠ paying customers, which Part 5's billing must (and does) treat knowingly.

9. Checkpoint

```
git add -A && git commit -m "feat: tenants + membership — invariants, transitions,
household surface (ADR-003)"
git tag lesson-2.8
```

You should now be able to: defend membership-as-entity against the simpler column; name both schema-enforced invariants and every transition that must conserve them; explain re-homing and whose data stays where (and why the model already decided); and argue result-enums-over-exceptions for domain outcomes.

Next: *Lesson 2.9 — Invitations, dissolve & the contributor seam.* How anyone ever joins someone *else's* household — and the decentralized-teardown seam that GDPR will thank us for in Part 7.

Lesson 2.9 — Invitations, dissolve & the contributor seam

> **Part 2 · Identity & tenancy — finale.** Two jobs close out the chassis. First, invitations: the only way a tenant ever gains a second member, built entirely from parts you already own (hashed single-use tokens, branded email, membership transitions). Second — the deeper one — *dissolve*: when a tenant dies, every feature's data must die with it, including the data of **features that don't exist yet**. That impossible-sounding requirement has a beautiful answer, the `ITenantDataContributor` seam — the platform's purest example of the Open/Closed principle, and the hook GDPR will hang the entire export/erasure epic on in Part 7.

Goal: an owner can invite by email; whoever proves control of the invite token joins (with correct re-homing of *their* old tenant); a dissolving tenant's data is wiped feature-by-feature through a discovered seam — with a test that fails any future feature that forgets to register.

Concepts & principles: composing prior seams into a feature; identity-at-redemption (the invite token authenticates the *invitation*, the login authenticates the *person*); Open/Closed via DI fan-out; teardown inside one transaction; guarding solo-owner data abandonment.

Maps to: ADR-003/ADR-011 (the seam GDPR extends) · repo: `src/Core/Entities/TenantInvitation.cs`, `src/Api/Services/TenantInvitationService.cs`, `src/Api/Controllers/HouseholdInvitationsController.cs`, `src/Core/Abstractions/ITenantDataContributor.cs`, `src/Api/Services/TenantDissolutionService.cs`, tests: `TenantInvitationTests` (Core), `WipeDataTests.cs`.

Prerequisites: 2.8 (membership + transitions), 2.4 (the token pattern), 2.3 (email).

---

1. Motivate — two lifecycle holes

Hole one: every tenant so far is a household of one. There is no path by which Ana's household ever contains Ben — and "Ben types Ana's household id" is obviously not it. Joining must be *invitation-shaped*: initiated by an owner, addressed to a person, consensual on both sides, expiring, revocable.

Hole two, quieter and worse: 2.8's `Dissolved` arm claims the tenant's data gets destroyed. *Which* data? Today, a `Note` here, a `Widget` there. Next month, files, notification prefs, webhook subscriptions, billing counters — written by features that don't exist yet. A central `WipeTenant()` method listing every table is doomed to incompleteness: each new feature must remember to edit it, and the one that forgets leaves orphaned rows — data that legally *must* die (wait for GDPR) quietly surviving. The platform needs teardown that **new features join automatically**.

2. Invitations — old parts, one new idea

The entity is 2.4's pattern wearing a new purpose — hashed single-use token, expiry, computed validity — plus a status lifecycle:

```
// src/Core/Entities/TenantInvitation.cs (the real shape)
public class TenantInvitation : ITenantScoped // ← note the marker!
{
    public Guid Id { get; set; } = Guid.CreateVersion7();
    public Guid TenantId { get; set; }
    public required string InvitedEmail { get; set; } // normalized lower-case
    public Guid InvitedByUserId { get; set; }
    public string Status { get; set; } = InvitationStatuses.Pending; //
pending|accepted|revoked|expired
    public required string TokenHash { get; set; } // raw token revealed once, at creation
    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset ExpiresAt { get; set; }

    public bool IsValidAt(DateTimeOffset now) => Status == InvitationStatuses.Pending &&
!IsExpiredAt(now);
}
```

`ITenantScoped` — the first *feature-owned* entity to wear the marker since the machinery went in. Invitations belong to the inviting tenant: the owner lists *their* pending invites, 2.6's filter scopes them for free, 2.7 stamps them on insert. The chassis just works; this is what building *on* it feels like.

The flow composes seams end to end: owner posts an email address → service invalidates prior pending invites for it, mints token via `ITokenGenerator`, stores the hash, lifespan from config (`Auth:Invitation:LifespanDays`, 1.4) → `BrandedEmail` renders the invite **in the inviter's saved locale** (2.3's explicit-culture design paying off — the recipient sees the language of the household inviting them) → `IEmailSender` delivers.

The one genuinely new idea is at redemption, and the entity's doc comment states it precisely: "*redeemed by whoever is signed in and presents the single-use token — **identity is the login, not the bare email***." Unpack that, because it's a subtle authorization design:

- The invite link lands on `/join`. The redeemer must **sign in first** (any method —

OTP, Google, whatever *they* have). The token proves they hold the invitation; the login proves who they are. Two proofs, two different questions.

- Consequence: the invite can be *forwarded*. Ana invites `ben@work.test`; Ben redeems

while signed in as `ben@personal.test` — and joins as his real, existing account rather than growing a duplicate. The addressed email is *delivery routing*, not identity. (Compare 2.4, where email possession *is* the identity claim — same token machinery, opposite trust conclusion, both correct for their question.)

- Joining triggers 2.8's transitions: Ben's old tenant-of-one is handled by the same

leave semantics — and here the contributor seam makes its entrance early. What if Ben's solo tenant *has data*? Silently discarding it on join would be the data-abandonment bug. The platform checks `HasDataAsync` across all contributors and requires explicit confirmation — "joining will delete your current household's data" — before dissolving it. The seam serves its first customer before dissolve even ships.

Revocation (owner cancels a pending invite) and expiry round out the lifecycle; `TenantInvitationTests` in Core.Tests drive the validity state machine with explicit clocks — pure domain logic, no database.

3. The contributor seam — teardown that scales with features

Now hole two. The platform's answer, in full:

```
// src/Core/Abstractions/ITenantDataContributor.cs — the real file
/// <summary>
/// A feature's contribution to tenant-data presence and teardown. Each vertical slice
/// that owns tenant-scoped tables registers one of these in DI; the platform queries
/// all contributors instead of a hard-coded method — so adding a feature never means
/// editing a central "has data" / "wipe data" method.
/// </summary>
public interface ITenantDataContributor
{
    /// <summary>True if this contributor holds any data for the tenant.</summary>
    Task<bool> HasDataAsync(Guid tenantId, CancellationToken ct = default);

    /// <summary>Removes this contributor's data for the tenant. Called within the
    /// dissolve transaction, so it shares the caller's unit of work — do not commit
here.</summary>
    Task WipeAsync(Guid tenantId, CancellationToken ct = default);

    /// <summary>The section name this contributor's data appears under in a tenant
    /// export (GDPR-1), e.g. "notes". Stable and unique across contributors.</summary>
    string ExportKey { get; }

    /// <summary>A JSON-serializable snapshot for a data export. Null when nothing to
    /// include. Identifiers and content only — never secrets.</summary>
    Task<object?> ExportAsync(Guid tenantId, CancellationToken ct = default);
}
```

The inversion is the entire idea. The platform doesn't know what features exist; it asks DI for *all* `ITenantDataContributor`s and fans out. A new feature ships its own contributor (five minutes: "do I have rows for this tenant / delete them / here's my export"), registers it, and is *automatically* part of dissolve, of the join-time data check, and — Part 7 — of GDPR export. **The platform is closed to modification, open to extension**: Open/Closed not as a poster slogan but as a DI query.

Notice the interface already carries `ExportKey`/`ExportAsync`. Historically the seam was born two-method and GDPR *widened* it — and because widening an interface breaks every implementor at compile time, each existing feature was forced, by the build, to answer "what do I export?" That's the seam working even while changing: the compiler runs the migration checklist. (The course teaches the final four-member shape from the start; you get the anecdote instead of the breakage.)

The orchestrator stays platform-small:

```
// src/Api/Services/TenantDissolutionService.cs (the real shape)
/// Fan out over every ITenantDataContributor so each feature wipes its own domain
/// data first, then run the platform's core teardown (memberships, invitations, the
/// tenant row). Deliberately does NOT open its own transaction — callers invoke it
/// inside their existing dissolve transaction, so the whole thing commits atomically.
public async Task DissolveAsync(Guid tenantId, CancellationToken ct = default)
{
    foreach (var contributor in contributors) // injected:
IEnumerable<ITenantDataContributor>
        await contributor.WipeAsync(tenantId, ct);
    await tenantRepository.WipeDataAsync(tenantId, ct); // core: memberships, invites,
tenant row
}
```

Two disciplines in the comments deserve promotion to principles. **Order:** features first, core last — contributors may need the tenant's rows (FKs) to still exist while they wipe. **Transactionality:** the service *enlists* in the caller's transaction rather than owning one — dissolve is one atomic event alongside the re-home and audit steps that surround it (2.8's leave flow); a crash mid-wipe rolls back to "still dissolved-nothing," never "half-dissolved." A teardown that can partially complete is a data-integrity bug with a delay timer. And who *calls* dissolve? Only system-level flows — the sole-owner leave and (later) account erasure — running on the system context from 2.7; the contributors' `WipeAsync` implementations are exactly the sanctioned cross-tenant-write code that trust level exists for.

4. Harden — the seam that can't be forgotten, provably

The seam's Achilles heel: a feature that *doesn't* register a contributor silently opts out of teardown. Convention again — unless a test closes it. The reference repo's `WipeDataTests` does exactly that, with 1.3's discovery trick pointed at a new target: walk the EF model for `ITenantScoped` entity types, dissolve a seeded tenant, then assert **zero rows remain in any tenant-scoped table**:

```
[Fact]
public async Task Dissolve_leaves_no_tenant_scoped_rows_behind()
{
    var tenantId = await SeedEveryFeaturesSampleDataAsync(); // incl. YOUR new feature

    await dissolution.DissolveAsync(tenantId);

    foreach (var entityType in db.Model.GetEntityTypes()
                 .Where(t => typeof(ITenantScoped).IsAssignableFrom(t.ClrType)))
        Assert.Equal(0, await CountRowsForTenantAsync(entityType, tenantId));
}
```

A feature that adds a tenant-scoped table without wiring teardown turns this red with the *table's name in the failure*. Model-derived, so it covers tables that don't exist yet — the same move as the fixture reset (R11) and the filter-coverage invariant (R2), now guarding deletion. Three guards, one pattern: **derive the checklist from the model; never hand-maintain it.**

5. Red — the tests

```
Scenario: invitation joins the redeemer's real account
  Given Ana invited ben@work.test
  And Ben signs in as ben@personal.test and presents the token
  Then Ben's account joins Ana's household as member
  And Ben's empty solo tenant is dissolved

Scenario: joining with data requires explicit consent to lose it
  Given Ben's solo household contains a Note
  When Ben redeems Ana's invite without confirmation
  Then the join is refused with has_data
  And nothing changed

Scenario: dissolve is total and atomic
  Given a tenant with data in every feature
  When the sole owner leaves with confirmation
  Then no tenant-scoped rows remain for that tenant
  And a wipe failure in any contributor rolls the entire dissolve back

Scenario: invitations are single-use and revocable
  Given a pending invitation
  When it is redeemed / revoked / expires
  Then subsequent redemption attempts fail with the same generic error
```

The third scenario's rollback half is worth the extra test plumbing (a contributor rigged to throw): atomicity claims are exactly the claims that rot silently.

6. Run it

```
dotnet test tests/Api.Tests --filter "Invitation|WipeData"
# live: invite yourself at a second address; open Mailpit; redeem on /join while
# signed in as another test account; GET /api/household → two members.
```

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how does per-feature data participate in tenant teardown (and later, export)? (a) A central `WipeTenant()` that each feature edits; (b) database cascade deletes from the tenant row; (c) a DI-discovered contributor seam + a model-derived completeness test.

Chosen: (c) (ADR-003 lineage; ADR-011 widens it). Fan-out inverts the dependency: the platform stops needing to know its features. The completeness test converts the seam's one weakness (forgetting to register) into a named build failure. The same seam answers three lifecycle questions — has-data (join guard), wipe (dissolve), export (GDPR) — so a feature declares its data relationship *once*.

Rejected: (a) — the forget-to-edit design; its failure mode is silent orphaned data with legal implications. (b) — cascades handle *rows* but not the rest (files on disk in Part 6, provider-side subscriptions in Part 5 — dissolve is not only SQL), offer no has-data or export answer, and hide teardown semantics in migration DDL where no test names them.

The trade: every feature carries ~30 lines of contributor boilerplate, and dissolve is O(features) sequential queries — perfectly acceptable for an event as rare as tenant death, and the boilerplate is the checklist (3.2's slice template includes it pre-written).

8. Checkpoint

```
git add -A && git commit -m "feat: invitations + dissolve — contributor seam, atomic
teardown, completeness gate"
git tag lesson-2.9
```

Part 2 complete. The chassis stands: identity (tokens, passwordless, OAuth), tenancy (read filter, write stamping, EnterTenant), the tenant model with its invariants, and lifecycle machinery that future features join by construction. Every lesson from here *rides* this.

You should now be able to: separate "who holds the invitation" from "who is joining" and say why forwarding is safe; implement the contributor seam and its completeness test from memory; explain wipe ordering and transaction enlistment; and recognize Open/Closed when it's a DI query rather than a slogan.

Next: *Part 3 — The slice pattern & the UI.* The chassis gets its assembly manual: the repository seam, the anatomy of a vertical slice, and the first pixels.

Part 3 — The slice pattern & the UI

Lesson 3.1 — The repository seam

> **Part 3 · The slice pattern & the UI.** Part 2 built the chassis; Part 3 is its *assembly manual* — the repeatable way features bolt on. This first lesson supplies the data-access seam every future slice uses, and it carries a quiet responsibility: it must give features an ergonomic way to read and write their own tables **without letting them undermine the tenant isolation** you fought for in 2.6/2.7. One generic interface, two query methods — one safe by default, one deliberately dangerous and greppable — and a commit model that makes atomicity a choice rather than an accident.

Goal: a generic `IRepository<T>` lets a slice read (tenant-filtered) and write its own entity with no bespoke repository code; an explicit, named escape hatch exists for the rare cross-tenant path; and a unit-of-work seam groups multi-write operations into a transaction.

Concepts & principles: the generic repository; open-generic DI registration; the safe-default / explicit-escape-hatch pattern; unit of work and the shared-context commit model; ergonomics that *preserve* guarantees rather than bypass them.

Maps to: ADR-003/ADR-004 · R5 (the escape-hatch ban, formalized 3.3) · repo: `src/Core/Repositories/IRepository.cs`, `IUnitOfWork.cs`, `src/Infrastructure/Repositories/EfRepository.cs`, `EfUnitOfWork.cs`, `tests/Api.Tests/RepositoryScopingTests.cs`.

Prerequisites: 2.6 (the query filter this seam must preserve), 2.7 (the write interceptor), 2.9 (the contributor wipe that first needed cross-tenant access).

---

1. Motivate — two ways to give features data access, both wrong

Wrong way one: a bespoke repository per entity. `INoteRepository`, `IWidgetRepository`, each with `GetById`, `List`, `Add`, `Delete` — ninety percent identical, hand-written every slice, and every one an opportunity to write a query that *forgets the tenant filter* or subtly diverges from its siblings. Multiply by thirty features and you have a maintenance tax paid in boilerplate and isolation risk.

Wrong way two: hand features the `DbContext` directly. Maximum flexibility, maximum danger. A feature holding `AppDbContext` can call `IgnoreQueryFilters()`, can `SaveChanges` on entities it shouldn't, can reach into other features' `DbSet`s. The chassis's guarantees become "please don't."

The seam threads the needle: **one generic repository** that eliminates the boilerplate *and* channels every feature read through the tenant-filtered path by default — while making the cross-tenant escape a separate, named, greppable method that a code reviewer (and an arch test) can see from orbit.

2. The interface — safe by default, dangerous on purpose

```
// src/Core/Repositories/IRepository.cs — the real interface
/// <summary>
/// A thin generic repository for FEATURE/domain entities, so a vertical slice can read
/// and write its own data without authoring a bespoke repository pair. Reads via Query()
/// are automatically tenant-scoped for ITenantScoped entities (the AppDbContext global
/// query filter), so a slice can't forget to scope.
/// </summary>
public interface IRepository<TEntity> where TEntity : class
{
    /// <summary>Queryable over the set — tenant-filtered for ITenantScoped entities.
    /// This is the normal feature read path; it cannot leak across tenants.</summary>
    IQueryable<TEntity> Query();

    /// <summary>Cross-tenant queryable: the global tenant filter is OFF, so this sees
    /// EVERY tenant's rows. The audited, deliberately-named escape hatch ... Feature
    /// slices use Query(); reach for this only when crossing tenants is the explicit
    /// intent, and always re-constrain with a Where(x => x.TenantId == ...).
    /// Greppable on purpose.</summary>
    IQueryable<TEntity> QueryAllTenants();

    Task AddAsync(TEntity entity, CancellationToken ct = default);
    void Update(TEntity entity);
    void Remove(TEntity entity);

    Task<int> SaveChangesAsync(CancellationToken ct = default);
}
```

The whole design philosophy is in the contrast between the two query methods:

- **`Query()` is the paved road.** It returns `db.Set<TEntity>()` — which, for anything `ITenantScoped`, already carries 2.6's global filter. A feature writes `Query().Where(n => n.Pinned).ToListAsync(ct)` and gets *only its tenant's* pinned notes, having written nothing about tenants. The guarantee rides underneath the ergonomics. `IQueryable` (not `Task<List<T>>`) is deliberate: the slice composes its own typed LINQ — filtering, projection, paging — and the repository never becomes a bag of `GetByFooBar` methods.
- **`QueryAllTenants()` is the clearly-marked exit.** Its whole job is to turn the filter *off*. Read the doc comment's tone — "EVERY tenant's rows," "deliberately-named," "Greppable on purpose." This is the sanctioned version of the escape hatch you first met in 2.9, when a dissolve contributor had to wipe a tenant *other* than the current one. The enforcement is more precise than "banned in features," and the precision is the lesson (three arch-test rules you'll write in 3.3): **raw `IgnoreQueryFilters()` is banned *everywhere* under `src/Api/Features/**`** — so the *only* way a slice can cross tenants is this named, greppable method; and **`QueryAllTenants()` itself is allowed only inside a slice's `*DataContributor.cs`** (dissolve and export are inherently cross-tenant), never on the request path — an endpoint or handler that reaches for it fails the build. So a feature's *read path* has no exit at all; its *teardown hook* has exactly one, and it announces itself in a file named for the purpose. The escape hatch and its bans are halves of one decision.

> **Forward glance (8.4).** The real `EfRepository.QueryAllTenants()` also chains `.TagWith(RlsTags.CrossTenant)`. That tag matters only once the Postgres row-level security backstop exists (Lesson 8.4) — it tells the *database* "this cross-tenant read is sanctioned." Until then your implementation is just `IgnoreQueryFilters()`; the tag is a seam we'll thread through later. It's here now so you recognize it when it returns.

3. The implementation — one registration covers every entity

```
// src/Infrastructure/Repositories/EfRepository.cs — the real file
public class EfRepository<TEntity>(AppDbContext db) : IRepository<TEntity> where TEntity :
class
{
    public IQueryable<TEntity> Query() => db.Set<TEntity>();

    public IQueryable<TEntity> QueryAllTenants() =>
        db.Set<TEntity>().IgnoreQueryFilters(); // + .TagWith(RlsTags.CrossTenant) in 8.4

    public async Task AddAsync(TEntity entity, CancellationToken ct = default) =>
        await db.Set<TEntity>().AddAsync(entity, ct);

    public void Update(TEntity entity) => db.Set<TEntity>().Update(entity);
    public void Remove(TEntity entity) => db.Set<TEntity>().Remove(entity);

    public Task<int> SaveChangesAsync(CancellationToken ct = default) =>
db.SaveChangesAsync(ct);
}
```

Thirty lines, and every entity type in the platform is served — because of one **open-generic registration**:

```
services.AddScoped(typeof(IRepository<>), typeof(EfRepository<>));
```

That single line means `IRepository<Note>`, `IRepository<Widget>`, and every future `IRepository<Whatever>` resolve with no per-entity wiring. Ask the container for `IRepository<Note>` and it constructs `EfRepository<Note>` on demand. This is the boilerplate-killer: adding a feature entity adds *zero* data-access code.

Notice what `EfRepository` injects: `AppDbContext` — the **request-scoped** one, the same instance the tenant filter (2.6) and stamping interceptor (2.7) are attached to. The repository is a thin pass-through, not a second layer of caching or logic. Its value is entirely in *what it refuses to expose*: a feature holding `IRepository<Note>` cannot reach `IRepository<OtherFeaturesThing>`'s data by accident, can't touch platform tables, and defaults to the filtered read path.

4. Unit of work — where atomicity lives

`SaveChangesAsync` on the repository flushes the shared context. But what about an operation that must write *several* things all-or-nothing? That's a different concern — transaction boundaries — and it gets its own seam, whose doc comment is the clearest statement of the platform's commit model:

```
// src/Core/Repositories/IUnitOfWork.cs
public interface IUnitOfWork
{
    Task<ITransactionScope> BeginTransactionAsync(CancellationToken ct = default);
}
public interface ITransactionScope : IAsyncDisposable
{
    Task CommitAsync(CancellationToken ct = default);
}
```

The subtlety the doc comment nails: **every repository shares the same request-scoped `DbContext`, so any `SaveChangesAsync` flushes *all* pending tracked changes, not just the caller's.** That sounds alarming until you see how the two layers compose:

- **A single self-contained write** — one entity, no cross-entity invariant — just calls `SaveChangesAsync` and rides EF's implicit per-call transaction. No ceremony.
- **A grouped operation** — several writes that must commit together — wraps them: `await using var tx = await uow.BeginTransactionAsync(ct); … ; await tx.CommitAsync(ct);`. Inside that scope, every intermediate `SaveChangesAsync` *enlists in the one open transaction*, and disposing the scope without committing rolls the whole group back.

You already saw this contract consumed before you built it: 2.8's atomic user+tenant+membership creation, and 2.9's dissolve that "enlists in the caller's transaction rather than opening its own." Both rely on exactly this model — one shared context, transactions layered on top when grouping is needed. The `await using` + commit-or-rollback shape is the same disposable-scope discipline as 2.7's `EnterTenant`; the platform reuses a small number of good shapes deliberately.

(The `EfUnitOfWork` implementation wraps EF's `Database.BeginTransactionAsync`; on a non-relational provider it degrades to a no-op so the same service code runs everywhere — though this course always tests on real Postgres, where the transaction is real.)

5. Red — prove the seam keeps the promise

The seam's entire reason for existing is that `Query()` *cannot leak*. So the test that matters re-proves 2.6's isolation, but *through the repository* a feature will actually use (`RepositoryScopingTests` in the reference repo):

```
[Fact]
public async Task Query_is_tenant_scoped_but_QueryAllTenants_sees_everything()
{
    await SeedNoteAsync(tenantA, "A's note");
    await SeedNoteAsync(tenantB, "B's note");

    var repoAsA = RepositoryActingAs(tenantA); // scoped context, tenant A

    Assert.Single(await repoAsA.Query().ToListAsync()); // only A's
    Assert.Equal(2, await repoAsA.QueryAllTenants().CountAsync()); // the escape hatch sees
both
}
```

Two assertions, two halves of the contract: the paved road is isolated; the marked exit genuinely exits. (The *bans* on taking that exit are enforced separately, by the arch tests in 3.3 — raw `IgnoreQueryFilters()` nowhere in features, `QueryAllTenants()` only in `*DataContributor.cs`. This test proves the mechanism works; those prove features can't misuse it.)

6. Run it

```
dotnet test tests/Api.Tests --filter Repository
```

No user-facing surface yet — this is infrastructure the next lesson consumes. The proof that it's right is the isolation test above plus the fact that 3.2's Notes slice will contain *no data-access code of its own*.

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how do feature slices access data? (a) A bespoke repository per entity; (b) the raw `DbContext` injected into handlers; (c) one generic `IRepository<T>` with a tenant-filtered `Query()` and a named, greppable `QueryAllTenants()` escape hatch, plus a separate `IUnitOfWork` for grouped transactions.

Chosen: (c) (ADR-004). It deletes per-entity boilerplate via open-generic registration, channels every feature read through 2.6's filter *by default*, and makes the dangerous cross-tenant path a distinct method that review and R5's arch test can police. Atomicity is factored into its own seam so the common single-write case stays ceremony-free while grouped writes get real transactions.

Rejected: (a) — boilerplate tax plus N chances to write an unfiltered query; the generic repo makes "forgot to scope" unreachable from the paved road. (b) — hands features the keys to the whole database and turns every chassis guarantee into a convention; the seam exists precisely to *not* do this.

The trade: a generic `IQueryable`-returning repository is less prescriptive than named query methods — a slice *can* compose an inefficient query. Accepted: the alternative (a method per access pattern) is the boilerplate we're eliminating, and slice code is reviewed. Platform/auth entities keep their dedicated repositories (they have bespoke needs and predate this seam); `IRepository<T>` is for feature tables.

8. Checkpoint

```
git add -A && git commit -m "feat: generic repository seam — tenant-safe Query, named
escape hatch, unit of work"
git tag lesson-3.1
```

You should now be able to: explain why a generic repository beats both per-entity repositories and raw `DbContext` access; describe how `Query()` inherits tenant isolation for free and why `QueryAllTenants()` is deliberately conspicuous; state the shared-context commit model and when a slice needs an explicit transaction; and register an open-generic service.

Next: *Lesson 3.2 — Anatomy of a vertical slice (Notes)* — the reference slice that consumes this seam, and the seven-step shape every feature you ever build will follow.

Lesson 3.2 — Anatomy of a vertical slice (Notes)

> **Part 3 · The slice pattern & the UI.** This is the lesson the whole platform was built to make possible. Every feature you add from here — and every feature a *real product* built on this template adds — follows one shape: a self-contained folder that bolts onto the chassis and reuses everything Part 2 gave you. We learn that shape from `Notes`, the reference slice, which exists to be copied and then deleted. Five small files, one `DbSet`, one registration — no changes to platform *behavior*. Master this and you can add a feature in an afternoon; the rest of the course is mostly this shape at higher stakes.

Goal: a complete CRUD feature — list/create/delete notes — reachable at `/api/notes`, tenant-isolated, wired into dissolve and export, driven by tests, and touching platform code only by *addition* — a `DbSet` on `AppDbContext` and a few registration lines in `Program.cs` — never changing platform *behavior*.

Concepts & principles: the vertical-slice feature folder; minimal-API feature groups vs controllers; auth-by-construction (`MapTenantFeatureGroup`); the thin handler; co-located DTOs; the four-member contributor; slice isolation rules; the request-path vs contributor escape-hatch boundary.

Maps to: ADR-004 · R6–R10, R34/R35 · `docs/WAYS_OF_WORKING.md` (the add-a-slice checklist) · repo: `src/Api/Features/Notes/*`, `src/Core/Entities/Note.cs`, `src/Api/Endpoints/FeatureEndpointExtensions.cs`, `src/Api/Program.cs` (the `MapNotes()` + DI lines), tests: `tests/Api.Tests/NotesSliceTests.cs`, `FeatureAuthorizationTests.cs`.

Prerequisites: 3.1 (the repository seam), all of Part 2 (the chassis this reuses).

---

1. Motivate — the shape is the product

The platform's value proposition, stated plainly: *adding a feature should be boring.* No decisions about auth, no decisions about tenant scoping, no editing a central router or a central teardown method, no fear of breaking another feature. If adding a feature is exciting, your architecture is charging you interest.

`docs/WAYS_OF_WORKING.md` distills the shape into a seven-step checklist — memorize it, because it *is* the rest of this course:

1. **Entity** → `src/Core/Entities/<Entity>.cs`, implementing `ITenantScoped`.
2. **DbSet + config** → add to `AppDbContext`, add an `IEntityTypeConfiguration`.
3. **Migration** → `dotnet ef migrations add Add<Entity>`.
4. **DI wiring** → register the handler + map the group in `Program.cs`.
5. **Contributor** → register the `ITenantDataContributor` (all four members).
6. **Fixture reset** → (automatic — the reset list is model-derived, 1.3).
7. **UI** → nav entry + component in the Shared.Ui RCL + resx strings (Part 3.4/3.5).

`Notes` is a worked example of steps 1–5. Let's build it, reading each file as an instance of a principle you already know.

2. The entity (step 1)

```
// src/Core/Entities/Note.cs
public class Note : ITenantScoped
{
    public Guid Id { get; set; } = Guid.CreateVersion7();
    public Guid TenantId { get; set; } // ← the marker's one requirement
    public required string Title { get; set; }
    public string Content { get; set; } = "";
    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset UpdatedAt { get; set; }
}
```

`: ITenantScoped` is the entire cost of joining the tenancy regime. The moment this compiles, 2.6's global filter scopes every read of `Note`, 2.7's interceptor stamps and guards every write, the R2 invariant test counts it as covered, and 2.9's wipe- completeness gate expects it to be torn down. One interface, and the chassis adopts your entity. Step 2 (a `DbSet<Note>` on `AppDbContext` + a `NoteConfiguration` setting the `Title` max length) and step 3 (`dotnet ef migrations add AddNotes`) are mechanical — the discovery patterns from 1.3 pick the configuration up automatically.

3. The endpoints — auth you can't forget (step 4a)

```
// src/Api/Features/Notes/NotesEndpoints.cs
public static class NotesEndpoints
{
    public static IEndpointRouteBuilder MapNotes(this IEndpointRouteBuilder app)
    {
        var group = app.MapTenantFeatureGroup("/api/notes"); // ← the one true way in

        group.MapGet("/", async (NotesHandler handler, CancellationToken ct) =>
            Results.Ok(await handler.ListAsync(ct)));

        group.MapPost("/", async (CreateNoteRequest request, NotesHandler handler,
CancellationToken ct) =>
        {
            var created = await handler.CreateAsync(request, ct);
            return created is null
                ? Results.BadRequest(new { error = "invalid_request", message = "A note
title is required" })
                : Results.Created($"/api/notes/{created.Id}", created);
        });

        group.MapDelete("/{id:guid}", async (Guid id, NotesHandler handler,
CancellationToken ct) =>
            await handler.DeleteAsync(id, ct) ? Results.NoContent() : Results.NotFound());

        return app;
    }
}
```

The load-bearing line is the first one. `MapTenantFeatureGroup` — recall its real definition from 3.1's neighbor file:

```
public static RouteGroupBuilder MapTenantFeatureGroup(this IEndpointRouteBuilder app,
string prefix) =>
    app.MapGroup(prefix).RequireAuthorization(AuthPolicies.TenantApi);
```

It is the *only* sanctioned way a feature registers routes, and the reason is R6: a slice must **not** write `app.MapGroup("/api/notes").RequireAuthorization(...)` by hand, because a hand-rolled group can forget the `.RequireAuthorization`, and a feature endpoint with forgotten auth is an unauthenticated data leak. The helper makes "authenticated, tenant-scoped" the *only* thing you can express — an arch test (3.3) fails any feature group not created through it. This is 2.6's philosophy applied to routing: don't ask developers to remember the guarantee; make the guaranteed path the only path.

Two more notes on style: features are **minimal-API groups**, while the *platform* stays controllers (auth, household, billing — the durable chassis). And the endpoints are *thin* — parse, delegate to the handler, map the result to an HTTP status. No logic lives here.

4. The handler — where the logic lives, and how little there is

```
// src/Api/Features/Notes/NotesHandler.cs
public class NotesHandler(IRepository<Note> notes, ICurrentTenant tenant, TimeProvider
clock)
{
    public async Task<IReadOnlyList<NoteResponse>> ListAsync(CancellationToken ct)
    {
        // Query() is tenant-scoped by the global filter — no explicit Where(TenantId)
needed.
        var rows = await notes.Query().OrderByDescending(n => n.CreatedAt).ToListAsync(ct);
        return rows.Select(NoteResponse.From).ToList();
    }

    public async Task<NoteResponse?> CreateAsync(CreateNoteRequest request,
CancellationToken ct)
    {
        if (tenant.TenantId is not { } tenantId) return null; // no tenant on the token
        if (string.IsNullOrWhiteSpace(request.Title)) return null; // endpoint maps null →
400

        var now = clock.GetUtcNow();
        var note = new Note { Id = Guid.CreateVersion7(), TenantId = tenantId,
            Title = request.Title.Trim(), Content = request.Content?.Trim() ?? "",
            CreatedAt = now, UpdatedAt = now };
        await notes.AddAsync(note, ct);
        await notes.SaveChangesAsync(ct);
        return NoteResponse.From(note);
    }

    public async Task<bool> DeleteAsync(Guid id, CancellationToken ct)
    {
        // Tenant-scoped lookup: a note from another tenant simply isn't found here.
        var note = await notes.Query().FirstOrDefaultAsync(n => n.Id == id, ct);
        if (note is null) return false;
        notes.Remove(note);
        await notes.SaveChangesAsync(ct);
        return true;
    }
}
```

Its own doc comment says it best: "*Note how little it needs.*" Three injected dependencies, all from the chassis: `IRepository<Note>` (3.1's seam), `ICurrentTenant` (2.6), and `TimeProvider` (the injected clock, formalized next lesson). Read the tenancy into it — this is where all of Part 2 pays out:

- **`ListAsync` writes no `Where(TenantId ==)`.** `Query()` is already filtered (2.6). A

developer *cannot* accidentally list another tenant's notes; the safe query is the natural one.

- **`DeleteAsync` gets tenant-scoped isolation for free.** `Query().FirstOrDefault(id)`

looks up by id *within the current tenant* — so a caller trying to delete another tenant's note by guessing its id gets `null` → 404, indistinguishable from "no such note." Cross-tenant access isn't forbidden with an error; it's *invisible*. That's the filter turning an authorization question into a not-found.

- **`CreateAsync` sets `TenantId` from the token's tenant**, and even if it didn't, 2.7's

interceptor would stamp it and refuse a foreign value. Belt and suspenders, by construction.

The handler contains exactly the logic that is *this feature's* — nothing about auth, tenancy plumbing, or persistence mechanics. That's the whole point of a chassis.

5. The models — co-located, on purpose (step 4b)

```
// src/Api/Features/Notes/NotesModels.cs
public record CreateNoteRequest(string? Title, string? Content);

public record NoteResponse(Guid Id, string Title, string Content, DateTimeOffset CreatedAt)
{
    public static NoteResponse From(Note n) => new(n.Id, n.Title, n.Content, n.CreatedAt);
}
```

The DTOs live *in the slice*, not in a shared `Models` project. This is deliberate (R7/R10): a shared DTO project becomes a coupling magnet — two features start sharing a DTO "to avoid duplication," and now they can't change independently or be deleted independently. A slice owns its request/response shapes; if two slices need a similar shape, they each have their own. The tiny duplication buys total independence — which is what makes the delete-me property real.

Note too that `NoteResponse` is a *projection*, not the entity: the API exposes `Id/Title/Content/CreatedAt`, deliberately omitting `TenantId` and `UpdatedAt`. The wire contract is chosen, never "just serialize the entity" — the entity is free to grow fields the API doesn't expose.

6. The contributor — and the escape-hatch boundary (step 5)

```
// src/Api/Features/Notes/NotesDataContributor.cs
public class NotesDataContributor(IRepository<Note> notes) : ITenantDataContributor
{
    public Task<bool> HasDataAsync(Guid tenantId, CancellationToken ct = default) =>
        // QueryAllTenants: dissolve runs for a tenant OTHER than the current one, so this
        // crosses tenants by design — the audited escape hatch, re-constrained to the
target.
        notes.QueryAllTenants().AnyAsync(n => n.TenantId == tenantId, ct);

    public async Task WipeAsync(Guid tenantId, CancellationToken ct = default) =>
        await notes.QueryAllTenants().Where(n => n.TenantId ==
tenantId).ExecuteDeleteAsync(ct);

    public string ExportKey => "notes";

    public async Task<object?> ExportAsync(Guid tenantId, CancellationToken ct = default)
=>
        await notes.QueryAllTenants().Where(n => n.TenantId == tenantId)
            .OrderBy(n => n.CreatedAt)
            .Select(n => new { n.Id, n.Title, n.Content, n.CreatedAt, n.UpdatedAt })
            .ToListAsync(ct);
}
```

Registering this (one line in `Program.cs`) is *all* it takes for `Notes` to participate in dissolve (2.9) and GDPR export (7.1) — no central method to edit. That's Open/Closed again, from the feature's side.

Now the subtle, precise thing this file teaches — the one I want you to carry into every slice. **The contributor uses `QueryAllTenants()`, and that is allowed here and *only* here.** Dissolve and export run for a tenant *other* than the ambient one (the system context has no current tenant), so they genuinely must cross tenants — hence the escape hatch, immediately re-constrained with `Where(n => n.TenantId == tenantId)`. But the arch test (3.3) enforces a three-part boundary that's worth stating exactly:

- Raw `IgnoreQueryFilters()` is banned **everywhere** in `Features/**` — even the contributor must go through the named `QueryAllTenants()`, never the raw EF call.
- `QueryAllTenants()` is permitted **only in `*DataContributor.cs`** — an *endpoint or handler* that calls it fails the build (v2 audit ADV-2). The request path has no exit.
- So `NotesHandler` (request path) uses `Query()`; `NotesDataContributor` (teardown) uses `QueryAllTenants()`. The boundary runs between the two files, and a machine patrols it.

That granularity — cross-tenant access allowed for teardown, forbidden for serving requests — is what mature tenant isolation looks like: not "never cross tenants" (some operations must) but "cross them only in named places a reviewer and a test can find."

7. Wiring and isolation — the rules that keep slices independent

Registration, the whole of step 4/5, in `Program.cs`:

```
builder.Services.AddScoped<NotesHandler>();
builder.Services.AddScoped<ITenantDataContributor, NotesDataContributor>();
// ... after app is built:
app.MapNotes();
```

And the isolation rules that make a slice a *unit* (R7–R9, each an arch test in 3.3):

- **A slice may not reference another slice's namespace** (R7) — `Features/Notes` can't

reach into `Features/Billing`. Slices compose only through the platform, never through each other, so any one can be deleted without breaking the rest.

- **Only `Program.cs` imports `Features.*`** — the composition root wires slices; nothing

else knows they exist.

- **Platform tests must not depend on the `Notes` sample** — the reference slice is

deletable *including from the test suite's assumptions*. (There's literally an arch test listing `Set<Note>`, `IRepository<Note>`, etc. as banned patterns outside the Notes tests themselves.)

Which is the last thing to internalize: `Notes` is marked □□ **DELETE-ME** in every file. It is a *worked example*, not a feature — you copy its shape for your first real slice and then delete it (Lesson 9.1's checklist includes the deletion, and the arch tests above guarantee nothing secretly depended on it). The reference implementation ships the teacher and a promise that the teacher can leave.

8. Red — the slice's tests

```
Scenario: notes are tenant-isolated end to end
  Given tenant A created a note and tenant B created a note
  When A lists /api/notes
  Then A sees only A's note
  And A deleting B's note id returns 404 (not 403 — it's invisible, not forbidden)

Scenario: a note requires a title
  When A posts a note with a blank title
  Then the response is 400 invalid_request

Scenario: the feature group requires auth
  When an unauthenticated caller hits /api/notes
  Then the response is 401
```

`NotesSliceTests` drives the behavior; `FeatureAuthorizationTests` proves the group is guarded (the auth-by-construction claim, tested). The isolation scenario is the one that matters — and note the assertion is **404, not 403**: cross-tenant access is invisible, per §4.

9. Run it

```
dotnet test tests/Api.Tests --filter "Notes|FeatureAuthorization"
dotnet run --project src/Api
# .http: POST /api/notes {"title":"first"} → 201 ; GET /api/notes → your note
```

10. Architecture Decision

ARCHITECTURE DECISION

The fork: how are app features structured? (a) Controllers alongside the platform's; (b) a service/repository layer cake shared across features; (c) self-contained vertical-slice folders (endpoints + handler + DTOs + contributor), minimal-API groups, DTOs co-located, one DI registration.

Chosen: (c) (ADR-004). A slice is independently understandable, independently testable, and independently *deletable*; it reuses the chassis and edits no platform code; `MapTenantFeatureGroup` makes auth unforgettable; the contributor plugs into lifecycle without a central method. The isolation arch tests (R7–R9) keep the independence real rather than aspirational.

Rejected: (a) — controllers work, but the platform reserves them for its durable surface and gives features the lighter minimal-API shape with the auth guarantee baked into the group helper; mixing both freely would blur "platform vs feature." (b) — shared layers are where coupling accretes: the shared DTO, the shared service "just this once," until slices can't move independently. Co-location's small duplication is the price of independence, and it's worth it.

The trade: minimal-API groups are less ceremonious than controllers (no attribute routing, model binding is more manual) and DTO co-location duplicates similar shapes across slices. Both accepted deliberately: the first keeps features light, the second keeps them independent. The platform's own controllers remain for the cases that want the heavier machinery.

11. Checkpoint

```
git add -A && git commit -m "feat: Notes reference slice — the vertical-slice shape every feature copies"
git tag lesson-3.2
```

You should now be able to: recite the seven-step add-a-slice checklist; explain why `MapTenantFeatureGroup` exists and what R6 prevents; describe how a handler stays tiny by reusing `Query()`/`ICurrentTenant`; state the request-path vs contributor escape-hatch boundary precisely; and justify co-located DTOs and the delete-me convention.

Next: *Lesson 3.3 — Injected clocks & the architecture tests* — the `TimeProvider` you've been quietly using gets its rationale, and the arch-test project that has been enforcing all these rules gets built for real.

Lesson 3.3 — Injected clocks & the architecture tests

> **Part 3 · The slice pattern & the UI.** Two threads that have been running quietly since Part 1 finally get their own lesson. First, the **injected clock**: every service you've written took a `TimeProvider` instead of calling `DateTime.UtcNow`, and it's time to say why — and to ban the alternative with a test. Second, the **architecture- test project**: throughout Parts 1–2 I kept promising "an arch test enforces this in 3.3." This is 3.3. We build the file that has been the invisible enforcer of every structural rule — and establish the pattern by which it *grows a test per rule* for the rest of the course.

Goal: all server code takes an injected clock; a build-failing test bans ambient `DateTime.UtcNow` in server projects; and the `ArchitectureTests` project exists, holding the invariants Parts 1–3 established, ready to accrete more.

Concepts & principles: deterministic time as a dependency; `TimeProvider` / `FakeTimeProvider`; architecture tests (source scans + model reflection); invariants as executable, build-failing assertions; audit findings hardened into permanent guards.

Maps to: R15 (+ R2, R6–R10, R13, R24, R34/R35 — the whole arch suite) · repo: `tests/Api.Tests/ArchitectureTests.cs`, and every service that injects `TimeProvider`.

Prerequisites: the Part-1 grep-gates (config sync 1.4, MailKit containment 2.3, error shape 1.5) — this lesson generalizes them; and 2.6/3.2 whose rules it enforces.

---

1. Motivate — the clock you can't test

Cast back to Lesson 2.2. You needed to prove a refresh token is rejected *after* it expires. How? With `DateTime.UtcNow` buried inside the service, your options are: sleep for the token's lifetime in the test (absurd — a 30-day token?), or monkey-patch a static (you can't, cleanly, in C#). Time-dependent logic built on ambient `UtcNow` is **structurally untestable** — the one input that determines the outcome is a global you can't control from the test.

That's the acute pain. The chronic one is subtler: ambient `UtcNow` scattered across a codebase means *many* clocks. A token minted reading one `UtcNow`, validated against another, logged with a third — and in a distributed system, on different machines with different skew. "What time is it" should be *one* answer, injected, so every collaborator in a request shares it. The fix is to make time a dependency like any other.

2. `TimeProvider` — time as a seam

.NET's `TimeProvider` (built into the modern BCL) is the abstraction. Inject it; call `GetUtcNow()`. You've been doing this since 2.1 without comment — here it is in the `NotesHandler` you just built:

```
public class NotesHandler(IRepository<Note> notes, ICurrentTenant tenant, TimeProvider
clock)
{
    public async Task<NoteResponse?> CreateAsync(CreateNoteRequest request,
CancellationToken ct)
    {
        var now = clock.GetUtcNow(); // ← the injected clock, not DateTimeOffset.UtcNow
        var note = new Note { /* ... */ CreatedAt = now, UpdatedAt = now };
        // ...
    }
}
```

In production you register the real one — `services.AddSingleton(TimeProvider.System)` — and `GetUtcNow()` returns the wall clock. In tests you inject `FakeTimeProvider` (from `Microsoft.Extensions.TimeProvider.Testing`, the package your test project already references), and time becomes a value you *control*:

```
var clock = new FakeTimeProvider(DateTimeOffset.Parse("2026-01-01T00:00:00Z"));
var svc = new PasswordlessService(/* ... */, clock);
var code = await svc.IssueOtpAsync("a@b.test", ct); // issued "now"

clock.Advance(TimeSpan.FromMinutes(11)); // jump past the 10-min lifespan

var result = await svc.RedeemOtpAsync("a@b.test", code, ct);
Assert.Equal(OtpStatus.Expired, result.Status); // deterministic, instant
```

Every expiry test you met in Part 2 — refresh-token expiry (2.2), OTP lifespan and lockout window (2.4), magic-link expiry — works *because* of this. `Advance` makes "wait eleven minutes" happen in microseconds, with no flake. Deterministic time isn't a testing nicety bolted on; it's what makes an entire category of logic testable at all.

3. Green — ban the alternative

A convention followed everywhere is one `DateTime.UtcNow` away from a hole. So the platform makes it a build failure, from the real `ArchitectureTests`:

```
[Fact]
public void ServerServices_UseInjectedClock_NotAmbientUtcNow()
{
    // R15: server code takes TimeProvider, so a cookie/URL/token lifetime can't drift
    // from the injected clock. Entity default-value helpers and the WASM client (no
    // injected clock) live in Core/Shared.Ui and are outside this scan; migrations
excluded.
    var dirs = new[] { Path.Combine(RepoRoot(), "src", "Api"),
                       Path.Combine(RepoRoot(), "src", "Infrastructure") };

    var offenders = dirs.SelectMany(d => SourceFiles(d))
        .Where(f =>
!f.Contains($"{Path.DirectorySeparatorChar}Migrations{Path.DirectorySeparatorChar}"))
        .Where(f => File.ReadAllText(f) is var t &&
                    (t.Contains("DateTime.UtcNow") || t.Contains("DateTimeOffset.UtcNow")))
        .Select(Path.GetFileName)
        .ToList();

    Assert.True(offenders.Count == 0,
        $"Server services must use an injected TimeProvider, not ambient UtcNow:
{string.Join(", ", offenders)}");
}
```

Write `DateTimeOffset.UtcNow` in any Api or Infrastructure file and the build goes red with the file's name. But read the comment's *scope*, because the exclusions are where the real thinking is — a blunt "ban UtcNow everywhere" would be wrong:

- **Core entity defaults are exempt.** `Note.CreatedAt` isn't set by a default helper, but

some entities use `= DateTimeOffset.UtcNow` as a property initializer. Core has no request, no DI, no injected clock — so a *default* there is acceptable; what matters is that the **service** overwrites it with `clock.GetUtcNow()` on the write path (as `NotesHandler` does). The rule targets the code that *makes decisions* from time.

- **The WASM client is exempt.** The Blazor client (Shared.Ui) runs in the browser with

no server clock to inject; "now" there is display, not a security decision.

- **Migrations are exempt.** Generated code, timestamped by tooling.

That list of exclusions is the difference between a rule people respect and one they route around. A guard that fires on legitimate cases gets suppressed; a guard scoped to exactly the risk (server-side *decisions* from time) stays trusted. You saw this instinct in 1.6's prohibited-list license gate — same discipline, applied to time.

4. Zoom out — the architecture-test project

That clock test is one of a family. `ArchitectureTests.cs` is where the platform's structural guarantees stop being prose in `CLAUDE.md` and become assertions the build runs. It has been referenced in nearly every lesson — "an arch test enforces this in 3.3" — and now you build it. Two mechanisms, both of which you've already met in miniature:

Source scans — grep with a verdict. The exact shape of 1.4's config-sync gate, 2.3's MailKit containment, 1.5's error-shape check. Read files, look for a forbidden (or required) pattern, fail with the offending file name. The tenant-filter-bypass test (the three-part `IgnoreQueryFilters` / `QueryAllTenants` / `RlsTags` rule from 3.1–3.2), the "only `Program.cs` imports `Features.*`"

isolation rule (R8), "features use `MapTenantFeatureGroup` not raw `MapGroup`" (R6), "each file declares a type matching its name" (R24) — all source scans.

Model reflection — ask the EF model about itself. This is the more powerful kind, because it covers entities *that don't exist yet*. The keystone example:

```
[Fact]
public void EveryTenantScopedEntity_HasAGlobalQueryFilter()
{
    using var ctx = new AppDbContext(modelOnlyOptions, new TestCurrentTenant());

    var missing = ctx.Model.GetEntityTypes()
        .Where(e => typeof(ITenantScoped).IsAssignableFrom(e.ClrType))
        .Where(e => e.GetDeclaredQueryFilters().Count == 0)
        .Select(e => e.ClrType.Name)
        .ToList();

    Assert.True(missing.Count == 0,
        $"ITenantScoped entities missing the global tenant query filter: {string.Join(", ",
missing)}");
}
```

This is 2.6's R2 invariant, and it's *self-extending*: add an `ITenantScoped` entity that somehow escapes the filter, and it names itself in the failure — no one has to remember to add a test for the new entity, because the test iterates the model. The same technique guards distinct table mappings (R35), unique export keys across contributors (R13), unique route prefixes (R35), and — arriving with their epics — that every user-keyed entity is wired into GDPR erasure (7.1) and every controller derives from a tenant/admin base (6.1).

5. The philosophy — invariants that can't regress

Here's the throughline of the entire course, stated once. Every structural guarantee in this platform exists at three levels: **prose** (the ADR / `CLAUDE.md` — the *why*), **a mechanism** (the query filter, the interceptor, the seam — the *how*), and **a test** (the arch test — the *proof it stays true*). The prose can be ignored; the mechanism can be bypassed; only the test, wired into 1.6's CI, makes the guarantee *permanent*.

Look at where these tests came from. Several cite `v2 audit ADV-1`, `ADV-2`, `TR-1` in their comments — they are **audit findings turned into guards**. Someone once found that a rotated-out row could be updated cross-tenant (2.7's ADV-1), or that a handler could call `QueryAllTenants` (3.2's ADV-2); the fix was code, but the *durable* fix was a test so the hole can never silently reopen. That is what "the architecture-test project grows a test per rule" means in practice: every time the platform learns something — from an audit, from a bug, from a new epic's invariant — the lesson is recorded as an executable assertion. Build the file now with the Part 1–3 rules; every part after this adds its own alongside the feature that motivates it.

6. Run it

```
dotnet test tests/Api.Tests --filter Architecture
# the whole structural suite, green
```

Then feel two of them bite (pain now, trust forever): write `DateTimeOffset.UtcNow` in a handler → the clock test names your file; add a raw `app.MapGroup("/api/x")` in a `Features/` file → the R6 test names it. Delete both; green returns. These failures are the platform refusing to let a guarantee rot.

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how are structural rules enforced? (a) Documentation + code review; (b) a dedicated architecture-testing library (NetArchTest, ArchUnitNET); (c) plain xUnit tests using source scans + EF-model reflection.

Chosen: (c). No new dependency (the scans are `File.ReadAllText` + a `Contains`; the model checks are `ctx.Model.GetEntityTypes()`), the model-reflection tests are *self-extending* (they cover future entities for free), and every rule reads as a plain test with a failure message that names the offender and the fix. They run in the same `dotnet test` as everything else, gated by CI.

Rejected: (a) — review is human and forgets; the whole point is guarantees that *can't* be forgotten. (b) — capable libraries, but they add a dependency and a second DSL to learn, and for the mostly-textual and EF-model-shaped rules here, a `Contains` scan is more transparent than a fluent `Types().That().ResideInNamespace()` chain. A larger codebase with deep layering rules might reasonably choose (b).

The trade: source scans are textual — they match strings, so they can be fooled (a `QueryAllTenants` in a comment trips the scan; an obfuscated call might slip it) and they're coarser than a real type analysis. Accepted: these guard against *accidents and drift*, not a determined adversary editing around them, and the model-reflection tests (which use real type info) cover the highest-stakes rules precisely.

8. Checkpoint

```
git add -A && git commit -m "test(arch): injected-clock ban + the architecture-test project (invariants as tests)"
git tag lesson-3.3
```

You should now be able to: explain why ambient `UtcNow` is structurally untestable and how `TimeProvider`/`FakeTimeProvider` fix it; justify the clock ban's *exclusions* as carefully as the ban; distinguish source-scan from model-reflection arch tests and say which is self-extending; and articulate the prose→mechanism→test trichotomy that makes a guarantee permanent.

Next: *Lesson 3.4 — The web client & auth UI* — the first pixels. Blazor WebAssembly, the shared RCL, and the client that consumes everything Part 2 built over HTTP.

Lesson 3.4 — The web client & auth UI

> **Part 3 · The slice pattern & the UI.** The first pixels. Everything so far has been API — testable, headless, invisible. Now we build the Blazor WebAssembly client that consumes it, and three architectural decisions dominate the lesson: **UI lives in a shared class library** (not the web app), so non-web clients get it for free; the **access token lives in memory only**, with sessions surviving reloads via the silent refresh that completes 2.2's rotation story; and the two things that differ between web and native are hidden behind **seams designed now, web-first**, so the MAUI shells in the appendix are cheap. The client is just another API consumer — held to the same clean-boundary rule as any other.

Goal: a Blazor WASM app where an unauthenticated visitor lands on a login page, signs in (magic link / OTP / OAuth) against the Part-2 API, stays signed in across reloads, and sees their notes — with every component living in the shared RCL.

Concepts & principles: Blazor WASM + the shared RCL (why UI-in-a-library); the client as an API client (clean boundary); in-memory access token + silent refresh; per-host seams (`ISessionStore`, `IOAuthInitiator`) designed web-first; delegating handlers and a DI-cycle-avoiding client split; single-origin readiness.

Maps to: ADR-C4/C5, Golden Rule 3 · repo: `src/Web/Program.cs`, `src/Web/Http/CookieHandler.cs/AuthHeaderHandler.cs/CookieSessionStore.cs`, `src/Shared.Ui/Auth/AuthService.cs/ISessionStore.cs/IOAuthInitiator.cs/AppClaims.cs`, `src/Shared.Ui/App.razor`, `Layout/MainLayout.razor`, `Pages/Login.razor`, `Pages/AuthCallback.razor`, arch test `WebApp_HasNoInlineBlazorComponents`.

Prerequisites: Part 2 (the whole auth API), 3.2 (the Notes API the UI will show).

---

1. Motivate — where does the UI live?

The obvious place for a Blazor page is the Blazor web app. The platform puts it somewhere else — a **Razor Class Library** (`Shared.Ui`) — and the web app becomes a nearly-empty host. Why fight the obvious?

Because of a promise made in Lesson 0.1: this platform ships (eventually) desktop and mobile shells (MAUI), and *web-first* means "build the feature on web, then extend the native shells cheaply." That's only possible if the native shells can *reuse the actual UI*. If your login page, your notes list, your settings screen live in the web app, the MAUI apps must reimplement all of it. If they live in a class library, every host — web and native — references the same components and gets the same UI for free. So the rule (Golden Rule 3): **Blazor components live in the shared RCL, never inline in the web app.** The web app carries only bootstrap markup.

This is enforced, of course — you built the mechanism in 3.3:

```
[Fact]
public void WebApp_HasNoInlineBlazorComponents()
{
    var allowed = new HashSet<string> { "_Imports.razor", "App.razor", "Routes.razor" };
    var inline = SourceFiles(Path.Combine(RepoRoot(), "src", "Web"), "*.razor")
        .Select(Path.GetFileName).Where(name => !allowed.Contains(name!)).ToList();
    Assert.True(inline.Count == 0,
        $"Blazor components belong in Shared.Ui, not the web app: {...}");
}
```

Put a `.razor` component in the web app and the build fails, naming it. The web-first promise is kept by a test, not a habit.

2. The client is an API client — the clean boundary, from the outside

The web app talks to the API the same way any third party would: HTTP + JSON, bearer token, no shortcuts. It has **no reference to Infrastructure, no `DbContext`, no shared entity types** — it can't; the project references forbid it (1.1). This is the clean API boundary (Golden Rule 2) seen from the client's side: the UI cannot reach around the API to the database even if a developer wanted to, because the types simply aren't there. When the Notes list renders, it does so from a `GET /api/notes` and a DTO it deserializes locally — never from a `Note` entity.

That constraint is a feature. It means the API's contract is the *only* contract, the same one the public API (7.3) and the native shells honor. There is no privileged client.

3. The session, client-side — memory + silent refresh

Here's the decision that surprises people: **the access token is never persisted.** Not in `localStorage`, not in `sessionStorage`, not anywhere on disk. From `AuthService`'s own summary:

> The access token is held in memory only — never persisted. Sessions survive reloads/restarts via the refresh token: on startup we silently exchange it for a fresh access token.

Why refuse to store the access token? Because `localStorage` is readable by any JavaScript on the page — one XSS and your bearer token is exfiltrated. Keeping it in a private field means a reload *loses* it — which sounds like a bug until you remember 2.2: the durable credential is the **HttpOnly refresh cookie**, which JavaScript *cannot* read and the browser *does* persist. So the session model is:

```
// src/Shared.Ui/Auth/AuthService.cs (trimmed to the web path)
private string? _accessToken; // memory only
private Task<bool>? _refreshInFlight;

public bool IsAuthenticated => !string.IsNullOrEmpty(_accessToken) &&
!IsTokenExpired(_accessToken);

/// Resolves the session on app startup: with no token in memory, silently exchange the
/// HttpOnly refresh cookie for an access token. Called ONCE from MainLayout.
public async Task InitializeAsync()
{
    if (IsAuthenticated) return;
    await TryRefreshAsync();
}

public Task<bool> TryRefreshAsync() => _refreshInFlight ??= RunRefreshAsync();

private async Task<bool> RunRefreshAsync()
{
    // Web: the CookieHandler attaches the HttpOnly refresh cookie; no body.
    var response = await httpClient.PostAsync("/api/auth/refresh", null);
    if (!response.IsSuccessStatusCode) { await ClearSessionAsync(); return false; }
    var payload = await response.Content.ReadFromJsonAsync<TokenResponse>();
    _accessToken = payload!.AccessToken; // fresh token, in memory
    return true;
}
```

Trace a page reload: memory is wiped, `_accessToken` is null, but the refresh cookie survived. `MainLayout` calls `InitializeAsync` before rendering the body; it silently `POST /api/auth/refresh`; the cookie rides along (via the handler in §4); the API rotates the token (2.2) and returns a fresh access token; the app is signed in again — invisibly. The user never sees a login screen they didn't ask for. This is the *client half* of the rotation dance you built server-side in 2.2; now you can see why the refresh token was worth all that care.

One subtlety worth its own sentence — `TryRefreshAsync` coalesces concurrent calls into one in-flight `Task`:

```
public Task<bool> TryRefreshAsync() => _refreshInFlight ??= RunRefreshAsync();
```

Because refresh **rotates** (2.2), two overlapping refreshes would race: the first burns the cookie, the second presents the now-revoked token and trips reuse detection — logging the user out of their own session. Sharing one `Task` makes concurrent callers await the same refresh. It's a direct consequence of a server-side decision rippling into client design — exactly the kind of cross-layer reasoning this course exists to build.

4. The HTTP plumbing — two clients, one subtle DI cycle

The client needs two things attached to its requests: the **refresh cookie** (for `/refresh`) and the **bearer token** (for everything authenticated). Delegating handlers do this. But there's a trap, and the platform's comment documents the escape:

```
// src/Web/Program.cs
// "Api" — general-purpose client used by components: credentials + Bearer token.
builder.Services.AddHttpClient("Api", c => c.BaseAddress = new Uri(apiBase))
    .AddHttpMessageHandler<CookieHandler>() // sends the HttpOnly cookie
    .AddHttpMessageHandler<AuthHeaderHandler>(); // attaches the in-memory JWT

// "ApiAuth" — used ONLY by AuthService for refresh/logout. Credentials only (no Bearer
// handler) to avoid a DI cycle (AuthHeaderHandler depends on AuthService).
builder.Services.AddHttpClient("ApiAuth", c => c.BaseAddress = new Uri(apiBase))
    .AddHttpMessageHandler<CookieHandler>();
```

The cycle: `AuthHeaderHandler` needs `AuthService` (to read the token), but `AuthService` needs an `HttpClient` (to call `/refresh`). If `AuthService`'s client carried the bearer handler, resolving it would loop. The fix is two clients: components use `"Api"` (cookie + bearer); `AuthService` uses `"ApiAuth"` (cookie only — it doesn't need a bearer to refresh, the cookie *is* its credential). Recognizing and breaking a DI cycle is a rite of passage; here it's a two-line decision with a comment so the next person doesn't "simplify" it back into a loop.

`CookieHandler` itself is tiny but non-obvious — WASM must *opt in* to sending cookies:

```
public class CookieHandler : DelegatingHandler
{
    protected override Task<HttpResponseMessage> SendAsync(HttpRequestMessage request,
CancellationToken ct)
    {
        request.SetBrowserRequestCredentials(BrowserRequestCredentials.Include);
        return base.SendAsync(request, ct);
    }
}
```

Without `Include`, the browser withholds cookies on the fetch and every refresh silently fails — the kind of one-liner that costs an afternoon if you don't know it exists.

(A registration detail with a real reason: `AuthService` is a **singleton**. `IHttpClientFactory` resolves handlers in their own DI scope, so a *scoped* `AuthService` would hand the bearer handler a different instance with an empty token — and the header would never attach. One token store, shared by app and handler, requires singleton. The comment in `Program.cs` spells this out; it's a subtle Blazor-DI fact worth filing away.)

5. The per-host seams — designed web-first, native for free

Two things genuinely differ between the web browser and a native MAUI shell, and the platform isolates *exactly those two* behind interfaces — introduced now, on web, so the appendix's native shells only implement the interface rather than re-architect the app.

Where the refresh token lives — `ISessionStore`:

```
// src/Shared.Ui/Auth/ISessionStore.cs
/// Abstracts where the refresh token lives, the one thing that differs between hosts.
public interface ISessionStore
{
    bool UsesBodyTransport { get; } // web: false (cookie) · native: true (body)
    Task<string?> GetRefreshTokenAsync(); // web: always null · native: from secure store
    Task SaveRefreshTokenAsync(string token); // web: no-op · native: persist rotated token
    Task ClearAsync(); // web: no-op · native: clear
}
```

On web, the browser owns the HttpOnly cookie, so the store is a **no-op** and `UsesBodyTransport` is false — the refresh call carries no body and relies on the cookie (`CookieSessionStore` in the web project). On native (appendix), there's no shared cookie jar, so the token lives in the OS secure store and travels in the request body. Look back at `RunRefreshAsync`: the branch on `sessionStore.UsesBodyTransport` is the *entire* web-vs-native difference in the auth flow, and it's one `if`. That's the payoff of finding the true seam.

How interactive OAuth starts — `IOAuthInitiator`: on web, "Sign in with Google" is a *full-page navigation* to the API's challenge URL (the browser handles the whole OAuth round-trip and returns to the app), so the web host **never registers or resolves** this service. On native (appendix), there's no full-page nav, so an implementation opens the system browser and captures the redirect. Designing the seam now, while building web, is what "web-first" operationally means — not "ignore native," but "shape the abstractions so native slots in."

A `warnings-as-errors` wrinkle worth internalizing (it bites the moment you try this): define the **interface** now — that's the seam — but do *not* yet add a constructor parameter for it to `AuthService`. The full reference takes `IOAuthInitiator? oauth = null` because its native methods *use* it; at this web-only stage those methods don't exist, so an injected-but-unread parameter trips **CS9113 ("parameter is unread") → build failure** under 1.1's `TreatWarningsAsErrors`. The parameter returns in the appendix, alongside the native code that reads it. The lesson: a seam is an *interface you define early*, not necessarily a *dependency you inject early* — inject it when something consumes it, or the compiler (rightly) objects to the dead wire.

6. The pages — login and the callback

Two components carry the auth UX (both in the RCL, both routable):

- **`Login.razor`** renders what the *deployment* supports: an email box for magic-link /

OTP, and OAuth buttons for exactly the providers `GET /api/auth/providers` reports (2.5's `EnabledAsync` — so an unconfigured provider shows no dead, 500-ing button; the UI fails closed to fewer options). Magic link and OTP post to the Part-2 endpoints (2.4); OAuth buttons do the full-page navigation of §5.

- **`AuthCallback.razor`** is where an OAuth (or magic-link) round-trip lands back in the

SPA. Because the browser did the redirecting, the app "boots fresh" here — so the callback simply runs the same `TryRefreshAsync` (the API set the refresh cookie during the callback), obtains an access token, and routes onward. The whole external-flow return reuses the silent-refresh machinery; there's no separate "accept the OAuth token" path on web.

One detail on the login page repays a callback to 2.4. When a sign-in *fails*, the client must choose what to say — and it must not undo the server's enumeration safety. A tiny shared helper decides:

```
// src/Shared.Ui/Auth/AuthErrorCopy.cs — maps a server error code to the copy key to show
public static string OtpErrorKey(string? errorCode) =>
    errorCode == "too_many_attempts" ? "Login_ErrTooManyAttempts" :
"Login_ErrCodeIncorrect";
```

Recall 2.4: the server deliberately collapses "wrong code" and "no active code" into one generic `invalid_code` (enumeration safety — CONF-6), but reports lockout as a *distinct* `too_many_attempts` (CONF-5, so the user learns retrying won't help until the window elapses). The client mirrors exactly that shape: lockout gets its own message, everything else stays on the deliberately-generic line — it does **not** invent a more specific "wrong code" message the server refused to give, which would leak the very distinction the server hid. The helper is shared across *every* sign-in surface (web now, native OTP in the appendix) so they can't drift into disagreeing about what an error means. (The two copy keys land as localized strings in 3.5.)

`MainLayout` is the single auth entry point: it `await`s `InitializeAsync` before rendering `@Body`, so every page below it can assume the session is resolved — no page triggers its own refresh, which (with the coalescing in §3) keeps rotation sane.

7. Single-origin, already

One line in `Program.cs` quietly prepares for deployment (Part 8):

```
var apiBase = builder.Configuration["ApiBaseUrl"] is { Length: > 0 } configured
    ? configured : builder.HostEnvironment.BaseAddress; // fall back to our own origin
```

In dev, `ApiBaseUrl` points the client at the API on a different port. In production (8.1), the API *serves this very bundle*, so the client and API share one origin and the fallback kicks in — same-origin, which deletes an entire class of cross-site-cookie pain (the reason 2.2's cookie fussed over `SameSite`). The client is written origin-agnostic so the deployment topology is a config value, not a code change.

8. Run it — the first end-to-end human flow

No new unit tests here (UI behavior is proven by the E2E harness, next lesson) — but this is the first lesson you *run and click*:

```
docker compose up -d
dotnet run --project src/Api # terminal 1
dotnet run --project src/Web # terminal 2 (dev: different port)
# browse to the Web dev URL → Login → request an OTP → read it from Mailpit (:8025)
# → you're in, you see your (empty) notes → reload the page → STILL in (silent refresh)
```

That reload-and-stay-signed-in moment is the whole session design working in front of you: memory was wiped, the cookie survived, the refresh was invisible.

9. Architecture Decision

ARCHITECTURE DECISION

The fork (two forks, really): (rendering) Blazor WebAssembly, Blazor Server, or a separate JS SPA framework? (token storage) access token in `localStorage`, or in memory with silent refresh?

Chosen: Blazor WASM + a shared RCL, with the access token in memory (ADR-C4/C5, Golden Rule 3). WASM keeps the UI a *pure API client* (no server-side render state, honors the clean boundary, and — decisively — the same C# components run in the MAUI shells). In-memory tokens + the HttpOnly refresh cookie put the durable credential where XSS can't reach it, at the cost of a silent refresh on startup — a cost the rotation design already required.

Rejected: *Blazor Server* — a stateful circuit per user, a persistent WebSocket, and a server that renders UI: it breaks "the UI is just a client," complicates scaling, and can't be a MAUI shell. *A JS SPA* (React/etc.) — fine technically, but a second language and toolchain for a .NET team, and no code-sharing with native. *`localStorage` tokens* — the standard footgun: convenient, and one XSS from total credential theft.

The trade: WASM ships a .NET runtime to the browser (a larger first load than a lean JS app) and needs the silent-refresh choreography (§3) instead of a token just sitting in storage. Both accepted: the download is one-time and cached, and the choreography buys XSS-resistant sessions plus one codebase across every client.

10. Checkpoint

```
git add -A && git commit -m "feat: web client + auth UI — RCL components, in-memory token + silent refresh, host seams"
git tag lesson-3.4
```

You should now be able to: explain why UI lives in the RCL and how the arch test keeps it there; describe the in-memory-token + silent-refresh model and connect it to 2.2's rotation; identify and break the AuthHeaderHandler↔AuthService DI cycle; and name the two genuine web-vs-native differences and the seams (`ISessionStore`, `IOAuthInitiator`) that isolate them.

Next: *Lesson 3.5 — Localization* — the UI (and the emails from 2.3) learn to speak more than one language, the right way.

Lesson 3.5 — Localization

> **Part 3 · The slice pattern & the UI.** A short but genuinely architectural lesson. "Support Spanish" sounds like a translation task; it's actually three separate decisions — where strings live, how the app *decides* which language to show, and how that decision survives a cold start on each host. Do it now, while the UI is small, and every future component is born translatable. Retrofit it later and you're hunting hard-coded English through fifty files. And there's a satisfying payoff: the emails you built in 2.3 already localized themselves; this lesson makes the *UI* rhyme with them.

Goal: the UI renders in English or Spanish from resource files; a language switcher persists the choice per host; a signed-in user's saved locale follows them across devices; and adding a language is adding a `.resx`.

Concepts & principles: externalized strings (resx + `IStringLocalizer`); the culture decision chain (stored → user claim → default); the web-first culture-persistence seam; UI vs email localization (ambient culture vs explicit culture); fail-soft culture resolution.

Maps to: ADR-C(i18n) · `docs/LOCALIZATION.md` · repo: `src/Shared.Ui/Resources/AppStrings.cs` + `AppStrings.resx`/`AppStrings.es.resx`, `src/Shared.Ui/Components/LanguageSwitcher.razor`, `ICulturePersistence.cs`, `LocalStorageCulturePersistence.cs`, `src/Web/Program.cs` (culture bootstrap), `src/Core/Entities/User.cs` (`Locale`), tests: `tests/E2E.Tests/I18nTests.cs`.

Prerequisites: 3.4 (the client + `MainLayout`), 2.1 (`User.Locale`), 2.3 (localized emails).

---

1. Motivate — the hard-coded string is a one-way door

Write `<h1>Sign in</h1>` in a component and you've made a decision you didn't know you were making: this app is English-only, and undoing it means finding every literal across the codebase. Externalize from the first component and the marginal cost of a second language is a translation file — no code changes. The choice is *when* you pay: a little now (look strings up by key), or a lot later (a codebase-wide string hunt). i18n is cheap only if it's early, which is why it's a Part-3 lesson and not a "someday" ticket.

2. Where strings live — resx + `IStringLocalizer`

.NET's localization is resource files keyed by string, resolved through a typed marker. The marker is a nearly-empty class whose only job is to *name* a resource set:

```
// src/Shared.Ui/Resources/AppStrings.cs — the whole file
/// <summary>Marker type for the app's shared UI strings. Inject
/// IStringLocalizer&lt;AppStrings&gt; and look up keys (e.g. Localizer["Login_Title"]).
/// Translations live in co-located AppStrings.resx (English) +
AppStrings.{culture}.resx.</summary>
public sealed class AppStrings;
```

Beside it: `AppStrings.resx` (neutral/English) and `AppStrings.es.resx` (Spanish), each a key→string map — `Login_Title` → "Sign in" / "Iniciar sesión". Components inject the localizer

and look strings up:

```
@inject IStringLocalizer<AppStrings> L
<h1>@L["Login_Title"]</h1>
<button>@L["Login_SendOtp"]</button>
```

Registered once (`builder.Services.AddLocalization()` in `Program.cs`, 3.4), and — because the components live in the RCL (3.4) — the *same* strings serve web and the native shells. Adding French is adding `AppStrings.fr.resx` with the same keys; no component changes. (The reference platform ships EN + ES live, with FR/DE/PT scaffolded — `docs/LOCALIZATION.md`.)

A discipline worth adopting now: **keys are structured, not sentences.** `Login_Title`, not `"Sign in"`, as the key — so a copy change ("Sign in" → "Log in") is a resx edit, not a key rename that ripples through components. The key names the *slot*; the resx fills it.

3. The decision chain — which language, and why

"Show Spanish" requires the app to *decide* Spanish, and the platform resolves it through a deliberate chain, most-specific-wins:

1. **A signed-in user's saved locale.** `User.Locale` (2.1) rides in the JWT as the `locale` claim; `MainLayout` reconciles the running culture to it after sign-in. So a user who chose Spanish sees Spanish **on every device** — the preference is server-side, following the account, not the browser. (This is the one per-*user* preference the platform allows, per Golden Rule 4's "only preferences are per-user" — localization is its canonical example.) Setting it needs a small endpoint: when a signed-in user picks a language, the switcher `PUT`s it to a `.../locale` endpoint (validated against the supported set — an unknown code is a `400 unsupported_locale`, never silently accepted) that updates `User.Locale`, so the *next* token carries the new claim. That endpoint is the first member of the account surface that GDPR (7.1) later extends with export and erasure.
2. **A stored host choice**, for anyone not signed in (a visitor on the login page). This is the `LanguageSwitcher`'s job, and where the host seam comes in (§4).
3. **Default** (English) when nothing else is known.

The important architectural point: locale is a *user* attribute that happens to have a *host* fallback — not a browser setting the server is unaware of. That's what makes "same language across devices" work, and it's why the claim exists.

4. Surviving a cold start — the web-first culture seam

Here's the subtle part. A Blazor WASM app must apply the culture **before it renders**, or the first paint flashes English then re-renders. But *where the chosen culture is stored* differs by host — and, exactly like 3.4's `ISessionStore`, that difference is the only one, so it gets a seam:

```
// src/Shared.Ui/ICulturePersistence.cs — the whole interface
/// <summary>Persists the user's chosen UI culture so the host can re-apply it on the next
/// cold start. Each host owns a store its startup path can actually read: web writes
/// localStorage["app_culture"] (read pre-render by Web/Program.cs); MAUI writes OS
/// Preferences (read in MauiProgram — WebView localStorage doesn't exist yet when the
/// native process boots). Callers still set the in-process culture themselves; this is
/// only the durable half.</summary>
public interface ICulturePersistence
{
    Task PersistAsync(string cultureCode);
}
```

The comment is the whole lesson in miniature — read *why* the seam exists rather than just *that* it does: on native (MAUI), the WebView's `localStorage` **doesn't exist yet** when the process boots, so the culture must live in OS `Preferences`, which the native startup path *can* read before the WebView spins up. Web can use `localStorage` because its startup (`Program.cs`) runs in the browser where `localStorage` is available. Same seam pattern as 3.4: the shared `LanguageSwitcher` component calls `PersistAsync`; each host's implementation puts the value where *its* boot path will find it. The web implementation and the bootstrap:

```
// src/Web/Program.cs — apply the saved culture BEFORE the app renders
var stored = await js.InvokeAsync<string?>("localStorage.getItem",
LocalStorageCulturePersistence.StorageKey);
try
{
    var culture = string.IsNullOrWhiteSpace(stored) ? "en" : stored;
    CultureInfo.DefaultThreadCurrentCulture = CultureInfo.DefaultThreadCurrentUICulture =
new CultureInfo(culture);
}
catch (CultureNotFoundException) { /* corrupt stored value — keep the default culture */ }
```

Note the **fail-soft** `catch`: a garbage stored value (hand-edited `localStorage`, version skew) falls back to English rather than crashing the boot. Same instinct as 2.5's `ResolveCulture` for emails — an unknown culture is a shrug, never a 500. Designing this seam now, on web, is once again what makes the native shell (appendix) a matter of *one small implementation* rather than a re-architecture.

5. UI vs email localization — the same resx, two resolution models

You've now localized strings twice — emails in 2.3, UI here — and the *difference* is instructive. Both use `.resx`, but they resolve culture differently, for a good reason:

- **The UI uses `IStringLocalizer` and the *ambient* thread culture.** There's one user

looking at one browser; "the current culture" is unambiguous, so the localizer reads it implicitly. `@L["Login_Title"]` just works.

- **Emails use `ResourceManager` keyed by an *explicit* culture** (2.3's `BrandedEmail`).

A server rendering an email has no meaningful "ambient" culture — it's processing many users' requests, and the recipient's language (the requester's for an OTP, the *inviter's* for an invite) is a parameter, not an ambient fact. So emails pass the culture in.

Same translation files, two resolution strategies — because "what language is this?" has an obvious answer in a browser and a *chosen* answer on a server. Recognizing when ambient context is legitimate versus when it's a bug is a recurring judgment; here you see both correct answers side by side.

6. Red — proving it end to end

Localization is a UI behavior, so its test is an E2E one (the harness arrives fully in 3.6; `I18nTests` is its first real journey):

```
Scenario: switching language re-renders the UI and persists
  Given a visitor on the login page in English ("Sign in")
  When they choose Español in the language switcher
  Then the page shows "Iniciar sesión"
  And after a full reload it is STILL in Spanish
```

The reload assertion is the one that proves the *seam* works, not just the switch — exactly the shape of 3.4's "reload and stay signed in." A unit-ish test also guards the resx themselves: assert `AppStrings.es.resx` has a value for every key in the neutral file, so a missing translation is caught before a user sees an English word in a Spanish page. (Keys present in one culture but not another is the classic i18n rot; a test that diffs the key sets stops it.)

7. Run it

```
dotnet run --project src/Api & dotnet run --project src/Web
# Login page → language switcher → Español → UI flips → reload → still Español
# Sign in as a user who saved "es" on another browser → lands in Spanish (the claim)
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how is language chosen and stored? (a) Browser `Accept-Language` only; (b) a per-host setting (cookie/localStorage), server-unaware; (c) a per-*user* locale (server-persisted, in the JWT) with a host-stored fallback for anonymous visitors, all resolving `.resx` via `IStringLocalizer`.

Chosen: (c). Language is a user preference that should follow the account across devices (hence the claim + `User.Locale`), with a host store so a not-yet-signed-in visitor still gets their choice. `IStringLocalizer` + resx is the platform-native path, and putting components in the RCL means web and native share one string set.

Rejected: (a) — `Accept-Language` is a decent *default* but can't honor an explicit user choice and doesn't cross devices predictably. (b) — host-only settings mean a user re-picks their language on every new device and the server can't localize *emails* to them (it doesn't know their language) — which is exactly why 2.3 needed a server-side locale to begin with.

The trade: a per-user locale adds a `User` column, a claim, and a reconcile step in `MainLayout` — more moving parts than a cookie. Accepted: it's the only model where a user's language is truly *theirs*, and it's what lets emails and UI agree.

9. Checkpoint

```
git add -A && git commit -m "feat: localization — resx + IStringLocalizer, per-user locale, web-first culture seam"
git tag lesson-3.5
```

You should now be able to: externalize a string and explain why the key is structured, not a sentence; describe the culture decision chain and why locale is per-user with a host fallback; explain the `ICulturePersistence` seam and the native cold-start reason behind it; and contrast ambient-culture (UI) with explicit-culture (email) resolution.

Next: *Lesson 3.6 — The E2E harness* — Playwright, the Page Object Model, and reading real OTPs out of Mailpit to drive sign-in through a real browser.

Lesson 3.6 — The E2E harness

> **Part 3 · The slice pattern & the UI.** The third leg of the test pyramid. You have unit tests (Core logic) and integration tests (the real app + real Postgres, via `WebApplicationFactory`). Now the top: **end-to-end tests that drive a real browser against the running stack**, proving the journeys a *user* actually takes. The star technique closes a loop three lessons in the making — the E2E suite reads the OTP code out of Mailpit, exactly as you did by hand in 2.4, so a full passwordless sign-in runs browser-to-inbox-to-browser with no human. And we settle, explicitly, the question of *which* tests belong here versus in the faster layers below.

Goal: a Playwright/NUnit harness that signs a user in through the real UI + API + database, using Mailpit as an assertable inbox; a Page Object Model that keeps journeys readable; and a clear rule for what earns an E2E test.

Concepts & principles: the test pyramid and its economics; E2E with Playwright; Page Object Model; stable `data-testid` selectors; the mail-trap-as-fixture trick; choosing the right test layer (the E2E-vs-integration division of labor).

Maps to: ADR-C14 · `docs/WAYS_OF_WORKING.md` (testing strategy) · repo: `tests/E2E.Tests/E2ETestBase.cs`, `Mailpit.cs`, `Pages/BasePage.cs`/`LoginPage.cs`, `AuthFlowTests.cs`, `playwright.runsettings`.

Prerequisites: 3.4 (the running UI), 2.4 (OTP sign-in), 0.2 (Mailpit in the stack).

---

1. Motivate — the layer that catches what the others can't

Your integration tests boot the real app and prove `/api/notes` is tenant-isolated. They do *not* prove that a human can click "Sign in," receive a code, type it, and land on their notes — because they never render a pixel or run a line of the Blazor client. The wiring *between* the browser and the API — routing, the silent-refresh choreography (3.4), a `data-testid` that got renamed, a CORS misstep — lives in a gap no server-side test sees. E2E tests live in that gap: they drive a real browser through the real UI against the real API and database, asserting the *journey*, not the unit.

They are also, honestly, the most expensive tests you own — slow (a browser boots, pages load), heavier to run (the whole stack must be up), and the most prone to flake (timing, async rendering). That cost is exactly why the *last* section of this lesson is about using them sparingly. First, build one.

2. The harness — a real browser against the real stack

Playwright drives Chromium/Firefox/WebKit from .NET; the NUnit integration gives each test a fresh browser context. The base class, from the real `E2ETestBase`:

```
public abstract class E2ETestBase : PageTest
{
    protected static string BaseUrl =>
        TestContext.Parameters.Get("PLAYWRIGHT_BASE_URL")
        ?? Environment.GetEnvironmentVariable("PLAYWRIGHT_BASE_URL")
        ?? "https://localhost:7008"; // the Web app's dev https profile

    // Dev runs on a self-signed cert, so ignore HTTPS errors for the test browser.
    public override BrowserNewContextOptions ContextOptions() => new()
    {
        BaseURL = BaseUrl,
        IgnoreHTTPSErrors = true,
    };

    protected static string UniqueEmail(string role) =>
$"e2e-{role}-{Guid.NewGuid():N}@example.com";
}
```

Two decisions already: the base URL is *overridable* (a `runsettings` parameter, then an env var, then the dev default) so the same suite runs against localhost and against CI's single-origin deployment (8.3) unchanged; and every test mints a **unique email**, so tests don't collide on account state — the E2E equivalent of 1.3's per-test database reset, achieved by never sharing an identity. Unlike the integration harness, E2E tests run against a stack *you* start (`docker compose up`, then the API, then the Web app) — they're testing the deployed reality, not an in-process factory.

3. Mailpit as an assertable inbox — the loop closes

This is the technique the whole passwordless design was quietly building toward. In 2.4 you signed in by reading an OTP out of Mailpit's web UI *by hand*. The E2E suite does the same thing *programmatically*, via Mailpit's REST API — from the real `Mailpit` helper:

```
/// Minimal client for the dev Mailpit REST API so E2E tests can read the OTP code / magic
/// link the app "sends" — the same trick a manual tester uses.
public static async Task<string> WaitForOtpAsync(string toEmail, TimeSpan timeout)
{
    var deadline = DateTime.UtcNow + timeout;
    while (DateTime.UtcNow < deadline)
    {
        var code = await TryGetLatestCodeAsync(toEmail); // GET /api/v1/messages, find,
regex the 6 digits
        if (code is not null) return code;
        await Task.Delay(500);
    }
    throw new TimeoutException($"No OTP email for {toEmail} within
{timeout.TotalSeconds:0}s.");
}
```

Look back at 0.2, where we chose Mailpit specifically because it "exposes an HTTP API your tests will query." *This is that.* The email seam (2.3) sends to Mailpit; the E2E test polls Mailpit's API for the message; a regex lifts the code; the browser types it. A real sign-in, no human, no real email provider — the mail trap is a **test fixture with an API**. One production-grade detail in the real code worth noting: it filters on the *subject* ("verification code") before regexing six digits, because an invitation email (delivered late by the outbox, Part 4) to the same address could otherwise satisfy a bare `\d{6}` — a real flake, prevented by matching the message you

actually want.

4. The Page Object Model — journeys that read like intent

Raw Playwright calls (`page.Locator("#email").FillAsync(...)`) scattered through tests are unreadable and brittle — every selector duplicated, every UI tweak a shotgun edit. The **Page Object Model** wraps each screen in a class exposing intent-named locators and actions. From the real `LoginPage`:

```
public class LoginPage(IPage page) : BasePage(page)
{
    public override string Path => "/login";

    public ILocator Email => Page.GetByTestId("login-email");
    public ILocator SendOtp => Page.GetByTestId("login-send-otp");
    public ILocator OtpCode => Page.GetByTestId("login-otp-code");
    public ILocator VerifyOtp => Page.GetByTestId("login-verify-otp");

    /// Requests an OTP, reads it from Mailpit, enters it, and submits.
    public async Task SignInWithOtpAsync(string email)
    {
        await Email.FillAsync(email);
        await SendOtp.ClickAsync();
        var code = await Mailpit.WaitForOtpAsync(email, TimeSpan.FromSeconds(20));
        await OtpCode.FillAsync(code);
        await VerifyOtp.ClickAsync();
    }
}
```

Two disciplines make this durable:

- **Selectors are `data-testid`, never CSS classes or visible text.** `GetByTestId("login-email")`

binds to a stable hook the component author placed *for testing*, not to a class name a restyle will change or to the label text (which 3.5 just made *translatable* — a test keyed on "Sign in" would break the moment the UI switches to Spanish!). This is the concrete reason data-testids matter here more than in most apps: your visible text is localized, so it is *by design* not a stable selector.

- **Tests speak in page methods, not clicks.** `E2ETestBase.SignInAsync` composes

`login.GotoAsync()` + `SignInWithOtpAsync` + a wait for the app shell (`GetByTestId("sign-out")`). A test reads "sign in as this user, then…" — the *what*, with the *how* encapsulated. When the login flow changes, one page object changes, not twenty tests.

5. Red — the first journey

`AuthFlowTests` is the first E2E file — one file per epic, mirroring `docs/stories/`:

```
Scenario: a new user signs in with a one-time code (happy path)
  Given a visitor on the login page
  When they request an OTP for a fresh email and enter the code from their inbox
  Then they land on the app shell, signed in

Scenario: an expired or wrong code is rejected
  When they enter a wrong code
  Then they see the login error and remain signed out
```

That happy path exercises *everything* at once: the Blazor client, the OTP endpoints (2.4), the email seam (2.3) → Mailpit, session issuance (2.2), the silent-refresh shell (3.4). One test, the entire vertical proven to work *together* — which no lower-layer test can claim. This is what E2E is *for*.

6. The economics — what earns an E2E test (the division of labor)

Now the rule this lesson exists to establish, because it governs every part after. E2E tests are slow, heavy, and flake-prone, so they are **rationed**: reserve them for critical user *journeys*, and push everything else down to the faster, sturdier layers. `docs/WAYS_OF_WORKING.md` draws the line, and it's worth internalizing:

Belongs in E2E (Playwright)	Belongs lower (xUnit + Testcontainers)
The happy path of a critical journey (sign-in, invite-and-join, upgrade, export)	Domain logic and derived rules (Core.Tests)
A key *user-visible* unhappy path (wrong OTP shows an error)	Tenant-isolation invariants, permission/entitlement boundaries
Cross-layer wiring only a browser exercises (silent refresh, i18n reload)	Concurrency, redelivery, quota atomicity — anything needing precise control

The principle: **an xUnit test that can prove it will always beat an E2E test that can.** Tenant isolation *could* be shown through a browser — but you proved it far faster and more reliably as an integration test in 2.6, seeding two tenants directly and asserting at the wire. Reserve the browser for what *only* the browser can see. Concretely, this is why the reference platform's billing quota logic lives in `QuotaServiceTests` (xUnit, precise concurrency control) while only the *402-becomes-a-visible-upgrade-prompt* moment is an E2E journey (5.3). Same feature, two layers, each doing what it's best at.

A corollary you'll feel in later parts: when an epic was built integration-first and lacks an E2E journey, that's a deliberate ration, not an oversight — the invariants are proven below, and a browser test would add cost without adding coverage. Write the E2E for the *journey*; trust the lower layers for the *logic*.

7. Run it

First-time Playwright setup installs the browser binaries; then the suite needs the full stack up (per `tests/E2E.Tests/README.md`):

```
dotnet build tests/E2E.Tests
pwsh tests/E2E.Tests/bin/Debug/net10.0/playwright.ps1 install # once
docker compose up -d
dotnet run --project src/Api & dotnet run --project src/Web &
dotnet test tests/E2E.Tests
# AuthFlowTests: a real Chromium signs in via an OTP read from Mailpit — green
```

One gotcha the reference `.env` note calls out: OTP E2E tests need email to land in *Mailpit*, so the API must use the dev SMTP default — if your local `.env` points SMTP at a real provider (Brevo), the tests can't read the code. Override back to Mailpit for E2E runs.

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how are user journeys tested? (a) Manual QA only; (b) component tests (bUnit) rendering Blazor components in isolation; (c) full E2E with Playwright against the running stack, Page Object Model, Mailpit as the inbox — reserved for critical journeys.

Chosen: (c), *rationed* (ADR-C14). Only a real browser against the real stack proves the cross-layer journeys that break in production; the POM + `data-testid` keep the suite readable and resilient to restyles and localization; Mailpit makes passwordless flows testable without a mail provider. The rationing (§6) keeps the slow layer small.

Rejected: (a) — manual QA doesn't scale or gate CI (though a *manual QA plan* still exists for release readiness — 8.3; automation and manual QA are complements, not substitutes). (b) — bUnit tests a component's render logic in isolation, which is useful but proves nothing about the component talking to the real API through the real client pipeline; it's a different (lower) layer, not a substitute for the journey.

The trade: E2E is the slowest, flakiest layer and needs the whole stack running — so it's excluded from the fast `build-test` CI job (1.6) and gated separately, and it's rationed hard. Accepted deliberately: a handful of journey tests catch the integration failures nothing else can, and the discipline of §6 keeps the cost bounded.

9. Checkpoint

```
git add -A && git commit -m "test(e2e): Playwright harness — POM, Mailpit-as-inbox, first
sign-in journey"
git tag lesson-3.6
```

You should now be able to: place a test on the right pyramid layer and defend the choice; build a Page Object with `data-testid` selectors and say why localized text can't be a selector; explain the Mailpit-as-fixture trick and trace the passwordless journey through every layer it touches; and state the rule for what earns an E2E test.

Next: *Lesson 3.7 — Build your own slice (capstone)* — no reference code this time. You apply the seven-step checklist and the full Red→Green→refactor loop, unaided, to a feature that's yours.

Lesson 3.7 — Build your own slice (capstone)

> **Part 3 · The slice pattern & the UI — capstone.** Every other lesson handed you the code. This one doesn't. You've read the chassis, reproduced the reference slice, and built the UI and test harness around it — now you build a *new* feature end to end, unaided, and discover that the platform makes it almost boring. That "almost boring" is the entire point of Parts 1–3: if adding a feature is calm, the architecture is doing its job. This is where the knowledge becomes a skill.

Goal: ship one complete, tested, tenant-isolated, localized feature slice — API to UI to E2E — adding to platform code only a `DbSet` and registrations (never changing platform behavior), and have every architecture test pass *because you followed the pattern*, not because you studied the tests.

Concepts & principles: synthesis. The seven-step checklist (3.2), TDD (1.2/3.3), the repository seam (3.1), tenancy (2.6/2.7), the contributor seam (2.9), RCL UI (3.4), localization (3.5), an E2E journey (3.6), and the arch tests as your safety net (3.3).

Maps to: everything in Parts 1–3 · `docs/WAYS_OF_WORKING.md` (slice + story format).

Prerequisites: all of Parts 1–3. Do not read ahead to Part 4 first — the point is to prove *this* foundation is solid in your hands.

---

1. The brief

Build a **Bookmarks** feature: a tenant can save, list, and delete links.

- A bookmark has a `Title`, a `Url`, and a `CreatedAt`.
- Anyone in the tenant can list the tenant's bookmarks (newest first), add one, delete one.
- Expose a **derived** value the API computes and never stores: the link's `Host` (e.g. `github.com` from `https://github.com/x/y`). This exercises Golden Rule 4 — derived, not stored — and gives you a genuine unit of domain logic to TDD.
- Validation: a blank title or a non-absolute URL is a `400`, using the shared error envelope (1.5).

If Bookmarks bores you, pick your own single-entity feature — Reminders, Tags, Saved Searches — the shape is identical. Keep it to **one entity**: the goal is to run the loop cleanly, not to design a subsystem. (Anything needing *new platform machinery* — files, background work, a second entity with relationships — is out of scope; those capabilities arrive in Parts 4+. A capstone uses the chassis you have.)

2. Story first (the habit from `WAYS_OF_WORKING.md`)

Before code, write `docs/stories/bookmarks.md` — intent + Gherkin acceptance criteria. This is the spec your tests will mirror:

```
### BOOK-1 — save and manage bookmarks

**As a** tenant member **I want** to save links **So that** my household keeps shared
references.

Scenario: save and list a bookmark
  Given I am signed in
  When I save a bookmark titled "Docs" for "https://learn.microsoft.com/dotnet"
  Then it appears at the top of my bookmark list
  And its displayed host is "learn.microsoft.com"

Scenario: bookmarks are tenant-isolated
  Given tenant A and tenant B each saved a bookmark
  When A lists bookmarks
  Then A sees only A's bookmark
  And A deleting B's bookmark id returns 404

Scenario: a bookmark needs a title and an absolute URL
  When I save a bookmark with a blank title (or "not-a-url")
  Then the response is 400 invalid_request
```

Writing the unhappy paths *now* — isolation, validation — is what stops you from "forgetting" to test them once the happy path works.

3. The seven steps (your worksheet)

Work the checklist from 3.2. For each step, the lesson that taught it is in parentheses — revisit it if you stall, but try from memory first:

1. **Entity** — `src/Core/Entities/Bookmark.cs`, `: ITenantScoped`, `TenantId` + `Title`/`Url`/`CreatedAt`. Compute `Host` as a property from `Url` (or in the response DTO) — never a stored column. (2.6, 3.2, Golden Rule 4)
1. **DbSet + config** — `DbSet<Bookmark>` on `AppDbContext`; a `BookmarkConfiguration` with a sensible `Title`/`Url` max length. (1.3)
1. **Migration** — `dotnet ef migrations add AddBookmarks`. (1.3)
2. **Slice folder** — `src/Api/Features/Bookmarks/`: `BookmarksEndpoints` (via `MapTenantFeatureGroup` — *not* raw `MapGroup`), `BookmarksHandler` (inject `IRepository<Bookmark>`, `ICurrentTenant`, `TimeProvider`; `Query()` for reads), `BookmarksModels` (co-located DTOs; the response carries `Host`). Register the handler and `app.MapBookmarks()` in `Program.cs`. (3.1, 3.2)
1. **Contributor** — `BookmarksDataContributor : ITenantDataContributor` with all four members; `ExportKey => "bookmarks"`; use `QueryAllTenants()` here (and *only* here). Register it. (2.9, 3.2)
1. **Fixture reset** — nothing to do; the reset list is model-derived (1.3). Confirm your entity is truncated by running the suite.
1. **UI + i18n** — a `Bookmarks.razor` page in **Shared.Ui** (not the web app), nav entry, strings via `IStringLocalizer<AppStrings>` with EN + ES resx keys. (3.4, 3.5)

4. Test-drive it — Red before Green, every step

- **Unit (Core.Tests):** the `Host` derivation. Feed it `https://a.b.com/x?y=1` → `a.b.com`

(that's `Uri.Host` — the *whole* host, matching §1's `github.com` definition; extracting a registrable domain like `b.com` needs a public-suffix list and is out of scope). Feed it junk → your defined behavior (empty? the raw string?). This is pure logic — no database — so it's a fast unit test, and writing it first *forces you to define* the edge cases. (1.2, 3.3)

- **Integration (Api.Tests):** the slice against real Postgres — the three Gherkin

scenarios. The isolation one is non-negotiable: seed two tenants, act as A, assert A sees one bookmark and gets `404` (not `403`) deleting B's. Use the harness patterns from 2.6/3.2. (1.3, 2.6, 3.2)

- **E2E (E2E.Tests):** *one* journey — sign in, add a bookmark, see it listed with its

host. Reuse `E2ETestBase.SignInAsync` and the POM style; add `data-testid` hooks to your page. Don't E2E the validation or isolation — those are proven faster above (§6 of 3.6 is the rule you're now applying for real). (3.6)

5. Your safety net — run the architecture tests

Here's the reward for everything Part 1–3 taught. You didn't study the arch tests to game them; you followed the pattern, so they should pass. Run them:

```
dotnet test tests/Api.Tests --filter Architecture
```

What they're checking about *your* slice, and what a failure would be telling you:

- `EveryTenantScopedEntity_HasAGlobalQueryFilter` / `EveryEntityWithATenantId_IsScopedOrAllowlisted` —

green means your `Bookmark` correctly wears `ITenantScoped`. Red means you added a `TenantId` without the marker — an isolation hole. (2.6)

- `FeatureSlices_DoNotBypassTheTenantFilter` — green means your handler used `Query()` and

only your `*DataContributor.cs` used `QueryAllTenants()`. Red means the request path reached for the escape hatch. (3.1, 3.2)

- `FeatureFiles_RegisterRoutesViaMapTenantFeatureGroup` — green means you didn't hand-roll

a `MapGroup`. Red means your endpoints could have forgotten auth. (3.2)

- `WebApp_HasNoInlineBlazorComponents` — green means your page is in the RCL. (3.4)
- `TenantDataContributors_HaveUniqueExportKeys` / `WipeDataTests` — green means your

contributor is registered with a unique key and your table tears down on dissolve. (2.9)

- `TenantScopedEntities_MapToDistinctTables` / `RouteGroupPrefixes_AreUnique` —

green means `bookmarks` didn't collide with an existing table or route. (3.3)

A red arch test here is not an obstacle — it's the platform catching a real mistake at build time, with the offending file named, exactly as designed. That's the whole thesis of the course made personal: the guardrails you built now guard *you*.

6. Definition of done

Check every box honestly (it's the PR template from 1.6):

- [ ] Story written first, with unhappy paths, before any code.
- [ ] Tests written before production code (Red→Green), at the right layers (§4).
- [ ] `dotnet build` — 0 warnings (1.1's bar).
- [ ] `dotnet test` — all green, including the full architecture suite (§5).
- [ ] Tenant isolation proven (the two-tenant integration test; `404`, not `403`).
- [ ] The derived `Host` is computed, never stored (Golden Rule 4).
- [ ] UI in Shared.Ui, EN + ES strings present, no hard-coded English.
- [ ] One E2E journey; validation/isolation left to the faster layers.
- [ ] Touched platform code only by addition — the `DbSet` on `AppDbContext` and the `Program.cs` registrations — and changed no platform *behavior*.
- [ ] Any real decision you made is recorded as a short ADR (0.1).

If all ten hold, you didn't just copy `Notes` — you *internalized* the platform. That last box matters: you made your own small decisions here (how to handle a malformed URL, what the empty-list UI says). Record them. The habit is the point.

7. Checkpoint

```
git add -A && git commit -m "feat(bookmarks): my first slice — end to end on the chassis"
git tag lesson-3.7
```

You should now be able to: build a complete feature slice from an empty folder without referring to `Notes`; sequence the work test-first across the three layers; and read a failing architecture test as feedback rather than an obstacle. If this felt calm — that calm is what a good architecture *feels like*, and it's what the rest of the course protects.

Part 3 complete. You have the assembly manual and the proof you can use it. From here the course returns to the chassis itself — the platform capabilities that make this a *product*, not just an app.

Next: *Part 4 — Reliability & operations.* The transactional outbox, idempotency, the scheduler, observability, and the append-only audit log — how the platform behaves when things fail, retry, and must be explained after the fact.

Part 4 — Reliability & operations

Lesson 4.1 — The transactional outbox

> **Part 4 · Reliability & operations.** The chassis can now authenticate, isolate tenants, and grow features. Part 4 makes it *reliable* — it teaches the platform how to behave when things fail, retry, and must be explained afterward. We open with the problem behind almost every "the charge went through but no email arrived" bug: the **dual-write problem**. You cannot atomically commit a database row *and* perform an external side effect — one of them will eventually be lost. The transactional outbox is the standard, load-bearing answer, and it's the foundation the next five lessons (email, webhooks, notifications, billing) all stand on.

Goal: a side effect enqueued *in the same transaction* as the business change that caused it, delivered later by a background dispatcher that claims work safely, retries with backoff, and dead-letters what it can't deliver.

Concepts & principles: the dual-write problem; the transactional outbox pattern; at-least-once delivery + mandatory idempotency; `FOR UPDATE SKIP LOCKED` work claiming; exponential backoff + dead-lettering; a testable core split from a background service.

Maps to: ADR-007 · repo: `src/Core/Abstractions/IOutbox.cs`, `IOutboxHandler.cs`, `src/Core/Entities/OutboxMessage.cs`, `src/Infrastructure/Outbox/EfOutbox.cs`, `OutboxProcessor.cs`, `OutboxDispatcher.cs`, `OutboxOptions.cs`, tests: `tests/Api.Tests/Outbox/OutboxProcessorTests.cs`.

Prerequisites: 1.3 (real Postgres — `SKIP LOCKED` is why the in-memory provider was banned), 3.1 (the unit of work whose transaction the outbox rides).

---

1. Motivate — the write you can't make atomic

A handler does the most natural thing in the world:

```
await repository.SaveChangesAsync(ct); // 1. persist the order
await emailSender.SendAsync(...); // 2. email the receipt
```

Now enumerate the failure modes. The process crashes between line 1 and line 2 → order saved, no receipt, *forever* (nothing will ever retry it). Line 2 throws → do you roll back the order that already committed? You can't; it's committed. You reorder them — email first, then save — and now a crash leaves a receipt for an order that doesn't exist. There is **no ordering that makes a database commit and an external call atomic**, because they're two different systems with two different notions of "done." This is the dual-write problem, and it's not a bug you can carefully code around — it's a property of distributed systems.

The insight that dissolves it: you *can* make two writes to the **same** database atomic — that's what a transaction is. So don't try to perform the side effect inline. Instead, in the same transaction as the business change, **write a durable record that the side effect is owed**. Commit both together or neither. Then a separate process reads those records and performs the effects. The atomic unit becomes "order + intent-to-email," which a single transaction *can* guarantee; the actual email happens later, driven by a record that cannot be lost.

2. The message — a durable IOU

```
// src/Core/Entities/OutboxMessage.cs (the real entity)
/// A durable record of a side effect to perform out-of-band (email, a webhook call, …).
/// It is written in the SAME transaction as the business change that produced it, so the
/// effect is atomic with the data — no "saved the row but lost the email" (ADR-007).
public class OutboxMessage
{
    public Guid Id { get; set; } = Guid.CreateVersion7();
    public required string Type { get; set; } // handler discriminator, e.g. "email"
    public required string Payload { get; set; } // opaque JSON for the matching handler
    public Guid? TenantId { get; set; } // context for the handler, NOT a scoping key
    public string Status { get; set; } = OutboxStatus.Pending; // pending | sent | dead
    public int AttemptCount { get; set; }
    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset NextAttemptAt { get; set; } // earliest dispatch; advanced on
retry (backoff)
    public DateTimeOffset? ProcessedAt { get; set; }
    public string? LastError { get; set; } // last failure detail; never a secret
}
```

Two decisions worth pausing on, both callbacks to earlier lessons:

- **It is deliberately *not* `ITenantScoped`.** Read the entity's own comment: the outbox

is *platform infrastructure* and may carry system (tenant-less) effects, so it sits **outside** the global tenant filter (2.6). And notice how it stays clear of the R2 invariant test (2.6/3.3) *without* an allowlist entry: that test flags an entity only when its `TenantId` is a **non-nullable `Guid`** (a real scoping key someone forgot to scope). Here `TenantId` is `Guid?` — *nullable context passed to the handler*, not a scoping key — so the test doesn't consider it a tenant-scoped table at all, and no allowlist entry is needed. (The allowlist is reserved for the genuinely awkward cases, like `TenantMembership`, that *do* carry a non-nullable `TenantId` yet mustn't be filtered.) Choosing `Guid?` here is thus both semantically right and what keeps the guardrail quiet — recognizing that a platform table is legitimately cross-tenant is the same judgment you exercised in 2.6.

- **`LastError` says "never a secret."** The same 1.5 discipline: failure detail is for

operators, and a payload or exception can carry sensitive data — so what lands in a stored, queryable column is bounded and scrubbed.

The `Type`/`Payload` shape is deliberately generic: the outbox doesn't know what an "email" *is*. It's a typed queue; the meaning lives in handlers (§4). That genericity is why the *same* outbox serves email (4.2), webhooks (7.4), and notifications (7.2).

3. Enqueue — the effect rides the caller's commit

```
// src/Infrastructure/Outbox/EfOutbox.cs (the whole enqueue side)
public async Task EnqueueAsync(string type, string payload, Guid? tenantId = null,
CancellationToken ct = default)
{
    var now = clock.GetUtcNow();
    await db.Set<OutboxMessage>().AddAsync(new OutboxMessage
    {
        Type = type, Payload = payload, TenantId = tenantId,
        Status = OutboxStatus.Pending, CreatedAt = now, NextAttemptAt = now,
    }, ct);
    // Intentionally no SaveChanges — persistence rides the caller's commit.
}
```

The most important line is the comment: **no `SaveChanges`.** `EnqueueAsync` *stages* the message on the shared request-scoped `AppDbContext` (3.1) and stops. It becomes durable when the caller's transaction commits — the *same* transaction as the business change. So a handler that does `repository.Add(order); outbox.EnqueueAsync("email", …); uow.Commit()` gets order-and-intent atomically, for free, because they share one unit of work (3.1's commit model, now paying off precisely as designed). The `IOutbox` interface documents the flip side too: a caller with *no* ambient transaction (the email decorator in 4.2) must persist explicitly — the outbox doesn't presume to own the commit.

4. Handlers — typed, and necessarily idempotent

Delivery is somebody's job, split by message type:

```
// src/Core/Abstractions/IOutboxHandler.cs
/// MUST be idempotent: delivery is at-least-once, so the same message may be handed to
/// HandleAsync more than once (e.g. after a crash between send and commit). Throw to
/// signal failure — the dispatcher retries with backoff and dead-letters after the cap.
public interface IOutboxHandler
{
    string Type { get; } // the type this handler claims
    Task HandleAsync(OutboxMessage message, CancellationToken ct = default);
}
```

Sit with the phrase **at-least-once**. The outbox guarantees an effect happens *at least* once — never zero times (that's the whole point) — but it *cannot* guarantee exactly once, because the handler could succeed at sending the email and then crash *before* recording that success, so the message stays `pending` and is retried. Therefore **every handler must be idempotent**: handling the same message twice must equal handling it once. This isn't a nicety; it's a contract the pattern forces on you. (It's also why 4.3 builds the *inbox* — the mirror-image dedup for effects you can't make naturally idempotent, like "charge this card.") One handler claims each `Type`; the dispatcher routes by it.

5. The processor — claiming work without stepping on yourself

Here's the crux, and it's a genuinely beautiful use of Postgres. Multiple dispatcher instances (or just concurrent polls) must not grab the *same* message and deliver it twice-over. The naive `SELECT ... WHERE pending LIMIT 1` then `UPDATE` has a race: two pollers read the same row before either claims it. The real `OutboxProcessor`:

```
// src/Infrastructure/Outbox/OutboxProcessor.cs (core)
public async Task<bool> ProcessNextAsync(CancellationToken ct = default)
{
    if (!db.Database.IsRelational())
        throw new InvalidOperationException("The outbox requires a relational provider (FOR
UPDATE SKIP LOCKED).");

    var now = clock.GetUtcNow();
    await using var tx = await db.Database.BeginTransactionAsync(ct);

    // Claim the oldest due+pending row, skipping any another poller already holds.
    var message = (await db.Set<OutboxMessage>()
        .FromSql($"""
            SELECT * FROM "OutboxMessages"
            WHERE "Status" = {OutboxStatus.Pending} AND "NextAttemptAt" <= {now}
            ORDER BY "CreatedAt"
            LIMIT 1
            FOR UPDATE SKIP LOCKED
            """)
        .ToListAsync(ct)).FirstOrDefault();

    if (message is null) { await tx.RollbackAsync(ct); return false; }

    try
    {
        if (!_handlers.TryGetValue(message.Type, out var handler))
            throw new InvalidOperationException($"No IOutboxHandler registered for outbox
type '{message.Type}'.");
        await handler.HandleAsync(message, ct);
        message.Status = OutboxStatus.Sent;
        message.ProcessedAt = now;
        message.LastError = null;
    }
    catch (Exception ex)
    {
        message.AttemptCount++;
        message.LastError = Truncate(ex.Message, 1000);
        if (message.AttemptCount >= options.MaxAttempts)
            message.Status = OutboxStatus.DeadLettered; // give up, keep the record
        else
            message.NextAttemptAt = now + Backoff(message.AttemptCount); // exponential
backoff
    }

    await db.SaveChangesAsync(ct);
    await tx.CommitAsync(ct);
    return true;
}
```

Everything here is load-bearing:

- **`FOR UPDATE SKIP LOCKED`** is the whole concurrency story in three words. `FOR UPDATE`

row-locks the claimed message inside the transaction; `SKIP LOCKED` tells Postgres "if a row is already locked by another poller, *skip it*, don't wait." So N pollers each grab a *different* due row and never block on each other — safe horizontal scaling with zero coordination code. This is a Postgres feature the EF in-memory provider cannot emulate, which is precisely why 1.3 banned it and why this method throws on a non-relational provider rather than pretending.

- **The `FromSql` + `ToListAsync().FirstOrDefault()` shuffle** has a real reason (the

comment explains it): the raw SQL already `LIMIT 1`s, but if you let EF apply `.FirstOrDefault()` server-side it layers its own order-less `First` operator on top and logs a noisy false-positive warning every poll. Materializing the (at most one) row, then taking `First` in memory, keeps the log clean. A small scar, labeled.

- **Backoff and dead-lettering** encode a failure policy: a transient failure (SMTP

hiccup) retries on an exponential schedule (`NextAttemptAt` pushed further out each time); after `MaxAttempts` the message is `dead`-lettered — *kept*, not deleted, so an operator can inspect `LastError` and decide. Failure is a state, not a disappearance.

- **The whole thing is one transaction.** Claim, handle, record-outcome, commit — so

either the message is durably marked `sent`/`retrying` or nothing happened and it stays claimable. No outcome is ever lost, which is the property the whole pattern exists for.

Note the deliberate split: `OutboxProcessor` is the *testable core* (no timing, no threads), separate from the `OutboxDispatcher` `BackgroundService` that merely loops it:

```
// src/Infrastructure/Outbox/OutboxDispatcher.cs (the loop)
using var scope = scopeFactory.CreateScope(); // fresh scope per pass (scoped DbContext)
var processor = scope.ServiceProvider.GetRequiredService<OutboxProcessor>();
var processed = await processor.ProcessDueAsync(stoppingToken);
if (processed == 0) await Task.Delay(options.PollInterval, stoppingToken); // idle only
when empty
```

A build detail you'll hit the moment you write this: `BackgroundService` and `IServiceScopeFactory` live in the ASP.NET Core shared framework, not the base runtime — so `Infrastructure` (a plain class library) can't see them yet. Add a framework reference to `Perezosoft.Infrastructure.csproj`, exactly as the reference project does:

```
<ItemGroup>
  <FrameworkReference Include="Microsoft.AspNetCore.App" />
</ItemGroup>
```

That's a deliberate, minimal opening — a `FrameworkReference` pulls in the shared framework's *hosting* abstractions without turning the library into a web app. (And per 1.1's locked-restore discipline: adding it changes the transitive graph, so regenerate `packages.lock.json` — `dotnet restore --use-lock-file` — and commit it, or CI's `--locked-mode` will fail.)

Its comment states the operational stance: one in-process poller is the baseline, the `SKIP LOCKED` claim keeps it correct *if* ever run multi-instance, and a real broker/scheduler is a documented swap-in — the seam is the `IOutbox`/`IOutboxHandler` pair, so replacing the *transport* never touches a caller. (A wrinkle the free-tier deployment must respect — an instance that sleeps pauses the poller — is a recorded trade-off you'll meet in Part 8, not a surprise.)

6. Red — testing the core without the clock or the thread

Because the processor is split out, its tests are fast and deterministic — real Postgres (1.3) for `SKIP LOCKED`, `FakeTimeProvider` (3.3) for backoff:

```
Scenario: a due message is delivered and marked sent
  Given a pending outbox message due now
  When the processor runs one pass
  Then the matching handler received it and the row is "sent"

Scenario: a failing handler retries with backoff, then dead-letters
  Given a handler that always throws and MaxAttempts = 3
  When the processor runs, advancing the clock past each NextAttemptAt
  Then attempts increment, NextAttemptAt pushes out exponentially, and after the 3rd the
row is "dead"

Scenario: two concurrent claims never take the same row
  Given two pending messages and two processors racing
  Then each processor claims a different row (SKIP LOCKED), none delivered twice
```

The third scenario is the one that *needs* real Postgres and would silently pass-but-lie on any faked database — the concrete payoff of 1.3's decision, cashed two parts later. `OutboxProcessorTests` drives all three; the backoff test is pure `FakeTimeProvider.Advance`, no waiting.

> **A `FakeTimeProvider` trap worth one sentence of warning.** A fresh `FakeTimeProvider` starts at the **year 2000**, not "now." So if you enqueue a message with the *real* system clock (`NextAttemptAt` = today) but run the processor with a fake clock at 2000, the message looks *far-future-dated* and is never "due" — the test hangs on nothing. The fix is the discipline 3.3 was really about: use the **same injected clock** to seed the message and to run the processor. Seed `NextAttemptAt` at the fake clock's instant, then `Advance` from there. Mixing a real clock into a fake-clock test is the single most common way these deterministic-time tests go confusingly wrong.

7. Run it

```
dotnet test tests/Api.Tests --filter Outbox
# then live: enqueue in a handler, watch the OutboxMessages table go pending → sent
# on the next dispatcher pass (and a forced failure climb attempts → dead)
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how are side effects performed reliably? (a) Inline, in the request (`Save` then `Send`); (b) fire-and-forget on a background `Task`/queue in memory; (c) a transactional outbox — persist intent in the business transaction, deliver asynchronously with at-least-once + idempotent handlers.

Chosen: (c) (ADR-007). It's the only option that makes the effect *atomic* with the data (dual-write solved) and *durable* across crashes; `SKIP LOCKED` gives safe concurrency on the database we already run, with no broker to operate; the processor/dispatcher split keeps the logic unit-testable. The genericity (`Type` + `Payload`) means every later async feature reuses it.

Rejected: (a) — the dual-write bug, unfixable by ordering; a crash loses effects silently. (b) — an in-memory queue evaporates on restart (same data-loss, just faster) and still isn't atomic with the commit. A dedicated broker (Rabbit/Kafka/SQS) is a fine *scale-up* — and the seam permits it — but adding an operational dependency to a Postgres-only, run-cheap platform is cost the baseline doesn't need to pay yet.

The trade: effects are now *eventually* consistent (a poll interval of latency) and every handler carries the idempotency burden (§4) — plus a background poller to run and, on a sleeping free-tier instance, to notice is paused (Part 8). Accepted: eventual-and- guaranteed beats immediate-and-lossy for side effects, every time.

9. Checkpoint

```
git add -A && git commit -m "feat: transactional outbox — atomic side effects, SKIP LOCKED
claiming, backoff + dead-letter"
git tag lesson-4.1
```

You should now be able to: state the dual-write problem and why no ordering fixes it; explain how staging-in-the-caller's-transaction makes an effect atomic with its cause; say what `FOR UPDATE SKIP LOCKED` buys and why it needs real Postgres; and justify why at-least-once delivery makes handler idempotency non-optional.

Next: *Lesson 4.2 — Email II: the outbox decorator* — the `IEmailSender` seam from 2.3 gets wrapped so that sending an email becomes crash-safe, without any caller noticing.

Lesson 4.2 — Email II: the outbox decorator

> **Part 4 · Reliability & operations.** Back in 2.3 I made a promise: the five-line `IEmailSender` seam would one day gain crash-safety "without touching a single call site." This is the lesson that keeps it. We wrap the email seam in a **decorator** that enqueues onto the outbox (4.1) instead of sending inline — and passwordless sign-in (2.4), invitations (2.9), and every future caller become reliable without changing one line. It's the shortest lesson in Part 4 and the clearest demonstration in the whole course of why seams are worth the discipline.

Goal: `IEmailSender.SendAsync` enqueues the email as an outbox message and returns immediately; the real SMTP send happens later on the dispatcher; no caller changes, and a request no longer blocks on — or fails because of — the mail server.

Concepts & principles: the decorator pattern; transparent behavior-swap via a shared interface; keyed DI registration to avoid a self-wrapping loop; the idempotency trade-off made concrete.

Maps to: ADR-007 · repo: `src/Infrastructure/Email/OutboxEmailSender.cs`, `EmailOutboxHandler.cs`, the DI registration in `ServiceCollectionExtensions`, tests: `tests/Api.Tests/Outbox/OutboxEmailTests.cs`.

Prerequisites: 2.3 (the `IEmailSender` seam + `SmtpEmailSender`), 4.1 (the outbox).

---

1. Motivate — the seam finally earns its keep

Recall what sending email looked like after 2.3: every caller injected `IEmailSender` and called `SendAsync`, which talked to SMTP *inline, during the request*. That has two problems 4.1 taught us to see. First, it's a dual-write: the magic-link token is saved to the database *and* the email is sent, two systems, no atomicity — crash between them and the user has a token row but no email (or an email for a token that rolled back). Second, it couples the *request's* fate to the *mail server's*: SMTP is slow, so the user waits; SMTP is down, so sign-in fails. 2.3 bounded that with a timeout, but a bounded failure is still a failure.

The outbox (4.1) fixes both — *if* email goes through it. The question is how to route every existing caller through the outbox without a find-and-replace across the codebase. The answer is the reason `IEmailSender` was an interface in the first place.

2. The decorator — same interface, new behavior

A **decorator** implements the same interface it wraps, adding behavior while remaining substitutable. `OutboxEmailSender` *is* an `IEmailSender`, but instead of sending, it enqueues:

```
// src/Infrastructure/Email/OutboxEmailSender.cs (the whole thing)
/// IEmailSender that ENQUEUES the email onto the transactional outbox instead of sending
/// it inline; the real SMTP send happens later on the dispatcher via EmailOutboxHandler →
/// SmtpEmailSender (ADR-007). This is the app-facing IEmailSender, so existing call sites
/// (passwordless, invitations) are unchanged — but a request no longer blocks on SMTP or
/// fails if the mail server is momentarily down.
public sealed class OutboxEmailSender(IOutbox outbox, AppDbContext db) : IEmailSender
{
    public const string MessageType = "email";

    public async Task SendAsync(string to, string subject, string htmlBody,
        IReadOnlyList<EmailInlineImage>? inlineImages = null, CancellationToken ct =
default)
    {
        var payload = JsonSerializer.Serialize(new EmailOutboxPayload(to, subject,
htmlBody, inlineImages));
        await outbox.EnqueueAsync(MessageType, payload, cancellationToken: ct);

        // The email call sites aren't wrapped in an explicit unit of work, so flush now to
make the
        // enqueue durable before the request returns. SaveChanges on the shared context
also commits
        // any pending business change alongside the message — exactly the atomicity we
want.
        await db.SaveChangesAsync(ct);
    }
}

public sealed record EmailOutboxPayload(
    string To, string Subject, string HtmlBody, IReadOnlyList<EmailInlineImage>?
InlineImages);
```

Look at what this *doesn't* require: `PasswordlessService` (2.4) still injects `IEmailSender` and calls `SendAsync`; it has no idea it's now talking to a decorator that enqueues rather than sends. The behavior of *every* email in the platform changed from "inline, fragile" to "queued, durable" by adding one class and changing one DI line. That is the entire return on the investment of making email a seam in 2.3 — a promise made three parts ago, paid in full here.

The `SaveChanges` deserves its comment (§1's atomicity, made concrete): the passwordless and invitation call sites don't run inside an explicit unit of work, so the decorator flushes to make the enqueue durable before returning. And because it's the *shared* context (3.1), that flush commits any pending business change — the `LoginToken`, the `TenantInvitation` — *in the same transaction* as the outbox row. Token-and-email-intent, atomic. The dual-write is gone, and the caller never knew there was one.

3. The handler — the real send, later

The dispatcher (4.1) eventually hands the message to the type-`"email"` handler, which does the actual SMTP send using the *undecorated* sender:

```
// src/Infrastructure/Email/EmailOutboxHandler.cs (the whole thing)
public sealed class EmailOutboxHandler(IEmailSender smtpSender) : IOutboxHandler
{
    public string Type => OutboxEmailSender.MessageType; // "email"

    public async Task HandleAsync(OutboxMessage message, CancellationToken ct = default)
    {
        var payload = JsonSerializer.Deserialize<EmailOutboxPayload>(message.Payload)
            ?? throw new InvalidOperationException($"Outbox message {message.Id} has an
unreadable email payload.");
        await smtpSender.SendAsync(payload.To, payload.Subject, payload.HtmlBody,
payload.InlineImages, ct);
    }
}
```

Now — a subtle DI trap, and its fix. The handler needs *an* `IEmailSender`, but if it resolved the app-facing one it would get `OutboxEmailSender` and enqueue the email *again* — an infinite loop of a message that re-queues itself forever. So the registration distinguishes the two by a **key**:

```
// the app-facing IEmailSender is the decorator; the real one is keyed "smtp"
services.AddKeyedScoped<IEmailSender, SmtpEmailSender>("smtp");
services.AddScoped<IEmailSender, OutboxEmailSender>(); // what callers get
// EmailOutboxHandler injects the KEYED "smtp" sender, not the decorator
```

Callers ask for `IEmailSender` and get the decorator (enqueue); the handler asks for the keyed `"smtp"` one and gets the real `SmtpEmailSender` (send). This is the same "break-the-cycle-with-two-registrations" move you met in 3.4 (the two HttpClients) — a recurring shape whenever a decorator and the thing it decorates share an interface. Recognize it once, and you'll reach for it automatically.

4. Idempotency, made concrete

4.1 insisted every outbox handler be idempotent because delivery is at-least-once. Here's what that means for *this* handler, from its own doc comment:

> At-least-once delivery means a duplicate dispatch can re-send a transactional email — accepted here (a second magic link / invitation copy is harmless); the dispatcher only retries on failure.

This is the honest, non-magical version of idempotency. `EmailOutboxHandler` doesn't do anything clever to *prevent* a double-send; it makes the *judgment* that a duplicate transactional email is harmless — worst case, a user gets two copies of the same magic link (both single-use; the first consumed wins, per 2.4). So "idempotent enough" is achieved by *choosing effects whose repetition doesn't matter*, not by deduplication machinery. Contrast that with an effect where repetition *does* matter — "charge this card twice" is not harmless — which is exactly the case the **inbox** (4.3) exists to handle. Email sits on the easy side of the idempotency line; knowing which side an effect is on is the design skill.

5. Red — proving the swap

```
Scenario: sending email enqueues rather than sends inline
  When a caller invokes IEmailSender.SendAsync
  Then no SMTP send has happened yet
  And one pending outbox message of type "email" exists carrying the payload

Scenario: the dispatcher performs the real send
  Given a pending "email" outbox message
  When the dispatcher runs a pass
  Then the underlying SmtpEmailSender received the exact to/subject/body/images
  And the message is marked sent
```

`OutboxEmailTests` drives both — using a fake `SmtpEmailSender` (recording sends) as the keyed sender, so the test asserts the round-trip payload without real SMTP. Note the first scenario proves the *decoupling* (the request did its work with zero SMTP contact); the second proves the *delivery*. Together they show the behavior swapped end to end while `SendAsync`'s signature — and every caller — stayed put.

6. Run it

```
dotnet test tests/Api.Tests --filter Outbox
dotnet run --project src/Api
# request a magic link → the request returns instantly (no SMTP wait); the OutboxMessages
# table shows a pending "email"; the next dispatcher pass sends it → Mailpit (:8025) shows
it
```

Stop your SMTP (or Mailpit) and request a link: the *request still succeeds* and the message waits as `pending`, retrying with backoff until the mail server returns. That's the reliability win made visible — sign-in no longer depends on email being up *right now*.

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how do existing email callers get outbox reliability? (a) Change every call site to enqueue instead of send; (b) put an "if async" branch inside `SmtpEmailSender`; (c) a decorator implementing `IEmailSender` that enqueues, swapped in via DI, with the real sender kept as a keyed dependency for the handler.

Chosen: (c) (ADR-007). The decorator is *transparent* — callers are untouched, the change is one class plus one registration, and the seam's substitutability (2.3) is what makes it possible. Keyed registration cleanly separates "what callers get" (enqueue) from "what the handler uses" (send), avoiding the self-wrapping loop.

Rejected: (a) — a codebase-wide edit that must be repeated for every future email caller, and easy to forget (a new caller would silently go back to fragile inline sends). (b) — a mode flag inside `SmtpEmailSender` conflates two responsibilities (talk to SMTP / decide whether to defer) in one class, and every caller would still need to know which mode to ask for. The decorator keeps each class doing one thing.

The trade: email is now *eventually* delivered (a poll interval late) and a transactional email can, rarely, arrive twice — both accepted in §4. And there are now two `IEmailSender` registrations to understand; the keying + comments carry that weight.

8. Checkpoint

```
git add -A && git commit -m "feat: email outbox decorator — crash-safe sends, zero
call-site changes"
git tag lesson-4.2
```

You should now be able to: define the decorator pattern and explain how it swaps behavior without touching callers; set up keyed DI to keep a decorator from wrapping itself; articulate "idempotent enough" as *choosing repetition-safe effects* rather than building dedup; and point to this lesson as the concrete payoff of 2.3's seam.

Next: *Lesson 4.3 — The inbox* — the mirror image: idempotency for inbound effects you *can't* just re-run safely, like a billing webhook that must not double-charge.

Lesson 4.3 — The inbox

> **Part 4 · Reliability & operations.** The outbox (4.1) handles effects *we* send; the inbox handles effects *we receive*. They're mirror images of one at-least-once problem. 4.2 got away with "idempotent enough" because a duplicate email is harmless — but the platform is about to receive webhooks where a duplicate is *not* harmless: a billing provider that redelivers "subscription activated" must not activate twice, and one that retries "payment succeeded" must not be read as two payments. When an inbound effect isn't naturally repetition-safe, you need a dedup gate. That's the inbox, and its correctness hinges on one subtle transaction rule most implementations get wrong.

Goal: an inbound delivery is processed *exactly once* even when the sender delivers it many times — claimed by a `(source, key)` ledger, race-free under concurrency, and rolled back with its work if the work fails so the inevitable redelivery re-processes.

Concepts & principles: inbound at-least-once & redelivery; idempotency via a dedup ledger; the claim-and-work-commit-together rule (and the classic autocommit bug); Postgres `INSERT … ON CONFLICT DO NOTHING`; the outbox-vs-inbox contrast (`SKIP LOCKED` vs `ON CONFLICT`).

Maps to: ADR-007 · repo: `src/Core/Abstractions/IInbox.cs`, `src/Core/Entities/InboxMessage.cs`, `src/Infrastructure/Inbox/EfInbox.cs`, tests: `tests/Api.Tests/Inbox/InboxTests.cs`.

Prerequisites: 4.1 (at-least-once, the shared-transaction commit model), 3.1 (unit of work).

---

1. Motivate — the redelivery you can't wish away

External systems that call you — payment providers, webhook senders — deliver **at-least-once**, for the same reason your outbox does: they can't tell "my call succeeded" from "my call succeeded but the response got lost," so when in doubt they resend. Stripe will deliver the same event twice; that's not a bug in Stripe, it's the honest behavior of any reliable sender. So your webhook endpoint *will* receive duplicates.

For some effects that's fine (4.2's email — send the receipt twice, shrug). For others it's a disaster: "activate the subscription" applied twice, "payment of $50 received" counted as $100, "grant admin" run again after it was revoked. These effects are *not* naturally idempotent, and — critically — you often *can't make them so* by choosing a safer effect the way 4.2 did, because the effect is dictated by the business event, not by you. So you need the other tool: a gate that recognizes "I've already handled this exact delivery" and refuses to do the work a second time. That gate is the inbox.

2. The ledger — one row per delivery, ever

```
// src/Core/Entities/InboxMessage.cs (the real entity)
/// A dedup ledger entry for an inbound at-least-once delivery (e.g. a Stripe webhook).
/// Recording a Source + IdempotencyKey exactly once is what makes inbound processing
/// idempotent: the first delivery claims a row and does the work in the same transaction;
/// a redelivery of the same key finds the row already present and is a no-op (ADR-007).
public class InboxMessage
{
    public Guid Id { get; set; } = Guid.CreateVersion7();
    public required string Source { get; set; } // namespaces the key, e.g. "stripe"
    public required string IdempotencyKey { get; set; } // the provider's delivery id, e.g.
evt_…
    public DateTimeOffset ReceivedAt { get; set; }
}
```

The design is almost entirely in one constraint: a **unique index on `(Source, IdempotencyKey)`**. Two decisions inside that:

- **`Source` namespaces the key.** Stripe's `evt_123` and some other provider's `evt_123`

must not collide, so the identity is the *pair*, not the key alone. Cheap now, essential the day a second integration arrives.

- **`IdempotencyKey` is the *provider's* id, not one you invent.** Stripe stamps each event

`evt_…` and reuses it across redeliveries of the same event — so "same event" is *their* identity, which is exactly what you must dedup on. Inventing your own key (hashing the body, say) is fragile: a re-serialized-but-identical redelivery could hash differently and slip through. Use the sender's stable id.

Like the outbox, `InboxMessage` is deliberately **not `ITenantScoped`** — its own comment: an inbound event is reconciled to its tenant *by the handler* (e.g. via the Stripe customer id), so the ledger is platform infrastructure on 2.6's allowlist. (That reconciliation is where 5.2's `EnterTenant` comes in — the webhook has no JWT, so it enters the tenant it resolves. Two Part-4 primitives converging in Part 5.)

3. The claim — race-free by construction

```
// src/Infrastructure/Inbox/EfInbox.cs (the whole implementation)
public async Task<bool> TryClaimAsync(string source, string idempotencyKey,
CancellationToken ct = default)
{
    var rows = await db.Database.ExecuteSqlAsync($"""
        INSERT INTO "InboxMessages" ("Id", "Source", "IdempotencyKey", "ReceivedAt")
        VALUES ({Guid.CreateVersion7()}, {source}, {idempotencyKey}, {clock.GetUtcNow()})
        ON CONFLICT ("Source", "IdempotencyKey") DO NOTHING
        """, ct);

    return rows == 1; // 1 = first delivery (claimed, do the work); 0 = duplicate (no-op)
}
```

The naïve approach — `SELECT` to check if the key exists, then `INSERT` if not — has a race: two concurrent redeliveries both `SELECT` (neither sees a row), both `INSERT`, and you've either double-processed or hit a duplicate-key crash. `INSERT … ON CONFLICT DO NOTHING` makes claim-or-skip a **single atomic statement**: the insert affects one row on the first delivery and zero on a duplicate, and concurrent claims of the same key *serialize on the unique index* — exactly one wins, no crash, no double. The return is just "did I insert a row" → "is this the first

time." Race-freedom comes from the database constraint, not from application locking.

This is a nice contrast to hold against 4.1. Both the outbox and the inbox solve concurrency with a Postgres feature rather than app-level locks, but *different* features for *different* shapes:

	Outbox (4.1)	Inbox (4.3)
Job	pick the next item **off a queue**	recognize a **duplicate** delivery
Postgres idiom	`FOR UPDATE SKIP LOCKED`	`INSERT … ON CONFLICT DO NOTHING`
"Winner"	one poller claims each *pending* row	one insert claims each *unique key*

Reaching for the right database primitive — a lock-and-skip for queue draining, a unique-constraint upsert for dedup — is the transferable skill; both beat hand-rolled locking.

4. The one rule that makes it correct — claim inside the work's transaction

Here is the subtlety, and it's the whole lesson. It is not enough to claim the key and then do the work. Read `IInbox`'s doc comment, which is really a warning:

> **Use inside the caller's transaction.** The claim and the work it guards must commit together: wrap `TryClaimAsync` + the processing in an `IUnitOfWork` transaction and commit only after the work succeeds. If the work throws and the transaction rolls back, the claim rolls back too, so the inevitable redelivery re-processes — that's the correct exactly-once-*effect* behaviour. (Claiming in autocommit and then doing the work would mark a delivery handled even if the work fails — don't.)

Walk the failure the wrong way exposes. Suppose you claim in its own committed statement, *then* start the work, and the work throws (a bug, a transient DB error). The claim is already committed — so the ledger says "handled" — but the work never happened. The sender redelivers (at-least-once, remember), your inbox sees the key is "already claimed," and **no-ops** — permanently dropping an event that was never processed. You've converted at-least-once into *at-most-once*, silently, for exactly the failures you most need to survive.

The fix is to bind claim and work into **one transaction** (3.1's unit of work):

```
await using var tx = await uow.BeginTransactionAsync(ct);
if (!await inbox.TryClaimAsync("stripe", evt.Id, ct))
    { await tx.RollbackAsync(ct); return; } // duplicate → no-op, safely
await ApplyTheEffect(evt, ct); // the business change
await tx.CommitAsync(ct); // claim + effect commit together, or neither
```

Now the claim's durability is *tied to* the work's. Work succeeds → both commit → future redeliveries correctly no-op. Work fails → transaction rolls back → the claim vanishes too → the redelivery re-processes, as it must. The term of art is that this gives **exactly-once *effect*** (the outcome happens once) on top of at-least-once *delivery* — you can't stop duplicate deliveries, but you can make their *effect* singular. That distinction — once delivery vs once effect — is worth carrying: nobody can promise the former across a network, and this is how you achieve the latter regardless.

5. Red — the tests that pin exactly-once-effect

```
Scenario: first delivery is claimed, duplicate is a no-op
  Given no ledger entry for ("stripe", "evt_1")
  When TryClaimAsync("stripe","evt_1") runs
  Then it returns true
  And a second TryClaimAsync("stripe","evt_1") returns false

Scenario: concurrent claims of the same key — exactly one wins
  When two claims of ("stripe","evt_1") race
  Then exactly one returns true, the other false, and no error is thrown

Scenario: failed work re-processes on redelivery (the correctness rule)
  Given the claim + work run in one transaction and the work throws
  When the transaction rolls back and the event is redelivered
  Then the second delivery is claimed and processed (the first claim did not survive)
```

`InboxTests` drives these on real Postgres (the `ON CONFLICT` and the concurrent-claim serialization are Postgres semantics — 1.3's decision again). The third scenario is the one that proves the §4 rule and the one a careless implementation fails: it asserts a rolled-back claim does *not* block reprocessing.

6. Run it

```
dotnet test tests/Api.Tests --filter Inbox
```

There's no user-facing surface yet — the inbox is a primitive. Its first real consumer is the billing webhook in 5.2, where you'll see `TryClaimAsync` wrapped exactly as §4 prescribes, deduping Stripe's redeliveries so a subscription event applies once.

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how is inbound at-least-once made safe? (a) Trust the sender to deliver once (it won't); (b) make every inbound handler naturally idempotent by hand; (c) a dedup-ledger gate — claim `(source, key)` via `ON CONFLICT`, inside the work's transaction, no-op on duplicates.

Chosen: (c) (ADR-007). It works for effects that *can't* be made naturally idempotent (the common webhook case), it's race-free via a database constraint rather than app locks, and binding the claim to the work's transaction delivers exactly-once- effect with the correct failure behavior (rollback → reprocess). It mirrors the outbox, so the two together cover both directions of at-least-once with one mental model.

Rejected: (a) — duplicates are guaranteed, not hypothetical; "assume once" is the bug. (b) — sometimes possible (4.2's email) and preferable *when* it is, but many business effects (apply a payment, activate a plan) have no repetition-safe form, so it can't be the general answer; the inbox is the fallback that always works.

The trade: an extra table and the discipline of remembering to wrap claim+work in a transaction — and getting that wrapping wrong fails silently and dangerously (§4), which is why the rule is documented on the interface and pinned by a test. The ledger also grows unbounded without pruning (a cleanup job — 4.4 — trims old entries, same as the outbox's `sent` rows).

8. Checkpoint

```
git add -A && git commit -m "feat: inbox — exactly-once-effect dedup for inbound
at-least-once deliveries"
git tag lesson-4.3
```

You should now be able to: explain why inbound deliveries are at-least-once and when that's dangerous; describe the `(source, key)` ledger and why the key is the sender's id; articulate the claim-inside-the-work's-transaction rule and the at-most-once bug that skipping it causes; and contrast `ON CONFLICT` (dedup) with `SKIP LOCKED` (queue-claim).

Next: *Lesson 4.4 — The scheduler* — the third reliability primitive: recurring in-process work (token cleanup, ledger pruning, the sweeps later lessons rely on).

Lesson 4.4 — The scheduler

> **Part 4 · Reliability & operations.** The third and last of the async primitives. The outbox pushes effects out; the inbox dedups effects coming in; the scheduler does work that no request triggers at all — recurring maintenance on a clock. It's the humblest of the three and the pattern is short, but it closes real loops: the expired tokens from 2.2/2.4, the `sent` outbox rows and inbox ledger from 4.1/4.3, and the billing sweeps Part 5 will lean on all need *something* to run them on a timer. That something is here, and it follows the same DI-discovered, resilient, testable-core shape as the dispatcher.

Goal: register a recurring job by dropping an `IScheduledJob` into DI — no host edits — and have it run on its own interval, isolated from other jobs' failures, with a testable core separate from the timer loop.

Concepts & principles: in-process scheduled work; DI-discovered jobs (Open/Closed again); failure isolation between jobs; per-run DI scope; the testable-core / background- service split (the outbox shape, reused); why *derived* cleanup, not stored flags.

Maps to: ADR-007 · repo: `src/Core/Abstractions/IScheduledJob.cs`, `src/Infrastructure/Scheduling/ScheduledJobsHost.cs`, `ScheduledJobsOptions.cs`, `ExpiredTokenCleanupJob.cs`, tests: `tests/Api.Tests/Scheduling/*`.

Prerequisites: 4.1 (the background-service + testable-core pattern this mirrors), 3.3 (injected clock), 2.9 (the contributor seam's discovery pattern this echoes).

---

1. Motivate — the work nobody asks for

Some work has no triggering request. Expired login and refresh tokens (2.2, 2.4) pile up in their tables and must be swept. The outbox's `sent` rows and the inbox's ledger (4.1, 4.3) grow forever without pruning. Later, trials expire and dunning sweeps must run (Part 5). None of this is initiated by a user clicking something — it's *time* that triggers it. You need a component that wakes on a schedule and runs due maintenance.

The temptation is a cron entry or a cloud scheduled-function per task. That works, but it scatters the platform's recurring logic *outside* the platform — into infrastructure config a developer can't see, test, or version alongside the code. For a platform meant to run cheap and self-contained (Postgres-only, one process), an **in-process scheduler** keeps the jobs *in the codebase*, testable and deployable as one unit. That's the choice here, with the honest caveat (§7) that a single sleeping instance has limits.

2. The job — an interface anyone can implement

```
// src/Core/Abstractions/IScheduledJob.cs (the real interface)
/// A recurring background job run by the ScheduledJobsHost on its own Interval (ADR-007).
/// Add a job by registering an IScheduledJob in DI — no host edits. Each run gets a fresh
/// DI scope, so a job may depend on scoped services (a new AppDbContext).
public interface IScheduledJob
{
    string Name { get; } // stable id, tracks last-run; unique across jobs
    TimeSpan Interval { get; } // minimum time between runs
    Task RunAsync(CancellationToken ct = default);
}
```

Three members, and the design intent is entirely in the doc comment: **add a job by registering it in DI — no host edits.** This is the *exact* Open/Closed move you've now seen three times — the data contributors (2.9), the outbox handlers (4.1), and now scheduled jobs. The host doesn't know what jobs exist; it asks DI for all `IScheduledJob`s and runs the due ones. A new sweep is a new class plus one registration; the scheduler is closed to modification, open to extension. When a pattern recurs this often in one codebase, it stops being a trick and becomes the house style — and recognizing it is half of reading the platform fluently.

3. The reference job — and a tenancy-flavored subtlety

```
// src/Infrastructure/Scheduling/ExpiredTokenCleanupJob.cs (the whole job)
/// Reference scheduled job (ADR-007): deletes expired passwordless and refresh tokens so
/// the auth tables don't grow without bound. Only rows whose ExpiresAt is in the past are
/// removed — a revoked-but-unexpired refresh token is kept because its hash still backs
/// rotation/replay detection until it expires. Copy this shape for trial-expiry sweeps,
etc.
public sealed class ExpiredTokenCleanupJob(AppDbContext db, TimeProvider clock) :
IScheduledJob
{
    public string Name => "expired-token-cleanup";
    public TimeSpan Interval => TimeSpan.FromHours(1);

    public async Task RunAsync(CancellationToken ct = default)
    {
        var now = clock.GetUtcNow();
        await db.LoginTokens.Where(t => t.ExpiresAt < now).ExecuteDeleteAsync(ct);
        await db.RefreshTokens.Where(t => t.ExpiresAt < now).ExecuteDeleteAsync(ct);
    }
}
```

Small, but two details are load-bearing:

- **It deletes only *expired* rows, not *revoked* ones.** The comment is a direct callback

to 2.2: a refresh token that's been rotated out (revoked) but hasn't yet expired is *deliberately kept*, because its hash is what powers reuse-detection — delete it early and a replayed stolen token would read as merely "unknown" instead of the theft signal 2.2 worked to produce. The cleanup respects the security design; "old" and "safe to delete" are not the same predicate, and getting that wrong would quietly reopen a hole.

- **The clock is injected** (3.3), so "what's expired" is testable by advancing a

`FakeTimeProvider` rather than waiting an hour. And note `ExecuteDeleteAsync` — a set-based delete. Recall from 2.7 that bulk operations bypass the change-tracker (and thus the stamping interceptor); that's *fine here* because this job runs as system maintenance on the system

context (no current tenant), which is exactly the trust level 2.7 reserved for platform code. A feature slice couldn't write this; a platform job can.

4. The host — resilient, scoped, and testable

```
// src/Infrastructure/Scheduling/ScheduledJobsHost.cs (the core)
public async Task RunDueJobsAsync(CancellationToken ct = default)
{
    using var scope = scopeFactory.CreateScope(); // fresh scope per tick (scoped
AppDbContext)
    var now = clock.GetUtcNow();

    foreach (var job in scope.ServiceProvider.GetServices<IScheduledJob>())
    {
        if (_lastRun.TryGetValue(job.Name, out var last) && now - last < job.Interval)
            continue; // not due yet

        try { await job.RunAsync(ct); }
        catch (OperationCanceledException) when (ct.IsCancellationRequested) { throw; } //
shutdown
        catch (Exception ex)
        {
            logger.LogError(ex, "Scheduled job {Job} failed; will retry next interval",
job.Name);
        }
        finally
        {
            // Advance even on failure so a broken job retries on its interval, not in a hot
loop.
            _lastRun[job.Name] = now;
        }
    }
}
```

Four decisions, each a small reliability lesson:

- **Failure isolation.** One job throwing is caught, logged, and *skipped* — it never stops

the other jobs or the host. A broken trial-sweep must not take down token cleanup. The blast radius of a bad job is that one job.

- **Advance last-run even on failure** (the `finally`). This is the subtle one: if a

failing job's timestamp *weren't* advanced, it would be "due" again on the very next tick and hammer in a hot loop (failing every few seconds, flooding logs, burning the DB). Advancing means a broken job retries on its normal interval — failure degrades to "this maintenance is delayed," not "the host is on fire."

- **A fresh DI scope per tick**, so jobs can depend on scoped services (a new

`AppDbContext`) — same reason the outbox dispatcher (4.1) scopes per pass. A background service is a singleton; the work it runs is scoped; the scope-per-tick bridges them.

- **The core is split from the loop.** `RunDueJobsAsync` (pure, testable) is separate from

the `ExecuteAsync` timer loop (`BackgroundService`) — the identical split as the outbox processor/dispatcher (4.1), for the identical reason: you can unit-test "did the due jobs run?" by calling the core with a `FakeTimeProvider`, with no real waiting.

5. Red — deterministic scheduling without waiting

```
Scenario: a job runs when due, then respects its interval
  Given a job with Interval = 1 hour, never run
  When RunDueJobsAsync runs
  Then the job ran once
  When RunDueJobsAsync runs again immediately
  Then the job did NOT run (not due)
  When the clock advances 1 hour and RunDueJobsAsync runs
  Then the job ran again

Scenario: a failing job is isolated and keeps its schedule
  Given job A throws and job B succeeds
  When RunDueJobsAsync runs
  Then B still ran, the host didn't crash, and A is scheduled to retry next interval (not
hot-looping)
```

`ScheduledJobsHostTests` + `ExpiredTokenCleanupJobTests` drive these with `FakeTimeProvider.Advance` — an hour passes in a microsecond, and the "isolation" scenario uses a deliberately-throwing fake job. No `Task.Delay`, no flake. The testable-core split (4.1) is what makes scheduling logic assertable instead of a timing gamble.

6. Run it

```
dotnet test tests/Api.Tests --filter Scheduling
# live: the host ticks on its interval; the LoginTokens/RefreshTokens tables shed expired
# rows within an hour of expiry (or immediately, if you shorten the interval in config to
watch it)
```

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how does recurring maintenance run? (a) OS cron / cloud scheduled functions calling an endpoint; (b) a distributed job framework (Hangfire, Quartz); (c) an in-process `BackgroundService` running DI-discovered `IScheduledJob`s.

Chosen: (c) (ADR-007). The jobs live *in the codebase* — versioned, unit-tested, deployed as one unit — with no external scheduler to configure or secure, matching the platform's Postgres-only, run-cheap baseline. DI-discovery keeps it Open/Closed; the core/loop split keeps it testable; failure isolation keeps one bad job cheap.

Rejected: (a) — scatters the platform's recurring logic into infra config that's invisible to the code, untestable in the suite, and drift-prone (the endpoint and the cron entry evolve separately). (b) — Hangfire/Quartz are excellent at *scale* (persistent schedules, multi-node coordination, dashboards) but add a dependency and operational surface the baseline doesn't need; the `IScheduledJob` seam is the documented swap-in point when multi-node arrives.

The trade — and it's a real one: in-process + single-instance means **if the process isn't running, jobs don't run.** On the free-tier deployment (Part 8) where the instance *sleeps* when idle, a scheduled sweep can be delayed until traffic wakes it — the same caveat as the outbox poller. That's acceptable for cleanup/sweeps (a token pruned an hour late harms nothing) and explicitly *not* acceptable for anything time-critical, which is a recorded deployment decision, not a surprise. And a naive multi-instance run would execute each job on every node — the seam's swap-in (a distributed scheduler) is the answer if that day comes.

8. Checkpoint

```
git add -A && git commit -m "feat: scheduler — DI-discovered recurring jobs,
failure-isolated, testable core"
git tag lesson-4.4
```

You should now be able to: recognize the DI-discovery Open/Closed pattern in its third guise and register a job without touching the host; explain failure isolation and why last-run advances even on failure; connect the token-cleanup predicate back to 2.2's reuse-detection design; and state the single-instance / sleeping-instance limitation and its documented swap-in.

Next: *Lesson 4.5 — Observability* — structured logging, tracing, and health checks: how you see what the platform is doing, and how a deploy knows it's alive.

Lesson 4.5 — Observability

> **Part 4 · Reliability & operations.** You've made the platform *reliable*; now make it *legible* — able to answer "what is it doing, for whom, and is it alive?" Three capabilities, one theme: every signal is enriched with *who* (tenant + user) so a production question ("why is this tenant seeing errors?") has an answer. Structured logs say what happened, traces say where the time went, and health checks tell an orchestrator whether to send traffic. And a discipline threads through all three: the signals carry *identifiers*, never secrets — observability must never become a leak.

Goal: every log line and trace span produced while handling a request carries `tenant_id`/`user_id`; the app exposes liveness and a database-backed readiness probe; and all of it runs with zero external dependencies by default (exporters are config-gated).

Concepts & principles: structured logging + per-request scope enrichment; distributed tracing (OpenTelemetry); liveness vs readiness; config-gated exporters (no-dependency default); identifiers-not-secrets in telemetry; enrichment that can never fail a request.

Maps to: ADR-008, ADR-C10 · repo: `src/Api/Observability/RequestLoggingScopeMiddleware.cs`, `TelemetryExtensions.cs`, `DatabaseHealthCheck.cs`, `src/Api/Program.cs` (logging setup + `/health` mapping from 1.2), tests: `tests/Api.Tests/Observability/*`.

Prerequisites: 1.2 (the health endpoint stub, now grown up), 2.6 (`ICurrentTenant`, the enrichment source), 1.4 (config-gating).

---

1. Motivate — a production question you can't answer

It's 2 a.m. and one tenant reports errors. With `Console.WriteLine`-style logging you have a wall of undifferentiated lines and no way to isolate *that tenant's* requests. With no tracing you can see a request was slow but not *where* the time went. With a naive health check that returns 200 as long as the process is up, your orchestrator keeps routing traffic to an instance whose database connection died. Observability is the difference between "the system is a black box that occasionally misbehaves" and "I can ask it questions and get answers." The three tools below are those three answers — *what*, *where*, *alive* — and the platform's specific contribution is enriching all of them with tenant identity so the questions can be *per-tenant*, which for a SaaS is the question that matters.

2. Structured logging — logs you can query, scoped to who

`Program.cs` (from 1.2, grown up) picks a log format by environment: a readable single-line console in dev, **JSON in production** so a log aggregator can index fields. That's the "structured" in structured logging — a log line isn't a string, it's an *event with fields*. And the platform attaches the fields that matter via a per-request scope:

```
// src/Api/Observability/RequestLoggingScopeMiddleware.cs (core)
/// Opens a per-request logging scope carrying tenant_id and user_id, so every log line
/// emitted while handling the request is filterable by tenant/user (OBS-1). ... Only
/// identifiers go in the scope — never tokens or other secrets.
internal static Dictionary<string, object> BuildScope(HttpContext context, ICurrentTenant
currentTenant)
{
    var scope = new Dictionary<string, object>();
    if (currentTenant.TenantId is { } tenantId) scope["tenant_id"] = tenantId;
    if (context.User.FindFirst(ClaimTypes.NameIdentifier)?.Value is { } userId)
scope["user_id"] = userId;
    return scope;
}
// ... using (logger.BeginScope(scope)) await next(context);
```

`BeginScope` means *every* log line written anywhere downstream during that request — in a handler, a service, EF — automatically carries `tenant_id` and `user_id`, without any of that code knowing about logging scopes. Now "show me this tenant's errors in the last hour" is a filter, not an archaeology dig. Three details are deliberate:

- **Identifiers only** — the comment is emphatic, and it's the recurring 1.5/4.1 rule:

never a token, never PII, in a log scope. Logs are widely readable and long-retained; a secret in a log is a secret leaked at scale.

- **Anonymous requests add no scope** (empty → skip `BeginScope`) — no noise on pre-auth

traffic.

- **Registered *after* authentication** so the claims exist to read. Middleware order is

itself a decision; the extension's comment says so.

3. Tracing — where the time actually went

Logs say *what*; traces say *where*. OpenTelemetry (the vendor-neutral standard) produces spans across the request, outbound HTTP, and the database, so a slow request decomposes into "80ms in Postgres, 200ms in that outbound call." The platform's setup, abridged:

```
// src/Api/Observability/TelemetryExtensions.cs (core)
services.AddOpenTelemetry()
    .ConfigureResource(r => r.AddService("Perezosoft.Api"))
    .WithTracing(tracing =>
    {
        tracing
            .AddAspNetCoreInstrumentation(o =>
                o.EnrichWithHttpResponse = (activity, response) => EnrichSpan(activity,
response.HttpContext))
            .AddHttpClientInstrumentation()
            .AddSource("Npgsql"); // stable built-in Npgsql tracing (NOT the beta EF
instrumentation)
        ApplyExporter(tracing, otlpEndpoint, useConsole);
    })
    .WithMetrics(...);
```

Three decisions with real reasoning behind them:

- **Same enrichment, on spans.** `EnrichSpan` tags each request span with

`tenant_id`/`user_id` (identifiers only, again), so traces filter per tenant just like logs — one consistent identity story across both signals. And note its defensive guard: `context.RequestServices` can be null for requests rejected before the pipeline runs (Kestrel bad requests, early aborts), so enrichment null-checks and *never throws* — **observability must not be able to fail the request it observes.** A telemetry bug taking down real traffic is the classic own-goal; the guard forbids it.

- **`AddSource("Npgsql")`, not EF Core instrumentation.** A pointed, documented choice

(ADR-C10): subscribe to Npgsql's *stable, built-in* tracing rather than the perpetually-beta EF Core instrumentation. "Latest stable, never previews" (Golden Rule 6) applied to telemetry — you don't want a beta package deciding whether your DB spans work.

- **Config-gated exporters, no-dependency default.** This is the elegant part: spans are

always *produced*, but *exported* only if configured — OTLP when `OpenTelemetry:Otlp:Endpoint` is set (production, pointing at a collector), the noisy console exporter only if you flip `OpenTelemetry:ConsoleExporter=true` for local debugging, and **nothing** otherwise. So a fresh checkout runs with a clean console and zero external dependency, while production wires a real backend by setting one config key (1.4's pattern). Instrumentation is always on; where it *goes* is deployment config. (Both keys must be declared in `appsettings.json` — the moment `TelemetryExtensions` reads them, 1.4's doc-sync gate will fail the build until they're documented. The gate doing its job is the reminder; add an `OpenTelemetry` section with the two keys.)

4. Health — liveness vs readiness

Remember the `/health` endpoint from 1.2, using the framework's health-check pipeline "so its growth is additive"? This is the growth. There are *two* questions an orchestrator asks, and conflating them is a classic bug:

- **Liveness** — "is the process up?" `/health` answers this with no dependency probe

(predicate `_ => false` → no checks run, just "the process answered"). If liveness fails, the orchestrator *restarts* the instance.

- **Readiness** — "can it actually serve?" `/health/ready` runs the checks *tagged*

`ready`, including the database probe. If readiness fails, the orchestrator *stops routing traffic* to the instance — but doesn't restart it (the DB might just be briefly unreachable).

```
// src/Api/Program.cs — the two probes (from 1.2, now with a real readiness check)
app.MapHealthChecks("/health", new HealthCheckOptions { Predicate = _ => false });
app.MapHealthChecks("/health/ready", new HealthCheckOptions { Predicate = c =>
c.Tags.Contains("ready") });
```

```
// src/Api/Observability/DatabaseHealthCheck.cs — the readiness probe
/// Readiness check (OBS-3): reports Healthy only when the database is reachable, so an
/// orchestrator won't route traffic to an instance that can't serve. Status-only — no
/// connection details are surfaced.
public async Task<HealthCheckResult> CheckHealthAsync(HealthCheckContext ctx,
CancellationToken ct = default)
{
    try
    {
        return await db.Database.CanConnectAsync(ct)
            ? HealthCheckResult.Healthy() : HealthCheckResult.Unhealthy("Database
unreachable");
    }
    catch { return HealthCheckResult.Unhealthy("Database unreachable"); }
}
```

The distinction is operationally load-bearing: a process that's *up* but can't reach its database is *live* (don't restart — restarting won't fix the DB) but *not ready* (don't send it traffic). Get this wrong by making `/health` probe the DB, and a brief DB blip triggers a restart storm across every instance. Note the readiness check is **status-only** — it returns "Database unreachable", never the connection string or error detail (the same no-leak discipline as §2, applied to a *public, unauthenticated* endpoint where it matters most). This `/health/ready` is exactly what Part 8's deploy pipeline polls as its post-deploy smoke gate — the endpoint born trivially in 1.2 becomes the thing that decides whether a release goes live.

5. Red — testing that telemetry enriches and health reports

```
Scenario: request logs carry tenant and user
  Given an authenticated request for tenant T by user U
  When the scope is built
  Then it contains tenant_id = T and user_id = U (and nothing else)

Scenario: an anonymous request adds no scope
  Then BuildScope returns empty (no noise on pre-auth traffic)

Scenario: readiness reflects the database
  Given the database is reachable
  Then /health/ready is Healthy
  Given it is not
  Then /health/ready is Unhealthy — and leaks no connection detail
```

`RequestLoggingScopeMiddlewareTests`, `TelemetryEnrichmentTests`, and `DatabaseHealthCheckTests` drive these — the enrichment logic is extracted into pure static methods (`BuildScope`, `EnrichSpan`) precisely so it's unit-testable without a running server, the same testable-core instinct as 4.1/4.4.

6. Run it

```
dotnet run --project src/Api
curl -k https://localhost:7160/health # Healthy (liveness — process up)
curl -k https://localhost:7160/health/ready # Healthy (readiness — DB reachable); stop
Postgres → Unhealthy
# sign in, make a request, watch the dev console: log lines carry tenant_id/user_id
# set OpenTelemetry:ConsoleExporter=true to see spans, incl. the Npgsql DB span
```

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how is the platform observed? (a) Ad-hoc `Console.WriteLine`/string logs, a single 200-OK health endpoint; (b) a specific vendor's SDK (Datadog/App Insights) wired throughout; (c) structured logging + OpenTelemetry with tenant/user enrichment, config-gated exporters, and split liveness/readiness probes.

Chosen: (c) (ADR-008). Structured + enriched makes signals *queryable per tenant* — the SaaS-critical axis; OpenTelemetry is vendor-neutral, so the backend is a config key (OTLP endpoint), not a code rewrite; config-gated exporters mean zero dependency by default and a real backend in prod without touching code; liveness/readiness split gives orchestrators the two distinct answers they need.

Rejected: (a) — unqueryable logs and a health check that lies (up ≠ ready) are exactly the 2 a.m. failure this lesson opened with. (b) — a vendor SDK threaded through the code couples you to that vendor (the anti-seam again); OTLP lets you point at Datadog, Jaeger, Grafana, or nothing by changing config — the vendor choice stays at the edge.

The trade: OpenTelemetry is more setup than `Console.WriteLine`, and enrichment middleware/span-tagging is machinery to maintain. Accepted: it's the difference between a system you can operate and one you can only pray to. Deliberately deferred (Golden Rule 6): the beta EF Core instrumentation, in favor of stable Npgsql tracing.

8. Checkpoint

```
git add -A && git commit -m "feat: observability — enriched structured logs, OTel tracing, liveness/readiness"
git tag lesson-4.5
```

You should now be able to: explain per-request scope enrichment and why `BeginScope` beats threading a logger; distinguish liveness from readiness and the restart-storm bug of conflating them; justify config-gated exporters and vendor-neutral OTLP over a vendor SDK; and state the "identifiers, never secrets" rule and where enrichment must never throw.

Next: *Lesson 4.6 — The append-only audit log* — the difference between *logs* (for operators, ephemeral) and an *audit trail* (for accountability, immutable), and enforcing immutability structurally.

Lesson 4.6 — The append-only audit log

> **Part 4 · Reliability & operations — finale.** One more record-keeping capability, and it is emphatically *not* logging. Lesson 4.5's logs are for operators: ephemeral, secret-scrubbed, thrown away after a retention window. An **audit trail** is for accountability: durable, tenant-owned, queryable, exportable, and — the defining property — *immutable*. "Who changed this member's role, and when?" is a question a customer, a compliance auditor, or a courtroom may ask months later, and the answer must be trustworthy, which means the record cannot have been altered. This lesson builds that trail and enforces its immutability **structurally**, closing Part 4 on the course's signature move.

Goal: security-relevant actions record a tenant-scoped audit event, atomically with the change they describe; the events are append-only — application code can insert but *cannot* update or delete them, enforced by an interceptor, not a policy.

Concepts & principles: audit trail vs operational logs; append-only immutability enforced by a `SaveChanges` interceptor; audit-as-tenant-data (`ITenantScoped`, exportable, dissolvable); staging on the unit of work (atomic with the audited change); the deliberate `ExecuteDelete` bypass for dissolve; identifiers-not-secrets, once more.

Maps to: ADR-008 · repo: `src/Core/Abstractions/IAuditLog.cs`, `src/Core/Entities/AuditEvent.cs`, `src/Infrastructure/Audit/AuditLog.cs`, `src/Infrastructure/Persistence/AuditAppendOnlyInterceptor.cs`, tests: `tests/Api.Tests/AuditLogTests.cs`.

Prerequisites: 4.5 (logs — the thing this is *not*), 2.7 (the interceptor pattern), 4.1 (staging-on-the-unit-of-work), 2.6 (`ITenantScoped`).

---

1. Motivate — the record that must not be editable

Consider the question "who removed this member?" asked six months after the fact. A log line *might* still exist, buried in an aggregator, scrubbed and rotated. But logs are the wrong tool for three reasons: they're operational (retained briefly, not forever), they're not *tenant data* (a tenant can't query or export their own log lines), and — decisively — they're not *trustworthy as evidence*, because operational logs are mutable by whoever runs the system. An audit trail is a different thing with different requirements:

	Operational logs (4.5)	Audit trail (4.6)
Audience	operators/SRE	the tenant, compliance, legal
Lifetime	short (rotated)	durable (retained, exported)
Ownership	the platform's	the *tenant's* data
Mutability	doesn't matter	**must be immutable**
Access	log aggregator	queryable/exportable via the app

The entity's own comment states the line: "*Audit ≠ logs: this is durable, queryable, exportable tenant data, not operational telemetry.*" Once you accept it's *tenant data*, two things follow automatically — it's `ITenantScoped` (a tenant sees only its own trail), and it participates in GDPR export and dissolve (Part 7). And once you accept it's *evidence*, immutability becomes a hard requirement — which §4 enforces in the one way this course trusts: structurally.

2. Recording — semantic events, staged on the caller's transaction

```
// src/Core/Abstractions/IAuditLog.cs (the real interface)
/// Records semantic audit events for the current tenant (OBS-4). The tenant is stamped
/// automatically (the event is ITenantScoped); pass the acting user explicitly. Like the
/// outbox, it STAGES the event on the current unit of work and does NOT save — so the audit
/// commits atomically with the change it records (or rolls back with it). Never put
/// secrets/PII in metadata.
public interface IAuditLog
{
    Task RecordAsync(string action, Guid? actorUserId = null, string? entityType = null,
        string? entityId = null, object? metadata = null, CancellationToken ct = default);
}
```

Two things you already know how to read:

- **It stages, it doesn't save** — exactly the outbox's move (4.1). `AuditLog.RecordAsync`

adds the event to the shared context and stops; it commits with the caller's transaction. This is *essential* for an audit trail: the audit of "role changed" must commit **in the same transaction as the role change itself**. Otherwise you get the audit-specific dual-write — the role change commits but the audit is lost (an unrecorded action), or the audit records something that then rolled back (a phantom event). Atomicity makes the trail *accurate by construction*: an event exists if and only if the thing it describes happened.

- **`action` is a stable, greppable verb** (`"member.role_changed"`), `metadata` is

structured JSON, and the rule is stamped on every relevant member: **identifiers only, never secrets or PII beyond actor refs.** An audit trail is *more* exposed than logs (the tenant can read and export it), so the no-secrets discipline (1.5/4.5) is if anything stricter here.

The implementation is unsurprising precisely because the patterns compose — it uses the generic repository (3.1), the injected clock (3.3), and lets the tenant interceptor (2.7) stamp `TenantId`:

```
// src/Infrastructure/Audit/AuditLog.cs (core)
await events.AddAsync(new AuditEvent
{
    Action = action, ActorUserId = actorUserId, EntityType = entityType, EntityId =
entityId,
    Metadata = metadata is null ? null : JsonSerializer.Serialize(metadata),
    CreatedAt = clock.GetUtcNow(),
}, ct);
// Intentionally no SaveChanges — the audit persists with the caller's unit of work.
```

Callers write `await audit.RecordAsync("member.role_changed", actor, "TenantMembership", id, new { from, to })` right beside the change, inside the same transaction (you'll see RBAC do exactly this in 6.1). One line, and the action is on the

permanent record.

3. Explicit recording, not magic — a recorded decision

A tempting alternative is to auto-audit *everything* by hooking `SaveChanges` and recording every entity change. The platform deliberately doesn't (ADR-008, with an amendment noting the deferral): audit events are **semantic**, not mechanical. "The `Role` column changed from `member` to `admin`" is a database fact; "*an owner promoted this member to admin*" is the *meaning*, and only the calling code knows the meaning — the intent, the actor, the business action. An auto-auditor produces a noisy change-feed no compliance officer wants to read; explicit `RecordAsync` calls produce a curated trail of things that *matter*. The cost is that a developer must remember to call it on security-relevant actions — accepted, and exactly the kind of decision the ADR log exists to record so a future maintainer doesn't "helpfully" add the auto-auditor back.

4. Immutability — enforced by the build, not the honor system

Here's the property that makes it an *audit* trail, and the course's signature move one last time in Part 4. It's not enough to *intend* that audit rows are never changed; a bug, a careless `Update`, or a malicious hand could rewrite history. So immutability is enforced by a `SaveChanges` interceptor — the same mechanism as tenant stamping (2.7):

```
// src/Infrastructure/Persistence/AuditAppendOnlyInterceptor.cs (core)
/// Enforces that AuditEvent is append-only (OBS-4): a tracked audit row in Modified or
/// Deleted state at SaveChanges throws, so application code can insert audit events but
/// never alter or remove them.
private static void Guard(DbContext? db)
{
    if (db is null) return;
    foreach (var entry in db.ChangeTracker.Entries<AuditEvent>())
        if (entry.State is EntityState.Modified or EntityState.Deleted)
            throw new InvalidOperationException(
                "AuditEvent is append-only: updates and deletes are not permitted from
application code.");
}
```

Registered in `OnConfiguring` alongside the tenant stamper (2.7), so — same reasoning — *every* context enforces it, including ones tests construct directly; the guarantee can't be sidestepped by a code path that forgot to opt in. Try to modify an audit row and `SaveChanges` throws before any SQL runs. Insert is allowed; mutate and delete are not. History is append-only *by construction*, not by hoping no one writes the wrong LINQ.

But notice the deliberate escape hatch, and why it's *safe*:

> Set-based deletes (`ExecuteDelete`) don't pass through the change tracker, so tenant dissolve can still purge a tenant's trail.

This is the *same* boundary you learned in 2.7: bulk `ExecuteDelete` bypasses the change tracker and therefore this interceptor. Here that bypass is a *feature*, not a leak — when a tenant is dissolved (2.9) or erased (7.1), its audit trail must go too (it's the tenant's data, and GDPR erasure means *all* of it). So dissolve's contributor uses the set-based delete to purge the trail,

which the interceptor can't and shouldn't block. The rule is precise: *application code* can never mutate an audit row (change-tracked writes are guarded); *platform teardown* can purge a whole tenant's trail (set-based, on the system context). "Append-only" means "no editing history," not "history is undeletable even when the tenant is destroyed" — and knowing exactly where the guarantee ends is, once again, the mark of enforcement done thoughtfully rather than dogmatically.

5. Red — proving append-only holds

```
Scenario: an action is recorded atomically with its change
  Given a role change made inside a transaction with a RecordAsync beside it
  When the transaction commits
  Then exactly one audit event exists for the change
  And if the transaction had rolled back, NO audit event would exist

Scenario: audit rows cannot be modified or deleted by application code
  Given an existing audit event
  When application code sets a field / removes it and calls SaveChanges
  Then SaveChanges throws "append-only" and the row is unchanged

Scenario: dissolve may purge a tenant's trail (set-based)
  When a tenant is dissolved
  Then its audit events are removed via ExecuteDelete (the interceptor doesn't block it)
```

`AuditLogTests` drives these on real Postgres. The middle scenario is the one that makes it an audit log rather than "a table we promise not to edit" — it asserts the *build-enforced* immutability, the same way 2.6/2.7's tests assert tenancy is structural. The third pins the deliberate boundary so no one later "fixes" the interceptor to block dissolve and thereby breaks erasure.

6. Run it

```
dotnet test tests/Api.Tests --filter Audit
# live (once RBAC's role-change lands in 6.1): change a member's role, then query
# AuditEvents for the tenant → one "member.role_changed" row with the actor + from/to
metadata
# try to UPDATE that row via EF → InvalidOperationException: append-only
```

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how are security/compliance actions recorded? (a) Rely on operational logs (4.5); (b) an audit table that application convention "doesn't modify"; (c) an `ITenantScoped`, append-only `AuditEvent` written by explicit `RecordAsync` calls, staged on the caller's transaction, immutability enforced by a `SaveChanges` interceptor.

Chosen: (c) (ADR-008). Audit is *tenant data* (queryable, exportable, dissolvable — so `ITenantScoped`), *evidence* (so structurally immutable, not convention-immutable), and *accurate* (staged on the audited change's transaction, so it exists iff the action happened). Explicit recording keeps the trail semantic and curated. Every piece reuses a pattern already built — repository, clock, interceptor, unit of work — so it's a composition, not new machinery.

Rejected: (a) — logs are ephemeral, operator-owned, and mutable: unfit as evidence and invisible to the tenant. (b) — an "audit table by convention" is one careless `Update` from a corrupted trail, and a corrupted audit trail is *worse than none* (it looks authoritative while lying); structural enforcement is the whole point. Also rejected: auto-auditing every `SaveChanges` — it produces mechanical noise, not semantic events (§3).

The trade: developers must *remember* to call `RecordAsync` on security-relevant actions (the price of semantic-over-mechanical), and append-only means corrections are themselves new events (you never edit a mistake, you record a compensating action) — which is exactly how a real ledger works, and the point.

8. Checkpoint

```
git add -A && git commit -m "feat: append-only audit log — tenant-scoped, atomic, immutable by interceptor"
git tag lesson-4.6
```

Part 4 complete. The platform is now *reliable* and *legible*: side effects survive crashes (outbox), inbound duplicates are harmless (inbox), maintenance runs on a clock (scheduler), operators can see per-tenant what's happening (observability), and every consequential action is on an immutable record (audit). Everything that follows — monetization, RBAC, GDPR — stands on these five primitives.

You should now be able to: articulate the audit-vs-logs distinction across all five axes; explain why audit must be staged on the audited change's transaction; describe interceptor- enforced append-only immutability and why `ExecuteDelete` is a deliberate, safe exception; and defend explicit semantic recording over mechanical auto-audit.

Next: *Part 5 — Monetization.* The billing abstraction, Stripe checkout and webhooks (where the inbox and `EnterTenant` finally meet), and plan entitlements + quotas — turning the platform into a product that charges money.

Part 5 — Monetization

Lesson 5.1 — Billing abstraction & entitlements

> **Part 5 · Monetization.** The platform can now do everything except the one thing a *business* must: charge money and gate features by what a tenant paid for. Part 5 adds that — and it opens with two decisions that keep you out of the two worst billing mistakes. First, **never touch card data**: payments happen on the provider's hosted checkout, behind a seam shaped exactly like `IEmailSender`. Second, **the provider is the source of truth for money; your database holds a projection** — and every uncertainty about that projection fails *closed*, to the free plan. Money is the one place where "fail open" isn't a bug, it's a theft.

Goal: a tenant's plan is resolved server-side from a local `Subscription` projection that fails closed to Free; an endpoint can be gated behind a plan entitlement with one `.RequireEntitlement(...)` call that returns **402**; and the app boots and is testable with no Stripe account at all.

Concepts & principles: the payment-provider seam (never build card forms); projection vs source-of-truth; fail-closed entitlement resolution; the 402-vs-403-vs-401 distinction; plans as code/config not tenant data; the dev fake that *must not* run in production (R1).

Maps to: ADR-006 · R1 · repo: `src/Core/Abstractions/IBillingProvider.cs`, `src/Core/Billing/PlanCatalog.cs`, `src/Core/Entities/Subscription.cs`, `src/Core/Abstractions/IEntitlementService.cs`, `src/Api/Services/EntitlementService.cs`, `src/Api/Endpoints/EntitlementEndpointExtensions.cs`, `src/Infrastructure/Billing/FakeBillingProvider.cs`, the registration guard in `ServiceCollectionExtensions`, tests: `tests/Api.Tests/Billing/EntitlementServiceTests.cs`, `RequireEntitlementFilterTests.cs`, `BillingProviderRegistrationTests.cs`.

Prerequisites: 2.6 (tenant-scoped `Query()`), 2.3 (the seam pattern this mirrors), 1.5 (status-code semantics), 1.4 (config-gating).

---

1. Motivate — two ways billing goes catastrophically wrong

Mistake one: touching cards. The instant your server *sees* a card number, you're in PCI-DSS scope — audits, compliance, liability, and a breach that ends the company. The entire industry's answer is: don't. The provider (Stripe) hosts the checkout page; the card never touches your servers; you get back a signed *event* saying "this tenant now has Pro." So `IBillingProvider` never has a "charge card" method — it has "create a hosted checkout session" and "parse a webhook." Building a card form is not a feature you're declining to build; it's a liability you're declining to accept.

Mistake two: trusting your own database about money. It's tempting to flip a tenant to Pro when checkout *completes*. But completion and payment aren't the same event, refunds happen, cards expire mid-subscription, disputes reverse charges — and **the provider knows all of that; your database doesn't.** So your `Subscription` row is a *projection* — a cache of the provider's truth, updated by webhooks (5.2) — and whenever it's stale, absent, or

ambiguous, you resolve to **Free**. Granting a paid feature on uncertain state is giving away the product; the safe default when unsure is *less* access, never more.

Both mistakes have the same antidote as the rest of this course: a seam for the first, and fail-closed defaults for the second.

2. The provider seam — money mutations live elsewhere

```
// src/Core/Abstractions/IBillingProvider.cs (shape)
/// Abstraction over the payment provider (ADR-006), mirroring IEmailSender's
Core-abstraction
/// shape so Core stays provider-agnostic and tests run offline. ... Money mutations happen
on
/// the provider's hosted Checkout — we never build card forms or store card data.
public interface IBillingProvider
{
    Task<BillingCheckoutSession> CreateCheckoutSessionAsync(BillingCheckoutRequest r,
CancellationToken ct = default);
    Task<BillingPortalSession> CreatePortalSessionAsync(BillingPortalRequest r,
CancellationToken ct = default);
    BillingWebhookEvent? ParseWebhookEvent(string payload, string? signature); // 5.2
    Task CancelSubscriptionAsync(string stripeSubscriptionId, CancellationToken ct =
default); // dissolve (5.3)
}
```

It's the `IEmailSender` seam again (2.3), applied to payments: a Core interface speaking the domain's language (checkout, portal, webhook), with the vendor (Stripe) quarantined in Infrastructure. Read one sentence from the checkout method's doc especially: "*Access is NOT granted by completing checkout — only the webhook flips the subscription.*" That's Mistake #2 encoded in the API — the checkout call returns a *URL to redirect to*, never a grant. The grant comes later, from a signed webhook, through the projection. Note too the `CreateCheckoutSessionAsync` carries the `TenantId` *into* the session, so the eventual webhook can reconcile back to the tenant — the thread that 5.2's `EnterTenant` will pull.

3. The projection — a cache that fails closed

```
// src/Core/Entities/Subscription.cs (shape)
/// A tenant's billing subscription — a local PROJECTION of the payment provider's state
/// (Stripe is the source of truth for money; ADR-006). A tenant has at most one (unique on
/// TenantId). Absence   Free, and any non-active status fails closed to Free —
entitlements
/// are never granted on stale/uncertain state.
public class Subscription : ITenantScoped
{
    public Guid TenantId { get; set; }
    public required string PlanKey { get; set; }
    public required string Status { get; set; } // active | trialing | past_due | canceled
    public string? StripeCustomerId { get; set; }
    public string? StripeSubscriptionId { get; set; }
    public DateTimeOffset? CurrentPeriodEnd { get; set; } // past = lapsed, even if
Status=active
    public DateTimeOffset? LastEventAt { get; set; } // newest applied event — the 5.2
ordering guard
    // ... LapseNotifiedAt (5.3), timestamps
}
```

It's `ITenantScoped` (2.6 scopes it automatically — a tenant sees only its own subscription), and it carries the provider's ids so the portal and cancel calls can find the customer. Two fields foreshadow later lessons — `LastEventAt` is the strictly-newer guard 5.2 needs (webhooks arrive out of order), `LapseNotifiedAt` is 5.3's dunning bookkeeping — but the property that matters *now* is the one repeated everywhere: **absence □ Free, non-active □ Free, lapsed □ Free.** A `Subscription` is never a reason to grant; it's only ever a *possible* reason, checked live.

Where does a tenant's Free-ness live if absence means Free? Nowhere — and that's the point. There's no "Free subscription" row to create or forget; the *absence* of a paid projection *is* the Free state. One less thing to keep consistent.

4. Plans are code, not data

```
// src/Core/Billing/PlanCatalog.cs (shape)
public sealed record Plan(string Key, IReadOnlySet<string> Entitlements,
    int? SeatLimit = null, IReadOnlyDictionary<string, int>? UsageLimits = null)
{
    public bool Includes(string entitlementKey) => Entitlements.Contains(entitlementKey);
    // null SeatLimit / absent usage key   unlimited
}

public static class PlanCatalog
{
    private static readonly IReadOnlyDictionary<string, Plan> Plans = new
Dictionary<string, Plan>
    {
        [PlanKeys.Free] = new(PlanKeys.Free, new HashSet<string>(), SeatLimit: 3, ...),
        [PlanKeys.Pro] = new(PlanKeys.Pro, new HashSet<string> { Entitlements.ProFeature },
SeatLimit: 10, ...),
    };
    /// Unknown keys fall back to Free (fail-closed).
    public static Plan Get(string planKey) => Plans.TryGetValue(planKey, out var p) ? p :
Plans[PlanKeys.Free];
}
```

Plans are **code/config, not tenant data** (ADR-006) — the catalog is a static dictionary, because a plan definition ("Pro grants X, caps seats at 10") is a property of *your product*, identical for every tenant, versioned with the code. What's per-tenant is only *which* plan they're on (the projection). And `Get` fails closed one more time: an unknown plan key resolves to Free, so a typo or a removed plan can never accidentally grant more than Free. (The `Entitlements`/`UsageKeys` here are marked EXAMPLE — a real app replaces them with its own gates; the quota fields, `SeatLimit`/`UsageLimits`, are 5.3's business.)

5. Entitlements — the server-side gate, failing closed

```
// src/Api/Services/EntitlementService.cs (core)
public async Task<bool> HasAsync(string entitlementKey, CancellationToken ct = default)
{
    var subscription = await subscriptions.Query().FirstOrDefaultAsync(ct); //
tenant-scoped (2.6)
    var planKey = ResolvePlanKey(subscription, clock.GetUtcNow());
    return PlanCatalog.Get(planKey).Includes(entitlementKey);
}

/// The active plan, or Free if the subscription is absent/inactive/lapsed (fail-closed).
internal static string ResolvePlanKey(Subscription? subscription, DateTimeOffset now)
{
    if (subscription is null) return PlanKeys.Free;
    var active = subscription.Status is SubscriptionStatus.Active or
SubscriptionStatus.Trialing
                 && (subscription.CurrentPeriodEnd is null || subscription.CurrentPeriodEnd
> now);
    return active ? subscription.PlanKey : PlanKeys.Free;
}
```

`ResolvePlanKey` is the whole fail-closed doctrine in five lines, and it's `internal static` precisely so it can be unit-tested exhaustively without a database: null → Free, `past_due` → Free, `canceled` → Free, `active`-but-`CurrentPeriodEnd`-in-the-past → Free, only `active`/`trialing`-and-unexpired → the real plan. Note it checks the *period end* even when the status says active — because a status can be stale between webhooks, but a lapsed period is lapsed. The clock is injected (3.3), so "is it expired?" is testable. Every path that isn't unambiguously paid-and-current returns Free. **Never trust the client for entitlement** — this reads the server's projection, never a claim or a request field.

6. The 402 gate — a distinct answer for "your plan doesn't include this"

```
// src/Api/Endpoints/EntitlementEndpointExtensions.cs (core)
public static TBuilder RequireEntitlement<TBuilder>(this TBuilder builder, string
entitlementKey)
    where TBuilder : IEndpointConventionBuilder
    => builder.AddEndpointFilter(async (context, next) =>
    {
        var entitlements =
context.HttpContext.RequestServices.GetRequiredService<IEntitlementService>();
        if (await entitlements.HasAsync(entitlementKey,
context.HttpContext.RequestAborted))
            return await next(context);

        var app =
context.HttpContext.RequestServices.GetRequiredService<IApplicationSettings>();
        return Results.Json(new
        {
            error = "payment_required",
            entitlement = entitlementKey,
            message = $"This feature requires a plan that includes '{entitlementKey}'.",
            upgrade_url = $"{app.ClientUrl.TrimEnd('/')}/billing",
        }, statusCode: StatusCodes.Status402PaymentRequired);
    });
```

A feature slice gates a paid capability by adding `.RequireEntitlement(Entitlements.Xxx)` beside its `MapTenantFeatureGroup` (3.2) — the same one-liner-guardrail style as

`.RequirePermission` (6.1). The decision worth dwelling on is the **status code**. Recall 1.5's three "no" answers, and now the set is complete:

- **401 Unauthorized** — "we don't know who you are" (2.1). Fix: sign in.
- **403 Forbidden** — "we know you; your *role* may not do this" (6.1). Fix: get permission.
- **402 Payment Required** — "we know you; your *plan* doesn't include this" (here). Fix: *upgrade* — so the response names the missing entitlement and includes an `upgrade_url`.

Three different problems, three different client reactions — and only by distinguishing them can the UI do the right thing (bounce to login / hide the button / show the upgrade page). Returning 403 for an unpaid feature would send the user to a permission dead-end instead of the billing page; returning 500 would look like a bug instead of a sales opportunity. This is 1.5's "the status code is the first bit of meaning" paying off — the E2E journey in 5.3 asserts that a 402 becomes a visible upgrade prompt, the whole point of choosing the right code.

7. The dev fake — and the rule that it must never ship

The app must boot and be testable with no Stripe account, so there's a `FakeBillingProvider`: deterministic fake checkout URLs, recorded requests, and a `ParseWebhookEvent` that trusts a literal `"valid"` signature so webhook flows can be driven offline. Convenient — and *dangerous if it ever ran in production*, because trusting a literal signature means accepting **forged, unauthenticated, cross-tenant billing writes.** So the registration fails closed at startup (R1):

```
// src/Infrastructure/ServiceCollectionExtensions.cs — the R1 guard
if (!string.IsNullOrEmpty(configuration["Billing:Stripe:SecretKey"]))
    services.AddScoped<IBillingProvider, StripeBillingProvider>();
else if (environment.IsDevelopment())
    services.AddScoped<IBillingProvider, FakeBillingProvider>();
else
    throw new InvalidOperationException(
        "No Billing:Stripe:SecretKey is configured and the environment is not Development.
The " +
        "in-memory FakeBillingProvider trusts a literal webhook signature and must never
run " +
        "outside Development (it would accept forged ... cross-tenant billing writes).
...");
```

Read the `else`: a production deploy with no Stripe key **refuses to boot** rather than silently falling back to the fake. This is the fail-fast-config doctrine (1.4) pointed at the highest-stakes case — better a loud startup crash than a live system accepting forged billing events. It's rule R1, machine-checked (`BillingProviderRegistrationTests`), and it's the sharpest example in the course of "the safe default when unsure is *refuse*, not *proceed*." A convenience that would be catastrophic in production is guarded so it *cannot* reach production.

8. Red — the tests

```
Scenario: entitlement resolution fails closed
  Given no subscription | a past_due one | an active-but-expired one
  Then the tenant resolves to Free and lacks the Pro entitlement
  Given an active, unexpired Pro subscription
  Then the tenant has the Pro entitlement

Scenario: a gated endpoint returns 402 for an unentitled tenant
  Given a Free tenant hitting a .RequireEntitlement("pro-feature") endpoint
  Then the response is 402 with error "payment_required", the entitlement named, and an
upgrade_url

Scenario: the fake provider must not register outside Development
  Given no Stripe key and a non-Development environment
  Then app startup throws (the fake can never serve production)
```

`EntitlementServiceTests` hammers `ResolvePlanKey`'s fail-closed matrix (pure, fast, no DB via the `internal static` split); `RequireEntitlementFilterTests` asserts the 402 shape; `BillingProviderRegistrationTests` pins R1. The fail-closed matrix is the one to write exhaustively — every "not clearly paid" input must land on Free.

9. Run it

```
dotnet test tests/Api.Tests --filter Billing
dotnet run --project src/Api # boots with the FakeBillingProvider (no Stripe key needed in
dev)
# hit a .RequireEntitlement endpoint as a Free tenant → 402 { payment_required, upgrade_url
}
```

10. Architecture Decision

ARCHITECTURE DECISION

The fork: how does the platform handle payments and paid gating? (a) Build card capture + charge logic in-house; (b) call the Stripe SDK directly throughout, treat the local DB as the truth; (c) a provider seam (hosted checkout, never card data) + a local `Subscription` *projection* that fails closed to Free + server-side entitlement gates returning 402.

Chosen: (c) (ADR-006). The seam keeps Core provider-agnostic and tests offline (mirroring `IEmailSender`); hosted checkout keeps card data — and PCI scope — entirely out of the platform; projection-not-truth + fail-closed-to-Free means uncertainty costs the *tenant* a feature, never the *business* revenue; 402 gives the client a distinct, actionable answer. The dev fake keeps the app bootable with no Stripe, guarded by R1 so it can't reach production.

Rejected: (a) — building card handling is accepting PCI liability and a breach surface for zero differentiation; nobody should. (b) — a direct-SDK, DB-is-truth design welds Stripe into every call site (anti-seam) *and* trusts local state about money, which drifts from the provider on every refund/dispute/expiry and fails *open* (grants access on stale "active" rows). Both rejected mistakes are the two this lesson opened with.

The trade: access is *eventually* consistent — completing checkout doesn't grant the feature; the webhook does (5.2), so there's a beat of latency and a hard dependency on webhook delivery. Accepted: it's the only design where your grant reflects the provider's actual money state. And the projection must be *reconciled* carefully against out-of-order webhooks — exactly the job of the next lesson.

11. Checkpoint

```
git add -A && git commit -m "feat: billing abstraction + entitlements — provider seam,
fail-closed projection, 402 gate"
git tag lesson-5.1
```

You should now be able to: explain why the platform never sees card data and what `IBillingProvider` therefore does and doesn't expose; articulate projection-vs-source-of-truth and why every ambiguity resolves to Free; place 402 among 401/403 and say what client reaction each drives; and justify R1's refuse-to-boot guard as fail-closed applied to money.

Next: *Lesson 5.2 — Stripe: checkout, webhook, portal* — where the projection actually gets updated, and the inbox (4.3) and `EnterTenant` (2.7) finally meet in the webhook handler.

Lesson 5.2 — Stripe: checkout, webhook, portal

> **Part 5 · Monetization.** This is where the projection from 5.1 actually gets written — and it's the lesson the whole reliability toolkit was quietly building toward. The billing webhook handler is the course's most satisfying *composition*: signature verification, the inbox (4.3), `EnterTenant` (2.7), the strictly-newer recency guard, an atomic transaction (3.1), and a dunning notification all cooperate in one method to turn an untrusted, at-least-once, out-of-order HTTP POST into a correct, exactly-once update of a tenant's plan. If Parts 2 and 4 felt like isolated primitives, this is where you see why they were built.

Goal: a tenant can start a hosted checkout and open a management portal; and an inbound Stripe webhook safely reconciles the `Subscription` projection — authentic-only, deduped, tenant-scoped, never regressed by a stale event, and atomic with its notification.

Concepts & principles: hosted checkout/portal (redirect, don't capture); webhook as the *only* source of grants; signature authenticity; inbox dedup on the event id; `EnterTenant` for a JWT-less but authenticated write; the out-of-order recency guard (strictly-newer); one-transaction claim+apply+notify; observable rejection of forged webhooks.

Maps to: ADR-006, ADR-007, ADR-003 · repo: `src/Api/Services/BillingService.cs`, `BillingWebhookHandler.cs`, `src/Api/Controllers/BillingWebhookController.cs`, `BillingController.cs`, `src/Infrastructure/Billing/StripeBillingProvider.cs`, tests: `tests/Api.Tests/Billing/BillingWebhookHandlerTests.cs`, `BillingWebhookControllerTests.cs`, `BillingServiceTests.cs`.

Prerequisites: 5.1 (the seam + projection), 4.3 (inbox), 2.7 (`EnterTenant`), 3.1 (unit of work).

---

1. Motivate — three interactions, one hard one

Billing has three moving parts, and their difficulty is wildly uneven:

- **Checkout** (start a subscription) and **portal** (manage/cancel) are *easy*: create a

hosted session at the provider, return its URL, redirect the user. The provider does the hard, dangerous work (card capture); you hand off and step back.

- **The webhook** is *hard*, because it's where the money-truth flows back *into* your

system — and it arrives as an untrusted HTTP POST from the public internet, delivered at-least-once, possibly out of order, possibly forged. Everything that makes billing correct or catastrophic lives in how you handle that one request.

So this lesson spends a paragraph each on the easy two and its whole heart on the webhook.

2. Checkout & portal — redirect, grant nothing

```
// src/Api/Services/BillingService.cs (core)
public async Task<CheckoutResult> CreateCheckoutAsync(Guid tenantId, string? planKey,
CancellationToken ct = default)
{
    if (!IsPurchasablePlan(planKey)) return new(CheckoutOutcome.InvalidPlan, null);
    var baseUrl = app.ClientUrl.TrimEnd('/');
    var session = await billing.CreateCheckoutSessionAsync(
        new BillingCheckoutRequest(tenantId, planKey!, $"{baseUrl}/billing/success",
$"{baseUrl}/billing/cancel"), ct);
    return new(CheckoutOutcome.Created, session.Url); // ← a URL to redirect to. No
Subscription written.
}
```

The load-bearing thing is what this method *doesn't* do: it writes **no** `Subscription`. Its doc comment is blunt — "*Checkout writes no Subscription — access is granted only by the BILLING-3 webhook.*" The client redirects the user to `session.Url`, the user pays *on Stripe*, and the grant arrives later, through the webhook (§3). Same for the portal: it looks up the tenant's `StripeCustomerId` from the projection (via tenant-scoped `Query()`, 2.6), opens a hosted portal, and any change the user makes there — upgrade, cancel — also comes back as a webhook. There is exactly one write path for subscription state, and it isn't here. `IsPurchasablePlan` fails closed too: only a real, non-Free catalog plan can start a checkout (unknown keys resolve to Free in 5.1, so they're rejected here). Owner-only access is enforced by the controller's `TenantApiControllerBase` (6.1) — checkout is not a thing any member may initiate.

3. The webhook handler — six primitives, one method

Here it is, the composition. Read it once whole, then we'll walk it — every line is a callback to something you built:

```
// src/Api/Services/BillingWebhookHandler.cs (the flow)
public async Task<WebhookResult> HandleAsync(string payload, string? signature,
CancellationToken ct = default)
{
    BillingWebhookEvent? evt;
    try { evt = provider.ParseWebhookEvent(payload, signature); } // 1. AUTHENTICITY
    catch (BillingWebhookSignatureException) { return WebhookResult.InvalidSignature; }
    if (evt is null) return WebhookResult.Ignored; // authentic but irrelevant

    await using var transaction = await unitOfWork.BeginTransactionAsync(ct); // 3.1

    // 2. DEDUP: claim the event id so a redelivery is a no-op (4.3)
    if (!await inbox.TryClaimAsync("stripe", evt.EventId, ct))
    {
        await transaction.CommitAsync(ct);
        return WebhookResult.Duplicate;
    }

    // 3. ENTER TENANT: no JWT here — enter the signature-authenticated tenant so the write
is
    // stamped + scoped by the normal interceptor/filter, not the escape hatch (2.7)
    bool applied;
    using (tenantContext.EnterTenant(evt.TenantId))
    {
        string? previousStatus;
        (applied, previousStatus) = await UpsertSubscriptionAsync(evt, ct); // 4. RECENCY
GUARD + upsert
        if (applied)
            await MaybeNotifyDunningAsync(evt, previousStatus, ct); // 5. dunning (5.3)
        await transaction.CommitAsync(ct); // 6. claim + projection + notification commit
ATOMICALLY
    }

    return applied ? WebhookResult.Applied : WebhookResult.Ignored;
}
```

Walk the six steps as the reunion they are:

1 **Authenticity first.** A webhook is an anonymous public POST — anyone can send one. So *before anything else*, `ParseWebhookEvent` verifies the provider signature; a bad signature returns `InvalidSignature` and applies **nothing**. This is the gate that makes the rest safe to trust: after this line, `evt` is genuinely from Stripe. (`null` means authentic-but-irrelevant — an event type we don't care about — acknowledged, not applied.)

1 **Dedup via the inbox (4.3).** Stripe delivers at-least-once, so `TryClaimAsync("stripe", evt.EventId) claims the event id; a redelivery finds it claimed and short-circuits to Duplicate`. This is the inbox's *first real customer* — and note it's used *exactly* as 4.3's correctness rule demanded: the claim is inside the transaction with the work it guards, so if the apply fails and rolls back, the claim rolls back too and the redelivery re-processes. The billing webhook is why the inbox exists.

1 **`EnterTenant` (2.7).** This is the reunion 4.3 and 2.7 were both pointing at. The webhook has **no JWT** — Stripe doesn't carry your access token — but it *does* carry a trusted tenant id (the metadata you stamped at checkout in 5.1, inside a signature-verified event). So the handler *enters* that tenant, and now the write goes through the **normal** tenant interceptor

and query filter (2.6/2.7): the `Subscription` is stamped to the entered tenant, scoped reads see only its row — the same structural isolation an authenticated request gets, achieved without a JWT. It does *not* reach for the `QueryAllTenants()` escape hatch. `EnterTenant` "scopes, doesn't authorize" (2.7) — the signature did the authorizing; this does the scoping.

1 **The recency guard** (inside `UpsertSubscriptionAsync`) — the subtlest correctness point, and one teams miss:

```
// apply only STRICTLY-NEWER events — webhooks are unordered
if (subscription is not null && subscription.LastEventAt is { } last && evt.OccurredAt <=
last)
    return (false, subscription.Status); // stale/out-of-order → acknowledged, NOT applied
```

Webhooks are not just at-least-once, they're **out of order**. Stripe can deliver a stale `canceled` event *after* a live `active` one (retries, network reordering). Without a guard, that stale event would clobber the tenant back to canceled — silently downgrading a paying customer. The guard: the projection remembers `LastEventAt` (the timestamp of the newest event *applied*), and an incoming event is applied only if it's **strictly newer**. A stale event is acknowledged (its id is claimed, so it won't be retried) but *not applied* — state never regresses. This is the projection-vs-truth discipline from 5.1 enforced against the messiness of real delivery: your cache converges to the provider's latest truth regardless of the order the news arrives in.

5 & 6. **Notify on transition, commit atomically.** If the event was applied *and* the status transitioned into a bad state (`past_due`/`canceled`), a dunning notification is staged (5.3) — only on a real transition, never on a stale event or a same-status redelivery. Then the whole thing commits in **one transaction**: the inbox claim, the projection write, and the notification are atomic. Either all three durably happened or none did — so there's never a claimed-but-unapplied event, or an applied change with a lost notification, or a notification for a change that rolled back. Every reliability lesson in Part 4 is load-bearing in this single commit.

4. The endpoint — a system surface that logs its attackers

```
// src/Api/Controllers/BillingWebhookController.cs (core)
[ApiController, AllowAnonymous, Route("api/billing/webhook")]
public class BillingWebhookController(BillingWebhookHandler handler, ILogger<...> logger) :
ControllerBase
{
    [HttpPost] public async Task<IActionResult> Receive(CancellationToken ct)
    {
        var payload = await new StreamReader(Request.Body).ReadToEndAsync(ct);
        var signature = Request.Headers["Stripe-Signature"].ToString();
        var result = await handler.HandleAsync(payload, signature, ct);
        if (result == WebhookResult.InvalidSignature)
        {
            var sourceIp = HttpContext.Connection.RemoteIpAddress?.ToString() ?? "unknown";
            logger.LogWarning("Rejected billing webhook with an invalid signature from
{SourceIp}.", sourceIp);
            return BadRequest(new { error = "invalid_signature" });
        }
        return Ok(); // Applied / Duplicate / Ignored all acknowledge → provider stops
retrying
    }
}
```

Three deliberate choices:

- **It's `[AllowAnonymous]` and does *not* derive from `TenantApiControllerBase`.** It's a

system endpoint — the provider calls it with no JWT — so it's the sanctioned exception to "every controller inherits the tenant/admin base" (the R4 arch test's allowlist, from 3.3). Authenticity is the *signature*, not a token; the handler enters the tenant itself.

- **A rejected signature is logged with the source IP** (never the payload). An anonymous

public endpoint receiving a forged webhook is a *probe* — surfacing it as a warning with the source makes the attack observable instead of a silent 400. (The same "log the source of a rejected inbound" instinct as the SSRF/webhook rules, R19.)

- **Applied / Duplicate / Ignored all return 200.** The provider only cares "did you

receive it?" — retrying is *its* at-least-once mechanism, and a duplicate or an irrelevant event is still "received." Only an *inauthentic* payload gets a 400 (and won't be retried, correctly — it was never a real event).

5. The provider implementation — Stripe behind the seam

`StripeBillingProvider` (Infrastructure) implements `IBillingProvider` using the Stripe SDK: `CreateCheckoutSessionAsync` builds a Checkout session stamping the `TenantId` into metadata; `ParseWebhookEvent` calls Stripe's signature verification (with the webhook secret from config) and maps the Stripe event to the provider-agnostic `BillingWebhookEvent`. The point isn't its detail — it's that *all* the Stripe-specific knowledge (SDK types, event-name strings, price-id ↔ plan-key mapping, signature crypto) lives in this one file, behind the seam, exactly as MailKit lives behind `SmtpEmailSender` (2.3). The handler in §3 never sees a Stripe type; it works against `BillingWebhookEvent`. Swap providers and §3 doesn't change.

6. Red — driving the whole flow with the fake

The `FakeBillingProvider` (5.1) makes this testable with no Stripe: it treats the literal signature `"valid"` as authentic and deserializes the payload as a `BillingWebhookEvent`, so tests drive the handler end-to-end offline.

```
Scenario: an authentic activation applies and grants the plan
  Given a signed "active/pro" event for tenant T
  When the webhook is handled
  Then T's Subscription projection is Pro/active and T now has the Pro entitlement (5.1)

Scenario: a redelivery is deduped
  When the same event id is delivered twice
  Then the second is Duplicate and applied nothing

Scenario: a stale out-of-order event does not regress state (the recency guard)
  Given T is active (LastEventAt = t2)
  When a canceled event with OccurredAt = t1 < t2 arrives
  Then it is acknowledged but NOT applied — T stays active

Scenario: a forged webhook is rejected and logged
  Given a bad signature
  Then the result is InvalidSignature → 400, the source IP is logged, nothing changed
```

The third scenario is the one that separates a correct handler from a plausible-but-wrong one — `BillingWebhookHandlerTests` drives it with `FakeTimeProvider` timestamps to make "older" and "newer" exact. Write that one first; it's the bug most billing integrations ship.

7. Run it

```
dotnet test tests/Api.Tests --filter Billing
# live (fake provider, dev): POST a signed event to /api/billing/webhook →
# the Subscription projection flips to active; a Free-gated endpoint now returns 200 (5.1's
402 is gone)
# for real Stripe: `stripe listen --forward-to localhost:7160/api/billing/webhook` replays
real events
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how does subscription state get into the platform? (a) Flip the projection when the checkout *success* redirect returns; (b) poll the provider's API periodically; (c) the provider's *webhook* is the sole write path — verified, deduped via the inbox, applied under `EnterTenant`, guarded against out-of-order delivery, atomic with its notification.

Chosen: (c) (ADR-006 + ADR-007 + ADR-003). The webhook is the provider's own truth pushed to you, so it's the only trustworthy source of a grant; verifying, deduping, and recency-guarding it turns an untrusted, at-least-once, unordered POST into an exactly-once, monotonic projection update; `EnterTenant` gives the JWT-less write the full isolation regime; one transaction keeps claim+apply+notify consistent. It's the reason the inbox and `EnterTenant` were built.

Rejected: (a) — the success redirect is *client-controlled* and fires before payment settles; trusting it grants access to anyone who can reach the success URL, and misses every later change (renewal, failure, cancel). (b) — polling is wasteful, laggy, and still needs all the same reconciliation logic; the webhook is push instead of pull for the same information.

The trade: grants are *eventually* consistent (a webhook round-trip after payment) and the platform depends on webhook delivery — a missed webhook means a stale projection until the next event or a manual re-sync. Accepted: it's the only design that reflects the provider's actual money-truth, and the recency guard + inbox make redelivery (the provider's own recovery mechanism) safe rather than dangerous.

9. Checkpoint

```
git add -A && git commit -m "feat: Stripe checkout/portal + webhook — inbox dedup,
EnterTenant, recency-guarded projection"
git tag lesson-5.2
```

You should now be able to: explain why checkout grants nothing and the webhook is the sole write path; trace the six primitives the handler composes and name the earlier lesson each comes from; articulate the strictly-newer recency guard and the silent-downgrade bug it prevents; and justify the anonymous, signature-authenticated, IP-logging webhook endpoint.

Next: *Lesson 5.3 — Quotas, dunning & dissolve* — the countable limits (seats, metered usage) enforced atomically under concurrency, the lapse sweep, and cleaning up a paid subscription when a tenant is dissolved.

Lesson 5.3 — Quotas, dunning & dissolve

> **Part 5 · Monetization — finale.** Entitlements (5.1) answer a *yes/no* question: does the plan include this feature? Quotas answer a *how many* question: seats used, exports this month — and "how many" is where **concurrency** bites. Two requests consuming the last unit of a quota at the same instant is the classic lost-update race, and getting it wrong lets tenants exceed limits they paid to stay under. This lesson makes quota consumption *atomic under concurrency* (the centerpiece), then closes the billing lifecycle: the lapse sweep that nudges a lapsed tenant, and the dissolve cleanup that stops charging a customer whose tenant no longer exists — both composed from primitives you already built.

Goal: seat and metered-usage limits enforced from the plan, with metered consumption that cannot be raced past the cap; a scheduled sweep that dun-notifies a lapsed tenant exactly once; and billing that cleans itself up on tenant dissolve (cancel the provider subscription, wipe the projection) without blocking a solo owner from leaving.

Concepts & principles: countable quotas vs boolean entitlements; the reserved-seat model; atomic check-and-consume via a conditional `UPDATE` (the lost-update race and its fix, R30); the first-row-insert race + unique-index retry; the lapse sweep on the scheduler; billing's dissolve contributor + provider-cancel-via-outbox.

Maps to: ADR-006, R30 · repo: `src/Api/Services/QuotaService.cs`, `src/Core/Entities/UsageCounter.cs`, `src/Api/Services/SubscriptionLapseSweepJob.cs`, `BillingNotifier.cs`, `BillingDataContributor.cs`, `src/Infrastructure/Billing/BillingCancelOutboxHandler.cs`, tests: `tests/Api.Tests/Billing/QuotaServiceTests.cs`, `SubscriptionLapseSweepJobTests.cs`, `BillingDissolveTests.cs`.

Prerequisites: 5.1 (plan resolution, `PlanCatalog` quotas), 4.4 (scheduler), 4.1/4.2 (outbox), 2.9 (the contributor seam), 1.3 (real Postgres for the concurrency).

---

1. Motivate — the last seat, sold twice

A plan caps a tenant at 10 seats. Nine are used. Two admins each invite someone in the same instant. The naive enforcement — *read* the count (9), *check* (9 < 10, OK), *write* (now 10) — runs twice concurrently: both read 9, both pass the check, both write, and the tenant now has 11 seats. That's a **lost update**, and for a quota it means a paying limit silently breached — the tenant got something they didn't pay for, and your revenue model has a hole. The same race applies to metered usage (the 100th export sold twice).

"Check then act" is never atomic across concurrent callers unless the database makes it so. The fix is to fold the check *into* the write — a single conditional statement the database executes atomically — so the limit is enforced by row-locking, not by hope. That's rule R30 ("quota consumption is atomic under concurrency"), and it's the heart of this lesson.

2. Two kinds of quota — seats and metered usage

Quotas come from the plan (5.1's `PlanCatalog`), and `null`/absent means *unlimited* — so the whole system is inert until a plan sets a real number:

- **Seats** are a *live count*: members + pending invites. `QuotaService.GetSeatUsageAsync`

computes it fresh (never a stored counter that drifts — Golden Rule 4). The subtle part is *pending invites count as seats*:

```
// src/Api/Services/QuotaService.cs
var members = (await tenants.GetMembersAsync(tenantId, ct)).Count;
var pending = (await invitations.GetPendingForTenantAsync(tenantId, ct)).Count;
return new SeatUsage(members + pending, plan.SeatLimit);
```

A pending invite is a **reserved seat** — otherwise a tenant at the cap could send N invitations (each individually "fits" against the current member count) and over-provision when they're all accepted. Counting pending invites reserves the seat at invite time, so the cap holds against the *committed future*, not just the present. (This is why the invite path from 2.9 calls `CanAddSeatsAsync` before creating an invite → a 402 when full.)

- **Metered usage** is a *monthly counter*: a `UsageCounter` row keyed `(TenantId, Key, Period)` `where` Period `is` "yyyy-MM". `Each metered action (an export, say) calls` TryConsumeAsync(key)` which increments the counter if it stays under the cap. This is the one with the race.

3. Atomic consume — the check is the write

Here's the centerpiece. `TryConsumeAsync`, and specifically its increment:

```
// src/Api/Services/QuotaService.cs — the atomic increment
private async Task<bool> TryIncrementAsync(string usageKey, string period, int amount, int
limit, DateTimeOffset now, CancellationToken ct) =>
    await usage.Query()
        .Where(c => c.Key == usageKey && c.Period == period && c.Count + amount <= limit)
// check…
        .ExecuteUpdateAsync(s => s
            .SetProperty(c => c.Count, c => c.Count + amount) // …and act, in ONE statement
            .SetProperty(c => c.UpdatedAt, now), ct) == 1; // affected 1 row? → consumed
```

Read what makes this correct: the limit check (`c.Count + amount <= limit`) is in the **`WHERE` clause of the `UPDATE` itself**. The database evaluates it while holding a row lock, so two concurrent consumers *serialize* on that row — the first increments 9→10, the second re-evaluates `10 + 1 <= 10` (false), matches no row, and `ExecuteUpdateAsync` returns 0 → not consumed. There is no window between check and write for a second caller to slip through, because there is no *between* — it's one atomic statement. The `== 1` (did we update exactly one row?) *is* the answer to "did we get to consume?" This is the check-then-act race dissolved by moving the check server-side into the write, and it needs real Postgres row-locking — the exact reason 1.3 banned the in-memory provider, cashed one last time in Part 5.

There's a second, subtler race the full method handles — the *first* consume of a period, when no counter row exists yet:

```
// (abridged) after TryIncrementAsync returns false with no existing row:
var row = new UsageCounter { Key = usageKey, Period = period, Count = amount, UpdatedAt =
now };
try
{
    await usage.AddAsync(row, ct);
    await usage.SaveChangesAsync(ct); // may collide with a concurrent first-insert
    return true;
}
catch (DbUpdateException)
{
    usage.Remove(row); // detach the failed insert
    return await TryIncrementAsync(usageKey, period, amount, limit, now, ct); // retry
against the now-existing row
}
```

Two requests both seeing "no row yet" would both try to `INSERT` — and the **unique index on `(TenantId, Key, Period)`** lets exactly one win; the loser catches `DbUpdateException`, detaches its doomed insert, and *retries the conditional increment* against the row that now exists. So even the create-the-first-row case is race-safe, via the same constraint-serialization idea as the inbox (4.3): let the database's unique index be the arbiter, then handle the loser gracefully. Belt (conditional update) and suspenders (unique index + retry).

4. Dunning — nudging a lapsed tenant, once

Payments fail; cards expire. "Dunning" is the polite process of telling a customer their subscription is in trouble. There are two triggers, and you've seen one:

- **Transition-driven** (5.2): when a webhook flips the projection *into* `past_due` or `canceled`, `MaybeNotifyDunningAsync` notifies the owner — on the transition only.
- **Time-driven** (this lesson): a subscription can simply *lapse* — its `CurrentPeriodEnd` passes with no renewal webhook. No event fires, so nothing transition-driven catches it. `SubscriptionLapseSweepJob` is a scheduled job (4.4) that periodically finds subscriptions whose period has ended and nudges the owner **once per lapse**, tracked by `Subscription.LapseNotifiedAt` (5.1): the sweep notifies only if `LapseNotifiedAt` predates the current `CurrentPeriodEnd`, then stamps it — so it fires once, not every run, and re-arms naturally when a new period starts. It composes 4.4 (the schedule) + the `IBillingNotifier` seam — no new machinery, just the reliability primitives pointed at a billing need.

> **Forward reference — the notifier is a seam, and delivery is deferred.** Both dunning paths depend on `IBillingNotifier`, but its *real* implementation — fanning a notification out to the owner's in-app center through the outbox — belongs to the notification epic you'll build in **Part 7**. That's not a problem, and *why* it isn't is the point: `IBillingNotifier` is a Core abstraction (the seam pattern again, 2.3), so you build every *caller* now — `MaybeNotifyDunningAsync`, `SubscriptionLapseSweepJob` — against the interface, and supply a **minimal stub** for the delivery. A faithful stub at this stage records the dunning event on the append-only audit log (4.6): it's tenant-scoped and staged on the caller's unit of work, so it commits atomically with the projection write / lapse stamp — *exactly* the transactional shape the real NOTIFY fan-out will have. When Part 7 lands, you swap only the one implementation class; not a single caller changes. This is the seam earning its keep a second time (after email in 2.3/4.2): it lets a later-taught subsystem be *called* before it's *built*, so the billing lifecycle is complete now and gets richer delivery later without a rewrite. Test the "notify on transition, once" logic against a recording test double, decoupling it from whatever delivers.

The "once per lapse" bookkeeping is the whole trick: a sweep that runs hourly must not email the owner hourly, so the *last-notified* timestamp gates it — the same advance-a-timestamp-to-avoid-repeating instinct as the scheduler's own last-run tracking (4.4).

5. Dissolve — stop charging a tenant that no longer exists

When a tenant is dissolved (2.9), its billing must clean up — or Stripe keeps charging a customer whose tenant is gone. Billing plugs into dissolve through the **contributor seam** (2.9), and its implementation makes three instructive choices:

```
// src/Api/Services/BillingDataContributor.cs (core)
public Task<bool> HasDataAsync(Guid tenantId, CancellationToken ct = default) =>
    Task.FromResult(false); // (1) billing is not "abandonable content"

public async Task WipeAsync(Guid tenantId, CancellationToken ct = default)
{
    var subscription = await subscriptions.QueryAllTenants()
        .FirstOrDefaultAsync(s => s.TenantId == tenantId, ct); // (2) cross-tenant: the
audited hatch
    if (subscription is null) return;

    if (!string.IsNullOrEmpty(subscription.StripeSubscriptionId))
    {
        await outbox.EnqueueAsync(BillingCancelOutboxHandler.MessageType, // (3) cancel via
the outbox
            JsonSerializer.Serialize(new
BillingCancelPayload(subscription.StripeSubscriptionId)), tenantId, ct);
        await subscriptions.SaveChangesAsync(ct);
    }
    await subscriptions.QueryAllTenants().Where(s => s.TenantId ==
tenantId).ExecuteDeleteAsync(ct);
}

public string ExportKey => "billing";
public async Task<object?> ExportAsync(...) => /* plan/status/period only — never Stripe
ids or card data */;
```

Each choice is a lesson recalled:

1 **`HasDataAsync` returns `false`.** Recall 2.9: a solo owner leaving is blocked if a contributor reports abandonable *data*. Billing is *plumbing*, not tenant content — a subscription isn't something the owner would lose regret. So it never blocks the leave; it just cleans itself up. Deciding what counts as "the user's data" versus "our plumbing" is a real modeling judgment, and this is the platform making it explicitly.

1 **`QueryAllTenants()` — and it's allowed here** because this is a `*DataContributor.cs` (3.2's precise rule: the escape hatch is permitted only in contributors, dissolve runs for a tenant other than the current one). Re-constrained to the target tenant, exactly as the rule requires.

1 **The provider cancel goes through the outbox (4.1), not an inline call.** This is the sharpest choice. Canceling at Stripe is an *external* call — slow, fallible — and it's happening *inside the dissolve transaction*. Making that call inline would either block the teardown on Stripe's latency or, worse, risk a partial dissolve if the call throws mid-transaction. So instead the cancel is **enqueued** on the outbox: staged with the dissolve (atomic — the cancel-intent commits iff the dissolve does), then delivered out-of-band with retry, and the provider cancel is *idempotent* (5.1) so at-least-once redelivery is safe. This is the outbox's whole reason for being (4.1: don't do external effects inline in a transaction) applied to the one external effect in teardown. The `SaveChanges` flushes the staged message before the set-based delete runs, so it survives the dissolve commit.

`ExportAsync` (GDPR, 7.1) returns plan/status/period — **never the Stripe ids or anything card-related** (5.1's no-card-data rule extends to exports). And its `ExportKey` is `"billing"`, unique per the R13 arch test (3.3).

6. Red — the tests, concurrency first

```
Scenario: metered usage cannot be raced past the cap (R30)
  Given a Free tenant with export cap 3 and 2 already used
  When 5 concurrent TryConsume("export") calls race
  Then exactly 1 succeeds (3rd unit) and 4 are refused — never 4 successes

Scenario: a pending invite reserves a seat
  Given a tenant at its seat cap via members + pending invites
  Then CanAddSeats(1) is false (a further invite is refused with 402)

Scenario: the lapse sweep nudges once per lapse
  Given a subscription whose CurrentPeriodEnd has passed and was never lapse-notified
  When the sweep runs twice
  Then the owner is notified exactly once (LapseNotifiedAt gates the second run)

Scenario: dissolve cancels the provider subscription and wipes the projection
  When a tenant with a live Stripe subscription is dissolved
  Then a provider-cancel is enqueued on the outbox and the Subscription row is gone
  And billing never blocked the (solo-owner) leave
```

The first scenario is the R30 correctness test and *must* run on real Postgres — `QuotaServiceTests` fires genuinely concurrent `TryConsumeAsync` calls and asserts the count never exceeds the cap. A test on a faked database would show four false successes and call it

green — the exact lie 1.3's decision exists to prevent. Write that one first and watch it *fail* against a naive read-check-write to feel why the atomic path matters.

7. Run it

```
dotnet test tests/Api.Tests --filter Billing
# live: consume a metered action past the Free cap → refused; invite past the seat cap →
402;
# dissolve a tenant → OutboxMessages shows a billing-cancel; the Subscription row is gone
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how are countable quotas enforced, and how does billing clean up on dissolve? (a) Read-check-write in app code; (b) an in-app lock/mutex around consumption; (c) an atomic conditional `UPDATE` (check in the `WHERE`) + unique-index retry for the first insert, plus dissolve via the contributor seam with the provider cancel enqueued on the outbox.

Chosen: (c) (ADR-006, R30). The conditional update makes check-and-consume a single row-locked statement — race-free by the database, not by app coordination; the unique index arbitrates the first-insert race with a graceful retry. Dissolve reuses the contributor seam (2.9) and pushes the one external effect (Stripe cancel) onto the outbox (4.1) so teardown stays fast, atomic, and retry-safe. Pending-invite-as-reserved-seat makes the seat cap hold against committed-but-unaccepted invites.

Rejected: (a) — the lost-update race from §1; it *looks* correct in single-threaded tests and breaks under real concurrency (which is why the R30 test is concurrent and Postgres-backed). (b) — an app-level lock doesn't work across multiple instances (each has its own mutex) and reintroduces the coordination the database already does correctly; pushing concurrency control to the DB is both correct and free. Inline Stripe-cancel in the dissolve transaction — rejected for the outbox, per 4.1's whole thesis.

The trade: the atomic path is less obvious than read-check-write (a conditional `UPDATE` whose return value is the decision) and leans on Postgres semantics — but it's the only version that's correct under concurrency, and the alternative's simplicity is a mirage that fails exactly when it matters (load). Metered usage is also *monthly* by the `Period` key — a deliberate, simple model; finer windows would need a different scheme.

9. Checkpoint

```
git add -A && git commit -m "feat: quotas + dunning + dissolve — atomic consume (R30), lapse
sweep, billing teardown"
git tag lesson-5.3
```

Part 5 complete. The platform is now a *product*: it charges money (hosted checkout), reflects the provider's truth safely (webhook + inbox + recency guard), gates features by plan (402), enforces countable limits without races (atomic quotas), chases lapsed payers (dunning), and cleans up billing when a tenant leaves (dissolve). And every piece composed from Parts 2–4 — the seam, the projection, `EnterTenant`, the inbox, the outbox, the scheduler, the contributor.

You should now be able to: explain the lost-update race and how a conditional `UPDATE` dissolves it; justify pending invites as reserved seats; describe once-per-lapse dunning and where its bookkeeping lives; and articulate why the provider cancel goes through the outbox and why billing reports no "abandonable data" to dissolve.

Next: *Part 6 — B2B essentials & security hardening* — RBAC (roles + the permission matrix + 403), file storage, the SSRF seam, and MFA/TOTP step-up.

Part 6 — B2B essentials & security

Lesson 6.1 — RBAC: roles & the permission matrix

> **Part 6 · B2B essentials & security hardening.** So far authorization has been binary — you're authenticated, or you're not — with the occasional ad-hoc "is this the owner?" check. That doesn't survive contact with a real team: an *admin* can invite members but must not touch billing; a plain *member* can look but not change. The wrong way to build this is a scatter of `if (role == "owner")` tests sprinkled across controllers — each one a place the rule can drift, and no single place to answer "what can an admin actually do?" This lesson builds the right way: **capabilities in one matrix**, checked by one gate, completing the 401/402/403 status trilogy with **403 Forbidden**.

Goal: every privileged action is gated by a named *permission*; a single role→permission matrix is the sole source of truth for who-can-do-what; a caller whose role lacks the permission gets a clean `403`; and unknown roles grant nothing (fail closed).

Concepts & principles: role-based access control; capabilities vs per-resource ACLs; a single authorization source of truth; fail-closed authorization; the 401 (who are you?) / 402 (did you pay?) / 403 (you may not) trilogy; server-authoritative UI (the client mirrors permissions, never enforces them).

Maps to: ADR-009, RBAC-1/2/3 · repo: `src/Core/Authorization/Permission.cs`, `RolePermissions.cs`, `src/Api/Services/PermissionService.cs`, `src/Api/Endpoints/PermissionEndpointExtensions.cs`, `src/Api/Authentication/RequireTenantPermissionAttribute.cs`, `src/Shared.Ui/Pages/Household.razor`, tests: `tests/Api.Tests/**/Permission*Tests.cs`.

Prerequisites: 2.8 (membership + roles on the tenant), 5.1 (the `RequireEntitlement` → 402 filter this parallels), 3.4 (the web client that consumes it).

---

1. Motivate — the scattered `role == "owner"` check

Picture the naive version. The rename endpoint checks `membership.Role == "owner" || membership.Role == "admin"`. The billing endpoint checks `role == "owner"`. The invite endpoint checks `role == "owner" || role == "admin"` — copy-pasted, and one of them has a typo. Now answer a simple question: *what can an admin do?* You can't, without grepping the whole codebase and reading every check. Add a fourth role and you edit twenty sites. Get one wrong and you've either locked out a legitimate user (annoying) or **granted a privilege to someone who shouldn't have it** (a security bug). Authorization scattered across call sites is authorization you cannot reason about — and authorization you cannot reason about is authorization you cannot trust.

The fix is to name the *capabilities* — the verbs of your app — put the role→capability mapping in exactly one place, and make every gate consult that place. That's role-based access control done as a **matrix**, and it's this lesson.

2. Capabilities, not ACLs — the `Permission` enum

The vocabulary is a flat enum of coarse capabilities:

```
// src/Core/Authorization/Permission.cs
public enum Permission
{
    ViewTenant, // read the tenant + roster. Every role has this.
    RenameTenant,
    ManageMembers, // invite/remove/manage members + invitations
    ManageRoles, // promote/demote (owner-only — privilege escalation)
    ManageBilling, // checkout + portal (owner-only — financial)
    ExportData, // "download my data" (owner-only; GDPR)
    TransferOwnership, // owner-only
    DissolveTenant, // owner-only
    ManageApiKeys, // owner-only; PUBAPI
    ManageWebhooks, // owner-only; HOOKS
}
```

Two words in the doc comment carry the design: capabilities, **not** ACLs. A *capability* is "may this role manage billing?" — a coarse verb attached to a role. An *ACL* (access-control list) is "may this user read *this specific document*?" — fine-grained, per-record sharing. They are different systems with different costs, and the platform deliberately builds the first and explicitly refuses to let it creep into the second: "*don't grow this into an ACL system without a new ADR.*" That refusal is itself a decision — per-record sharing is a large, invasive feature, and pretending a capability enum can quietly become one is how you end up with a half-built ACL system nobody designed. Note too that the enum already names capabilities for features you haven't built (`ManageApiKeys`, `ManageWebhooks`, Part 7): the *vocabulary* of what's protectable can lead the features, so when the public API lands its gate already has a name.

3. The matrix — the single source of truth

Here is the whole authorization policy of the platform, in one table:

```
// src/Core/Authorization/RolePermissions.cs
private static readonly IReadOnlyDictionary<string, IReadOnlySet<Permission>> ByRole =
    new Dictionary<string, IReadOnlySet<Permission>>(StringComparer.OrdinalIgnoreCase)
    {
        [TenantRoles.Owner] = Enum.GetValues<Permission>().ToHashSet(), // owner:
everything
        [TenantRoles.Admin] = new HashSet<Permission> { ViewTenant, RenameTenant,
ManageMembers },
        [TenantRoles.Member] = new HashSet<Permission> { ViewTenant }, // member: look,
don't touch
    };

public static bool Grants(string? role, Permission permission) =>
    PermissionsFor(role).Contains(permission); // null/unknown role → empty set → false
```

Everything worth noticing is here:

- **Roles are ordered by power** — owner □ admin □ member — but the code doesn't *encode* an

ordering; it lists each role's set explicitly. That's deliberate: an explicit set per role is impossible to misread, whereas "admin ≥ member so admin inherits member's rights plus…" is the kind of cleverness that hides a bug. Owner gets `Enum.GetValues<Permission>()` — *by construction* every permission, including ones added later, so a new capability is owner-accessible the

moment it's defined.

- **The sensitive verbs are owner-only by design** — billing (financial) and role/ownership

changes (privilege escalation) live only in the owner's set. An admin can run the team's day-to-day (invite, rename) but cannot spend money or promote themselves. That last point is the one that matters: `ManageRoles` being owner-only is what stops an admin from making *themselves* an owner — the classic privilege-escalation hole.

- **Unknown roles grant nothing.** `PermissionsFor` returns an empty set for null or unrecognized

roles, so `Grants` is `false`. This is **fail-closed** authorization: a corrupted token, a renamed role, a bug that passes garbage — every one of them denies rather than grants. The same instinct as the billing projection failing to Free (5.1): when in doubt, deny.

"Change authorization by editing this matrix — never by adding a `role == "admin"` check elsewhere." That sentence *is* the architecture. One file answers "what can each role do?", and code review of an authorization change is code review of one small table.

4. Two gates, one matrix — 403

The matrix is consulted through a request-scoped service that fails closed on every missing piece:

```
// src/Api/Services/PermissionService.cs
public async Task<bool> HasAsync(Permission permission, CancellationToken ct = default)
{
    var userIdValue =
accessor.HttpContext?.User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
    if (!Guid.TryParse(userIdValue, out var userId)) return false; // no user → false
    var membership = await tenants.GetMembershipAsync(userId, ct);
    return membership is not null && RolePermissions.Grants(membership.Role, permission);
}
```

The platform has two surface styles — minimal-API endpoints and MVC controllers — so it supplies two thin gates over the *same* service, deliberately mirroring each other:

```
// minimal API: src/Api/Endpoints/PermissionEndpointExtensions.cs
group.MapPost("/rename", …).RequirePermission(Permission.RenameTenant);
// → 403 { error = "forbidden", permission = "RenameTenant", message = "…" } if the role
lacks it

// MVC: src/Api/Authentication/RequireTenantPermissionAttribute.cs
[RequireTenantPermission(Permission.ManageBilling, "Only the owner can manage billing.")]
// → 401 invalid_token if no membership; 403 forbidden if the role lacks it
```

The `.RequirePermission(...)` filter is the deliberate twin of 5.1's `.RequireEntitlement(...)`: same shape, same place in the pipeline, different question. That symmetry is the point of the whole numeric-status story — the platform now speaks the full HTTP authorization trilogy, and each status means exactly one thing:

- **401 Unauthorized** — *who are you?* No valid token; re-authenticate.
- **402 Payment Required** — *did you pay?* Authenticated, but the plan doesn't include this

(5.1).

- **403 Forbidden** — *you may not.* Authenticated and paid, but your **role** lacks the capability. Re-authenticating won't help; a higher-privileged teammate must act.

Conflating these is a real bug — a 403 dressed as a 401 sends the user to log in again for a problem logging in can't fix. Distinct statuses give the client an unambiguous next step.

5. Server-authoritative UI — the roster mirrors, never enforces

`Household.razor` (3.4) is *admin-aware*: it shows role-change controls only to a user who holds `ManageRoles`, hides the dissolve button from non-owners, and so on. This is good UX — don't show buttons that will 403. But the lesson is the discipline underneath: **the UI hint is a convenience, not the authorization.** The server re-checks the permission on every request regardless of what the client rendered, because the client is untrusted — a user can craft the request the hidden button would have sent. The roster reads its permissions from the server (a `GET` that returns the caller's effective capabilities) and mirrors them; it never *decides* them. "Never trust the client" (2.2's clean boundary) applied to authorization: the button is a courtesy, the server filter is the wall.

6. Red — the matrix under test

```
Scenario: an admin cannot manage billing
  Given a caller whose role is admin
  When they POST /api/billing/checkout
  Then the response is 403 forbidden

Scenario: a member can view but not rename
  Given a caller whose role is member
  Then GET /api/household succeeds
  And POST /api/household/rename is 403

Scenario: an unknown role grants nothing (fail closed)
  Then RolePermissions.Grants("wizard", ViewTenant) is false

Scenario: the owner holds every permission, including newly added ones
  Then PermissionsFor(owner) equals the full Permission enum
```

The matrix tests are pure and fast — `RolePermissions.Grants` is a static function of (role, permission), so its truth table is unit-tested with no server at all — while the gate tests drive a real request to assert the `403` envelope. The last scenario is a *regression guard on the design*: asserting owner = the full enum means adding a permission can't silently forget to grant it to the owner.

7. Run it

```
dotnet test tests/Api.Tests --filter Permission
# live: sign in as an admin, try to open the billing portal → 403; the roster hides the
# role-change dropdown for non-owners, but curl-ing the role-change endpoint as an admin
still 403s
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how is "who can do what" expressed? (a) Inline `role == "owner"` checks at each call site; (b) a full per-resource ACL system (grant user U access to record R); (c) a single role→**capability** matrix, consulted by one gate, fail-closed.

Chosen: (c) (ADR-009). Named capabilities give authorization a vocabulary; the matrix is one auditable source of truth ("what can an admin do?" is one file); the shared gate makes every endpoint enforce it the same way and return a clean 403; fail-closed means unknown roles and corrupt tokens deny. Owner-only billing/role-change closes the privilege-escalation surface by construction.

Rejected: (a) — unauditable, un-refactorable, and the source of drift bugs; the exact thing §1 opened with. (b) — ACLs are a *much* larger system (per-record grants, inheritance, sharing UI) and this app doesn't need per-record sharing; building one uninvited is over-engineering and a maintenance sink. The enum's doc comment explicitly fences it off behind a new ADR.

The trade: coarse capabilities can't express "user A may edit document 42 but not 43" — that's the ACL feature deliberately not built. Accepted: coarse team-level roles are what a B2B tenant actually needs, and the matrix stays legible precisely because it's coarse. If per-record sharing is ever required, it's a *new* ADR and a *new* system layered beside this one, not a mutation of it.

9. Checkpoint

```
git add -A && git commit -m "feat: RBAC — permission enum + role matrix + RequirePermission gate (403)"
git tag lesson-6.1
```

You should now be able to: explain capabilities vs ACLs and why the platform builds the former and fences off the latter; describe the role→permission matrix as the single source of truth and why owner-only billing/roles closes privilege escalation; distinguish 401/402/403 and give each a distinct client action; and state why the UI mirrors permissions but the server enforces them.

Next: *Lesson 6.2 — File storage* — a second seam in the shape of `IEmailSender`, storing binary content with tenant-scoped keys that a crafted path can't escape.

Lesson 6.2 — File storage

> **Part 6 · B2B essentials & security hardening.** Every real app eventually needs to store a blob — an avatar, an attachment, an export. The temptation is to reach for `System.IO` in dev and an AWS SDK call in production, sprinkled through feature code. That couples your features to a storage vendor and — far worse for a multi-tenant SaaS — makes it *your* problem, in every call site, to ensure one tenant's file can never be read under another tenant's key. This lesson builds file storage the way the platform builds every external capability: **one seam, tenant-scoped by construction, with the isolation enforced in the storage layer, not hoped for in the callers.**

Goal: a single `IFileStorage` abstraction that features depend on; a local-disk backend for dev and an S3-compatible backend for production, selected by config; object keys namespaced under the tenant so a crafted key cannot escape into another tenant's space; and streaming, never whole-file buffering.

Concepts & principles: the storage seam (mirror of `IEmailSender`); tenant-scoped object keys as the blob analogue of the query filter; path-traversal defense (validate + resolved-path containment); fail-closed without a tenant; streaming I/O; config-selected implementation; the injected clock in a cloud SDK.

Maps to: ADR-010, FILES-1/3 · repo: `src/Core/Abstractions/IFileStorage.cs`, `src/Infrastructure/Files/LocalDiskFileStorage.cs`, `S3FileStorage.cs`, `StorageKeys.cs`, tests: `tests/Api.Tests/Files/LocalDiskFileStorageTests.cs`, `S3FileStorageMinioTests.cs`.

Prerequisites: 2.3 (`IEmailSender` — the seam this copies), 3.1 (`ICurrentTenant`), 1.4 (config-selected impl), 1.3 (real backends under test, here MinIO via Testcontainers).

---

1. Motivate — the leak that isn't in your code

You store user uploads. The obvious key is the filename the user gave you: `storage/{filename}`. Tenant A uploads `report.pdf`; tenant B uploads `report.pdf`; one overwrites the other, or worse, B downloads A's file. So you prefix with the tenant: `storage/{tenantId}/{filename}`. Better — until a user uploads a file named `../{otherTenantId}/report.pdf`, and your naive `Path.Combine` resolves it *out* of their namespace and into someone else's. The blob store has exactly the cross-tenant-leak problem the database had (2.6) — and just like the database, the answer is not "remember to be careful in every feature" but "make the isolation structural, in one place, so features *can't* get it wrong."

That one place is a seam. Features call `IFileStorage`; the seam's implementation owns tenant namespacing and key validation; a feature literally cannot ask for a key outside its tenant, because the implementation rejects it before touching disk or S3.

2. The seam — `IFileStorage`, shaped like `IEmailSender`

```
// src/Core/Abstractions/IFileStorage.cs
public interface IFileStorage
{
    Task PutAsync(string key, Stream content, string contentType, CancellationToken ct =
default);
    Task<FileObject?> GetAsync(string key, CancellationToken ct = default); // null if
absent; caller disposes
    Task<bool> ExistsAsync(string key, CancellationToken ct = default);
    Task DeleteAsync(string key, CancellationToken ct = default); // delete-missing is a
no-op
    Task<Uri> GetDownloadUrlAsync(string key, TimeSpan lifetime, CancellationToken ct =
default);
}
```

If this shape feels familiar, that's the point — its doc comment says so outright: *"Mirrors the `IEmailSender` shape: a Core abstraction with a config-selected Infrastructure impl."* The recurring platform move: name the capability as an interface in Core (where features can depend on it without depending on a vendor), and put the vendor-specific work in one Infrastructure class behind it. Features never see `System.IO` or an S3 type — they see `IFileStorage`. Three shape decisions worth reading:

- **`Stream`, never `byte[]`.** `PutAsync`/`GetAsync` deal in streams, and the doc comment is emphatic: "*Streams, never buffers whole files.*" A `byte[]` API invites loading a 2 GB upload entirely into memory — a trivially exploitable memory-exhaustion DoS. Streaming makes large files a constant-memory operation.
- **`GetAsync` returns `null` for absent, not throw.** Absence is an expected outcome, not an exceptional one; the caller branches on `null` rather than catching.
- **`FileObject` is `IAsyncDisposable`.** The returned handle owns an open stream (a file handle or an S3 response body); `await using` releases it. The type makes the resource ownership explicit and hard to leak.

`DeleteAsync` treating a missing key as a no-op is a small but deliberate *cloud-parity* choice — S3 deletes are idempotent, so the local backend matches, and callers don't need provider-specific error handling.

3. Tenant-scoped keys — the blob query filter

The isolation lives in one shared helper both backends call:

```
// src/Infrastructure/Files/StorageKeys.cs
public static void Validate(string key)
{
    if (string.IsNullOrWhiteSpace(key)) throw new InvalidStorageKeyException("…must not be
empty.");
    if (key.Contains('\\')) throw new InvalidStorageKeyException("…use '/' not '\\'.");
    if (Path.IsPathRooted(key)) throw new InvalidStorageKeyException("…must be relative.");
    var segments = key.Split('/', StringSplitOptions.RemoveEmptyEntries);
    if (segments.Length == 0) throw new InvalidStorageKeyException("…must reference a
file.");
    if (segments.Any(s => s is "." or "..")) throw new InvalidStorageKeyException("…no
'.'/'..' segments.");
}
public static string ForTenant(Guid tenantId, string key) { Validate(key); return
$"{tenantId:D}/{key}"; }
```

Read what's banned: empty keys, backslashes (Windows-path smuggling), rooted/absolute paths, and any `./..` segment (traversal). *A client path is never trusted.* `ForTenant` then namespaces the validated key under the tenant id — so every object physically lives at `{tenantId}/…`, and there is no key a caller can pass that lands outside their own prefix. This is the exact analogue of the `ITenantScoped` global query filter (ADR-003): there, every read is silently scoped to the current tenant; here, every object key is silently scoped to the current tenant. One is enforced by EF, the other by the storage layer — same principle, different substrate. And crucially, both backends call the *same* `StorageKeys`, so local disk and S3 cannot enforce *different* isolation rules — a class of bug ("safe in prod, leaky in dev, or vice-versa") designed out.

4. Two backends — local disk (dev) and S3 (prod)

`LocalDiskFileStorage` is the dev/test default. It stores blobs under `{root}/blobs/{tenantId}/{key}` and content types in a *parallel* `{root}/meta/{tenantId}/{key}` tree (two trees so a user's key can never collide with another object's metadata), streams to/from `FileStream`, and — belt and suspenders — after `StorageKeys.Validate` it *also* asserts the resolved absolute path stays within the tenant directory:

```
// src/Infrastructure/Files/LocalDiskFileStorage.cs (the defense-in-depth check)
var blob = Path.GetFullPath(Path.Combine(tenantBlobRoot, key));
if (!IsWithin(blob, tenantBlobRoot) || !IsWithin(meta, tenantMetaRoot))
    throw new InvalidStorageKeyException($"Key '{key}' resolves outside the tenant
namespace.");
```

Why check *again* when `Validate` already rejected `..`? Because path resolution is subtle and platform-dependent, and a leak here is catastrophic — so the code validates the *input* string **and** verifies the *resolved* path, two independent defenses against the same threat. If a key somehow survives validation and still resolves out of the tenant root, the containment check catches it. Defense in depth: two locks on the one door that must never open.

`S3FileStorage` is the production backend, selected when `Storage:S3` is configured, and works against AWS S3, MinIO, Cloudflare R2, or Backblaze B2 — any S3-compatible endpoint — via endpoint/region/path-style settings. It namespaces keys with the same `StorageKeys.ForTenant`, streams via the SDK (`AutoCloseStream = false` so the caller keeps owning the input stream), maps a `404` from S3 to a `null` `GetAsync` (absence, not exception),

and hands out a **native presigned GET URL** for downloads. One detail is a callback to Part 3:

```
// S3FileStorage.GetDownloadUrlAsync — the presigned URL's expiry uses the INJECTED clock
Expires = clock.GetUtcNow().UtcDateTime.Add(lifetime), // injected TimeProvider (3.3), not DateTime.UtcNow
```

Even here, at the edge against a cloud SDK, the platform's "no ambient clock" rule (3.3) holds: expiry is computed from the injected `TimeProvider`, so a test can mint a URL, advance the clock, and assert it expired — deterministically, without sleeping.

5. `GetDownloadUrlAsync` — a fork resolved in the next lesson

Notice the two backends answer `GetDownloadUrlAsync` differently. S3 returns a *native presigned URL* — the object is served straight from the cloud, bypassing your API entirely. Local disk has no such facility, so it returns a URL pointing at a platform endpoint, `GET /api/files/{token}`, where `token` is a signed, time-limited, tenant+key-bound blob. That signing — and the endpoint that serves it — is the whole of Lesson 6.3, so it's deferred there; what matters *now* is that the seam exposes one method (`GetDownloadUrlAsync`) and each backend satisfies the contract its own way. The feature calling it doesn't know or care which backend is wired; it gets a URL that downloads exactly one object and expires. That's the seam paying off: the *contract* is "a signed, time-limited, single-object URL," and how it's honored is an implementation's business.

> **A build-order note.** `LocalDiskFileStorage` takes an `IFileDownloadTokenizer` in its constructor — and that dependency's implementation is built in 6.3. So two things wait one lesson: **registering `IFileStorage` in DI **(it can't resolve until the tokenizer exists) and the** download-URL test** (it needs a tokenizer to assert the URL shape). Build the seam, both backends, and the round-trip/traversal/fail-closed tests here in 6.2 with a tiny in-test stand-in tokenizer; wire the DI and the real signed-URL path in 6.3 when the tokenizer lands. This is the honest cost of teaching in dependency order — occasionally a class is *usable* before its last collaborator exists, and you scaffold the collaborator just enough to keep the current lesson's tests green.

6. Red — real backends, not fakes

```
Scenario: a stored object round-trips within its tenant
  Given tenant T's context
  When I Put "avatars/me.png" then Get it
  Then I read back the same bytes and content type

Scenario: a crafted key cannot escape the tenant namespace
  When I Put "../otherTenant/secret.png"
  Then it is rejected with InvalidStorageKeyException — nothing is written outside T

Scenario: with no current tenant, storage fails closed
  Then Put/Get without a tenant context throws — never a global/un-namespaced write
```

`LocalDiskFileStorageTests` drives the disk backend against a temp directory; crucially, `S3FileStorageMinioTests` drives the S3 backend against a **real MinIO container** (Testcontainers, 1.3) — because the S3 path is exactly where a fake would lie about presigned URLs, key namespacing, and 404-mapping. The traversal-rejection scenario is the security test;

write it first and watch the crafted key bounce off `Validate`.

7. Run it

```
dotnet test tests/Api.Tests --filter FileStorage
# dev (local disk): Put an object, find it under ./storage/blobs/{tenantId}/…; a "../" key
is refused
# prod-like: point Storage:S3 at a local MinIO and the same feature code stores to the
bucket
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how does the platform store blobs? (a) `System.IO` in dev and cloud-SDK calls in production, inline in feature code; (b) always a cloud SDK, even locally; (c) an `IFileStorage` seam with a local-disk dev backend and an S3-compatible prod backend, selected by config, both enforcing tenant-scoped keys through one shared validator.

Chosen: (c) (ADR-010). The seam keeps features vendor-agnostic (mirroring `IEmailSender`); the shared `StorageKeys` makes tenant isolation structural and *identical* across backends; local disk means a fresh checkout stores files with zero cloud setup, while production flips one config section to real object storage. Streaming and the injected clock carry the platform's DoS-safety and testability rules into the storage layer.

Rejected: (a) — couples every feature to a vendor and scatters the tenant-isolation obligation across call sites, exactly the leak §1 described. (b) — forcing a cloud dependency for local dev/tests is friction with no benefit; the seam lets dev be trivial and prod be real without feature code knowing.

The trade: two backends means the S3 path must be tested against a real S3-compatible server (MinIO), not just disk — more test infrastructure. Accepted: the alternative is shipping a storage layer whose production behavior (presigned URLs, key handling, error mapping) was never exercised until a customer hit it.

9. Checkpoint

```
git add -A && git commit -m "feat: file storage — IFileStorage seam, tenant-scoped keys,
local + S3 backends"
git tag lesson-6.2
```

You should now be able to: explain why blob storage needs the same tenant-isolation discipline as the database and how tenant-scoped keys provide it; justify streaming over buffering and returning null for absence; describe the validate-then-verify-resolved-path defense in depth; and articulate why the same `StorageKeys` must back both backends.

Next: *Lesson 6.3 — Data protection & signed downloads* — the DB-persisted keyring that signs the local download token from §5, the endpoint that serves it, and the encryption pattern MFA and webhook secrets all reuse.

Lesson 6.3 — Data protection & signed downloads

> **Part 6 · B2B essentials & security hardening.** The last lesson left a loose end: the local-disk backend promised a signed, time-limited download URL but didn't sign anything. This lesson ties that off — and in doing so introduces the platform's *secret-handling substrate*, ASP.NET Core Data Protection with a **database-persisted keyring**. That one facility signs the download token, encrypts the MFA secret (6.5), signs the MFA challenge, and encrypts webhook secrets (7.4). Learn it once here, recognize it four more times. The theme: never invent your own crypto or key management — stand on a keyring the framework rotates for you, and persist it somewhere every instance shares.

Goal: a Data Protection keyring persisted to the database (so tokens/secrets survive restarts and are shared across instances); a signed, time-limited, tenant+key-bound token that backs local-disk downloads; an anonymous endpoint that serves exactly one object from a valid token and reveals nothing on failure; and the client seam that turns the URL into a download on web and native.

Concepts & principles: ASP.NET Core Data Protection; keyring persistence & sharing across instances; `ITimeLimitedDataProtector` (encrypt + MAC + expiry); the purpose string as a key-derivation input (and the rename trap); token-as-authorization; `Content-Disposition: attachment`; a client seam for divergent host behavior.

Maps to: ADR-010, FILES-2, NATIVE-3 · repo: `src/Infrastructure/ServiceCollectionExtensions.cs` (the keyring), `src/Infrastructure/Persistence/AppDbContext.cs` (`IDataProtectionKeyContext`), `src/Infrastructure/Files/FileDownloadTokenizer.cs`, `src/Api/Controllers/FilesController.cs`, `src/Shared.Ui/IFileDownloadLauncher.cs` (+ `Browser*`/`Share*` impls), tests: `tests/Api.Tests/Files/FileDownloadTokenizerTests.cs`.

Prerequisites: 6.2 (the storage seam that mints the URL), 2.7 (`EnterTenant`, reused by the serving endpoint), 3.4 (the web client + the RCL seam pattern).

---

1. Motivate — you need a signed token, so you need keys

The local backend must serve a download without a login (an `<img src>` or a click can't carry a JWT), but it must not serve *any* object to *anyone* who guesses a path. The standard answer is a **signed, time-limited token**: a string that says "bearer may fetch tenant T's object K until time X," cryptographically signed so it can't be forged or altered, and expiring on its own. Minting one means you need a signing key. And the moment you have a signing key you have three real problems most people get wrong: *where does it live* (a hard-coded key is a leaked key), *what happens on restart* (a key regenerated at boot invalidates every token in flight), and *what about multiple instances* (each with its own key can't read the others' tokens). Solving these by hand is how crypto goes wrong. So you don't — you use the framework's Data Protection stack, and you tell it to persist its keyring where every instance can share it: the database.

2. The keyring — persisted to the database

Three lines wire the whole substrate:

```
// src/Infrastructure/ServiceCollectionExtensions.cs
services.AddDataProtection()
    .PersistKeysToDbContext<AppDbContext>() // keyring lives in a DB table, not on disk
    .SetApplicationName("template"); // stable app identity → keys are shared, not
per-deploy
```

```
// src/Infrastructure/Persistence/AppDbContext.cs
public class AppDbContext : DbContext, IDataProtectionKeyContext // ← the contract DP looks
for
{
    public DbSet<DataProtectionKey> DataProtectionKeys => Set<DataProtectionKey>();
}
```

Each line answers one of §1's problems:

- **`PersistKeysToDbContext<AppDbContext>()`** puts the keyring in a `DataProtectionKeys`

table. Data Protection generates, rotates, and retires keys automatically; persisting them to the DB means the key that signed a token yesterday is still there today (survives restarts and redeploys) and is visible to *every* instance (a token minted by instance 1 verifies on instance 2). This is what makes signed tokens durable and horizontally scalable — the single most common Data Protection bug (works locally, breaks on the second replica or after a redeploy) is designed out by not using the on-disk default.

- **`SetApplicationName("template")`** pins a stable application identity. Data Protection

isolates keys by app name; without a fixed one it can derive a per-deploy identity and stop recognizing older keys. A constant name means the keyring is one continuous lineage. (Note it's `"template"`, not the rebranded product name — deliberately *not* changed during the Perezosoft rename, because changing it would orphan everything already encrypted. Same trap as the purpose strings below; §4.)

`IDataProtectionKeyContext` is the small interface Data Protection looks for to find the `DbSet` to store keys in. `AppDbContext` implements it, so the keyring rides along in your existing database and migrations — no separate store to provision.

3. The tokenizer — encrypt + MAC + expiry, fail closed

With a keyring in place, minting a download token is a few lines:

```
// src/Infrastructure/Files/FileDownloadTokenizer.cs
private readonly ITimeLimitedDataProtector _protector =
    provider.CreateProtector("Template.Files.Download.v1").ToTimeLimitedDataProtector();

public string Mint(Guid tenantId, string key, TimeSpan lifetime) =>
    _protector.Protect($"{tenantId:D}/{key}", lifetime); // payload signed+encrypted,
expires on its own

public bool TryRead(string token, out Guid tenantId, out string key)
{
    try
    {
        var payload = _protector.Unprotect(token); // throws if expired OR tampered
        var slash = payload.IndexOf('/'); // a Guid never contains '/', so the first splits
it
        if (slash <= 0 || !Guid.TryParse(payload[..slash], out tenantId)) return false;
        key = payload[(slash + 1)..];
        return key.Length > 0;
    }
    catch (CryptographicException) { return false; } // expired/altered/garbage → fail
closed
}
```

`ToTimeLimitedDataProtector()` gives you a protector whose `Protect(payload, lifetime)` bakes an expiry into the token, and whose `Unprotect` *throws* if the token has expired, been altered, or was never valid. So `TryRead` needs no clock logic and no signature-checking code of its own — the protector encrypts, MACs (tamper-detects), and enforces expiry, and any failure surfaces as a `CryptographicException` that becomes a plain `false`. The payload format is a deliberate micro-decision: `{tenantId}/{key}` split on the *first* `/`, safe because a GUID's canonical form never contains a slash, so the key keeps its own slashes intact. And the doc comment names the reason one class owns both `Mint` and `TryRead`: the minting side (storage) and the reading side (the endpoint) *can't drift* if they share one token format.

4. The purpose string is a key — and renaming it is a data-loss bug

`CreateProtector("Template.Files.Download.v1")` — that string is not a label, it's an **input to key derivation**. Data Protection derives a distinct sub-key per purpose string, so tokens minted for `"Template.Files.Download.v1"` can *only* be read by a protector created with the exact same string. That gives free isolation (a file token can never be mis-read as an MFA secret, §6.5), and it comes with a sharp trap the codebase flags everywhere it appears: **rename the purpose string and everything encrypted under the old one becomes undecryptable.** For a download token that's a shrug (they expire in minutes). For the MFA secret encrypted under `"Template.Mfa.Secret.v1"` (6.5), renaming the string locks every user out of their own second factor. That's why these strings kept their `"Template.*"` prefix through the product rename — the `.v1` suffix is the escape hatch: you bump the version *only* alongside a deliberate re-encrypt migration, never casually. A string that looks like a name but behaves like a cryptographic key is exactly the kind of thing a lesson has to point at.

5. Serving the download — the token is the authorization

```
// src/Api/Controllers/FilesController.cs
[ApiController, AllowAnonymous, Route("api/files")]
public class FilesController(IFileStorage storage, ITenantContext tenantContext,
IFileDownloadTokenizer tokenizer)
    : ControllerBase
{
    [HttpGet("{token}")]
    public async Task<IActionResult> Download(string token, CancellationToken ct)
    {
        if (!tokenizer.TryRead(token, out var tenantId, out var key)) return NotFound();
        FileObject? file;
        using (tenantContext.EnterTenant(tenantId)) // the token authorized this tenant;
scope to it
            file = await storage.GetAsync(key, ct);
        if (file is null) return NotFound();
        return File(file.Content, file.ContentType, Path.GetFileName(key)); //
Content-Disposition: attachment
    }
}
```

This is the third `EnterTenant` reunion (after the billing webhook and — you'll see — admin impersonation): an **anonymous** endpoint that is nonetheless *tenant-scoped*, because the **token is the authorization**. `TryRead` cryptographically proves the caller holds a valid, unexpired token for tenant T's key K; the endpoint then *enters* T so the storage layer scopes correctly (6.2), reads the one object, and streams it. Two disciplines from earlier lessons are load-bearing:

- **Every failure is `404`** — bad token, expired token, missing object, all identical. The

endpoint never distinguishes "invalid token" from "no such file," because the difference is a hint an attacker could use to probe what exists. Silence is the security posture (the same no-leak instinct as the readiness probe in 4.5 and the erasure path in 7.1).

- **`Content-Disposition: attachment`** (the `File(..., fileName)` overload) tells the browser

to *download*, not render inline — so a signed URL opened in the same tab downloads the file instead of replacing your SPA, and a malicious HTML upload can't execute in your origin. The filename comes from the key's basename (server-controlled), never from client input.

6. The client seam — one URL, two host behaviors

A signed URL is only useful if the client can act on it, and *how* differs by host — which is one more seam (this time in the RCL, 3.4):

```
// src/Shared.Ui/IFileDownloadLauncher.cs
public interface IFileDownloadLauncher { Task LaunchAsync(string url, string
fallbackFileName); }
```

A **browser** downloads natively — because the API sends `Content-Disposition: attachment`, `BrowserFileDownloadLauncher` just navigates to the URL and the browser saves it (a same-tab nav never disturbs the page). A **WebView** (the MAUI native shells, appendix) can't trigger a browser download, so `ShareFileDownloadLauncher` fetches the bytes and opens the OS **share sheet** instead (NATIVE-3). Same feature code calls `LaunchAsync`; the host-appropriate impl is injected. It's the identical seam discipline as `IEmailSender`/`IFileStorage`, now applied to a

client-side behavioral difference — proof the pattern isn't backend-only: any point where "the right thing to do depends on the host" becomes an interface in shared code with a per-host implementation.

7. Red — tokens that expire, tamper, and fail closed

```
Scenario: a fresh token round-trips to its tenant and key
  When I Mint(T, "docs/a.pdf", 5m) then TryRead it
  Then tenantId = T and key = "docs/a.pdf"

Scenario: an already-expired token is rejected
  Given a token minted with a negative (already-past) lifetime
  Then TryRead returns false

Scenario: a tampered token is rejected
  Given a valid token with one character flipped
  Then TryRead returns false

Scenario: the download endpoint reveals nothing on a bad token
  When GET /api/files/{garbage}
  Then 404 — indistinguishable from a missing file
```

`FileDownloadTokenizerTests` drives these. One implementation note that trips people up: `ITimeLimitedDataProtector` reads the *ambient* clock internally — you can't hand it a `FakeTimeProvider` the way the rest of the platform injects `TimeProvider` (3.3). So you test expiry the honest way the reference does: mint the token with a **negative lifetime** (`TimeSpan.FromSeconds(-1)`) so it's born already-expired, and assert `TryRead` refuses it — deterministic, no sleeping, no clock injection. It's a small reminder that a framework primitive sometimes doesn't take your injected clock, and the test adapts rather than pretends. The expiry and tamper scenarios are the security core — write them first; they prove the protector, not your code, is enforcing integrity and lifetime.

8. Run it

```
dotnet test tests/Api.Tests --filter FileDownloadTokenizer
# live (local disk): request a download URL for an object → GET it in the browser → it
downloads
# (never renders); wait past the lifetime → 404; flip a character in the token → 404
# inspect the DataProtectionKeys table → the keyring is really in your database
```

9. Architecture Decision

ARCHITECTURE DECISION

The fork: how are signed tokens and at-rest secrets protected, and where do the keys live? (a) Roll your own HMAC with a key in config/appsettings; (b) Data Protection with the **default on-disk** keyring; (c) Data Protection with the keyring **persisted to the database** and a stable application name, accessed through purpose-scoped protectors.

Chosen: (c) (ADR-010). Data Protection gives you audited encryption + MAC + automatic key rotation — you never touch raw crypto. DB persistence makes keys durable across restarts and *shared* across instances (the multi-replica correctness fix); the stable app name keeps the keyring one lineage; purpose strings isolate each use (downloads vs MFA vs challenges) for free. One substrate serves four features.

Rejected: (a) — hand-rolled crypto and a config-file key is the classic footgun: no rotation, a leaked key in source control, and subtle MAC mistakes. (b) — the on-disk default silently breaks the moment you run a second instance or redeploy to fresh storage (each has its own keyring), and it's the bug that only shows up in production.

The trade: the purpose string becomes a load-bearing cryptographic input — rename it and you orphan data (§4), a real operational hazard the code mitigates with the `"Template.*"` freeze and `.v1` versioning. Accepted: the alternative is worse crypto with worse operations. A missed re-encrypt migration is a known, documented risk, not a surprise.

10. Checkpoint

```
git add -A && git commit -m "feat: data protection keyring (DB-persisted) + signed download tokens + serving endpoint"
git tag lesson-6.3
```

You should now be able to: explain why a signing key must be persisted (durable) and shared (multi-instance) and how `PersistKeysToDbContext` + a stable app name achieve both; describe `ITimeLimitedDataProtector`'s encrypt/MAC/expiry and why `TryRead` fails closed; explain the purpose string as a key-derivation input and the rename trap; and justify the token-as-auth, always-404, `Content-Disposition: attachment` download endpoint.

Next: *Lesson 6.4 — The SSRF seam* — a guard that decides whether the server may fetch a client-supplied URL at all, built now because Part 7's outbound webhooks depend on it.

Lesson 6.4 — The SSRF seam

> **Part 6 · B2B essentials & security hardening.** This is the shortest lesson in the part and one of the most important — a single small interface that stands between your server and a whole class of attack. The moment your platform will fetch a URL *the tenant chose* — an outbound webhook target, an avatar-by-URL, an import-from-link — you've handed a customer the ability to point your server at things *it* can reach but *they* can't: your internal network, your database, your cloud's metadata endpoint. That's **Server-Side Request Forgery (SSRF)**, and the fix is a guard that every such fetch must pass. It's built here, one lesson ahead of Part 7's webhooks, because that's the feature that needs it.

Goal: one abstraction, `IOutboundUrlGuard`, that answers "may the server POST to this client-supplied URL, right now?" — allowing only HTTPS outside development, rejecting any host that resolves to a private/loopback/link-local/metadata address, and re-resolving at call time so DNS rebinding can't slip through.

Concepts & principles: SSRF as a threat; allowlist-by-property (public-routable only) vs blocklist; DNS rebinding and why you re-resolve at call time; check-at-use not check-at-create; fail-closed on unresolvable hosts; a dev-permissive / prod-strict guard.

Maps to: FOUNDATION_RULES R3, v2 audit GAP-2 · repo: `src/Core/Abstractions/IOutboundUrlGuard.cs`, `src/Infrastructure/Http/OutboundUrlGuard.cs`, tests: `tests/Api.Tests/Webhooks/OutboundUrlGuardTests.cs`.

Prerequisites: 1.4 (the dev-vs-prod environment split this leans on), 2.3 (the Core-seam / Infrastructure-impl pattern).

---

1. Motivate — the request your server can make that the attacker can't

Your platform lets a tenant register a webhook URL — "POST to `https://my-app.example.com/hook` whenever X happens." Innocent. But your *server* makes that POST, and your server sits inside your infrastructure. So a malicious tenant registers `http://169.254.169.254/latest/meta-data/` — the cloud metadata endpoint — and now your server dutifully fetches your instance's IAM credentials and delivers them into the webhook body the attacker controls. Or they register `http://localhost:5432` and probe your database port, or `http://10.0.0.5/admin` to reach an internal service that trusts anything on the private network. The attacker can't reach any of these from the outside — but *your server can*, and you just made it do so on their behalf. That's SSRF: the confused-deputy problem, where you're the deputy with network access and the attacker supplies the target.

The defense is not "validate the URL looks nice." It's a guard that resolves where the URL actually *points* and refuses anything that isn't a public, internet-routable address — applied at the moment of every fetch, no exceptions.

2. The seam — a yes/no question in Core

```
// src/Core/Abstractions/IOutboundUrlGuard.cs
public interface IOutboundUrlGuard
{
    /// True if the URL is well-formed and safe to POST to from the server right now.
    ValueTask<bool> IsAllowedAsync(string? url, CancellationToken ct = default);
}
```

One method, one boolean. Making this an interface in Core (not a static helper) is the usual seam move for two concrete payoffs: features depend on the *question*, not the answer, so tests can inject a permissive guard (`AllowAllUrlGuard`) to exercise webhook logic without wrestling DNS; and the real policy lives in one Infrastructure class that every caller routes through. The doc comment sets the rule that Part 7 will obey: "*Every outbound call to a tenant-supplied URL (webhook create + every send) must pass through this.*" Every send — not just at registration. §4 explains why that word is load-bearing.

3. The policy — allowlist public, resolve the host

```
// src/Infrastructure/Http/OutboundUrlGuard.cs
public async ValueTask<bool> IsAllowedAsync(string? url, CancellationToken ct = default)
{
    if (!Uri.TryCreate(url, UriKind.Absolute, out var uri)) return false; // malformed → no
    if (environment.IsDevelopment())
        return uri.Scheme is "http" or "https"; // permissive locally
    if (uri.Scheme != Uri.UriSchemeHttps) return false; // https-only in prod

    IPAddress[] addresses;
    try
    {
        addresses = IPAddress.TryParse(uri.Host, out var literal)
            ? [literal]
            : await Dns.GetHostAddressesAsync(uri.Host, ct); // resolve the host NOW
    }
    catch { return false; } // unresolvable → refuse, don't guess

    return addresses.Length > 0 && Array.TrueForAll(addresses, IsPublic); // EVERY address
must be public
}
```

And `IsPublic` is a careful, documented blocklist of everything non-routable — for IPv4: loopback, `10/8`, `172.16/12`, `192.168/16`, link-local `169.254/16` (which *includes* the metadata endpoint), CGNAT `100.64/10`, and `0/8`; for IPv6: link-local `fe80::/10`, unique-local `fc00::/7`, and the unspecified address (with IPv4-mapped IPv6 normalized first so `::ffff:10.0.0.1` can't sneak past). Four decisions are worth their weight:

- **HTTPS-only outside development.** A tenant webhook target must be `https` in production —

no cleartext delivery of signed payloads. Dev is permissive (`http` + loopback) so your local endpoints and the test suite work; the environment split (1.4) draws the line.

- **Resolve the host, then judge the address — not the string.** `http://internal.evil.com`

looks like a public hostname but might resolve to `10.0.0.5`. You cannot judge safety from the URL's text; you must resolve it to IPs and check *those*. This is allowlisting by *property* (is it public-routable?), not by name.

- **Every resolved address must pass.** A host with two A-records — one public, one `127.0.0.1`

— is rejected. `Array.TrueForAll` means one bad address fails the whole URL; there's no "well, one of them was fine." Fail-closed on the *set*.

- **Unresolvable → refuse.** A host that won't resolve returns `false`, not "probably fine." The

guard never guesses in the attacker's favor.

4. DNS rebinding — why the check is at use, not create

Here is the subtle attack the design specifically defeats. Suppose you validated the webhook URL *once*, at registration: the attacker's `evil.com` resolves to a harmless public IP, passes, and you store it as trusted. Then the attacker changes `evil.com`'s DNS to point at `169.254.169.254`. Every future delivery now fetches the metadata endpoint — and your create-time check waved it through days ago. That's **DNS rebinding**, and it's why the guard re-resolves the host *at call time, on every send*, not once at registration. The freshest resolution is the only one that matters, because it's the one your server is about to actually connect to. "Check at use, not at create" is a general secure-design principle — a decision made in the past about a mutable external fact (DNS) is not a decision you can still trust — and SSRF is its sharpest illustration. This is exactly why §2's contract says the guard runs on *every* webhook send, not just at subscription time.

> **An honest caveat.** Re-resolving and then connecting is still a small TOCTOU window (the IP could change between the guard's lookup and the HTTP client's own lookup). Fully closing it means pinning the guard-approved IP into the actual connection. The platform accepts the residual window as a documented trade — the re-resolve defeats the practical rebinding attack, and pinning adds real complexity to the HTTP stack. Naming the limit is part of teaching the defense: a guard is a strong mitigation, not a proof.

5. Red — the guard's truth table

```
Scenario: a public HTTPS URL is allowed (prod)
  Then IsAllowed("https://api.stripe.com/…") is true

Scenario: the cloud metadata endpoint is blocked (prod)
  Then IsAllowed("https://169.254.169.254/…") is false

Scenario: private, loopback, and CGNAT ranges are blocked (prod)
  Then IsAllowed for 10.x / 192.168.x / 127.0.0.1 / 100.64.x is false

Scenario: plain HTTP is blocked in production
  Then IsAllowed("http://api.example.com") is false

Scenario: a host that resolves to both a public and a private IP is blocked
  Then the whole URL is rejected (one bad address fails all)
```

`OutboundUrlGuardTests` drives the truth table against the production policy (it constructs the guard with a non-development environment). The metadata-endpoint and mixed-resolution scenarios are the ones that prove the design; the mixed case in particular is the bug a naive "check the first IP" guard would ship. Because the guard is pure policy over an environment +

DNS, most of it is testable with literal IPs and no network.

6. Run it

```
dotnet test tests/Api.Tests --filter OutboundUrlGuard
# there's nothing user-facing yet — this seam has no endpoint. It's infrastructure that
Part 7's
# webhook create + every webhook send will call before touching the network.
```

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how does the platform stop SSRF when it fetches tenant-supplied URLs? (a) Trust the URL / validate only its syntax; (b) a name-based blocklist ("reject anything with `localhost` or `169.254`"); (c) an `IOutboundUrlGuard` that resolves the host to IPs and allows only public-routable addresses, HTTPS-only in prod, re-resolved on every use.

Chosen: (c) (R3, GAP-2). Judging the *resolved address* is the only defense that can't be fooled by a public-looking hostname pointing inward; re-resolving at use defeats DNS rebinding; requiring *all* resolved addresses to be public closes the multi-record trick; HTTPS-only protects the payload in transit. One seam, called by every outbound fetch, keeps the policy in one auditable place.

Rejected: (a) — syntactic validation is no defense; `http://10.0.0.5` is perfectly well-formed. (b) — a string blocklist is trivially bypassed (`http://0x7f000001`, a hostname that resolves to a private IP, IPv6 forms, decimal-encoded IPs); you cannot enumerate every textual spelling of an internal address, which is exactly why you resolve and judge the IP.

The trade: re-resolving on every send costs a DNS lookup per delivery, and a small TOCTOU window remains (§4) short of connection-level IP pinning. Accepted: the lookup is cheap relative to the HTTP call it guards, and the residual window is documented — a strong, practical mitigation beats a perfect one that never ships.

8. Checkpoint

```
git add -A && git commit -m "feat: SSRF guard — IOutboundUrlGuard, public-routable-only,
re-resolve at use"
git tag lesson-6.4
```

You should now be able to: explain SSRF as a confused-deputy attack and name concrete targets (metadata, loopback, RFC-1918); argue why you must resolve the host and judge the address rather than the string; explain DNS rebinding and why the check belongs at use, not at create; and state the fail-closed rules (unresolvable → deny, one private address → deny, HTTP in prod → deny) and the honest TOCTOU limit.

Next: *Lesson 6.5 — MFA / TOTP* — a second authentication factor: an encrypted secret, hashed single-use recovery codes, anti-replay time-steps, and step-up enforced on **every** sign-in path.

Lesson 6.5 — MFA / TOTP

> **Part 6 · B2B essentials & security hardening — finale.** A password (or a magic link, or an OAuth token) proves *one* factor: something the user knows or an account they control. Multi-factor authentication adds a *second* — something they *have*, a phone running an authenticator app — so a stolen password alone can't get in. This lesson builds authenticator-app TOTP from scratch: an encrypted secret (on the keyring from 6.3), hashed single-use recovery codes, RFC-6238 anti-replay, and — the architecturally hardest part — **step-up enforced on every single sign-in path**, so there's no back door that skips the second factor. It's also a capstone: it composes Data Protection (6.3), the token hasher (2.4), the injected clock (3.3), and the contributor seam (2.9) into one feature.

Goal: a user can enroll an authenticator app; the TOTP secret is stored encrypted and never re-exposed; recovery codes are hashed and single-use; a login by *any* method challenges for the second factor when MFA is on; a TOTP can't be replayed within its validity window; and disabling MFA requires proving possession.

Concepts & principles: TOTP (RFC-6238); factor-of-possession; encrypt-the-secret / hash-the-recovery-codes (why the asymmetry); the enrollment provisioning URI + QR; anti-replay via last-accepted time-step; the login step-up as a *choke point* every auth path funnels through; the signed single-use challenge; MFA state as erasable user data.

Maps to: ADR-012, MFA-1/2/3/4, R32 · repo: `src/Api/Services/MfaService.cs`, `MfaChallengeService.cs`, `MfaLoginService.cs`, `src/Core/Entities/UserMfa.cs`, `MfaRecoveryCode.cs`, `src/Api/Services/MfaUserDataContributor.cs`, `src/Shared.Ui/Components/MfaCard.razor`, tests: `tests/Api.Tests/Mfa/*`, `tests/Api.Tests/Integration/MfaStepUpIntegrationTests.cs`.

Prerequisites: 6.3 (Data Protection — encrypts the secret + signs the challenge), 2.4 (`ITokenHasher`/`ITokenGenerator` — recovery codes), 3.3 (injected clock — TOTP + anti-replay), 2.5–2.6 (the sign-in paths this wraps), 2.9 (the contributor seam — erasure).

---

1. Motivate — the second factor, and the back door problem

Adding TOTP looks easy: generate a secret, show a QR code, verify a 6-digit code at login. The authenticator-app part genuinely is easy — a well-worn library (Otp.NET) does the RFC-6238 math. The *hard* part, the part that separates real MFA from theater, is a question of completeness: **does every way of logging in enforce it?** A platform with password login, magic links, email OTP, and three OAuth providers has *six* front doors. If five of them challenge for the second factor and one — say, the OAuth callback — quietly issues a session without it, then MFA is worthless: an attacker with a stolen Google session skips it entirely. The security of MFA is exactly the security of its *weakest* sign-in path. So the architectural work isn't the TOTP check; it's arranging the code so that **there is one place a session gets issued, and that place enforces step-up**, making a skip structurally impossible rather than a thing you hope every path remembered. That's §5, and it's the heart of the lesson.

2. Enrollment — an encrypted secret you show once

```
// src/Api/Services/MfaService.cs — begin enrollment
var secret = Base32Encoding.ToString(KeyGeneration.GenerateRandomKey(20)); // 160-bit TOTP
secret
await mfa.AddAsync(new UserMfa
{
    UserId = userId,
    EncryptedSecret = _protector.Protect(secret), // Data Protection (6.3) — never stored
in plaintext
    Enabled = false, // not on until a code proves possession
    EnrolledAt = clock.GetUtcNow(),
}, ct);
var uri = $"otpauth://totp/{label}?secret={secret}&issuer={issuer}&digits=6&period=30";
return new MfaEnrollment(uri, secret); // the ONLY time the secret leaves the server
```

The design decisions are dense here:

- **The secret is *encrypted at rest*, not hashed.** This is the deliberate asymmetry with

passwords and recovery codes (§4). A TOTP secret is *shared* — both your server and the authenticator app must know it to compute the same rolling code — so the server has to be able to *recover* it to verify. Recoverable means encrypted (reversible with the key), via the `"Template.Mfa.Secret.v1"` protector on the DB keyring (6.3). And recall 6.3's trap: rename that purpose string and every enrolled user's secret becomes undecryptable — which is why it's frozen through the rebrand.

- **`Enabled = false` until confirmed.** Enrollment stores the secret but does *not* turn MFA

on. The user must first prove their app is correctly configured by returning a valid code (§3). This prevents the lockout-on-enroll disaster: turning MFA on before confirming the app works would strand a user who scanned the QR wrong.

- **The secret is returned exactly once**, as the `otpauth://` provisioning URI (which

`MfaCard.razor` renders as a QR via `mfa-qr.js`) and as the raw string for manual entry. After this call it never leaves the server again — `GET`ting the MFA status returns *enabled/not*, never the secret. "Show once, never again" is the same discipline as recovery codes and API keys.

- **The clock is injected** (`clock.GetUtcNow()`, 3.3) — `EnrolledAt`, and more importantly the

TOTP verification window, are all computed from `TimeProvider`, so tests drive time exactly.

3. Confirm & verify — TOTP with a skew window, and anti-replay

Confirmation checks one code and, on success, flips `Enabled` and issues recovery codes (§4). The *login* check is `VerifyAsync`, and it hides the lesson's second subtlety:

```
// src/Api/Services/MfaService.cs — the login verification (abridged)
if (TryVerifyTotp(record, code, out var timeStep))
{
    // Anti-replay (R32 / v2 audit LOGIC-S1): a code is valid for ~90s (±1 step). The same
code
    // must not mint two sessions. Reject a step already used (or older); record the
accepted step.
    if (record.LastVerifiedTimeStep is { } last && timeStep <= last) return false;
    record.LastVerifiedTimeStep = timeStep;
    mfa.Update(record); await mfa.SaveChangesAsync(ct);
    return true;
}
// else: try a single-use recovery code (§4)
```

with the TOTP math behind a small helper:

```
private static readonly VerificationWindow Window = new(previous: 1, future: 1); // ±1 step
for clock skew
return new Totp(Base32Encoding.ToBytes(secret)).VerifyTotp(code.Trim(), out timeStep,
Window);
```

Two things:

- **The ±1 step window** accepts the code for the previous and next 30-second step, not just

the current one. Real device clocks drift and users type slowly; without a small window, legitimate codes get rejected at the boundary. One step each way (~90 seconds total) is the standard skew tolerance — wide enough to be usable, narrow enough to be safe.

- **Anti-replay via `LastVerifiedTimeStep`.** That skew window creates a problem: a single code

is valid for ~90 seconds, so an attacker who snoops one code could replay it to mint a *second* session before it expires. The fix: the `UserMfa` row remembers the *time-step* of the last code it accepted for a login, and rejects any code whose step is `<=` that. So a given code works exactly once; a replay within its window matches an already-consumed step and fails. This is rule R32, and it's the kind of correctness bug that a naive "does the code verify?" check ships without noticing. (Enrollment-confirm deliberately does *not* record the step — it isn't a login, and conflating the two once caused a real bug the comment memorializes.)

4. Recovery codes — hashed, single-use, and the asymmetry

Lose your phone and TOTP alone locks you out forever — so confirmation issues ten recovery codes:

```
// src/Api/Services/MfaService.cs — on confirm
var codeRaw = tokenGenerator.GenerateToken(); // shown to the user once
raw.Add(codeRaw);
await recoveryCodes.AddAsync(new MfaRecoveryCode { UserId = userId, CodeHash =
hasher.HashToken(codeRaw) }, ct);
// … on verify: look up by hash where UsedAt == null, then stamp UsedAt (consume)
```

Here's the asymmetry that makes MFA a *teaching* feature: the TOTP secret is **encrypted** (§2), but recovery codes are **hashed**. Why treat two secrets differently? Because of what the server must *do* with each. The TOTP secret must be *recovered* to recompute the rolling code, so it must be reversible — encrypted. A recovery code only ever needs to be *compared* to what the

user types — the server never needs the original back — so it's stored as a one-way hash (the exact `ITokenHasher` treatment as magic-link and OTP tokens, 2.4), and even a full database leak reveals no usable code. **Hash what you only compare; encrypt what you must recover.** That one sentence is a security principle worth more than the feature. Recovery codes are also single-use (`UsedAt` stamped on redemption) and re-issued fresh on every enable, so a used set can't be replayed and an old set can't linger.

5. The step-up choke point — one place issues a session

Now the architecturally hard part from §1. The platform funnels **every** sign-in path through one service at the exact moment a session would be issued:

```
// src/Api/Services/MfaLoginService.cs
public async Task<(AccessSession?, string?)> CompleteOrChallengeAsync(
    User user, string provider, string ip, bool native, CancellationToken ct = default)
{
    if (await mfa.IsEnabledAsync(user.Id, ct))
        return (null, challenges.Mint(user.Id, provider, native)); // MFA on → NO session,
a challenge instead
    var session = await sessionService.IssueAsync(user, provider, ip, native, ct);
    return (session, null); // MFA off → session as before
}
```

The rule is structural: **no path calls `sessionService.IssueAsync` directly for a primary login — every one calls `CompleteOrChallengeAsync`.** Password login, magic-link verify, email OTP, each OAuth callback, and the native variants all resolve a user and then hand it to this one method (`AuthController`, `NativeAuthController`). If MFA is enabled it returns a *challenge* and *no session*; the sign-in is not complete until the client posts the challenge plus a code to `VerifyChallengeAsync`. Because there's exactly one door to a session and it checks MFA, you cannot add a sixth sign-in path that *forgets* the second factor — the only way to get a session is through the choke point. That's the difference between MFA that's enforced and MFA you *hope* is enforced everywhere: the reference tests this on JSON, redirect-based OAuth/magic-link, and native paths (MFA-2/3/4) precisely to prove no door was left unlocked.

(The `native` bool threaded through the signature only distinguishes how the completed session is *returned* — a cookie for the redirect-based native shells vs. a JSON body for the web SPA — and is inert until the MAUI shells exist, appendix. Building web-first (Golden Rule 5), you can omit it entirely and add it back when the native shells arrive; the choke-point logic is identical either way. The redirect-based OAuth/magic-link step-up is likewise the *matured* shape — a web-only baseline can complete the challenge over JSON and grow the redirect later.)

The challenge itself reuses 6.3's substrate:

```
// src/Api/Services/MfaChallengeService.cs — signed, short-lived, single-use
_protector.Protect($"{userId:D}|{provider}|{native}|{challengeId}", Lifetime); //
DP-signed, 5-min expiry
// challengeId tracked in IMemoryCache; Consume() removes it → a second redemption of the
same challenge fails
```

It's a signed, 5-minute, **single-use** token that carries *no secret* — it only asserts "this user still owes a second factor for this login." Signed (Data Protection, 6.3) so it can't be forged; single-use (an `IMemoryCache` id consumed on redemption) so a captured challenge can't be

replayed. And `VerifyChallengeAsync` orders its steps carefully: verify the code *before* consuming the challenge, so a *wrong* code doesn't burn the challenge (the user can retry), but a *correct* code consumes it before issuing the session (no double-redemption). That ordering is a small correctness gem worth pausing on.

6. Disable & erase — closing the lifecycle

Disabling MFA requires a valid code (`DisableAsync` calls `VerifyAsync` first) — you must prove possession to *remove* the factor, exactly as you did to add it, so a walk-up attacker at an unlocked screen can't quietly strip protection. And MFA state is **user-scoped identity data**: a user's `UserMfa` row and recovery codes must be wiped on account erasure. That erasure hook, `MfaUserDataContributor`, is the *per-user* analogue of the tenant contributor seam (2.9) — and here's a forward reference exactly like Part 5's `IBillingNotifier`: the per-user erasure seam (`IUserDataContributor`) itself is built in **Part 7's GDPR lesson (7.1)**, where a build-time gate (R12) will *fail the compile* if any piece of per-user data lacks an eraser. So in teaching order you note the obligation now and satisfy it there — `MfaUserDataContributor` becomes the seam's first implementation the moment 7.1 introduces it, wiping the `UserMfa` row and recovery codes, with no change to the MFA feature you built here. The lifecycle (enroll, confirm, verify, disable, *erase*) is designed now and completed one part later — the seam pattern letting a later-taught subsystem reach back and finish an earlier feature.

7. Red — the security-critical scenarios first

```
Scenario: a stolen password alone can't log in when MFA is on
  Given a user with MFA enabled
  When they complete primary auth
  Then no session is issued — a challenge is returned instead

Scenario: a TOTP can't be replayed within its window (R32)
  Given a code that just minted a session (time-step S)
  When the same code is presented again
  Then it is rejected (its step ≤ LastVerifiedTimeStep)

Scenario: every sign-in path enforces step-up
  Then password, magic-link, OTP, and each OAuth callback all return a challenge, not a
session

Scenario: a recovery code works once
  When an unused recovery code is presented
  Then it succeeds and is consumed; presenting it again fails

Scenario: disabling MFA requires a valid code
  When disable is requested with a wrong code
  Then MFA stays enabled
```

`MfaServiceTests` covers the TOTP/anti-replay/recovery logic; `MfaChallengeServiceTests` the signed single-use challenge; and — the one that proves §1 — `MfaStepUpIntegrationTests` drives *real* sign-ins across the paths to assert each returns a challenge. Write the replay scenario and the every-path scenario first; they're the two that catch the bugs that matter.

8. Run it

```
dotnet test tests/Api.Tests --filter Mfa
# live: enroll (scan the QR in the MFA card with any authenticator app) → confirm with a
code →
# sign out → sign back in by ANY method → you're challenged for a code; reuse the same code
twice → refused;
# spend a recovery code → it won't work a second time
```

9. Architecture Decision

ARCHITECTURE DECISION

The fork: how is a second factor added and *enforced*? (a) TOTP verified only on the password path; (b) TOTP enforced by adding a check to each sign-in path; (c) TOTP with the secret encrypted + recovery codes hashed, anti-replay on the time-step, and step-up enforced at a single session-issuing **choke point** every path funnels through.

Chosen: (c) (ADR-012, R32). One choke point (`CompleteOrChallengeAsync`) makes "every path enforces MFA" structural, not a per-path promise; encrypt-vs-hash matches each secret to what the server must do with it; the last-accepted time-step closes the replay window the skew tolerance opens; the signed single-use challenge carries no secret and can't be forged or replayed; the contributor seam guarantees erasure.

Rejected: (a) — MFA is only as strong as the weakest sign-in path; leaving OAuth or magic links unprotected is a back door that defeats the whole feature (§1). (b) — a check per path *works* until someone adds a seventh path and forgets; correctness-by-remembering is the bug factory the choke point eliminates. Storing the TOTP secret hashed — impossible; the server must recompute the code, so it must be recoverable (encrypted).

The trade: the choke point means every auth path must be routed through it (a real refactor, and a rule to hold in review), and TOTP inherits authenticator-app UX friction (clock skew, lost devices → recovery codes). Accepted: enforced-everywhere MFA is the entire value proposition, and the alternative — MFA with a hole — is worse than no MFA because it *feels* secure.

10. Checkpoint

```
git add -A && git commit -m "feat: MFA/TOTP — encrypted secret, hashed recovery codes,
anti-replay, step-up on every path"
git tag lesson-6.5
```

Part 6 complete. The platform now speaks the full authorization trilogy (401/402/403 via the RBAC matrix), stores tenant-isolated blobs behind a seam, protects secrets and signs tokens on a DB-persisted keyring, guards every outbound fetch against SSRF, and enforces a second factor on every door in.

You should now be able to: explain TOTP and the factor-of-possession; justify encrypting the secret while hashing recovery codes ("hash what you compare, encrypt what you recover"); describe the ±1 skew window and the anti-replay time-step that closes it; and — the big one — explain why a single session-issuing choke point makes step-up enforceable everywhere and why per-path checks don't.

Next: *Part 7 — Compliance & extensibility* — GDPR export/erasure on the contributor seams, in-app notifications, the config-gated public API + API keys, outbound webhooks (composing the SSRF guard and the keyring from this part), and the audited admin back-office.

Part 7 — Compliance & extensibility

Lesson 7.1 — GDPR: export & erasure

> **Part 7 · Compliance & extensibility.** Two rights a modern platform must honor: *access* (give me a copy of my data) and *erasure* (delete me). They sound like features you bolt on at the end — a script that dumps some tables, a `DELETE` somewhere. They're not. Done naively, export leaks secrets and erasure either misses data (a compliance failure) or strands a tenant ownerless (a correctness failure). This lesson builds both on the *contributor seam* you already have — the same pattern that cleans up billing on dissolve (5.3) now assembles an export bundle and drives per-feature teardown — plus a new per-*user* contributor whose absence **fails the build**. Compliance becomes a structural property, not a checklist someone hopes was followed.

Goal: a tenant owner can download a complete, secret-free JSON export of the tenant's data; a user can erase their account, honoring the single-owner invariant; every feature's data is included in the export and removed on erasure *automatically*, via contributor loops that a new feature plugs into — with a build-time gate that fails if a new user-keyed table lacks an eraser.

Concepts & principles: the right to access / right to erasure; the contributor seam on two axes (tenant + user); export and wipe as two faces of one contract; secret-free exports; the single-owner invariant in erasure; audit-survives-erasure (actor ids, never PII); one transaction; a build-time completeness gate (R12).

Maps to: ADR-011, GDPR-1/2, R12 · repo: `src/Core/Abstractions/IUserDataContributor.cs`, `src/Api/Services/TenantExportService.cs`, `AccountErasureService.cs`, `MfaUserDataContributor.cs`, `NotificationUserDataContributor.cs`, tests: `tests/Api.Tests/Gdpr/*`.

Prerequisites: 2.9 (`ITenantDataContributor` + dissolve), 5.3 (billing's contributor — export + wipe), 6.2/6.3 (file storage + signed URL — the export's delivery), 4.6 (the audit log that survives erasure), 6.5 (MFA — its deferred eraser lands *here*).

---

1. Motivate — two rights, four ways to get them wrong

Export looks trivial: serialize the tenant's rows to JSON, hand it over. But which rows? Miss a feature's data and the export is incomplete. Include the invitation *token hashes* and you've handed a user cryptographic material. Hard-code the list of what to export and every new feature silently falls out of the bundle until someone remembers to add it.

Erasure is worse. `DELETE FROM users WHERE id = …` orphans everything that referenced the user, and if that user was the *sole owner* of a tenant with three other members, you've just decapitated a live team. Or you delete the user but forget their MFA secret and notification rows — a compliance failure that says "forgotten" but isn't. Or you delete the audit trail too, destroying the record of *who did what* right when you might need it.

Both rights share one root problem: **completeness across a growing set of features.** The answer is not a hand-maintained list — it's a seam every feature registers with, so "all the data" is "whatever's registered," and a new feature that forgets to register is caught by a gate, not by a customer.

2. One seam, two faces — export and wipe

You already built `ITenantDataContributor` for dissolve (2.9/5.3): each feature implements `HasDataAsync`/`WipeAsync` so teardown covers it without a central list. GDPR grows that same interface two more members — `ExportKey` and `ExportAsync` — so **the one contributor a feature registers now answers both "wipe yourself" and "describe yourself for export."** That's the elegant part: export and erasure are two faces of the same question ("what data does this feature own for a tenant?"), so they ride one seam. Billing's contributor (5.3) already returns a `"billing"` export section next to its wipe; the export service just asks every contributor:

```
// src/Api/Services/TenantExportService.cs — the contributor loop
foreach (var contributor in contributors)
    bundle[contributor.ExportKey] = await contributor.ExportAsync(tenantId,
cancellationToken);
```

Add a feature with tenant data, implement the one contributor, and it appears in the export *and* is wiped on dissolve — no edit to the export or dissolve service. The `ExportKey` is unique per contributor (an arch test enforces it, 3.3), so sections can't collide.

3. Export — complete, secret-free, delivered as a signed URL

`TenantExportService` assembles core tenant data plus every contributor's section, and two disciplines govern it:

```
// src/Api/Services/TenantExportService.cs (core)
var bundle = new Dictionary<string, object?>
{
    ["exported_at"] = clock.GetUtcNow(),
    ["tenant"] = new { tenant.Id, tenant.Name, tenant.CreatedAt },
    ["members"] = members.Select(m => new { m.UserId, m.Email, m.DisplayName, m.Role,
m.JoinedAt }),
    // Token hashes are deliberately omitted — never export secrets (ADR-011).
    ["invitations"] = pending.Select(i => new { i.InvitedEmail, i.Status, i.CreatedAt,
i.ExpiresAt }),
};
// … contributor sections …
using (var stream = new MemoryStream(payload))
    await files.PutAsync(key, stream, "application/json", ct); // stored via IFileStorage
(6.2)
var url = await files.GetDownloadUrlAsync(key, LinkLifetime, ct); // signed, 15-min URL
(6.3)
await audit.RecordAsync("tenant.exported", actorUserId, nameof(Tenant),
tenantId.ToString(), new { key }, ct);
```

- **Secret-free.** Invitation *token hashes*, API-key hashes, MFA secrets — none appear. The

export gives a user *their data*, not the platform's cryptographic material; the omission is explicit and commented so a future edit doesn't casually add a hash "for completeness." (Note the billing contributor's own `ExportAsync` in 5.3 returns plan/status only — never Stripe ids — the same rule pushed into each feature.)

- **Delivered as a signed URL, not an HTTP body.** The bundle is written to `IFileStorage`

(6.2) and returned as a 15-minute signed download URL (6.3). This is the entire files+data-protection arc from Part 6 paying off: an export can be large (stream it to storage, don't buffer

it in a response), and the download is a tenant-scoped, expiring, single-object URL — the exact capability 6.2/6.3 built. The export is **owner-gated** (`ExportData`, the permission from 6.1) and **audited** (the tenant sees that an export was taken).

One thing the abridged snippet doesn't shout but a builder must know: `ExportAsync` runs *within the tenant's scope*. The members and pending-invitations reads go through the ordinary tenant-scoped queries (2.6), so they only return data when a tenant is entered — which the owner-gated endpoint provides for free (the caller's JWT carries the tenant). Test the service in isolation without a tenant context and the `invitations` section comes back empty; that's not a bug, it's the tenant filter correctly seeing "no tenant, no rows." Bind the context to the tenant (as the endpoint does) and it fills in.

4. Erasure — the single-owner invariant, in one transaction

`AccountErasureService.EraseAsync` is where the correctness lives. It first classifies the caller, then acts — all inside one transaction:

```
// src/Api/Services/AccountErasureService.cs (shape)
if (isOwner && memberCount > 1) return MustTransferFirst; // never strand a tenant
ownerless
var soloOwner = isOwner && memberCount == 1;
if (soloOwner && !confirmDissolve) return DissolveConfirmationRequired; // destructive →
confirm

await using var scope = await unitOfWork.BeginTransactionAsync(ct);
if (soloOwner) await dissolution.DissolveAsync(membership.TenantId, ct); // wipes tenant
data via contributors
else if (member) { await audit.RecordAsync("account.erased", userId, …); await
tenants.RemoveMemberAsync(membership, ct); }

foreach (var contributor in userDataContributors) // per-USER data (MFA, notifications, …)
    await contributor.WipeAsync(userId, ct);
await refreshTokens.…ExecuteDeleteAsync(ct); await logins.… await loginTokens.… await
users.… // identity-core
await scope.CommitAsync(ct);
```

Four decisions worth their weight:

- **Never strand a tenant ownerless.** A sole owner with other members must *transfer first*

(`MustTransferFirst`) — the single-owner invariant (ADR-003) that also governs leaving (2.8). A solo owner's erasure *dissolves* the tenant, but only with explicit confirmation (`DissolveConfirmationRequired`), because it's destructive. A plain member is removed and — unlike *leaving* — **not re-homed**, because the account is going away entirely.

- **Two contributor axes.** Dissolve runs the *tenant* contributors (§2, wiping tenant data);

then the *user* contributors (`IUserDataContributor`, §5) wipe the caller's per-user PII. Two seams for two ownership axes, both looped, neither hard-coded.

- **The audit trail survives.** Erasing a member records `account.erased` in the *surviving*

tenant before dropping the membership — actor id, never PII. Accountability outlives the account; "who did what" is not itself personal data you erase. (When a solo owner dissolves, the tenant's audit goes *with* the tenant — there's no surviving place to keep it.)

- **One transaction.** Every delete and every contributor wipe enlists in one unit of work, so erasure is all-or-nothing — no half-erased account if something fails midway.

5. The per-user contributor — and the gate that fails the build

Here's the seam 6.5 deferred to:

```
// src/Core/Abstractions/IUserDataContributor.cs
public interface IUserDataContributor
{
    Task WipeAsync(Guid userId, CancellationToken ct = default); // runs inside the erasure transaction
}
```

It's `ITenantDataContributor` applied to the *user* axis: any concern storing user-keyed PII *outside* identity-core registers one, and `AccountErasureService` wipes it via the loop instead of a hard-coded list. Two implementations exist immediately — `NotificationUserDataContributor` (this part's feature) and **`MfaUserDataContributor`**, the eraser 6.5 promised. The moment this seam exists, MFA's obligation is satisfied by one small class that wipes the `UserMfa` row and recovery codes — no change to the MFA feature, exactly as 6.5 said. The forward reference closes.

And the safety net: rule **R12** is an arch test that **fails the build** if a user-keyed entity has no registered `IUserDataContributor` (or isn't identity-core, which the service handles directly). So adding a new table of per-user PII and forgetting to erase it isn't a compliance bug you discover in an audit — it's a *red build* you discover in CI. That's the lesson's deepest point: **completeness enforced by the compiler beats completeness enforced by diligence.** A checklist rots; an arch test fails loudly the day the gap appears.

6. Red — completeness and the invariant

```
Scenario: the export includes every feature's section and no secrets
  When an owner exports the tenant
  Then the bundle has core + each contributor's section
  And it contains no token/secret hashes

Scenario: a sole owner with members must transfer first
  When such an owner tries to erase their account
  Then the result is MustTransferFirst — nothing is deleted

Scenario: erasing a member removes their PII but keeps the audit actor id
  When a member erases their account
  Then their user/logins/tokens are gone and MFA/notification rows are wiped (contributors)
  And the tenant's audit still shows account.erased by that actor id

Scenario: a new user-keyed table without an eraser fails the build (R12)
  Then the arch test flags it
```

`TenantExportTests` and `AccountErasureTests` drive these on real Postgres (the multi-table transactional teardown is exactly what a fake would get wrong), and the R12 arch test is the completeness guard. Write the invariant scenario first — it's the one whose absence decapitates a team.

7. Run it

```
dotnet test tests/Api.Tests --filter Gdpr
# live: as an owner, request an export → download the signed URL → inspect the JSON (all sections, no hashes)
# erase a member account → their rows are gone, MFA/notifications wiped, the tenant's audit retains the actor id
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how are export and erasure kept *complete* as features grow? (a) A hand-maintained central list of tables to export/delete; (b) per-feature ad-hoc export and delete code, called from a coordinator; (c) the contributor seam on both axes — one interface a feature registers with covers export *and* wipe (tenant) / erasure (user) — plus a build-time gate (R12) that fails if a user-keyed table has no eraser.

Chosen: (c) (ADR-011, R12). Registration means "all the data" is "whatever registered," so a new feature is included automatically; export and wipe sharing one contract removes duplicate what-data-do-I-own logic; the build-time gate turns a compliance omission into a red build. Secret-free exports and audit-survives-erasure are the correctness rules layered on top.

Rejected: (a) — a central list is the thing everyone forgets to update; the gap is silent until a regulator or a customer finds it. (b) — ad-hoc per-feature code with a coordinator still needs the coordinator to *know* every feature, reintroducing the list; and without a shared contract, export and erasure drift apart.

The trade: every feature with tenant/user data must implement a contributor (a small, enforced obligation), and the export path depends on file storage + data protection (Part 6). Accepted: a small per-feature tax buys automatic, gate-verified compliance — the alternative is a manual process that fails exactly when the stakes are highest.

9. Checkpoint

```
git add -A && git commit -m "feat: GDPR export + erasure — contributor seam (tenant+user axes), R12 gate, single-owner invariant"
git tag lesson-7.1
```

You should now be able to: explain export and erasure as completeness problems and why a registration seam solves them; describe export and wipe as two faces of one contributor contract; justify secret-free exports and audit-survives-erasure; state the single-owner invariant in erasure; and explain why the R12 build gate beats a manual checklist.

Next: *Lesson 7.2 — In-app notifications* — the sanctioned per-user (not per-tenant) exception examined, fan-out through the outbox, and the real `IBillingNotifier` that Part 5 stubbed finally landing.

Lesson 7.2 — In-app notifications

> **Part 7 · Compliance & extensibility.** Golden Rule 1 says the platform is *tenant-scoped, not user-scoped* — app data belongs to the tenant, and "only preferences are per-user." This lesson builds the **second** sanctioned per-user thing — notifications — and the interesting part is *why* it's allowed to break the tenant-scoping default and what that costs. Along the way it delivers a satisfying payoff: the `IBillingNotifier` you stubbed in Part 5 (because the notification center didn't exist yet) finally gets its real implementation, proving the seam-defers-delivery bet. Notifications also show the outbox (4.1) and the email decorator (4.2) composing into a two-channel fan-out that stays transactional with the event that triggered it.

Goal: a per-user in-app notification center (list, unread count, mark-read) plus a delivery preference (in-app / email); a `NotifyAsync` seam any feature calls that stages the in-app row on the caller's transaction and sends the email copy through the reliable outbox; and the retro-fill of `IBillingNotifier` onto this service.

Concepts & principles: the per-user exception to tenant-scoping (and its justification); notification as a staged, transactional side effect (like the audit log); two channels driven by preferences, never hard-coded; read-side scoped to the caller; composing outbox + email decorator; closing a forward reference (`IBillingNotifier`).

Maps to: ADR-013, NOTIFY-1/2 · repo: `src/Api/Services/NotificationService.cs`, `src/Core/Entities/Notification.cs`, `NotificationPreference.cs`, `src/Api/Controllers/NotificationsController.cs`, `NotificationUserDataContributor.cs`, `src/Shared.Ui/Components/NotificationBell.razor`, tests: `tests/Api.Tests/Notify/*`.

Prerequisites: 4.2 (the outbox-backed `IEmailSender`), 4.6 (the audit log — the staging pattern this copies), 7.1 (`IUserDataContributor` — notifications register one), 5.3 (`IBillingNotifier`, the stub retro-filled here).

---

1. Motivate — the per-user exception, and why it's earned

Everything the platform stores is tenant-scoped: notes belong to a tenant, subscriptions to a tenant, files to a tenant. This is the invariant that keeps tenants isolated (2.6). But a *notification* — "your payment failed," "you were invited," "an admin messaged your team" — is addressed to a **person**, not an organization. A user in three tenants has one notification bell, not three. So notifications are keyed by `UserId`, not `TenantId`, and that's a deliberate, examined exception to Golden Rule 1 — the *second* one after preferences.

Why is the exception earned here and not elsewhere? Because a notification is *about* the user's relationship to the platform across whatever tenants they're in — it's genuinely user-owned data, not tenant-owned data misfiled. The discipline is to make such exceptions *rare and conscious*: each one is an ADR (ADR-013), each one is a place the automatic tenant filter doesn't apply, and each one therefore carries its own scoping obligation — the read side must scope to *the calling user* by hand, because the query filter that normally guarantees isolation isn't watching this axis. Naming the exception is how you keep it from quietly multiplying into a user-scoped app.

2. `NotifyAsync` — a staged, transactional side effect

The seam a feature calls to notify someone:

```
// src/Api/Services/NotificationService.cs
public async Task NotifyAsync(Guid userId, string kind, string title, string body, object?
metadata = null, CancellationToken ct = default)
{
    var prefs = await GetPreferencesAsync(userId, ct); // defaults to both channels on
    if (prefs.InApp)
        await notifications.AddAsync(new Notification { UserId = userId, Kind = kind, Title
= title,
            Body = body, Metadata = …, CreatedAt = clock.GetUtcNow() }, ct); // STAGED — no
SaveChanges
    if (prefs.Email)
    {
        var user = await users.GetByIdAsync(userId, ct);
        if (user is not null)
        {
            var emailBody = BrandedEmail.Notification(title, body,
BrandedEmail.ResolveCulture(user.Locale));
            await emailSender.SendAsync(user.Email, emailBody.Subject, emailBody.Html,
emailBody.InlineImages, ct);
        }
    }
}
```

The load-bearing word is **staged**. `NotifyAsync` adds the in-app row to the caller's unit of work and does *not* save — exactly like `IAuditLog.RecordAsync` (4.6). Why does that matter? Because a notification should be *consistent with the thing it announces*. When the billing webhook flips a subscription to `past_due` and notifies the owner (5.2), the projection write and the notification must commit together: if the webhook transaction rolls back, the "your payment failed" notification must roll back too — otherwise you've told a user about a change that didn't happen. Staging on the caller's transaction makes notification an atomic part of the triggering change, not a separate effect that can succeed when the change failed. This is the *audit-log staging pattern* (4.6) reused for a second purpose — you've seen it before, and recognizing it is the point.

3. Two channels, driven by preferences — never hard-coded

`NotifyAsync` sends across whatever channels the user's preferences allow, and the two channels have *different reliability shapes* by design:

- **In-app** is staged on the caller's transaction (§2) — it commits with the triggering change, synchronously and transactionally.
- **Email** goes through `IEmailSender` — which, since 4.2, is the **outbox-backed decorator**.

So the email isn't sent inline (that would be a slow, fallible external call inside the transaction); it's *enqueued* on the outbox and delivered out-of-band with retry. The in-app row and the email-intent both ride the same transaction, and the actual SMTP send happens reliably afterward.

The email copy is branded through the shared template helper — `BrandedEmail.Notification(title, body, culture)`. That method is a *new* branded-email

template you add here, following the exact pattern of the `Otp/MagicLink/Invitation` templates from Part 2 (2.3): same `BrandedEmail` class, same HTML-encode-the-inputs discipline, same inline-logo handling — the notification email is just one more branded template, not a new mechanism. (It reuses existing localization keys, so no new resx entries.) A builder following along adds the method; it doesn't pre-exist.

Crucially, "which channels?" is a *preference lookup*, never a hard-coded decision in the calling feature. Billing doesn't know or care whether the owner wants email; it calls `NotifyAsync` and the service consults `GetPreferencesAsync` (absence □ both on — a sensible default that requires no row until a user opts out). Channels are policy, centralized here, so adding an SMS channel later is one edit to this service, not a change to every caller. That's the same "decision-in-one-place" instinct as the RBAC matrix (6.1) and the plan catalog (5.1).

4. The read side — scoped to the caller, by hand

Because notifications live on the user axis where the tenant filter doesn't apply (§1), every read *manually* scopes to the calling user:

```
// list (newest first, cursor-paginated, capped at 100), unread count, mark-read — all
filtered by UserId
notifications.Query().Where(n => n.UserId == userId) …
```

`MarkReadAsync` looks up by *both* the notification id **and** `UserId`, so a user can't mark someone else's notification read by guessing an id — the ownership check is in the query, not a separate `if`. This hand-scoping is the *cost* of the per-user exception named in §1: on the tenant axis the global filter does this for you and forgetting is impossible; on the user axis you must remember, on every query, and the tests must prove cross-user isolation because no filter guarantees it. A worked reminder that every escape from an automatic invariant buys you manual obligations.

5. Closing the loop — the real `IBillingNotifier`

Remember Part 5: `BillingWebhookHandler` and `SubscriptionLapseSweepJob` both called `IBillingNotifier`, but its real implementation depended on this notification center, which didn't exist yet — so you built the callers against the interface and stubbed delivery to the audit log, documenting that the real impl would land in Part 7. **This is Part 7.** The real `BillingNotifier` now delegates to `INotificationService.NotifyAsync`:

```
// BillingNotifier (real) — the dunning event becomes a genuine user notification
await notifications.NotifyAsync(ownerUserId, kind, title, body, metadata: null, ct);
```

Nothing in Part 5 changes — not the webhook handler, not the sweep job, not the `IBillingNotifier` signature. Only the *implementation* behind the seam is swapped, from audit-log-stub to notification-fan-out. That's the seam-defers-delivery bet from 5.3 paying off exactly as promised: a later-taught subsystem reached back and completed an earlier feature without touching it. And because the stub already staged on the caller's transaction (the audit log did too), the swap is transactionally identical — the dunning notification commits with the projection write, just as it always described. Feeling this click is the reward for having built the seam honestly two parts ago.

6. Erasure, and the UI

Notifications are per-user PII, so — per 7.1 — `NotificationUserDataContributor` implements `IUserDataContributor` and wipes a user's notifications and preference row on account erasure. It's the second implementation of the seam 7.1 introduced (alongside MFA's), and the R12 gate is what guarantees it exists. The UI is a `NotificationBell` (unread count + dropdown list, mark-read) and a `NotificationPrefsCard` (the two channel toggles), both in the RCL (3.3) so every client shares them — the bell polls the scoped read endpoints, the card writes preferences.

7. Red — transactional, scoped, preference-driven

```
Scenario: an in-app notification commits with its triggering change
  Given a feature stages a NotifyAsync inside a transaction that then rolls back
  Then no notification row exists (it wasn't a separate effect)

Scenario: preferences gate the channels
  Given a user with email disabled
  When they are notified
  Then an in-app row is created and NO email is enqueued on the outbox

Scenario: a user sees only their own notifications
  Then list/unread/mark-read for user A never touch user B's rows

Scenario: dunning notifies through the real service
  When billing marks a subscription past_due
  Then the owner gets a notification via INotificationService (not the Part-5 stub)
```

`NotificationServiceTests` and `NotificationFanOutTests` drive these. The rollback scenario proves the staging (write it first — it's the property that makes notifications trustworthy), and the own-notifications-only scenario proves the hand-scoping the per-user axis demands.

8. Run it

```
dotnet test tests/Api.Tests --filter Notify
# live: trigger something that notifies (e.g. an admin announcement, 7.5) → the bell shows
an unread count;
# toggle email off in the prefs card → the next notification makes an in-app row but
enqueues no outbox email
```

9. Architecture Decision

ARCHITECTURE DECISION

The fork: how are users notified, and where does that data live? (a) Send email inline at the call site, no in-app history; (b) an in-app center keyed per *tenant*; (c) a per-*user* notification center with a `NotifyAsync` seam that stages the in-app row transactionally and sends email via the outbox decorator, channels driven by per-user preferences.

Chosen: (c) (ADR-013). A notification is addressed to a person across their tenants, so the per-user key is correct — an examined, ADR-logged exception to tenant-scoping. Staging makes the notification atomic with its trigger (the audit-log pattern); routing email through the outbox makes delivery reliable without blocking the transaction; preference-driven channels keep the decision in one place and let callers stay ignorant of delivery. The seam is what let Part 5 call it before it existed.

Rejected: (a) — inline email is unreliable (no retry, blocks the transaction) and leaves no history a user can revisit; (b) — a per-tenant center is simply the wrong ownership model (one user, many tenants, one bell) and would fragment a person's notifications.

The trade: the per-user axis escapes the automatic tenant filter, so every read must hand-scope to the caller and be tested for cross-user isolation — a real, permanent obligation. Accepted: it's the honest cost of a genuinely user-owned concern, and keeping such exceptions rare and ADR-logged is what stops them from eroding the tenant-scoping default.

10. Checkpoint

```
git add -A && git commit -m "feat: in-app notifications — per-user center, staged fan-out,
preference-driven channels, real IBillingNotifier"
git tag lesson-7.2
```

You should now be able to: justify the per-user exception to tenant-scoping and name its cost (hand-scoped reads); explain staged/transactional notification and why it mirrors the audit log; describe the two channels' differing reliability (staged in-app vs outbox email) and preference-driven routing; and explain how the real `IBillingNotifier` closes Part 5's forward reference with zero caller changes.

Next: *Lesson 7.3 — Public API & API keys* — a config-gated, default-off programmatic surface: hash-only keys, a key that authenticates *into* a tenant, and scoped access.

Lesson 7.3 — Public API & API keys

> **Part 7 · Compliance & extensibility.** Until now every request came from *your* web app, carrying a JWT your login flow issued. A public API is different: it lets a tenant's own scripts and integrations call you *programmatically*, authenticating with a long-lived **API key** instead of a session. That raises three questions this lesson answers — how do you store a credential that must survive for months without becoming a liability if the DB leaks; how does a bare key, carrying no session, select and scope to its tenant; and how do you ship a whole new attack surface *safely by default*? The answers reuse machinery you have (token hashing, the tenant-claim scoping) and add one new discipline: **config-gated, default-off** — a feature that doesn't exist until an operator turns it on.

Goal: an owner can mint, list, and revoke tenant API keys; only a one-way *hash* of each key is stored; a presented key authenticates as a second auth scheme, resolving to a `tenant_id`-scoped principal so the normal query filter isolates it with no extra wiring; scopes limit what a key can do; and the entire surface is off unless explicitly enabled.

Concepts & principles: config-gated default-off (R21); hash-only credential storage (reused from tokens); a second authentication scheme; key→tenant principal via the `tenant_id` claim; pre-tenant-scope resolution (the key selects its tenant); scopes with reject-not-grant-all; per-key rate limiting; the key as the audit actor.

Maps to: ADR-015, PUBAPI-1/2, R21 · repo: `src/Api/Services/ApiKeyService.cs`, `src/Api/Authentication/ApiKeyAuthenticationHandler.cs`, `src/Core/Entities/ApiKey.cs`, `src/Api/Configuration/PublicApiSettings.cs`, `src/Api/Endpoints/ApiKeyEndpoints.cs`, tests: `tests/Api.Tests/PublicApi/ApiKeyServiceTests.cs`.

Prerequisites: 2.4 (`ITokenHasher`/`ITokenGenerator` — hashed single-use tokens, reused), 2.6 (the `tenant_id` claim drives tenant scoping), 6.1 (owner-only `ManageApiKeys`), 1.4 (config-gating).

---

1. Motivate — a credential that lives for months

A session token lives for minutes and rotates (2.7). An API key lives for *months* — a customer pastes it into a CI pipeline and forgets it. That longevity changes the threat model in three ways. First, **you can't store it recoverably**: a leaked database of session tokens is bad but they expire; a leaked database of long-lived API keys is a catastrophe, so keys must be stored as one-way hashes, never decryptable. Second, **the key carries no session** — there's no login flow, no JWT, no user; the bare string arriving in a header must somehow select *which tenant* it acts for and scope everything to it. Third, **a public API is a new, permanent attack surface** — so it must not exist for tenants who never asked for it; it ships *off*, and an operator turns it on deliberately.

Each of those is a decision this lesson makes, and two of the three reuse machinery you already built — the token hasher (2.4) and the tenant-claim scoping (2.6). Only "default-off" is new.

2. Default-off — a feature that isn't there until you enable it

```
// src/Api/Configuration/PublicApiSettings.cs — off unless configured
public sealed class PublicApiSettings { public bool Enabled { get; set; } /* default false
*/ }
```

The public API, the API-key auth scheme, and the `/api/public` endpoints are all gated on `PublicApi:Enabled`, which defaults to **false**. On a fresh deployment the entire surface simply doesn't respond — the routes aren't mapped, the scheme isn't registered. This is rule **R21**: *extensibility features ship config-gated and default-off.* The reasoning is a security principle — **minimize the attack surface to what's actually in use.** A tenant who never wanted a programmatic API shouldn't be running one; a vulnerability in a disabled feature can't be exploited because the feature isn't there. Turning it on is a conscious operator action (a config key, 1.4), logged and deliberate, not a default everyone inherits. The same gate governs outbound webhooks (7.4) and the admin console (7.5) — all three are power that most deployments don't need, so all three are dark until lit.

3. Hash-only storage — the token discipline, reused

Minting a key reuses the *exact* machinery from passwordless tokens (2.4):

```
// src/Api/Services/ApiKeyService.cs — create
var raw = RawPrefix + tokenGenerator.GenerateToken(); // "pk_" + cryptographically random
token
var key = new ApiKey
{
    KeyHash = tokenHasher.HashToken(raw), // ONLY the hash is stored
    Prefix = raw[..Math.Min(raw.Length, 12)], // "pk_ab12cd34" — non-secret, for display
    Scopes = string.Join(',', granted),
    …
};
return new ApiKeyCreated(key, raw); // the raw key is returned ONCE, never again
```

Everything here is a callback to 2.4's "never store a secret you only need to compare":

- **Only the hash is persisted.** `HashToken` is the same one-way hash the magic-link and OTP

tokens use. The raw key is returned exactly once at creation and never stored — lose it, mint a new one. A full DB leak reveals hashes, not usable keys.

- **The `pk_` prefix** identifies a string as an API key on sight — useful for secret-scanners

(GitHub's push protection can recognize the format) and for the auth handler to distinguish a key from a JWT (§4).

- **A non-secret `Prefix`** (first ~12 chars) is stored *in the clear* purely for display, so

the management UI can show "pk_ab12cd34…" to help an owner identify which key is which without ever revealing the whole thing. Hash what you compare; keep a harmless fragment for humans.

4. Authenticating into a tenant — the second scheme

A key carries no session, so how does it scope? Through a second authentication scheme that turns the key into a principal carrying the *same `tenant_id` claim* a JWT would:

```
// src/Api/Authentication/ApiKeyAuthenticationHandler.cs (core)
var result = await service.AuthenticateAsync(raw, Context.RequestAborted);
if (result is null) return AuthenticateResult.Fail("Invalid or expired API key.");
var claims = new List<Claim>
{
    new(ClaimTypes.NameIdentifier, result.KeyId.ToString()), // the AUDIT ACTOR is the key
itself
    new(JwtClaims.TenantId, result.TenantId.ToString()), // drives the normal tenant query
filter
    new(KeyNameClaim, result.Name),
};
claims.AddRange(result.Scopes.Select(s => new Claim(ScopeClaim, s)));
```

This is the elegant reuse. The tenant query filter (2.6) scopes on the `tenant_id` *claim* — it doesn't care whether that claim came from a JWT or an API key. So by minting a principal with the key's tenant in that claim, **every downstream query is tenant-isolated with zero extra wiring** — the same filter, the same interceptor, the same isolation an authenticated web request gets. The auth handler is the only new piece; the entire scoping regime just works. Two details:

- **`NoResult` vs `Fail`.** No key presented → `NoResult` (let anonymous or the JWT scheme handle it); a *bad* key → `Fail` (401). This lets the two schemes coexist — a request with a JWT falls through the key handler untouched, and the key extractor even accepts `Authorization: Bearer pk_…` but only if it's a `pk_` key, so JWTs still route to the Bearer scheme.
- **The key is the audit actor.** `NameIdentifier` is set to the *key id*, not a user, so audit entries for API traffic record *which key* acted — the right granularity when there's no human in the loop.

And how does a bare key find its tenant? `AuthenticateAsync` runs **before any tenant scope exists** (nothing has selected a tenant yet — the key is what selects it), so it resolves across all tenants by hash:

```
// src/Api/Services/ApiKeyService.cs — resolve pre-scope
var key = await keys.QueryAllTenants().FirstOrDefaultAsync(k => k.KeyHash == hash, ct); //
sanctioned escape hatch
if (key is null || !key.IsActive(now)) return null; // unknown / revoked / expired → no
principal
```

`QueryAllTenants()` is the sanctioned escape hatch (3.1) — legitimately used here because this is genuinely a *pre-auth, cross-tenant* read: the key hasn't told us its tenant yet, that's what we're resolving. It's exactly the kind of pre-tenant-scope read the hatch exists for, and it's in a service the arch rules permit.

5. Scopes — reject, never grant-all

A key can be limited to a subset of capabilities (`read`, `write`, …), and the mint logic makes a pointed fail-safe choice:

```
// null request → all known scopes; named-but-all-invalid → REJECT (null), never silently
grant all
if (scopes is null) return ApiScopes.All;
var requested = scopes.Select(normalize).Where(ApiScopes.All.Contains).Distinct().ToList();
return requested.Count == 0 ? null : requested; // provided but nothing valid → refuse to
create
```

The trap this avoids: a caller asks for scope `"wrte"` (typo), the code filters it out, the list is empty — and a naive implementation treats "empty scopes" as "no restriction" and mints an **all-powerful** key. That's a privilege-escalation-by-typo bug. The platform instead *rejects* the request (returns null → the endpoint 400s): if you named scopes and none were recognized, that's an error, not a grant. Fail-closed applied to authorization scope — the same instinct as RBAC's unknown-role-grants-nothing (6.1). (An *absent* scope list is the explicit "give me everything this key's owner can do" case, which is fine.) Per-key **rate limits** (PUBAPI-2) cap each key's request rate independently, and an anonymous public OpenAPI document describes the surface — both layered on once the scheme exists.

6. Management, and the anatomy

Key *management* (create/list/revoke) is ordinary tenant-scoped work: owner-gated (`ManageApiKeys`, 6.1), and because it runs inside the current tenant's scope, `CreateAsync` just adds the row and the interceptor stamps `TenantId` automatically (2.7) — the management side needs no escape hatch, only `AuthenticateAsync` does. Revocation stamps `RevokedAt` (soft-delete, so `IsActive` returns false and the key stops working immediately) rather than hard-deleting, preserving the audit trail of what existed. The `ApiKey` entity is `ITenantScoped` like everything else, so it also ships its RLS policy (ADR-020, 8.4) and appears in the tenant export/wipe — a new entity carries all the platform's obligations.

7. Red — hashing, scoping, fail-closed

```
Scenario: only the hash is stored; the raw key works once to authenticate
  When a key is minted
  Then the DB row has a hash (not the raw), and AuthenticateAsync(raw) resolves its tenant
+ scopes

Scenario: a key scopes to exactly its tenant
  Given key K for tenant T
  When a request authenticates with K
  Then queries see only T's data (the tenant_id claim drives the filter)

Scenario: a revoked or expired key fails
  Then AuthenticateAsync returns null → 401

Scenario: named-but-all-invalid scopes are rejected, not granted-all
  When a key is requested with only unrecognized scopes
  Then creation is refused (no all-powerful key)

Scenario: the surface is absent when disabled
  Given PublicApi:Enabled = false
  Then /api/public routes are not mapped and the key scheme is not registered
```

`ApiKeyServiceTests` drives the hashing/scoping/fail-closed logic on real Postgres. The scopes-reject scenario is the security test (write it first — it's the privilege-escalation footgun), and the disabled-surface scenario proves R21.

8. Run it

```
dotnet test tests/Api.Tests --filter ApiKey
# enable PublicApi in config → mint a key (copy the raw pk_… shown once) →
# curl -H "X-Api-Key: pk_…" /api/public/… → scoped to that key's tenant; revoke it → next
call 401s
# disable PublicApi → the routes 404 and the scheme is gone
```

9. Architecture Decision

ARCHITECTURE DECISION

The fork: how do programmatic clients authenticate, and how is the surface kept safe? (a) Reuse user JWTs / long-lived session tokens for scripts; (b) API keys stored encrypted (so they're recoverable); (c) hash-only API keys resolving to a `tenant_id`-scoped principal via a second auth scheme, scoped by capability, and the whole surface config-gated default-off.

Chosen: (c) (ADR-015, R21). Hash-only storage means a DB leak yields no usable keys (reusing the token discipline); the second scheme reuses the tenant-claim scoping so isolation is automatic; scopes with reject-not-grant-all fail closed; default-off minimizes the attack surface to deployments that actually want it. The key as audit actor gives the right accountability granularity.

Rejected: (a) — user sessions are short-lived and tied to a person, wrong for a months-long machine credential, and reusing them blurs human vs machine in the audit trail. (b) — encrypting keys makes them recoverable, which is exactly what you *don't* want for a long-lived credential; hashing is strictly safer because the server never needs the original back (unlike the webhook secret in 7.4, which it must recompute).

The trade: hash-only means a lost key can't be recovered, only reissued (a minor UX cost, the right security default), and default-off means operators must consciously enable the API (friction that is the point). Accepted: both costs buy a materially smaller, safer attack surface.

10. Checkpoint

```
git add -A && git commit -m "feat: public API + API keys — hash-only, key→tenant principal,
scopes, config-gated default-off"
git tag lesson-7.3
```

You should now be able to: explain why a long-lived key must be hashed (not encrypted) and how that reuses the token discipline; describe how a bare key selects and scopes to its tenant via the `tenant_id` claim and pre-scope `QueryAllTenants` resolution; justify reject-not-grant-all for scopes; and state the default-off (R21) attack-surface argument.

Next: *Lesson 7.4 — Outbound webhooks* — four seams composing: an encrypted secret (6.3), an HMAC signature, reliable outbox delivery (4.1), and the SSRF guard (6.4).

Lesson 7.4 — Outbound webhooks

> **Part 7 · Compliance & extensibility.** This lesson builds almost no new machinery — and that's the lesson. Outbound webhooks (notify a tenant's URL when something happens in their account) are the point where *four* things you already built snap together: the **encrypted secret** (6.3's Data Protection), the **HMAC signature** (new, but tiny), **reliable delivery** (4.1's outbox with retry/backoff), and the **SSRF guard** (6.4, re-checked at send). If Part 5's billing webhook was the *inbound* composition — turning an untrusted POST into a safe write — this is its outbound mirror: turning "tell the customer's server" into a signed, reliable, safe delivery. Watching the seams compose is the reward for having built each one properly.

Goal: a tenant owner can register a webhook subscription (a URL + a generated signing secret); when a platform event fires, a signed POST is delivered to that URL through the outbox with retry; the receiver can verify authenticity via an HMAC signature; the secret is encrypted at rest; every target URL is SSRF-checked at send time; and the whole feature is config-gated default-off.

Concepts & principles: the four composing seams (secret encryption, HMAC signing, outbox delivery, SSRF guard); encrypt-not-hash for a secret you must recompute with; HMAC-SHA256 request signing; at-least-once delivery + receiver idempotency; re-check-at-send for SSRF; a sender shared by async delivery and a sync "send test"; delivery log + replay.

Maps to: ADR-016, HOOKS-1/2, R21 · repo: `src/Core/Webhooks/WebhookSignature.cs`, `src/Infrastructure/Webhooks/WebhookSender.cs`, `WebhookSecretProtector.cs`, `WebhookOutboxHandler.cs`, `src/Api/Services/WebhookService.cs`, `src/Core/Entities/WebhookSubscription.cs`, `WebhookDelivery.cs`, tests: `tests/Api.Tests/Webhooks/*`, `tests/Core.Tests/WebhookSignatureTests.cs`.

Prerequisites: 4.1 (the outbox — delivery + retry), 6.3 (Data Protection — the secret), 6.4 (`IOutboundUrlGuard` — SSRF), 7.3 (config-gated default-off), 5.2 (the *inbound* webhook — this is its mirror).

---

1. Motivate — "just POST to their URL" is four problems

A tenant wants a POST to `https://their-app.com/hook` when an event fires. Naively: loop the subscriptions, `httpClient.PostAsync(url, json)`. Now count the ways that's wrong. **Authenticity:** the receiver has no way to know the POST is really from you and not a forger who guessed the URL — you need to *sign* it. **Reliability:** their server is down for thirty seconds; a fire-and-forget POST is simply lost — you need retry with backoff. **Security:** the URL is tenant-supplied, so a malicious tenant points it at `169.254.169.254` and your server fetches cloud metadata — you need the SSRF guard. **Secret handling:** the signing secret has to be stored somewhere, and stored wrong it leaks. Four distinct problems — and you built a tool for each in earlier lessons. This lesson is where they compose, and the shape of the composition *is* the architecture.

2. The signature — HMAC-SHA256, GitHub/Stripe-style

The one genuinely new piece is tiny and pure:

```
// src/Core/Webhooks/WebhookSignature.cs
public static string Compute(string secret, string body)
{
    var hash = HMACSHA256.HashData(Encoding.UTF8.GetBytes(secret),
Encoding.UTF8.GetBytes(body));
    return "sha256=" + Convert.ToHexStringLower(hash); // wire format like GitHub/Stripe
}
```

An HMAC over the *raw request body* with a secret shared between you and the receiver. The receiver recomputes it and compares — a match proves two things at once: the delivery is *from you* (only you and they know the secret) and *untampered* (any change to the body changes the hash). The `sha256=<hex>` wire format deliberately mirrors GitHub and Stripe, so a developer integrating your webhooks applies muscle memory they already have. It's pure and deterministic — hence in `Core`, unit-tested in isolation, and usable from both the delivery path and your own documentation examples. This is exactly the *inverse* of Part 5's inbound signature check: there you *verified* Stripe's signature on a POST *to* you; here you *produce* a signature on a POST *from* you. Same primitive, opposite direction.

3. The secret — encrypted, because you must recompute with it

The signing secret is stored via a Data Protection protector (6.3):

```
// src/Infrastructure/Webhooks/WebhookSecretProtector.cs
private readonly IDataProtector _protector =
dataProtection.CreateProtector("Template.Webhook.Secret.v1");
public string Protect(string secret) => _protector.Protect(secret);
public string Unprotect(string encrypted) => _protector.Unprotect(encrypted);
```

Here the encrypt-vs-hash decision goes the *other* way from API keys (7.3), and the contrast is the teaching point. An API key you only ever *compare* to what's presented, so you hash it (irreversible, safest). A webhook secret you must *recompute an HMAC with* on every delivery, so the server needs the plaintext back — which means it must be **encrypted** (reversible with the key), not hashed. It's the identical rule from 6.5's MFA secret: *hash what you only compare, encrypt what you must recover.* Same rule, and note it reuses the *same substrate* — the DB keyring from 6.3, a distinct purpose string (`"Template.Webhook.Secret.v1"`, with the same rename-orphans-data trap the comment flags). One keyring, now protecting its third kind of secret (download tokens, MFA secrets, webhook secrets).

4. Delivery — through the outbox, signed, SSRF-checked

The delivery composes the remaining two seams. A single `WebhookSender` does one signed POST and is shared by two callers:

```
// src/Infrastructure/Webhooks/WebhookSender.cs
public async Task<int> SendAsync(string url, string secret, string eventType, string
eventId, string body, CancellationToken ct)
{
    // 1. SSRF re-check AT SEND (6.4) — DNS rebinding could have re-pointed a once-valid
URL inward.
    if (!await urlGuard.IsAllowedAsync(url, ct))
        throw new InvalidOperationException("Refusing to send a webhook to a disallowed
URL.");

    using var request = new HttpRequestMessage(HttpMethod.Post, url) { Content = new
StringContent(body, …, "application/json") };
    request.Headers.TryAddWithoutValidation(WebhookSignature.IdHeaderName, eventId);
    request.Headers.TryAddWithoutValidation(WebhookSignature.EventHeaderName, eventType);
    request.Headers.TryAddWithoutValidation(WebhookSignature.HeaderName,
WebhookSignature.Compute(secret, body)); // 2. sign
    using var response = await httpClient.SendAsync(request, ct);
    return (int)response.StatusCode;
}
```

Two seams visible in one method:

- **SSRF re-check at send (6.4).** The URL was checked when the subscription was created — but

6.4's whole lesson was that a create-time check is defeated by DNS rebinding, so the guard runs **again here, at the moment of the actual POST**. This is precisely the "check at use, not at create" principle 6.4 argued for, now cashed: the freshest resolution is the only one that matters. The subscription-create path checks *too* (fail fast on an obviously-bad URL), but the send-time check is the one that's load-bearing.

- **Signing (§2).** The body is signed and the `X-Webhook-{Id,Event,Signature}` headers stamped.

And the delivery is driven by the **outbox** (4.1), not called inline. `WebhookOutboxHandler` runs each delivery and — the key detail — **throws on a non-2xx response**, which is exactly how the outbox's retry/backoff engages: a failed delivery isn't lost, it's retried with exponential backoff by the machinery from 4.1. The receiver therefore sees *at-least-once* delivery (the `X-Webhook-Id` header lets them dedup — the same idempotency contract Stripe imposed on *you* inbound in 5.2, now imposed by *you* outbound). `WebhookSender` being shared means the synchronous **"send test"** button (owner clicks it, sees the live status code) and the async retried delivery run *identical* signing/SSRF logic — no chance of the test succeeding while real deliveries use a different, buggier path.

5. Fan-out, log, and replay

`WebhookService` (well, `IWebhookPublisher`) fans an event out to the tenant's matching subscriptions by enqueuing one outbox message per subscription — so a feature just "publishes an event" and the delivery machinery does the rest, decoupled and reliable. Every attempt is recorded in a `WebhookDelivery` **log** (HOOKS-2): timestamp, status code, attempt count — so an owner can *see* whether their endpoint is healthy and **replay** a failed delivery on demand. That observability is what makes webhooks operable rather than a black box; a delivery that silently failed three days ago is a support nightmare, a delivery you can see and replay is a self-service fix. Subscriptions are `ITenantScoped`, owner-gated (`ManageWebhooks`,

6.1), and the whole feature is **config-gated default-off** (R21, 7.3) — same as the public API, because it's the same class of power-most-deployments-don't-need.

> **An arch-test beat you'll hit — the `WebhookDelivery` log is *not* `ITenantScoped`.** The delivery-log entity carries a `TenantId` *column* (so you can see a tenant's deliveries) but is deliberately **not** marked `ITenantScoped` — exactly like `OutboxMessage` back in 4.1. It's a system-written log, appended by the delivery machinery outside any request's tenant scope, so subjecting it to the global query filter would be wrong. That means it trips the same tenant-invariant arch test `OutboxMessage` did ("an entity with a `TenantId` must be `ITenantScoped` *or* explicitly allowlisted"), and the fix is the same: add `WebhookDelivery` to the allowlist, declaring "yes, this one has a `TenantId` and is intentionally unscoped." When you hit the red build, that's not a bug — it's the invariant doing its job, asking you to *justify* an unscoped `TenantId` rather than letting one slip in silently (the lesson of 4.1, cashed a second time). The `WebhookService` source-file-naming allowlist entry (3.3) is the same category of one-line, arch-test-demanded declaration.

6. Red — the composition, tested at its seams

```
Scenario: a delivery is signed so the receiver can verify it
  When an event is delivered to a subscription
  Then the POST carries X-Webhook-Signature = sha256=<hmac of the body with the secret>

Scenario: a failed delivery is retried, not lost
  Given the receiver returns 500
  Then the handler throws → the outbox retries with backoff → the delivery log shows the
attempts

Scenario: SSRF is re-checked at send
  Given a subscription whose URL now resolves to a private address
  Then SendAsync refuses (the create-time pass doesn't grandfather it in)

Scenario: the secret is encrypted at rest, recoverable to sign
  Then the stored secret is ciphertext, and Unprotect yields the plaintext used for the
HMAC

Scenario: send-test and async delivery sign identically
  Then both paths produce the same signature for the same body + secret
```

`WebhookSignatureTests` (Core, pure) pins the HMAC; `WebhookDeliveryTests` and `WebhookDeliveryLogTests` drive delivery, retry, and the log on real Postgres; the SSRF-at-send scenario is the security test (write it first — it's the DNS-rebinding defense actually firing).

7. Run it

```
dotnet test tests/Api.Tests --filter Webhook
# enable Webhooks in config → register a subscription (copy the secret) → click "send test"
→ see the status code
# trigger a real event → the delivery log shows the attempt; point the URL at a down server
→ watch retries/backoff
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how are outbound webhooks delivered safely and reliably? (a) Inline fire-and-forget `httpClient.Post`, no signature; (b) inline POST with a signature but no retry and a create-time-only URL check; (c) compose four seams — HMAC signing, an encrypted secret, outbox-driven at-least-once delivery with retry, and an SSRF guard re-checked at send — behind a config-gated default-off feature with a delivery log + replay.

Chosen: (c) (ADR-016). Each problem (authenticity, reliability, SSRF, secret handling) is solved by a seam already built and battle-tested elsewhere, so webhooks are *assembly*, not invention; the outbox gives real retry, the send-time SSRF check gives real rebinding defense, the shared sender guarantees test and prod deliver identically, and the log makes it operable.

Rejected: (a) — unsigned, unreliable, and an open SSRF hole; every failure mode from §1 at once. (b) — a signature without retry loses deliveries on any transient receiver hiccup, and a create-time-only URL check is defeated by DNS rebinding (6.4) — the two subtlest failures, both shipped.

The trade: at-least-once delivery pushes an idempotency obligation onto the *receiver* (dedup by `X-Webhook-Id`) — but that's the industry norm (Stripe/GitHub do the same, and 5.2 made *you* do it inbound), so it's a familiar contract, not a surprise. Accepted: exactly-once outbound is impossible over an unreliable network; at-least-once + an id is the correct, standard answer.

9. Checkpoint

```
git add -A && git commit -m "feat: outbound webhooks — HMAC signing + encrypted secret + outbox delivery + SSRF-at-send, log & replay"
git tag lesson-7.4
```

You should now be able to: describe the four seams webhooks compose and name the earlier lesson each comes from; explain HMAC request signing and the receiver's verify; justify encrypting (not hashing) the secret and contrast it with API keys; explain outbox-driven at-least-once delivery + receiver idempotency; and explain why the SSRF check must run at send, not just at create.

Next: *Lesson 7.5 — Admin back-office* — platform-staff power made accountable: a config allowlist, cross-tenant reads without loosening the filter, and short-lived audited impersonation.

Lesson 7.5 — Admin back-office

> **Part 7 · Compliance & extensibility — finale.** Someone at your company will eventually need to look inside a tenant — to debug a support ticket, verify a billing dispute, or sign in *as* a user to reproduce a bug. This is the most dangerous capability in the whole platform: cross-tenant access, the exact thing every other lesson worked to prevent. So the design question isn't "how do we let staff bypass tenant isolation?" — it's "how do we grant this power *without* loosening the isolation, and make every use of it *accountable*?" The answer reuses `EnterTenant` (2.7) for the fourth and final time, gates staff by an out-of-band allowlist, and makes every privileged action leave a *loud trail in the affected tenant* — so the tenant's own owner can see that a platform admin looked.

Goal: a config-gated platform-staff surface where staff (identified by an out-of-band email allowlist, not a DB role) can inspect any tenant's data *through the normal filter* by entering it, send tenant-wide announcements via the notification fan-out, and "sign in as" a user with a short-lived, non-refreshable, audited token — every action logged inside the target tenant.

Concepts & principles: privileged access without loosening the invariant (`EnterTenant`, not `IgnoreQueryFilters`); an out-of-band staff allowlist (fail-closed); accountability by loud in-tenant auditing; short-lived non-refreshable impersonation + the `impersonated_by` claim; config-gated default-off; composing the notification fan-out (7.2).

Maps to: ADR-014, ADMIN-1/2/3, R21 · repo: `src/Api/Services/PlatformStaffService.cs`, `src/Api/Controllers/AdminController.cs`, `AdminApiControllerBase.cs`, `src/Api/Configuration/PlatformAdminSettings.cs`, tests: `tests/Api.Tests/Admin/*`.

Prerequisites: 2.7 (`EnterTenant` — the whole lesson turns on it), 4.6 (the audit log — accountability), 7.2 (the notification fan-out — announcements), 7.3 (config-gated default-off), 2.7's JWT service (impersonation token minting).

---

1. Motivate — the most dangerous feature, done accountably

Every lesson so far defended tenant isolation. Now you must *deliberately* let a small set of people cross it — because real operations require it. Get this wrong three ways: (1) implement cross-tenant access by loosening the query filter (`IgnoreQueryFilters()`), and you've punched a hole in the isolation everything else depends on, one bug away from leaking across tenants for *everyone*; (2) gate "staff" by a database role, and a privilege-escalation bug anywhere can mint staff; (3) let staff act invisibly, and you have insider access no one can audit — a compliance and trust catastrophe. The platform answers each: **don't loosen the filter — enter the tenant**; **don't store staff in the DB — read an out-of-band allowlist**; **never act invisibly — audit loudly, inside the affected tenant.** The through-line is *accountable power*: the capability exists, but it's narrow, fail-closed, and leaves a trail the victim tenant can see.

2. Who is staff — an out-of-band allowlist, fail closed

Staff identity does **not** live in the database:

```
// src/Api/Services/PlatformStaffService.cs
private readonly HashSet<string> _staff = new(options.Value.StaffEmails,
StringComparer.OrdinalIgnoreCase);
public async Task<bool> IsStaffAsync(Guid userId, CancellationToken ct = default)
{
    if (_staff.Count == 0) return false; // empty allowlist → nobody is staff
    var user = await users.GetByIdAsync(userId, ct);
    return user is not null && _staff.Contains(user.Email); // must resolve to an
allowlisted email
}
```

Staff is a **config allowlist** (`Admin:StaffEmails` — the `PlatformAdminSettings` class binds from the `Admin` config section, so the key is `Admin:StaffEmails`, env `Admin__StaffEmails`), set out-of-band by whoever controls deployment — not a role a user can be *granted* through any in-app flow. This is deliberate: the most powerful capability in the system should not be reachable by *any* code path that manipulates data, because that path could have a bug. A config value can only be changed by someone with deploy access, a much smaller and more trusted set than "anyone who can find a role-assignment bug." And it **fails closed** twice over: an empty allowlist means *nobody* is staff (so a fresh/misconfigured deployment grants no one god-mode), and a user who doesn't resolve to an allowlisted email is not staff. The whole admin surface is also **config-gated default-off** (R21, 7.3) — no allowlist, no admin console at all.

3. Inspection — enter the tenant, don't loosen the filter

Here's the core move, and it's the fourth reunion with `EnterTenant` (after the billing webhook, the file-download endpoint, and — coming — impersonation's audit):

```
// src/Api/Controllers/AdminController.cs — tenant detail (staff only)
using (tenantContext.EnterTenant(id)) // scope INTO the target tenant — the filter still
applies
{
    var tenant = await tenants.GetByIdAsync(id, ct);
    var members = await tenants.GetMemberDetailsAsync(id, ct);
    var subscription = await subscriptions.Query().FirstOrDefaultAsync(ct); // sees ONLY
this tenant's row
    // Loud, in-tenant audit: the tenant's owner can see a platform admin accessed the
account.
    await audit.RecordAsync("admin.tenant.viewed", staffUserId, nameof(Tenant),
id.ToString(), null, ct);
    await auditEvents.SaveChangesAsync(ct);
    return Ok(/* detail */);
}
```

Read what this does *not* do: it never calls `IgnoreQueryFilters()`. Instead the staff member **enters** the target tenant (2.7), and every query inside that scope runs through the *normal* tenant filter — scoped to that tenant. The isolation machinery isn't bypassed; it's *pointed* at a different tenant. This is the whole reason `EnterTenant` exists as a distinct primitive from the escape hatch (2.7: "scopes, doesn't authorize"): the staff check did the authorizing, `EnterTenant` does the scoping, and the invariant that "reads are filtered to a tenant" holds even for admin reads — an admin can't accidentally write a query that spans tenants, because inside the scope the filter is still on. The tenant *list* view (which counts across tenants) reads genuinely non-tenant-scoped tables directly; the moment it touches tenant-scoped data, it enters a tenant first.

4. Accountability — loud, in the tenant that was accessed

Every privileged action records an audit event **inside the target tenant** (note the `EnterTenant` around the `audit.RecordAsync` even for actions that otherwise wouldn't need a scope). That placement is the entire ethic of the feature: the audit lands where the tenant's *own owner* can see it (4.6's append-only trail), so "a platform admin viewed your account" / "…signed in as you" / "…sent an announcement" is visible to the people affected, not buried in a staff-only log they can't read. Insider access that the subject can audit is a fundamentally different trust proposition from invisible insider access — the first is a documented, accountable operation; the second is surveillance. The audit records the *staff* user id as actor (never impersonated identity), so the trail always names the real human who acted.

5. Impersonation — short-lived, non-refreshable, audited

"Sign in as" is the sharpest tool, and its safety is in three constraints:

```
// src/Api/Controllers/AdminController.cs — impersonate (staff only)
var token = jwt.IssueImpersonationToken(
    target.Id, target.Email, staffUserId, ImpersonationLifetime, target.DisplayName,
tenantName, tenantId);
// … loud, in-tenant audit …
return Ok(new ImpersonationResponse { AccessToken = token, ExpiresIn =
(int)ImpersonationLifetime.TotalSeconds });
```

with `ImpersonationLifetime = 15 minutes`. Three deliberate limits:

- **Short-lived (15 min).** Long enough to reproduce a bug, short enough that a leaked

impersonation token is a small window, not a standing key.

- **Non-refreshable.** No refresh token is issued (contrast the normal login flow, 2.7), so the

token *cannot* be extended — it expires on its own and that's final. This is the single most important property: impersonation can't silently become a permanent session.

- **Carries an `impersonated_by` claim.** The token embeds *who* is impersonating, so anything the

impersonator does is attributable to the real staff member, not just the target user — the audit trail never loses the real actor even while the session *acts as* someone else.

And it's **loudly audited in the target's tenant** (`admin.impersonation.started`), so the tenant owner can see that a platform admin signed in as one of their users. Every safety property here is about bounding and attributing a genuinely dangerous power, not removing it.

6. Announcements — reusing the fan-out

Staff can message a whole tenant, and — tellingly — this adds *no* new delivery code:

```
// src/Api/Controllers/AdminController.cs — announce (staff only), all in one transaction
using (tenantContext.EnterTenant(id))
{
    var members = await tenants.GetMembersAsync(id, ct);
    foreach (var member in members)
        await notifications.NotifyAsync(member.UserId, "announcement", title, body,
metadata: null, ct);
    await audit.RecordAsync("admin.announcement.sent", staffUserId, nameof(Tenant),
id.ToString(), new { member_count = members.Count }, ct);
    await auditEvents.SaveChangesAsync(ct);
    await scope.CommitAsync(ct);
}
```

It enters the tenant, loops its members, and calls `INotificationService.NotifyAsync` (7.2) for each — so announcements flow through the *exact same* per-user, preference-respecting, in-app-plus-outbox-email fan-out as any other notification. Staff don't get a special delivery path; they get the platform's one notification path, pointed at a tenant's roster. Notifications, outbox emails, and the in-tenant audit all commit in **one transaction** — the same staged-and-atomic discipline as everywhere else. Reuse over reinvention, one more time.

7. Red — power that's narrow, scoped, and accountable

```
Scenario: a non-staff user is refused everywhere
  Given a signed-in user not on the allowlist
  Then every /api/admin action (except /me) returns 403

Scenario: an empty allowlist grants no one
  Given Admin:StaffEmails is empty
  Then IsStaff is false for everybody

Scenario: inspecting a tenant is scoped and audited in that tenant
  When staff view tenant T
  Then they see only T's data (entered, not filter-bypassed)
  And T's audit log gains admin.tenant.viewed by the staff actor id

Scenario: an impersonation token is short-lived and non-refreshable
  Then it expires in 15 minutes, carries impersonated_by, and has no refresh token
  And T's audit shows admin.impersonation.started
```

`PlatformStaffServiceTests` pins the fail-closed allowlist; `AdminControllerTests` drives the 403-for-non-staff, the entered-and-audited inspection, and the impersonation constraints. Write the empty-allowlist and non-staff-403 scenarios first — they prove the gate that guards the most dangerous surface in the system.

8. Run it

```
dotnet test tests/Api.Tests --filter Admin
# configure Admin:StaffEmails with your email → the admin console appears → view a tenant →
# that tenant's audit log now shows your visit; impersonate a user → a 15-min token, no
refresh;
# clear the allowlist → the console is gone and every admin call 403s
```

9. Architecture Decision

ARCHITECTURE DECISION

The fork: how is platform-staff cross-tenant access granted? (a) A DB `is_staff` flag + `IgnoreQueryFilters()` for cross-tenant reads; (b) a separate admin database/service with its own copy of the data; (c) an out-of-band config allowlist + `EnterTenant` (never loosen the filter) + loud in-tenant auditing + short-lived non-refreshable impersonation, all config-gated default-off.

Chosen: (c) (ADR-014). The config allowlist keeps the most powerful capability off any data-manipulation code path (fail-closed on empty/unknown); `EnterTenant` grants access without breaching the isolation invariant (the filter stays on, just re-pointed); in-tenant auditing makes every use visible to the affected tenant; impersonation is bounded (short, non-refreshable) and attributed (`impersonated_by`). Announcements reuse the notification fan-out. Power exists, but it's narrow and accountable.

Rejected: (a) — `IgnoreQueryFilters()` is banned in features (2.6) precisely because it's a hole in the core invariant; a DB staff flag is grantable by any privilege-escalation bug. (b) — a parallel admin datastore means data duplication, drift, and a *second* place to secure; entering the live tenant reads the real data through the real, tested filter.

The trade: staff management is a deploy-time config change, not a self-service in-app screen (intentional friction — you *want* granting god-mode to require deploy access), and impersonation expiring in 15 minutes can interrupt a long debugging session (re-impersonate — the audit trail is the point). Accepted: both frictions are the safety features, not obstacles to remove.

10. Checkpoint

```
git add -A && git commit -m "feat: admin back-office — config allowlist, EnterTenant
inspection, audited impersonation, announcements"
git tag lesson-7.5
```

Part 7 complete. The platform now honors data-subject rights (export + erasure on the contributor seams), notifies users (closing Part 5's `IBillingNotifier` forward reference), exposes a safe programmatic API, delivers signed reliable webhooks, and grants accountable staff access — every one of them composed from primitives built earlier and gated or audited to stay safe.

You should now be able to: explain why staff access uses `EnterTenant` rather than loosening the filter, and why that preserves the core invariant; justify an out-of-band fail-closed allowlist over a DB role; describe the three impersonation constraints (short-lived, non-refreshable, `impersonated_by`) and why in-tenant auditing makes insider access accountable; and see how announcements reuse the notification fan-out.

Next: *Part 8 — Ship it* — single-origin hosting, the container + staging bring-up, the deploy pipeline & release readiness, and the RLS tenancy backstop that puts a second wall under the query filter at the database itself.

Part 8 — Ship it

Lesson 8.1 — Single-origin hosting

> **Part 8 · Ship it.** The app works on your machine — two dev servers, the API on one port, the Blazor client on another, CORS gluing them together. That topology is convenient locally and *wrong* in production, for one specific and painful reason: your refresh token lives in a cookie, and a cookie shared across two origins is a third-party cookie — increasingly blocked by browsers, and fragile even when allowed. This lesson makes the deployed app a **single origin**: the API itself serves the compiled WASM client, so the browser sees one site, the refresh cookie is first-party, and CORS disappears. It's a small amount of code (config-gated, off by default) that eliminates an entire class of production auth failure.

Goal: in production the ASP.NET Core API serves the published Blazor WASM bundle from its own origin — static files, SPA deep-link fallback, and a guard that keeps `/api/*` returning real API 404s — all config-gated so local dev keeps its separate dev server; plus config-gated forwarded-headers handling so the app behaves correctly behind a reverse proxy.

Concepts & principles: single-origin vs cross-origin; first-party vs third-party cookies; why single-origin kills the cross-site refresh-cookie failure class; SPA fallback routing; `/api/*` must 404 as API, not serve the shell; config-gated hosting (dev unchanged); forwarded headers behind a proxy (real client IP + scheme).

Maps to: ADR-017, DEPLOY-1 · repo: `src/Api/Program.cs` (the `Hosting:ServeWebClient` block + `AddProxyForwarding`/`UseProxyForwarding`), tests: E2E deep-link + `/api/version` smoke.

Prerequisites: 3.4 (the WASM client + its in-memory-token/refresh-cookie model), 2.7 (refresh tokens in a cookie), 1.4 (config-gating), 4.5 (`/health` liveness/readiness — the deploy smoke in 8.3 also polls these; `/api/version` itself arrives in 8.3).

---

1. Motivate — the third-party cookie that breaks login in production

Recall the auth design (2.7, 3.4): the *access* token lives in memory in the WASM app, and the *refresh* token lives in an `HttpOnly` cookie the browser sends automatically on the silent-refresh call. That's secure and it works beautifully in dev — where, if you squint, both servers are "localhost." But deploy the API to `api.example.com` and the client to `app.example.com`, and the refresh cookie is now set by `api.example.com` while the page is `app.example.com`: a **third-party cookie**. Modern browsers block third-party cookies by default (Safari already, Chrome heading there), so the silent refresh silently fails — users get logged out every few minutes and nobody can tell why. Even the workarounds (SameSite=None, explicit CORS credentials) are fragile and leave you fighting the browser's privacy roadmap forever.

The clean fix isn't to fight it — it's to remove the cross-origin split entirely. If the API and the client are the *same origin*, the refresh cookie is **first-party**, always sent, never blocked, and CORS isn't needed at all because there's nothing "cross" about the requests. So in production the API serves the WASM bundle itself.

2. The API serves the client — config-gated

```
// src/Api/Program.cs — serve the published WASM from the API's own origin (DEPLOY-1)
var serveWebClient = app.Configuration.GetValue("Hosting:ServeWebClient", false); //
default OFF
if (serveWebClient)
{
    app.UseBlazorFrameworkFiles(); // the Blazor runtime + framework files
    app.UseStaticFiles(); // the app's static assets (wwwroot)
}
```

Two static-file middlewares, gated behind `Hosting:ServeWebClient` which **defaults to false**. That default is the whole ergonomics story: *local dev is unchanged* — you still run the separate `src/Web` dev server with hot reload, and the API doesn't pretend to host anything. But the *deployed container* sets `Hosting:ServeWebClient=true` and ships the published `wwwroot` alongside the API (the Dockerfile folds the WASM output into the API's `wwwroot`, 8.2). Same binary, two behaviors, chosen by config — the 1.4 pattern applied to hosting topology. Static assets are served *before* auth in the pipeline (they're public), which is why this block sits where it does.

3. The SPA fallback — and why `/api/*` must not become the shell

A single-page app has client-side routes: `app.example.com/settings` is *not* a file on disk, it's a route the WASM router handles after `index.html` loads. So any unmatched path must fall back to `index.html` — *except* API paths, which must stay API-shaped:

```
// src/Api/Program.cs — the two fallbacks, order-sensitive
if (serveWebClient)
{
    app.MapFallback("/api/{**rest}", () => Results.NotFound()); // an unknown /api/* → a
real 404
    app.MapFallbackToFile("index.html"); // everything else → the SPA shell
}
```

The subtle bug this prevents: with only `MapFallbackToFile("index.html")`, a request to a mistyped or removed endpoint like `/api/does-not-exist` would fall through to the catch-all and return `index.html` with a **200** — so an API client (or your own `fetch`) asking for JSON gets a page of HTML and a success code. That's maddening to debug and breaks content negotiation. The more-specific `/api/{**rest}` fallback out-precedences the file fallback for API paths, so an unknown API route returns a genuine `404` while `/settings` correctly loads the shell. The deploy smoke (8.3) asserts exactly this pair: `spa-deeplink → 200` *and* `api-not-shadowed → 404`. Route precedence is a design decision here, not an accident.

4. Behind a proxy — forwarded headers, config-gated

A hosted platform (Render, nginx, any load balancer) terminates TLS and forwards your app plain HTTP, so from the app's point of view every request looks like it arrived over `http` from the *proxy's* IP — not the user's. That breaks two things: the rate limiter (2.4/CONF-5) would throttle the proxy's single IP instead of per-client, and OAuth redirect URIs would be built with `http` instead of `https` (a broken, insecure callback). The fix is to honor the proxy's `X-Forwarded-For`/`X-Forwarded-Proto` headers — but *only* when actually behind a trusted proxy:

```
// registration + middleware, both config-gated on Proxy:Enabled (default OFF)
builder.Services.AddProxyForwarding(builder.Configuration);
// …
app.UseProxyForwarding(app.Configuration); // MUST run FIRST — before HTTPS-redirect, auth,
rate limiter
```

Two disciplines: it's **default-off** (blindly trusting forwarded headers when you're *not* behind a proxy lets a client spoof its own IP/scheme — a security hole), enabled only where a proxy really sits in front (staging/prod set `Proxy:Enabled=true`); and the middleware runs **first**, before anything reads the IP or scheme, so the corrected values are in place before the rate limiter, HTTPS-redirect, and auth all consume them. Order is load-bearing, and the code comments say so.

5. Red — the topology, proven by smoke

Single-origin hosting is verified less by unit tests than by the deploy smoke (8.3) and E2E:

```
Scenario: the deployed origin serves the SPA and the API together
  Then GET / → 200 (the WASM shell)
  And GET /settings → 200 (deep link falls back to the shell, client-router takes over)
  And GET /api/version → 200 (the API on the same origin)
  And GET /api/nope → 404 (unknown API route is NOT shadowed by the shell)

Scenario: the refresh cookie is first-party
  Given the deployed app on one origin
  When the client silently refreshes
  Then the refresh cookie is sent (same-origin) and login persists — no third-party-cookie
block
```

The first scenario is exactly the assert-block the 8.3 deploy pipeline runs against the live staging URL after every deploy; the deep-link + api-not-shadowed pair is the one that proves the fallback routing is correct. The second is the *reason the whole lesson exists* — verified end-to-end against the real deployed origin, because it's precisely the thing that works in dev and would have broken in a cross-origin production.

6. Run it

```
# dev is unchanged — two servers:
dotnet run --project src/Api # API only; Hosting:ServeWebClient defaults off
dotnet run --project src/Web # the WASM dev server with hot reload

# single-origin (what production does): publish the WASM into the API's wwwroot, run the
API alone
# with Hosting__ServeWebClient=true → open the API's origin and the whole app loads from
it.
# (The 8.2 container does this folding for you.)
```

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how are the API and the WASM client deployed? (a) Two origins (`api.example.com` + `app.example.com`) glued with CORS + cross-site cookies; (b) the client on a CDN/static host, the API separate, tokens in `localStorage`; (c) single origin — the API serves the published WASM bundle, refresh cookie first-party, config-gated so dev keeps two servers.

Chosen: (c) (ADR-017). Single origin makes the refresh cookie first-party — immune to the third-party-cookie blocking that breaks (a) — and removes CORS entirely (nothing is cross-origin). One deployable unit (one container, one TLS cert, one URL) is simpler to operate on a free tier. Config-gating keeps local dev's fast two-server hot-reload loop intact.

Rejected: (a) — third-party cookies are being deprecated across browsers; building auth on them is building on sand, and the fragile SameSite=None/CORS-credentials workarounds are a permanent maintenance fight. (b) — moving the refresh token to `localStorage` to dodge the cookie problem trades it for an XSS-exfiltration risk (a script can read `localStorage`; it can't read an `HttpOnly` cookie), which is a worse trade for a long-lived credential.

The trade: single-origin means the API process also serves static files (a little extra work per request) and the two can't scale independently on separate hosts. Accepted: for this app the operational simplicity and the first-party-cookie correctness dominate, and the topology is a config flag — a large deployment that genuinely needs independent scaling can split them and re-introduce a same-site cookie strategy deliberately.

8. Checkpoint

```
git add -A && git commit -m "feat: single-origin hosting — API serves the WASM bundle, SPA fallback, config-gated forwarded headers"
git tag lesson-8.1
```

You should now be able to: explain the third-party-cookie failure that breaks cross-origin refresh and why single-origin fixes it; describe the two-fallback routing and why `/api/*` must 404 rather than serve the shell; justify config-gated forwarded headers and why the middleware must run first; and explain why hosting topology is a config flag that leaves dev untouched.

Next: *Lesson 8.2 — Container & staging* — the Dockerfile that folds the WASM into the API, the compose `app` profile as a local "does it really boot?" parity check, and the Render blueprint.

Lesson 8.2 — Container & staging

> **Part 8 · Ship it.** "It works on my machine" is not a deployment. This lesson packages the single-origin app (8.1) into a **container** — one reproducible image that builds the WASM, builds the API, folds them together, and boots anywhere Docker runs — then brings it up on a free-tier host (Render + Neon + Brevo). Two ideas carry the lesson: a Dockerfile is a *build that must be reproducible* (pinned SDK, locked restores, cached layers), and the compose `app` profile is a **local parity check** — a way to answer "does the exact image that ships to production actually boot, migrate, and serve?" *before* you push it. And the deployment runbook itself is treated as an artifact you write and maintain, not tribal knowledge.

Goal: a multi-stage Dockerfile that produces one image serving API + WASM single-origin, reproducibly (pinned SDK, `--locked-mode` restore, layer-cached dependencies, non-root, graceful shutdown, `$PORT`-aware); a `docker-compose.yml` whose default is dev dependencies (Postgres + Mailpit) and whose gated `app` profile boots the production image locally for parity; and a Render blueprint (`render.yaml`) with secrets kept out of git — brought up as a live staging environment.

Concepts & principles: multi-stage builds; reproducible builds (pinned base + locked restore + committed lockfiles); Docker layer caching (manifests before sources); folding the WASM into the API image; non-root + graceful shutdown (`exec`, SIGTERM) + `$PORT`; the compose profile as a boot-migrate-serve parity gate; infrastructure-as-code (`render.yaml`); secrets never in git; free-tier trade-offs as recorded decisions; the runbook as a maintained artifact.

Maps to: ADR-017, DEPLOY-2 · repo: `Dockerfile`, `.dockerignore`, `docker-compose.yml`, `render.yaml`, `docs/DEPLOYMENT.md`.

Prerequisites: 8.1 (single-origin — what the image serves), 1.3 (Docker Postgres for dev/tests), 1.6 (CI + lockfiles — the reproducibility this mirrors), 5.1 (the Production Stripe-key guard the blueprint must satisfy).

---

1. Motivate — reproducibility and the "does it really boot?" gap

Two failure modes this lesson closes. First, **the build that isn't reproducible**: a Dockerfile that says `FROM …/sdk:latest` and `dotnet restore` with no lockfile will, on some future rebuild, pull a different SDK patch or a different transitive package and either break subtly or fail `--locked-mode` — the classic "worked last week, red today, nothing changed." A production image must build the *same* bits every time. Second, **the gap between CI-green and boots-in-prod**: your tests pass, but does the *packaged image* — not `dotnet run`, the actual container with its published bits and startup migration — really come up against a real database and serve? You don't want to discover the answer from Render's logs after a push. Both are solved here: pin and lock the build, and give yourself a one-command local run of the production image against real sibling containers.

2. The Dockerfile — a reproducible, multi-stage build

```
# ---- build stage: the SDK image ----
FROM mcr.microsoft.com/dotnet/sdk:10.0.301 AS build # PINNED to the lockfile-generating SDK
WORKDIR /src
COPY Directory.Build.props Directory.Packages.props ./ # warnings-as-error + Central
Package Mgmt
# manifests + lockfiles FIRST, so restore is its own cached layer
COPY src/Core/Perezosoft.Core.csproj src/Core/packages.lock.json src/Core/
# … (Infrastructure, Api, Shared.Ui, Web) …
RUN dotnet restore src/Api/Perezosoft.Api.csproj --locked-mode \
 && dotnet restore src/Web/Perezosoft.Web.csproj # Web without locked-mode (see note)
COPY src/ src/ # NOW the sources
RUN dotnet publish src/Web/… -c Release -o /publish/web \
 && dotnet publish src/Api/… -c Release -o /publish/api \
 && cp -r /publish/web/wwwroot/. /publish/api/wwwroot/ \ # FOLD the WASM into the API's
wwwroot
 && echo '{}' > /publish/api/wwwroot/appsettings.json # drop any baked ApiBaseUrl → default
to own origin
```

Every line is a reproducibility or single-origin decision:

- **Pinned SDK (`10.0.301`), not `latest`.** The comment is explicit: the WASM SDK injects

patch-sensitive implicit package refs, so a floating tag breaks `--locked-mode`. The image builds with the *exact* SDK that generated the committed lockfiles (1.6) — same guarantee as CI.

- **Manifests + lockfiles copied *before* the sources.** Docker caches each layer; by restoring

from just the `.csproj` + `packages.lock.json` before copying `src/`, the expensive `restore` layer is reused on every build where dependencies didn't change — only a *dependency* edit re-runs it, not every source edit. Layer ordering is a build-speed decision.

- **`--locked-mode` on the API side** — the restore fails if the lockfile doesn't match, the same

supply-chain gate as CI. (The Web project restores *without* locked-mode, documented at length: the Blazor SDK injects an implicit patch-specific asset package that differs between the lockfile-writing SDK and the base image; CI's build-test job is the authoritative lockfile gate for Web, so the Dockerfile just needs a reproducible publish.)

- **Folding the WASM into the API's `wwwroot`** is what makes 8.1's single-origin real: one

published tree, API + client together. Overwriting the client's `appsettings.json` with `{}` removes any baked-in API base URL so the WASM defaults to *its own origin* — exactly the single-origin case.

Then a slim **runtime** stage (`aspnet:10.0`, no SDK) copies only the published output — a smaller, smaller-attack-surface image — and three production hygiene choices:

```
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
USER app # non-root (the .NET image ships this user)
ENV Hosting__ServeWebClient=true # turn on 8.1's single-origin hosting
HEALTHCHECK … CMD curl -fsS "http://localhost:${PORT:-8080}/health" || exit 1 # liveness
ENTRYPOINT ["sh", "-c", "exec dotnet Perezosoft.Api.dll --urls
http://0.0.0.0:${PORT:-8080}"]
```

- **Non-root `USER app`** — never run a public-facing process as root; a container escape is far

worse from root.

- **`exec dotnet …`** so `dotnet` becomes PID 1 and receives `SIGTERM` directly — the platform can

stop the container *gracefully* (drain requests) instead of killing it. A shell that doesn't `exec` would swallow the signal.

- **`$PORT`-aware** (default 8080) — hosts like Render assign a port via `$PORT`; the entrypoint

binds it. The `HEALTHCHECK` runs 8.1/4.5's `/health` liveness probe from inside the container.

3. The compose `app` profile — a local parity check

`docker-compose.yml`'s default services are the *dev dependencies* — Postgres and Mailpit — so the normal loop (`docker compose up -d`) starts only what dev needs (1.3). The production image is behind a **profile** so it does *not* run by default:

```
app:
  profiles: ["app"] # only runs with `docker compose --profile app up`
  build: { context: ., dockerfile: Dockerfile }
  environment:
    ConnectionStrings__DefaultConnection: "Host=db;…" # reach the sibling Postgres by
service name
    Email__Smtp__Host: mail # reach the sibling Mailpit
  depends_on:
    db: { condition: service_healthy }
    mail: { condition: service_healthy }
```

The comment names its purpose exactly: "*the local parity check — does the container that ships to Render actually boot + migrate + serve?*" Run `docker compose --profile app up` and you get the *real production image* running against a real Postgres and a real SMTP catcher, migrating on startup and serving the single-origin app — locally, before you ever push. It's the missing rung between "tests pass" and "it's live": a fast, honest answer to whether the packaged artifact works, not just the source. Keeping it behind a profile means it never slows the everyday `up -d`.

The word **migrate** in that parity claim needs a piece you add now — a startup migration step, so a freshly-created database (the container's first boot, or after a `docker compose down -v`) gets its schema with no manual `dotnet ef database update`:

```
// src/Api/Program.cs — apply EF migrations on startup (idempotent: a no-op when already
current)
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    if (db.Database.IsRelational()) // guard: non-relational test providers are untouched
        db.Database.Migrate();
}
```

This is what makes the container *self-sufficient* — it schemas itself on boot, so the compose parity run genuinely exercises "boot **and migrate** and serve," and staging (8.3) needs no separate migration step in the deploy. `Migrate()` is idempotent (it only applies what's pending), so running it on every start is safe. Two operational refinements the reference layers on, worth knowing about: gate it behind a config flag (`Hosting:MigrateOnStartup`, default on)

so an integration-test harness that builds its schema a different way can turn it off; and, once RLS's two-role topology exists (8.4), run this DDL over the *owner/migrator* connection (`ConnectionStrings:Migrations`) rather than the low-privilege runtime role — a migration creates tables, which the runtime role isn't allowed to do. (Both are small conditionals; the core is the four lines above.)

4. Staging on the free tier — `render.yaml`, secrets out of git

`render.yaml` is the infrastructure-as-code blueprint — the staging environment as a committed, reviewable file rather than dashboard clicks:

```
services:
  - type: web
    runtime: docker
    healthCheckPath: /health/ready # readiness = DB reachable; Render restarts if it fails
    autoDeploy: false # 8.3 drives deploys via a hook, not every push
    envVars:
      - key: Proxy__Enabled
        value: "true" # behind Render's TLS proxy (8.1's forwarded headers)
      - key: Jwt__Secret
        generateValue: true # minted ONCE, persisted → sessions survive redeploys
      - key: ConnectionStrings__DefaultConnection
        sync: false # SECRET — set in the dashboard, never committed
      - key: Billing__Stripe__SecretKey
        sync: false # REQUIRED in Production (5.1's fail-closed guard)
```

The disciplines:

- **Secrets are `sync: false`** — declared in the blueprint but their *values* live only in the

Render dashboard, never in git. The auth rule from Part 2 (secrets never in source) enforced at the deployment layer. Non-secret config (`Proxy__Enabled`, the readiness path) *is* in the file.

- **`Jwt__Secret: generateValue: true`** — Render mints one strong secret once and persists it, so

refresh tokens survive redeploys (a regenerated signing key would invalidate every session).

- **The Production Stripe-key guard (5.1) must be satisfied** — `Billing__Stripe__SecretKey` is

required because the app *refuses to boot* in Production without it (fail-closed). The blueprint documents it with "a Stripe *test* key for staging." The deploy would fail loudly otherwise — the guard doing its job.

- **Free-tier trade-offs are recorded, not hidden.** The header comments spell out that the free

instance sleeps after ~15 min idle (cold-start ~30–60s, outbox/scheduler pause while asleep) — "*fine for staging, never for a paid prod.*" Naming a constraint as a decision (with its blast radius) is the platform's documentation ethic, applied to ops.

5. The runbook is an artifact — you write `DEPLOYMENT.md`

The deliverable here isn't only the YAML — it's `docs/DEPLOYMENT.md`, the step-by-step runbook for standing up Neon (Postgres), Brevo (SMTP), and Render (host), wiring the secrets, and

verifying all four sign-in paths against the live URL. Treating the runbook as a *maintained artifact* — written, reviewed, kept current — is itself the lesson: deployment knowledge that lives only in one engineer's head is a bus-factor-of-one outage waiting to happen. A learner following this course *writes their own* `DEPLOYMENT.md` as they bring the environment up, so the ops knowledge is captured the same way the code is. (Note one Brevo-specific gotcha the runbook records: Render's free tier blocks outbound ports 25/465/587, so SMTP goes over Brevo's **2525** — the kind of detail that costs an afternoon if it's not written down.)

6. Red — parity, not unit tests

Containerization is verified by *running* it, not asserting in xUnit:

```
Scenario: the production image boots, migrates, and serves (compose parity)
  When `docker compose --profile app up` runs against sibling db + mail
  Then the app applies migrations on startup, /health is 200, and the SPA loads from its
origin

Scenario: the build is reproducible
  Given the committed lockfiles + pinned SDK
  Then `dotnet restore --locked-mode` succeeds and produces the same dependency set

Scenario: staging comes up and all four sign-in paths work
  When the Render blueprint is applied and secrets set
  Then password, magic-link, OTP, and OAuth all complete against the live https URL
```

The compose-parity scenario is the one that earns its keep — it's the manual/local gate CI can't cheaply run (it needs a database and a full image build), and it's the honest rehearsal of the production boot. 8.3 adds the *automated* post-deploy smoke on top; this is the local dress rehearsal.

7. Run it

```
docker compose up -d # dev deps only (Postgres + Mailpit) — the everyday loop
docker compose --profile app up --build # PARITY: build + boot the real production image
locally
# → watch it migrate on startup and serve http://localhost:8080 single-origin; Ctrl-C to
stop
# staging: push render.yaml, apply the blueprint in Render, set the sync:false secrets, hit
the live URL
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how is the app packaged and deployed? (a) Publish to the host directly (no container), configure the box by hand; (b) a container, but with a floating SDK tag and unlocked restore; (c) a pinned, locked, multi-stage container image (API + WASM folded, non-root, graceful, `$PORT`-aware), a compose profile for local parity, and an IaC blueprint with secrets out of git.

Chosen: (c) (ADR-017). Pinned SDK + locked restore + committed lockfiles make the build reproducible (the CI guarantee carried into the image); the multi-stage build ships a slim runtime; the compose `app` profile answers "does it really boot?" locally; `render.yaml` makes staging reviewable and its secrets stay in the dashboard. Free-tier limits are recorded decisions.

Rejected: (a) — hand-configured hosts drift, aren't reproducible, and "works on the box" is the problem, one level up. (b) — a floating tag/unlocked restore reintroduces the non-reproducible build that breaks unpredictably; the pin is the point.

The trade: a pinned SDK must be bumped deliberately (a small maintenance task, coupled to regenerating lockfiles), and the free tier sleeps (cold starts, paused background work) — fine for staging, explicitly not for paid prod. Accepted: reproducibility is worth the pin, and the free-tier limits are documented so nobody mistakes staging for production-grade.

9. Checkpoint

```
git add -A && git commit -m "feat: containerize — reproducible multi-stage Dockerfile
(single-origin), compose app-profile parity, render.yaml"
git tag lesson-8.2
```

You should now be able to: explain multi-stage builds and the reproducibility trio (pinned base + locked restore + committed lockfiles); describe Docker layer caching and why manifests are copied before sources; justify non-root + `exec`/SIGTERM + `$PORT`; explain the compose `app` profile as a boot-migrate-serve parity gate; and articulate keeping secrets out of git (`sync:false`) and the runbook as a maintained artifact.

Next: *Lesson 8.3 — The deploy pipeline & release readiness* — the CI from lesson 1.6 grows deploy stages (auto-to-staging with a version-gated smoke, gated-to-prod) and the manual half: a QA plan as a maintained artifact.

Lesson 8.3 — The deploy pipeline & release readiness

> **Part 8 · Ship it.** The DevOps thread that started in lesson 1.6 — a CI workflow born at the first commit and grown a job per part — reaches its destination here: it learns to *deploy*. Push to `develop` and the app auto-deploys to staging and then **proves the new build is actually live** before calling it green; push to `main` and prod deploys only behind a human approval. But automation is only half of release readiness. The other half is a **QA test plan** — a maintained, human-run checklist across web and the native shells — because some regressions (a broken OAuth redirect on a real device, a visual glitch) no automated gate will ever catch. The lesson's thesis: *automation gates regressions; the QA pass gates releases.*

Goal: a CI pipeline where `develop`→staging is automatic and immediately followed by a version-gated post-deploy smoke that waits for *this commit* to be live before asserting; `main`→prod is gated behind a GitHub Environment approval; concurrent deploys don't spuriously fail each other; and a QA test plan exists as a maintained artifact that gates releases the automation can't.

Concepts & principles: continuous delivery (auto-to-staging, gated-to-prod); the version-gated smoke (why health alone can't tell old build from new); post-deploy assertions; environment-approval gates; deploy concurrency (cancel-in-progress); regressions-vs-releases (what automation gates vs what a human QA pass gates); the QA plan as a versioned artifact.

Maps to: ADR-017, DEPLOY-3 · repo: `.github/workflows/ci.yml` (the `deploy-staging` / `deploy-prod` jobs), `src/Api/Program.cs` (`/api/version`), `docs/QA_TEST_PLAN.md`.

Prerequisites: 1.6 (the CI workflow this extends), 8.1/8.2 (single-origin + the container + `render.yaml` being deployed), 4.5 (`/health` liveness/readiness — two of the smoke's probes; the third, `/api/version`, is built in this lesson).

---

1. Motivate — "deployed" is not "live", and green tests aren't a release

Two subtle truths shape this lesson. First: triggering a deploy is not the same as the new build serving traffic. On a rolling host the *old* instance keeps answering while the new one builds and boots — so if your pipeline triggers a deploy and immediately curls `/health`, it gets a cheerful 200 *from the old build* and reports success before your change is even live. A deploy pipeline that can't tell "old build still up" from "new build live" is lying to you. Second: a full green CI run means your *code* is correct against your *tests* — it does not mean the *release* is good. Does Google sign-in actually complete on a physical Android device? Does the invite email render in a real inbox? Does the desktop shell's window restore correctly? Automated tests can't economically cover all of that, and pretending they do is how a "green" release ships a broken OAuth flow. So the pipeline gets a *version-gated* smoke that proves liveness of the right build, and release readiness gets a *human* QA pass for what automation can't reach.

2. Deploy to staging — automatic, on `develop`

```
# .github/workflows/ci.yml — deploy-staging
deploy-staging:
  needs: [build-test, secret-scan, qa-artifacts, license-scan, docker-build, e2e] # only
after all gates green
  if: github.event_name == 'push' && github.ref == 'refs/heads/develop'
  concurrency: { group: deploy-staging, cancel-in-progress: true }
  steps:
    - name: Trigger Render deploy
      run: curl -fsS -X POST "$RENDER_DEPLOY_HOOK_STAGING" > /dev/null
```

The shape: staging deploys **automatically** on every push to `develop`, but *only after* the full quality bar is green — the `needs:` list gates the deploy behind build/test, secret-scan, license-scan, the E2E suite, and the docker image build. A deploy is downstream of proof. The `concurrency` block with `cancel-in-progress: true` handles back-to-back merges: if two pushes land close together, the older deploy job is *cancelled* rather than left to race — because (§3) the older run's version-gated smoke could never see its own commit go live once a newer build superseded it, and would fail spuriously. Cancelling reads as neutral; the newest push still runs its full smoke. Concurrency control here is a correctness fix, not just tidiness.

3. The version-gated smoke — prove the new build is live

This is the clever, load-bearing part. After triggering the deploy, the pipeline **polls `/api/version` until it reports *this workflow's commit SHA*** and readiness is 200 — only then does it run assertions:

```
# wait for the NEW build to actually be live (the old instance serves during the build)
EXPECT=${{ github.sha }}
for i in $(seq 1 50); do
  got=$(curl -s "$BASE/api/version" | jq -r '.commit')
  code=$(curl -s -o /dev/null -w '%{http_code}' "$BASE/health/ready")
  if [ "$got" = "$EXPECT" ] && [ "$code" = "200" ]; then ready=1; break; fi
  sleep 15 # free-tier build + cold boot can take minutes
done
[ "$ready" = 1 ] || { echo "::error::new build did not go live in time"; exit 1; }
```

`/api/version` is a small endpoint you add *in this lesson* (DEPLOY-3) — anonymous, returning the commit the running instance was built from, read from the platform's environment (`APP_BUILD_COMMIT`, or Render's injected `RENDER_GIT_COMMIT`, else `"unknown"`):

```
// src/Api/Program.cs — the build-identity probe the smoke polls
app.MapGet("/api/version", () => Results.Ok(new
{
    commit = Environment.GetEnvironmentVariable("APP_BUILD_COMMIT")
             ?? Environment.GetEnvironmentVariable("RENDER_GIT_COMMIT") ?? "unknown",
})).AllowAnonymous();
```

It sits next to `/health` (4.5) but answers a *different* question — not "am I alive?" but "*which build* am I?" Polling it until it equals `github.sha` is what distinguishes "my change is live" from "the old build is still answering 200s" — the exact trap §1 opened with. Health alone can't do this; you need an endpoint that reports *identity*, not just liveness. Once the right build is confirmed live, a compact assert-block checks the single-origin topology (8.1) end-to-end

against the real URL:

```
assert liveness 200 /health
assert readiness 200 /health/ready
assert build-version 200 /api/version
assert spa-shell 200 /
assert spa-deeplink 200 /settings # deep link falls back to the WASM shell (8.1)
assert api-not-shadowed 404 /api/does-not-exist # unknown API route is NOT the shell (8.1)
assert providers 200 /api/auth/providers
```

That `spa-deeplink 200` + `api-not-shadowed 404` pair is precisely the 8.1 fallback-routing correctness, now guarded on *every deploy* — a regression in the fallback order fails the pipeline, not a user. The smoke is a *post-deploy* gate: it runs against the live environment, after the deploy, and turns "we think it deployed" into "we verified the new build serves correctly."

4. Deploy to prod — gated behind a human

```
deploy-prod:
  needs: [build-test, secret-scan, qa-artifacts, license-scan, docker-build, e2e]
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  environment: production # ← the gate: add a required reviewer to this GitHub Environment
```

Prod deploys only on a push to `main`, and the `environment: production` line is the safety gate: by adding a *required reviewer* to the `production` Environment (repo Settings → Environments), a prod deploy **pauses for human approval** before it runs. This is why `main` is "deploy-only and behind develop" (the git-workflow rule from the start of the course): work flows develop → staging → (verified) → main → prod, and the last hop is a deliberate, approved action, not an automatic consequence of a merge. Continuous *delivery*, not blind continuous deployment — the pipeline makes prod releasable at any time, a human decides *when*.

5. Release readiness — the QA plan is an artifact

Automation gates *regressions*; a human QA pass gates *releases*. The platform ships `docs/QA_TEST_PLAN.md` — a maintained checklist of **117 cases** spanning web plus all four native platforms (Windows/Android/iOS/macOS): smoke tests, regression tests, and a §13 native release checklist. It's a *versioned artifact*, edited alongside the code (a new feature adds its QA cases), with generated PDF run-logs a tester fills in (the course's tooling regenerates them from the Markdown). Treating manual QA as an *artifact* — not an ad-hoc "click around before shipping" — is the lesson: it makes release readiness repeatable and reviewable, and it honestly scopes what automation covers versus what a person must verify (real-device OAuth, email rendering, native window behavior, visual correctness). The two halves compose: CI's automated gates keep `develop` continuously releasable; the QA pass is the deliberate human sign-off before a prod release.

6. Red — the pipeline gates, exercised

The pipeline itself is the test, but its behaviors are observable and were verified:

```
Scenario: staging deploys only when the full bar is green
  Given a push to develop with a failing test
  Then deploy-staging never runs (needs: build-test failed)

Scenario: the smoke waits for the new build, not the old one
  Given a deploy in progress (old build still serving 200s)
  Then the smoke keeps polling /api/version until it equals the pushed SHA before asserting

Scenario: back-to-back pushes don't fail each other
  Given two pushes to develop seconds apart
  Then the older deploy job is cancelled (neutral), the newer runs its full smoke

Scenario: prod waits for a human
  Given a push to main
  Then deploy-prod pauses on the production environment until a reviewer approves
```

The version-gated-smoke scenario is the one that captures the lesson's core insight; it's what makes the pipeline honest about "live" versus "deployed."

7. Run it

```
# push to develop → watch Actions: the gates run, then deploy-staging triggers Render and
polls
# /api/version until your commit is live, then asserts the smoke block against the staging
URL.
# push to main → deploy-prod appears "Waiting" until you approve it in the production
environment.
# regenerate the QA run-log PDFs from docs/QA_TEST_PLAN.md, then run the smoke checklist by
hand.
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how does code reach production, and how is a release judged ready? (a) Manual deploys (someone runs a script / clicks the dashboard); (b) fully automatic deploy to prod on every green merge, health-check only; (c) auto-to-staging with a *version-gated* post-deploy smoke, gated-to-prod behind a human approval, plus a maintained QA plan for what automation can't cover.

Chosen: (c) (ADR-017, DEPLOY-3). Auto-to-staging keeps `develop` continuously verified against the real environment; the version-gated smoke proves the *right build* is live (not the old one still serving); the prod environment-approval keeps releases deliberate; deploy concurrency stops back-to-back merges from failing spuriously; and the QA plan honestly scopes and gates the human-only checks. Automation catches regressions fast; a person signs off releases.

Rejected: (a) — manual deploys are error-prone, unaudited, and bus-factor-one. (b) — fully automatic prod-on-merge with only a health check ships the "old build still 200s" lie and removes the human judgment that catches what tests don't; too much blast radius for this app's stage.

The trade: a human approval on prod adds latency to releasing (intentional — that's the deliberation), and a maintained QA plan is real ongoing work (117 cases to keep current). Accepted: the latency is the safety, and a versioned QA artifact is how release readiness stays repeatable instead of vibes. The version-gated smoke costs several minutes of polling on the free tier — a fair price for never mistaking a stale instance for a live deploy.

9. Checkpoint

```
git add -A && git commit -m "feat: deploy pipeline — auto-staging + version-gated smoke,
gated prod, QA plan as release-readiness artifact"
git tag lesson-8.3
```

You should now be able to: explain continuous delivery (auto-staging, gated-prod) and why the deploy is downstream of the full green bar; describe the version-gated smoke and why `/api/version` (not `/health`) is what proves the new build is live; justify the environment-approval prod gate and deploy concurrency; and articulate the regressions-vs-releases split — what CI gates versus what the QA plan gates.

Next: *Lesson 8.4 — The RLS tenancy backstop* — Postgres row-level security as a second wall under the EF query filter, so a single missed `.Where(tenant)` can't leak across tenants.

Lesson 8.4 — The RLS tenancy backstop

> **Part 8 · Ship it — finale.** The entire platform rests on one invariant: tenant data never leaks across tenants (Golden Rule 1). Since lesson 2.6 that's been enforced by an EF Core global query filter — good, but it lives in *your application*, and it's exactly one forgotten `IgnoreQueryFilters()`, one raw SQL query, one set-based `ExecuteUpdate` from being bypassed. A single bug in the app layer and the invariant is gone. This lesson adds a **second wall, underneath the app, at the database itself**: Postgres row-level security (RLS) that re-enforces tenant isolation even if the application filter is bypassed. Defense in depth for the one invariant you cannot afford to get wrong — and a masterclass in making a security control *fail closed*, *impossible to drift*, and *verified by CI*.

Goal: every `ITenantScoped` table carries a fail-closed Postgres RLS policy so the database itself refuses cross-tenant reads *and* writes; the app carries the ambient tenant to the DB per command; sanctioned cross-tenant operations declare themselves explicitly; the policy set is derived from the EF model so it can't drift; a CI gate fails if a new tenant entity ships without its policy; and the app refuses to boot if its DB role would silently bypass RLS.

Concepts & principles: defense in depth; row-level security; fail-closed DB policy (unset GUC → no rows); `FORCE` RLS + the superuser/owner exemption; `USING` + `WITH CHECK` (reads *and* writes); per-command session GUCs via an interceptor; explicit bypass (never missing = allow); the two-role topology (owner/migrator vs runtime); model-derived DDL (no drift); a migration-parity CI gate; a fail-closed startup posture guard.

Maps to: ADR-020 · repo: `src/Infrastructure/Persistence/RlsDdl.cs`, `RlsSessionInterceptor.cs`, `RlsTags.cs`, `RlsPostureGuard.cs`, the `RlsTenancyBackstop` migration, `docker/db/provision-rls-runtime-role.sql`, tests: `tests/Api.Tests/Rls/*`.

Prerequisites: 2.6 (the EF query filter this backstops), 2.7 (`EnterTenant` + the escape hatch), 3.1 (`QueryAllTenants()`), 4.1 (the tenant-less system context), 8.2 (the container + compose this lesson's provisioning script mounts into, and the startup-migration step it builds on).

---

1. Motivate — one wall is one bug from gone

The EF query filter (2.6) is a genuinely good defense — it scopes every LINQ read to the current tenant automatically, and the arch tests (2.6/3.3) ban raw `IgnoreQueryFilters()` in feature code. But think about where it *doesn't* reach: a raw SQL query bypasses it; `ExecuteUpdate`/`ExecuteDelete` (set-based writes) don't run the filter's predicate the same way; a sanctioned `QueryAllTenants()` that's accidentally left un-re-scoped; a future contributor who reaches for the escape hatch and forgets to constrain it. Every one of those is a *single mistake in application code* away from a cross-tenant leak — and a cross-tenant leak in a multi-tenant SaaS is the worst bug you can ship, the one that ends customer trust. Relying on one wall, in one layer, written by fallible humans, is not enough for *this* invariant.

Defense in depth says: put a *second, independent* wall in a *different layer*, so a breach of the first doesn't breach the system. The database is that layer. Postgres RLS lets the DB itself refuse to return or accept rows that don't belong to the current tenant — enforced below the application, so even a query that escaped the EF filter still hits this wall. The app filter is the

fast, everyday guard; RLS is the backstop that turns "we forgot a `.Where`" from a breach into a no-op.

2. The policy — fail-closed, reads and writes, model-derived

`RlsDdl` generates the DDL — one policy per tenant-scoped table:

```
-- for each ITenantScoped table (RlsDdl.StatementsFor):
ALTER TABLE "Notes" ENABLE ROW LEVEL SECURITY;
ALTER TABLE "Notes" FORCE ROW LEVEL SECURITY; -- subject even the table owner
CREATE POLICY rls_tenant_isolation ON "Notes"
    AS PERMISSIVE FOR ALL
    USING ("TenantId" = NULLIF(current_setting('app.tenant_id', true), '')::uuid
                OR current_setting('app.rls_bypass', true) = 'on')
    WITH CHECK ("TenantId" = NULLIF(current_setting('app.tenant_id', true), '')::uuid
                OR current_setting('app.rls_bypass', true) = 'on');
```

Every clause is a deliberate safety property:

- **Fail-closed predicate.** The tenant is carried in a Postgres *GUC* (a session setting),

`app.tenant_id`. If it's **unset or empty**, `NULLIF(current_setting(...), '')::uuid` is `NULL`, and `"TenantId" = NULL` matches **no rows**. So the default — no tenant set — is *see nothing*, not *see everything*. This is the single most important line: a misconfiguration or a forgotten scope yields zero rows, never a leak. Fail-closed, at the database.

- **`USING` *and* `WITH CHECK`.** `USING` filters what a query can *read*; `WITH CHECK` constrains

what it can *write*. Both mirror each other, so the DB rejects a foreign-tenant `INSERT`/`UPDATE` too — the database-level counterpart to the `TenantStampingInterceptor` (2.7). Reads *and* writes are walled.

- **`FORCE ROW LEVEL SECURITY`.** By default even RLS exempts a table's *owner*; `FORCE` subjects

the owner too. (Superusers and `BYPASSRLS` roles *always* bypass — which is why local dev's superuser sees no change, while a properly non-privileged runtime role gets real enforcement. That gap is exactly what §5's posture guard closes.)

- **Explicit bypass GUC.** `app.rls_bypass = 'on'` disables the policy for a command — the

sanctioned cross-tenant path (§4). Note it must be *exactly* `'on'`; anything else (including unset) leaves the policy in force. Bypass is an explicit opt-in, never a default.

And crucially, the policy list is **derived from the EF model**, not hand-maintained: `RlsDdl.TenantTables(model)` reflects over the model for everything implementing `ITenantScoped`. So the set of protected tables *cannot drift* from the set of tenant entities — the same "one source of truth" instinct as the RBAC matrix and the contributor seams, applied to DB policy.

3. Carrying the tenant to the DB — the session interceptor

RLS reads `app.tenant_id` from the connection; something must *set* it to the app's ambient tenant before each command. That's `RlsSessionInterceptor`, and its mechanics are a lesson in doing this correctly:

```
// src/Infrastructure/Persistence/RlsSessionInterceptor.cs (essence)
// Before each command executes, set the GUCs via a SEPARATE parameterized command:
guc.CommandText = "SELECT set_config('app.tenant_id', $1, false),
set_config('app.rls_bypass', $2, false);";
// tenant = the AppDbContext's current tenant (or empty); bypass = "on" when tenant-less OR
the
// command carries the cross-tenant tag (§4).
```

The subtleties, each a real bug avoided:

- **A separate command, never a prepended statement.** You can't just glue `SET …;` in front of

EF's SQL — a prepended statement occupies a result-set position and corrupts EF's positional reading of `SaveChanges` batches (its rows-affected checks). So the GUCs are set by their own parameterized command first.

- **Session-level `set_config` (not `SET LOCAL`)** so it applies to reads *outside* a transaction

too. Safe under connection pooling because Npgsql resets session state when a physical connection returns to the pool, and the interceptor re-asserts per logical open.

- **Per-command, not per-request.** The tenant can *change mid-request* (`EnterTenant`, 2.7 — the

admin inspection and billing webhook do this), and individual queries can be sanctioned cross-tenant. So the desired GUC state is recomputed per command, with a per-connection cache to skip the round-trip when nothing changed — and **invalidated** exactly where Postgres can silently revert the GUCs: connection (re)open, transaction rollback, savepoint rollback (`set_config` is transactional). Getting those invalidation points wrong would leave a stale tenant on the connection — a leak — so they're enumerated and handled.

4. Sanctioned bypass — the escape hatch declares itself

The platform *does* have legitimate cross-tenant operations: the outbox dispatcher and scheduled jobs run tenant-less (4.1/4.4); `QueryAllTenants()` resolves an API key's tenant pre-auth (7.3); dissolve/export contributors read across the target tenant (7.1). RLS must not break these — but it also must not have a blanket "off switch." The resolution is **explicit, per-query declaration**:

```
// src/Infrastructure/Persistence/RlsTags.cs
public const string CrossTenant = "rls:cross-tenant"; // an EF query tag → rendered as a
leading SQL comment
```

A sanctioned cross-tenant query is written `.TagWith(RlsTags.CrossTenant)`. EF renders the tag as a leading SQL comment, which survives deferred execution and reaches the database command; the interceptor sees the comment and sets `app.rls_bypass = 'on'` **for that command only**. The tenant-less system context (no tenant at all) also bypasses. Everything else stays subject to the policy. This is ADR-020's rule verbatim: **explicit bypass, never missing-setting = allow.** An untagged query that somehow escaped the EF filter is *still* walled by RLS, because escaping the app filter doesn't set the bypass GUC — the two walls are independent, and only a *deliberate*, tagged, in-Infrastructure query gets through the second one. (This is why the webhook/outbox handlers you saw in 7.4 carry the tag — they

genuinely run cross-tenant, and they say so.)

5. The posture guard — refuse to boot into a false sense of security

RLS has a treacherous property: as noted in §2, Postgres exempts **superusers** and **`BYPASSRLS` roles unconditionally, and table** owners** unless `FORCE` (and even under FORCE, an owner can alter/drop policies). So if the app connects as a privileged role, *all these carefully-built policies silently do nothing* — you have a wall you believe in that isn't there. That's more dangerous than no wall, because it breeds false confidence. The platform's answer is the **two-role topology** plus a fail-closed startup guard:

- **Two roles** — provisioned by a `docker/db/provision-rls-runtime-role.sql` script you add *with

this lesson* (the 8.2 `docker-compose.yml` mounts the `docker/db` directory so a fresh dev volume auto-runs it; in staging/prod you run it once by hand): an *owner/migrator* role that owns the schema and runs migrations (DDL), and a **non-privileged runtime role** the app actually connects as — subject to RLS. `Program.cs` uses `ConnectionStrings:Migrations` (owner) for the startup migration (8.2) and the default runtime connection for serving.

- **`RlsPostureGuard`** — config-gated on `Rls:EnforceRuntimeRole`, and when enabled it **refuses to

start** if the runtime role is a superuser, has `BYPASSRLS`, or owns the tables:

```
// RlsPostureGuard — fail closed at boot, exactly like the Production Stripe-key guard
(5.1)
if (privileged)
    throw new InvalidOperationException(
        "Rls:EnforceRuntimeRole is enabled, but the app's DB role is a superuser / has
BYPASSRLS / "
        + "owns the tables — RLS would be silently bypassed. Connect as the non-privileged
runtime role …");
```

This is the same fail-closed instinct as 5.1's refuse-to-boot-without-a-Stripe-key: *if a security control you rely on can't actually be in force, don't run pretending it is.* The guard turns "silent bypass" into "loud startup failure," which is precisely the trade you want for a wall you're depending on.

6. No drift — the migration-parity CI gate

A new `ITenantScoped` entity must ship its RLS policy in the *same migration* that creates its table — otherwise the new table has no second wall and nobody notices. Since the policy set is model-derived (§2), CI can *check* this mechanically:

```
Scenario: a new tenant entity without its RLS policy fails CI (RlsMigrationGateTests)
  Given a migrated database and the current EF model
  When the parity gate compares the RLS policies present vs RlsDdl.StatementsFor(model)
  Then any ITenantScoped table missing its policy fails the build
```

`RlsMigrationGateTests` asserts a *migrated* database matches what `RlsDdl` derives from the *current model* — so adding a tenant entity and forgetting its policy migration is a **red build**, not a production gap. It's the same "completeness enforced by the compiler/CI, not by

diligence" principle as the GDPR R12 gate (7.1) and the RLS-DDL-derived-from-the-model design: the invariant is structural, and the day someone forgets, CI says so.

7. Red — the second wall, proven

```
Scenario: RLS blocks a cross-tenant read even when the app filter is bypassed
  Given tenant A's row and a connection with app.tenant_id = B (subject to RLS)
  When a raw query (no EF filter) selects the row
  Then zero rows come back — the database refused it

Scenario: an unset tenant sees nothing (fail-closed)
  Given no app.tenant_id set and no bypass
  Then every tenant-scoped table returns zero rows

Scenario: WITH CHECK rejects a foreign-tenant write
  When a command scoped to tenant A tries to INSERT a row stamped tenant B
  Then the database rejects it

Scenario: a tagged cross-tenant query is allowed; an untagged one is not
  Then .TagWith(rls:cross-tenant) reads across tenants; the same query untagged is walled

Scenario: the app refuses to boot as a privileged role (posture guard)
  Given Rls:EnforceRuntimeRole=true and a superuser connection
  Then startup throws
```

`RlsBackstopTests` drive these against **real Postgres as a non-privileged role** (`RlsTestSetup` provisions it) — because a superuser connection would bypass RLS and the test would prove nothing, the exact false-confidence trap the posture guard exists for. This is the ultimate vindication of 1.3's "test against real Postgres, never the in-memory provider": RLS *is* a Postgres feature, and only a real, correctly-privileged Postgres can prove it works.

8. Run it

```
dotnet test tests/Api.Tests --filter Rls
# locally you're a superuser, so RLS is exempt — the tests provision a non-privileged role
to see it.
# in staging/prod (two-role topology + Rls:EnforceRuntimeRole=true): the app boots as the
runtime role,
# the posture guard verifies it's really subject to RLS, and a bypassed app filter still
leaks nothing.
```

9. Architecture Decision

ARCHITECTURE DECISION

The fork: how is the tenant-isolation invariant enforced? (a) The EF query filter alone (one wall, in the app); (b) RLS alone (drop the app filter, let the DB do it); (c) both — the app filter as the everyday guard *and* a fail-closed, model-derived Postgres RLS backstop, with explicit per-query bypass, a two-role topology, a boot-time posture guard, and a CI parity gate.

Chosen: (c) (ADR-020). Defense in depth: a single missed `.Where`/`IgnoreQueryFilters`/raw-SQL in the app is caught by the DB wall, and a hypothetical RLS misconfiguration is still fronted by the app filter. Fail-closed policies (unset tenant → no rows) make the safe state the default; model-derived DDL + the CI parity gate make the policy set impossible to drift; the posture guard makes "silently bypassed" a loud boot failure; explicit tags keep the sanctioned cross-tenant paths working without a blanket off-switch.

Rejected: (a) — one wall in one fallible layer is one bug from a catastrophic leak; too thin for the invariant that matters most. (b) — RLS *alone* means every query pays the GUC round-trip and loses EF-level ergonomics, and it's all-or-nothing at the DB with no fast app-layer scoping; the app filter is the better everyday mechanism, with RLS as the backstop beneath it.

The trade: real operational complexity — a two-role database topology to provision, a per-command GUC interceptor to maintain, a posture guard and parity gate to keep green, and local dev (superuser) not exercising RLS so tests must provision a non-privileged role. Accepted: for a multi-tenant SaaS, a cross-tenant leak is existential, and a second independent wall in the database layer is proportionate insurance — the complexity is bounded, tested, and fails closed.

10. Checkpoint

```
git add -A && git commit -m "feat: RLS tenancy backstop — fail-closed model-derived
policies, session interceptor, posture guard, parity gate"
git tag lesson-8.4
```

Part 8 complete. The app now ships: single-origin so auth survives the browser's cookie rules, in one reproducible container brought up on a free-tier staging host, through a pipeline that auto-deploys and version-gates staging while gating prod behind a human — and its core invariant is guarded by two independent walls, the app filter and a fail-closed database backstop.

You should now be able to: explain defense in depth and why the tenant invariant warrants a second wall in the DB layer; describe the fail-closed RLS policy (`USING+WITH CHECK`, unset-GUC→no-rows, `FORCE`); explain how the session interceptor carries the tenant per command and why bypass is explicit-only; justify the two-role topology + posture guard (the superuser-exemption trap); and explain how model-derived DDL + the parity gate keep policies from drifting.

Next: *Part 9 — Make it yours* — rebranding every touchpoint, the "OUT list" and scope discipline, and turning this reference platform into *your* product.